

CS8165.001 Complete Lecture Readers

Readable commentary for each lecture in the course's primary language. This document is designed to make the raw slides human-readable while keeping links back to the extracted source chunks.

Contents

Lecture 01 - Introduction

Lecture 02 - Rendering Pipeline

Lecture 03 - Geometric Transformations

Lecture 04 - Geometric Projection

Lecture 05 - Clipping

Lecture 06 - Rasterization

Lecture 07 - Visibility Determination

Lecture 08 - Local Illumination

Lecture 09 - Texturing

Lecture 10 - Shadows

Lecture 01 - Introduction

This lecture is the orientation layer for the whole course. It explains why interactive computer graphics is a pipeline problem: a scene must be represented, processed under time constraints, sampled into an image, and displayed fast enough for interaction.

Source chunk	course_text_parts/03_lectures/01_introduction.txt
Extracted slide pages	51
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.

Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Course Learning Outcomes
Source cue	- Learn to program computer graphics applications using OpenGL / - Learn to realize efficient computer graphics applications on the GPU / - Implement your own computer graphics project / [Image Credits: Wikipedia User Mimigu] [Crassin et al., PG2011]
Professor-style explanation	The slide 'Course Learning Outcomes' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Learn to program computer graphics applications using OpenGL / - Learn to realize efficient computer graphics applications on the GPU / - Implement your own computer graphics project / [Image Credits: Wikipedia User Mimigu] [Crassin et al., PG2011]. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations. For examination purposes, the important content is the connection between lecture concept and practiced skill: the course does not separate theory from implementation.
Technical commentary	This slide is about Course Learning Outcomes. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Learn to program computer graphics applications using OpenGL / - Learn to realize efficient computer graphics applications on the GPU / - Implement your own computer graphics project / [Image Credits: Wikipedia User Mimigu] [Crassin et al., PG2011]. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Exam-grade answer	A strong answer for 'Course Learning Outcomes' connects the course object to a concrete learning output: code, exercise, project work, or assessed concept.
Common trap	Do not treat 'Course Learning Outcomes' as administrative filler if it points to exercises, tools, or expected implementation skills.
Check yourself	Can you connect 'Course Learning Outcomes' to an exercise, project task, or exam-preparation action?

Page 3	Real-Time Rendering Contest Results 1/3
Source cue	Students: Frank Richter, Manuel Herbert Güntzel
Professor-style explanation	The slide 'Real-Time Rendering Contest Results 1/3' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Students: Frank Richter, Manuel Herbert Güntzel. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input. For examination purposes, the important content is not the date alone but the reason the milestone matters: every historical item solved a limitation in representation, interaction, display, or computation.
Technical commentary	This slide is about Real-Time Rendering Contest Results 1/3. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Students: Frank Richter, Manuel Herbert Güntzel. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Exam-grade answer	A strong answer for 'Real-Time Rendering Contest Results 1/3' states the milestone and the graphics limitation it helped overcome, such as limited interaction, limited displays, missing 3D representation, or slow rendering.
Common trap	Do not learn 'Real-Time Rendering Contest Results 1/3' as trivia only; connect each historical item to the technical capability it introduced or made practical.
Check yourself	Can you name the graphics capability or idea represented by 'Real-Time Rendering Contest Results 1/3' and why it mattered historically?
Page 4	Real-Time Rendering Contest Results 2/3
Source cue	Students: Andreas Rottach, Matthias Englert

Professor-style explanation	The slide 'Real-Time Rendering Contest Results 2/3' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Students: Andreas Rottach, Matthias Englert. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input. For examination purposes, the important content is not the date alone but the reason the milestone matters: every historical item solved a limitation in representation, interaction, display, or computation.
Technical commentary	This slide is about Real-Time Rendering Contest Results 2/3. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Students: Andreas Rottach, Matthias Englert. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Exam-grade answer	A strong answer for 'Real-Time Rendering Contest Results 2/3' states the milestone and the graphics limitation it helped overcome, such as limited interaction, limited displays, missing 3D representation, or slow rendering.
Common trap	Do not learn 'Real-Time Rendering Contest Results 2/3' as trivia only; connect each historical item to the technical capability it introduced or made practical.
Check yourself	Can you name the graphics capability or idea represented by 'Real-Time Rendering Contest Results 2/3' and why it mattered historically?
Page 5	Real-Time Rendering Contest Results 3/3
Source cue	Students: Jan Eric Haßler, Kai Viktor Freissler

Professor-style explanation	The slide 'Real-Time Rendering Contest Results 3/3' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Students: Jan Eric Haßler, Kai Viktor Freissler. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input. For examination purposes, the important content is not the date alone but the reason the milestone matters: every historical item solved a limitation in representation, interaction, display, or computation.
Technical commentary	This slide is about Real-Time Rendering Contest Results 3/3. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Students: Jan Eric Haßler, Kai Viktor Freissler. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Exam-grade answer	A strong answer for 'Real-Time Rendering Contest Results 3/3' states the milestone and the graphics limitation it helped overcome, such as limited interaction, limited displays, missing 3D representation, or slow rendering.
Common trap	Do not learn 'Real-Time Rendering Contest Results 3/3' as trivia only; connect each historical item to the technical capability it introduced or made practical.
Check yourself	Can you name the graphics capability or idea represented by 'Real-Time Rendering Contest Results 3/3' and why it mattered historically?
Page 6	1.1 Course Organization
Source cue	Details about the lecture course

Professor-style explanation	The slide '1.1 Course Organization' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: Details about the lecture course. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations. For examination purposes, the important content is the connection between lecture concept and practiced skill: the course does not separate theory from implementation.
Technical commentary	This slide is about 1.1 Course Organization. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: Details about the lecture course. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Exam-grade answer	A strong answer for '1.1 Course Organization' connects the course object to a concrete learning output: code, exercise, project work, or assessed concept.
Common trap	Do not treat '1.1 Course Organization' as administrative filler if it points to exercises, tools, or expected implementation skills.
Check yourself	Can you connect '1.1 Course Organization' to an exercise, project task, or exam-preparation action?
Page 7	Teachers
Source cue	- Lectures / timo.ropinski@uni-ulm.de / Office hours upon request / - Exercises / Poonam Aditya Sawant

Professor-style explanation	The slide 'Teachers' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Lectures / timo.ropinski@uni-ulm.de / Office hours upon request / - Exercises / Poonam Aditya Sawant. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations. For examination purposes, the important content is the connection between lecture concept and practiced skill: the course does not separate theory from implementation.
Technical commentary	This slide is about Teachers. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Lectures / timo.ropinski@uni-ulm.de / Office hours upon request / - Exercises / Poonam Aditya Sawant. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Exam-grade answer	A strong answer for 'Teachers' connects the course object to a concrete learning output: code, exercise, project work, or assessed concept.
Common trap	Do not treat 'Teachers' as administrative filler if it points to exercises, tools, or expected implementation skills.
Check yourself	Can you connect 'Teachers' to an exercise, project task, or exam-preparation action?
Page 8	Course Format
Source cue	- Lectures / - Tuesday (14-16) and every other Wednesdays (12-14) / - Room O28/1002 / - Specific dates/modalities in Moodle (password: ICG26SUMMER) / - Lecture slides

Professor-style explanation	The slide 'Course Format' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Lectures / - Tuesday (14-16) and every other Wednesdays (12-14) / - Room O28/1002 / - Specific dates/modalities in Moodle (password: ICG26SUMMER) / - Lecture slides. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations. For examination purposes, the important content is the connection between lecture concept and practiced skill: the course does not separate theory from implementation.
Technical commentary	This slide is about Course Format. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Lectures / - Tuesday (14-16) and every other Wednesdays (12-14) / - Room O28/1002 / - Specific dates/modalities in Moodle (password: ICG26SUMMER) / - Lecture slides. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Exam-grade answer	A strong answer for 'Course Format' connects the course object to a concrete learning output: code, exercise, project work, or assessed concept.
Common trap	Do not treat 'Course Format' as administrative filler if it points to exercises, tools, or expected implementation skills.
Check yourself	Can you connect 'Course Format' to an exercise, project task, or exam-preparation action?
Page 9	Course Format
Source cue	- Exercises / - Every other Wednesday (12-14) / - Room O28/1002 / - Programming in C/C++ (introduction available upon request) / - Enroll in Moodle

Professor-style explanation	The slide 'Course Format' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Exercises / - Every other Wednesday (12-14) / - Room O28/1002 / - Programming in C/C++ (introduction available upon request) / - Enroll in Moodle. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations. For examination purposes, the important content is the connection between lecture concept and practiced skill: the course does not separate theory from implementation.
Technical commentary	This slide is about Course Format. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Exercises / - Every other Wednesday (12-14) / - Room O28/1002 / - Programming in C/C++ (introduction available upon request) / - Enroll in Moodle. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Exam-grade answer	A strong answer for 'Course Format' connects the course object to a concrete learning output: code, exercise, project work, or assessed concept.
Common trap	Do not treat 'Course Format' as administrative filler if it points to exercises, tools, or expected implementation skills.
Check yourself	Can you connect 'Course Format' to an exercise, project task, or exam-preparation action?
Page 10	Exercises
Source cue	- Exercise / - Exercise 1 - Rendering Pipeline / - Exercise 2 - Primitive Types and Shaders / - Exercise 3 - Geometric Transformations / - Exercise 4 - Projection and Clipping

Professor-style explanation	The slide 'Exercises' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Exercise / - Exercise 1 - Rendering Pipeline / - Exercise 2 - Primitive Types and Shaders / - Exercise 3 - Geometric Transformations / - Exercise 4 - Projection and Clipping. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations. For examination purposes, the important content is the connection between lecture concept and practiced skill: the course does not separate theory from implementation.
Technical commentary	This slide is about Exercises. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Exercise / - Exercise 1 - Rendering Pipeline / - Exercise 2 - Primitive Types and Shaders / - Exercise 3 - Geometric Transformations / - Exercise 4 - Projection and Clipping. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Exam-grade answer	A strong answer for 'Exercises' connects the course object to a concrete learning output: code, exercise, project work, or assessed concept.
Common trap	Do not treat 'Exercises' as administrative filler if it points to exercises, tools, or expected implementation skills.
Check yourself	Can you connect 'Exercises' to an exercise, project task, or exam-preparation action?
Page 11	Successful Course Completion
Source cue	- Pass the oral/written exam / - Dates to be announced / - "Notenbonus" possible / - 50% success rate of exercise solutions for n-1 sheets / - Only running solutions are marked

Professor-style explanation	The slide 'Successful Course Completion' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - Pass the oral/written exam / - Dates to be announced / - "Notenbonus" possible / - 50% success rate of exercise solutions for n-1 sheets / - Only running solutions are marked. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations. For examination purposes, the important content is the connection between lecture concept and practiced skill: the course does not separate theory from implementation.
Technical commentary	This slide is about Successful Course Completion. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - Pass the oral/written exam / - Dates to be announced / - "Notenbonus" possible / - 50% success rate of exercise solutions for n-1 sheets / - Only running solutions are marked. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Exam-grade answer	A strong answer for 'Successful Course Completion' connects the course object to a concrete learning output: code, exercise, project work, or assessed concept.
Common trap	Do not treat 'Successful Course Completion' as administrative filler if it points to exercises, tools, or expected implementation skills.
Check yourself	Can you connect 'Successful Course Completion' to an exercise, project task, or exam-preparation action?
Page 12	Course Overview
Source cue	Background / Journey along the / Rendering Pipeline / Lighting Effects / Extrahierte Tabellen:

Professor-style explanation	The slide 'Course Overview' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: Background / Journey along the / Rendering Pipeline / Lighting Effects / Extrahierte Tabellen. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations. For examination purposes, the important content is the connection between lecture concept and practiced skill: the course does not separate theory from implementation.
Technical commentary	This slide is about Course Overview. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: Background / Journey along the / Rendering Pipeline / Lighting Effects / Extrahierte Tabellen. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Exam-grade answer	A strong answer for 'Course Overview' connects the course object to a concrete learning output: code, exercise, project work, or assessed concept.
Common trap	Do not treat 'Course Overview' as administrative filler if it points to exercises, tools, or expected implementation skills.
Check yourself	Can you connect 'Course Overview' to an exercise, project task, or exam-preparation action?
Page 13	Literature
Source cue	- P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics / (4th Edition), AK Peters 2016. / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013.

Professor-style explanation	The slide 'Literature' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics / (4th Edition), AK Peters 2016. - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.
Technical commentary	This slide is about Literature. The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics / (4th Edition), AK Peters 2016. - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Literature' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Literature' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Literature' points to for deeper study?
Page 14	Relevant Chapters and Link to Lecture Course
Source cue	- For self study

Professor-style explanation	The slide 'Relevant Chapters and Link to Lecture Course' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. The listed course objects are connected as workflow components: - For self study. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations. For examination purposes, the important content is the connection between lecture concept and practiced skill: the course does not separate theory from implementation.
Technical commentary	This slide is about Relevant Chapters and Link to Lecture Course. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. The listed course objects are connected as workflow components: - For self study. Lecture topics introduce concepts, exercises convert them into practice, and project or assessment elements test whether those concepts can be used.
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Exam-grade answer	A strong answer for 'Relevant Chapters and Link to Lecture Course' connects the course object to a concrete learning output: code, exercise, project work, or assessed concept.
Common trap	Do not treat 'Relevant Chapters and Link to Lecture Course' as administrative filler if it points to exercises, tools, or expected implementation skills.
Check yourself	Can you connect 'Relevant Chapters and Link to Lecture Course' to an exercise, project task, or exam-preparation action?
Page 15	(1.1 Course Organization)
Source cue	1.2 Computer Graphics Overview / 1.3 Pixel-based Representations / 1.4 3D Models / 1.5 Algorithmic Paradigms

Professor-style explanation	The outline '(1.1 Course Organization)' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: 1.2 Computer Graphics Overview / 1.3 Pixel-based Representations / 1.4 3D Models / 1.5 Algorithmic Paradigms. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about (1.1 Course Organization). The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: 1.2 Computer Graphics Overview / 1.3 Pixel-based Representations / 1.4 3D Models / 1.5 Algorithmic Paradigms. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for '(1.1 Course Organization)' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize '(1.1 Course Organization)' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where '(1.1 Course Organization)' fits in the lecture order and what later section depends on it?
Page 16	1.2 Computer Graphics Overview
Source cue	Relation of the field with respect to other disciplines
Professor-style explanation	The slide '1.2 Computer Graphics Overview' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Relation of the field with respect to other disciplines. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about 1.2 Computer Graphics Overview. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Relation of the field with respect to other disciplines. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for '1.2 Computer Graphics Overview' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer '1.2 Computer Graphics Overview' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn '1.2 Computer Graphics Overview' into a causal sentence instead of repeating the slide title?

Page 17	Computer Graphics History
Source cue	- Origins in art / - Perspective, color, etc. / - Computer graphics milestones [Reichardt, 1968] / - 1960 - "Computer Graphics", William Fetter (Boeing) / - 1962 - Spacewars, Steve Russel (MIT) Computer
Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: - Origins in art / - Perspective, color, etc. / - Computer graphics milestones [Reichardt, 1968] / - 1960 - "Computer Graphics", William Fetter (Boeing) / - 1962 - Spacewars, Steve Russel (MIT) Computer. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input. For examination purposes, the important content is not the date alone but the reason the milestone matters: every historical item solved a limitation in representation, interaction, display, or computation.

Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: - Origins in art / - Perspective, color, etc. / - Computer graphics milestones [Reichardt, 1968] / - 1960 - "Computer Graphics", William Fetter (Boeing) / - 1962 - Spacewars, Steve Russel (MIT) Computer. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Exam-grade answer	A strong answer for 'Computer Graphics History' states the milestone and the graphics limitation it helped overcome, such as limited interaction, limited displays, missing 3D representation, or slow rendering.
Common trap	Do not learn 'Computer Graphics History' as trivia only; connect each historical item to the technical capability it introduced or made practical.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?

Page 18	Computer Graphics History
Source cue	Sutherland, Utah (1970) Wireframe
Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Sutherland, Utah (1970) Wireframe. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input. For examination purposes, the important content is not the date alone but the reason the milestone matters: every historical item solved a limitation in representation, interaction, display, or computation.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Sutherland, Utah (1970) Wireframe. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.

Exam-grade answer	A strong answer for 'Computer Graphics History' states the milestone and the graphics limitation it helped overcome, such as limited interaction, limited displays, missing 3D representation, or slow rendering.
Common trap	Do not learn 'Computer Graphics History' as trivia only; connect each historical item to the technical capability it introduced or made practical.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?

Page 19	Computer Graphics History
Source cue	Sutherland, Utah (1970) Turner Whitted, Siggraph (1979)
Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Sutherland, Utah (1970) Turner Whitted, Siggraph (1979). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input. For examination purposes, the important content is not the date alone but the reason the milestone matters: every historical item solved a limitation in representation, interaction, display, or computation.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Sutherland, Utah (1970) Turner Whitted, Siggraph (1979). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Exam-grade answer	A strong answer for 'Computer Graphics History' states the milestone and the graphics limitation it helped overcome, such as limited interaction, limited displays, missing 3D representation, or slow rendering.
Common trap	Do not learn 'Computer Graphics History' as trivia only; connect each historical item to the technical capability it introduced or made practical.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?

Page 20	Computer Graphics History
Source cue	The Abyss (1989) Terminator 2 (1991) Jurassic Park (1993)

Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: The Abyss (1989) Terminator 2 (1991) Jurassic Park (1993). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input. For examination purposes, the important content is not the date alone but the reason the milestone matters: every historical item solved a limitation in representation, interaction, display, or computation.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: The Abyss (1989) Terminator 2 (1991) Jurassic Park (1993). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Exam-grade answer	A strong answer for 'Computer Graphics History' states the milestone and the graphics limitation it helped overcome, such as limited interaction, limited displays, missing 3D representation, or slow rendering.
Common trap	Do not learn 'Computer Graphics History' as trivia only; connect each historical item to the technical capability it introduced or made practical.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?
Page 21	Computer Graphics History
Source cue	Ikea (2014)

Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: Ikea (2014). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input. For examination purposes, the important content is not the date alone but the reason the milestone matters: every historical item solved a limitation in representation, interaction, display, or computation.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: Ikea (2014). Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Exam-grade answer	A strong answer for 'Computer Graphics History' states the milestone and the graphics limitation it helped overcome, such as limited interaction, limited displays, missing 3D representation, or slow rendering.
Common trap	Do not learn 'Computer Graphics History' as trivia only; connect each historical item to the technical capability it introduced or made practical.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?
Page 22	Related Disciplines
Source cue	- Human-Computer Interaction / - Virtual and Augmented Reality / - Visualization / - Image Processing / - Computer Vision

Professor-style explanation	The slide 'Related Disciplines' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Human-Computer Interaction / - Virtual and Augmented Reality / - Visualization / - Image Processing / - Computer Vision. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Related Disciplines. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Human-Computer Interaction / - Virtual and Augmented Reality / - Visualization / - Image Processing / - Computer Vision. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Related Disciplines' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Related Disciplines' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Related Disciplines' into a causal sentence instead of repeating the slide title?

Page 23	Applications
Source cue	- Computer games / - Animation movies / - Special effects in movies / - CAD/CAM [CryEngine 5.7, CryTek 2022] / - Simulation
Professor-style explanation	The slide 'Applications' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Computer games / - Animation movies / - Special effects in movies / - CAD/CAM [CryEngine 5.7, CryTek 2022] / - Simulation. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Applications. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Computer games / - Animation movies / - Special effects in movies / - CAD/CAM [CryEngine 5.7, CryTek 2022] / - Simulation. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Applications' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Applications' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Applications' into a causal sentence instead of repeating the slide title?

Page 24	1.3 Pixel-based Representations
Source cue	What is a pixel?
Professor-style explanation	The slide '1.3 Pixel-based Representations' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: What is a pixel?. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations. For examination purposes, the important content is the chain width x height -> pixel count -> bits per pixel -> memory size; this chain also explains bandwidth and performance pressure.
Technical commentary	This slide is about 1.3 Pixel-based Representations. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: What is a pixel?. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Exam-grade answer	A strong answer for '1.3 Pixel-based Representations' includes the formula relation width x height x bits per pixel, distinguishes bits from bytes, and links larger images to memory bandwidth.
Common trap	Do not confuse bits with bytes in '1.3 Pixel-based Representations', and do not ignore that alpha or higher precision changes memory size.

Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in '1.3 Pixel-based Representations'?
Page 25	Raster Images
Source cue	["Bibliothek" by press office Ulm University]
Professor-style explanation	The slide 'Raster Images' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: ["Bibliothek" by press office Ulm University]. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations. For examination purposes, the important content is the chain width x height -> pixel count -> bits per pixel -> memory size; this chain also explains bandwidth and performance pressure.
Technical commentary	This slide is about Raster Images. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: ["Bibliothek" by press office Ulm University]. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Exam-grade answer	A strong answer for 'Raster Images' includes the formula relation width x height x bits per pixel, distinguishes bits from bytes, and links larger images to memory bandwidth.
Common trap	Do not confuse bits with bytes in 'Raster Images', and do not ignore that alpha or higher precision changes memory size.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'Raster Images'?
Page 26	Pixel
Source cue	- Name derived from Word for Picture Element / - Pixel is a sample with associated sample values (=pixel components) / - Color / - Transparency (=alpha value) / - Depth (i.e., distance from viewer (=z value))

Professor-style explanation	The slide 'Pixel' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Name derived from Word for Picture Element / - Pixel is a sample with associated sample values (=pixel components) / - Color / - Transparency (=alpha value) / - Depth (i.e., distance from viewer (=z value)). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Pixel. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Name derived from Word for Picture Element / - Pixel is a sample with associated sample values (=pixel components) / - Color / - Transparency (=alpha value) / - Depth (i.e., distance from viewer (=z value)). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Pixel' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Pixel' works per object, per image region, per ray, or per fragment?
Page 27	Color
Source cue	- In real world color is given / by color spectrum Day Light Light Bulb / Fluorescent Tube White LED / - In computer graphics color is represented through color models / - RGB color model (additive)

Professor-style explanation	The slide 'Color' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - In real world color is given / by color spectrum Day Light Light Bulb / Fluorescent Tube White LED / - In computer graphics color is represented through color models / - RGB color model (additive). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Color. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - In real world color is given / by color spectrum Day Light Light Bulb / Fluorescent Tube White LED / - In computer graphics color is represented through color models / - RGB color model (additive). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Color' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Color'?
Page 28	LCD Raster Screen
Source cue	- Liquid crystal displays exist since the 90s and enable flat screens / - Liquid crystals affect the polarization of light / - Based on polarization, subpixel visibility is regulated

Professor-style explanation	The slide 'LCD Raster Screen' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Liquid crystal displays exist since the 90s and enable flat screens / - Liquid crystals affect the polarization of light / - Based on polarization, subpixel visibility is regulated. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about LCD Raster Screen. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Liquid crystal displays exist since the 90s and enable flat screens / - Liquid crystals affect the polarization of light / - Based on polarization, subpixel visibility is regulated. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'LCD Raster Screen' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'LCD Raster Screen'?
Page 29	Raster Image Sizes
Source cue	- Assume a raster image with 1024x768 pixels and 24-bit color depth / - How many different colors can be represented? / - $2^{24} = 2^8 \times 2^8 \times 2^8 = 16,777,216$ / - How much memory does the image need? / - $1024 \times 768 \times 24 = 18,874,368$ bits = 2,359,296 bytes = 2.25 MiB

Professor-style explanation	The slide 'Raster Image Sizes' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: - Assume a raster image with 1024x768 pixels and 24-bit color depth / - How many different colors can be represented? / - $2^{24} = 2^8 \times 2^8 \times 2^8 = 16.777.216$ / - How much memory does the image need? / - $1024 \times 768 \times 24 = 18,874,368$ bits = 2,359,296 bytes = 2.25 MiB. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations. For examination purposes, the important content is the chain width x height -> pixel count -> bits per pixel -> memory size; this chain also explains bandwidth and performance pressure.
Technical commentary	This slide is about Raster Image Sizes. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: - Assume a raster image with 1024x768 pixels and 24-bit color depth / - How many different colors can be represented? / - $2^{24} = 2^8 \times 2^8 \times 2^8 = 16.777.216$ / - How much memory does the image need? / - $1024 \times 768 \times 24 = 18,874,368$ bits = 2,359,296 bytes = 2.25 MiB. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Exam-grade answer	A strong answer for 'Raster Image Sizes' includes the formula relation width x height x bits per pixel, distinguishes bits from bytes, and links larger images to memory bandwidth.
Common trap	Do not confuse bits with bytes in 'Raster Image Sizes', and do not ignore that alpha or higher precision changes memory size.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'Raster Image Sizes'?
Page 30	1.4 3D Models
Source cue	Modeling with triangles and other geometric primitives

Professor-style explanation	The slide '1.4 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: Modeling with triangles and other geometric primitives. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about 1.4 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: Modeling with triangles and other geometric primitives. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for '1.4 3D Models' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe '1.4 3D Models' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by '1.4 3D Models' before rendering begins?
Page 31	3D Model Types
Source cue	- Models specify the structures to be displayed / - Three commonly used model types exist / - Implicit surfaces / - Polygonal models / - Volumetric models

Professor-style explanation	The slide '3D Model Types' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Models specify the structures to be displayed / - Three commonly used model types exist / - Implicit surfaces / - Polygonal models / - Volumetric models. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about 3D Model Types. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Models specify the structures to be displayed / - Three commonly used model types exist / - Implicit surfaces / - Polygonal models / - Volumetric models. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for '3D Model Types' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe '3D Model Types' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by '3D Model Types' before rendering begins?
Page 32	Implicit Surfaces
Source cue	- Set of all points that fulfill $f(x, y, z) = 0$ / - All points with $f(x, y, z) < 0$ define solid bound by $f(x, y, z) = 0$ / - Display possibilities / - Search for null points / - Approximate with polygons

Professor-style explanation	The slide 'Implicit Surfaces' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Set of all points that fulfill $f(x, y, z) = 0$ / - All points with $f(x, y, z) < 0$ define solid bound by $f(x, y, z) = 0$ / - Display possibilities / - Search for null points / - Approximate with polygons. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Implicit Surfaces. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Set of all points that fulfill $f(x, y, z) = 0$ / - All points with $f(x, y, z) < 0$ define solid bound by $f(x, y, z) = 0$ / - Display possibilities / - Search for null points / - Approximate with polygons. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Implicit Surfaces' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Implicit Surfaces' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Implicit Surfaces' into a causal sentence instead of repeating the slide title?
Page 33	Quadrics
Source cue	- Surfaces represented by quadric equations / Ellipsoid Single Hyperboloid Double Hyperboloid / $x^2 y^2 z^2$ / $f(x, y, z) = + - 1$ $f(x, y, z) = + - - 1$ $f(x, y, z) = + - + 1$ / $\alpha^2 \beta^2 \gamma^2$ $\alpha^2 \beta^2 \gamma^2$ $\alpha^2 \beta^2 \gamma^2$

Professor-style explanation	The slide 'Quadrics' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Surfaces represented by quadric equations / Ellipsoid Single Hyperboloid Double Hyperboloid / $x^2 y^2 z^2 x^2 y^2 z^2 x^2 y^2 z^2 / f x, y, z = + + - 1 f x, y, z = + - - 1 f x, y, z = + - + 1 / \alpha^2 \beta^2 \gamma^2 \alpha^2 \beta^2 \gamma^2 \alpha^2 \beta^2 \gamma^2$. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Quadrics. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Surfaces represented by quadric equations / Ellipsoid Single Hyperboloid Double Hyperboloid / $x^2 y^2 z^2 x^2 y^2 z^2 x^2 y^2 z^2 / f x, y, z = + + - 1 f x, y, z = + - - 1 f x, y, z = + - + 1 / \alpha^2 \beta^2 \gamma^2 \alpha^2 \beta^2 \gamma^2 \alpha^2 \beta^2 \gamma^2$. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Quadrics' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Quadrics' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Quadrics' into a causal sentence instead of repeating the slide title?
Page 34	Object Representation
Source cue	Point Cloud (3D Scan) Binary Data (MRT) Signed Distance Fields (SDF) Meshes
Professor-style explanation	The slide 'Object Representation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Point Cloud (3D Scan) Binary Data (MRT) Signed Distance Fields (SDF) Meshes. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Object Representation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Point Cloud (3D Scan) Binary Data (MRT) Signed Distance Fields (SDF) Meshes. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Object Representation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Object Representation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Object Representation' into a causal sentence instead of repeating the slide title?

Page 35	Polygonal Models 1/3
Source cue	- Surface representation is approximated by polygons / - Polygons are made up of vertices and edges
Professor-style explanation	The slide 'Polygonal Models 1/3' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Surface representation is approximated by polygons / - Polygons are made up of vertices and edges. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Polygonal Models 1/3. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Surface representation is approximated by polygons / - Polygons are made up of vertices and edges. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Polygonal Models 1/3' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.

Common trap	Do not describe 'Polygonal Models 1/3' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 1/3' before rendering begins?
Page 36	Polygonal Models 1/4
Source cue	- Surface representation is approximated by polygons (mostly triangles/quads) / - Polygons are made up of vertices and edges / Valence / Degree Adjacent Edges 1-Ring Neighborhood
Professor-style explanation	The slide 'Polygonal Models 1/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Surface representation is approximated by polygons (mostly triangles/quads) / - Polygons are made up of vertices and edges / Valence / Degree Adjacent Edges 1-Ring Neighborhood. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Polygonal Models 1/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Surface representation is approximated by polygons (mostly triangles/quads) / - Polygons are made up of vertices and edges / Valence / Degree Adjacent Edges 1-Ring Neighborhood. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Polygonal Models 1/4' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Polygonal Models 1/4' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 1/4' before rendering begins?
Page 37	Polygonal Models 2/4

Source cue	- Surface representation are often approximated by triangular polygons / - Triangles consist of three vertices and three edges / - Display achieved through rendering by raster conversion of triangles
Professor-style explanation	The slide 'Polygonal Models 2/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Surface representation are often approximated by triangular polygons / - Triangles consist of three vertices and three edges / - Display achieved through rendering by raster conversion of triangles. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Polygonal Models 2/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Surface representation are often approximated by triangular polygons / - Triangles consist of three vertices and three edges / - Display achieved through rendering by raster conversion of triangles. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Polygonal Models 2/4' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Polygonal Models 2/4' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 2/4' before rendering begins?
Page 38	Polygonal Models 3/4
Source cue	- Various attributes are associated with vertices of a polygonal model / - Colors / - Material properties / - Textures / - ...

Professor-style explanation	The slide 'Polygonal Models 3/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Various attributes are associated with vertices of a polygonal model / - Colors / - Material properties / - Textures / - . Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Polygonal Models 3/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Various attributes are associated with vertices of a polygonal model / - Colors / - Material properties / - Textures / - . Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Polygonal Models 3/4' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Polygonal Models 3/4' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 3/4' before rendering begins?
Page 39	Polygonal Models 4/4
Source cue	- Surface normal required in many cases / - Normal is calculated through cross product: $\mathbf{n} = \frac{\mathbf{a} \times \mathbf{b}}{\ \mathbf{a} \times \mathbf{b}\ }$ / - Example use cases / $\mathbf{a} \times \mathbf{b} = a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x$

Professor-style explanation	The slide 'Polygonal Models 4/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Surface normal required in many cases / - Normal is calculated through cross product: $\mathbf{n} = (\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})$ / - Example use cases / a b – a b / 2 3 3 2. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Polygonal Models 4/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Surface normal required in many cases / - Normal is calculated through cross product: $\mathbf{n} = (\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})$ / - Example use cases / a b – a b / 2 3 3 2. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Polygonal Models 4/4' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Polygonal Models 4/4' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 4/4' before rendering begins?
Page 40	Volumetric Models
Source cue	- Space filling object representation through voxels (volume elements) / - Display possibilities / - Rendering of extracted surfaces / - Direct rendering through specialized algorithms

Professor-style explanation	The slide 'Volumetric Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Space filling object representation through voxels (volume elements) / - Display possibilities / - Rendering of extracted surfaces / - Direct rendering through specialized algorithms. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Volumetric Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Space filling object representation through voxels (volume elements) / - Display possibilities / - Rendering of extracted surfaces / - Direct rendering through specialized algorithms. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Volumetric Models' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Volumetric Models' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Volumetric Models' before rendering begins?
Page 41	Model Generation
Source cue	- Polygonal models are generated through 3D scanning or 3D modeling / (e.g., LIDAR, Blender, Cinema 4D, Maya) / - Volumetric models are generated through imaging or simulation / (e.g., CT, MRI, PET, CFD, seismic surveys)

Professor-style explanation	The slide 'Model Generation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Polygonal models are generated through 3D scanning or 3D modeling / (e.g., LIDAR, Blender, Cinema 4D, Maya) / - Volumetric models are generated through imaging or simulation / (e.g., CT, MRI, PET, CFD, seismic surveys). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Model Generation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Polygonal models are generated through 3D scanning or 3D modeling / (e.g., LIDAR, Blender, Cinema 4D, Maya) / - Volumetric models are generated through imaging or simulation / (e.g., CT, MRI, PET, CFD, seismic surveys). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Model Generation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Model Generation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Model Generation' into a causal sentence instead of repeating the slide title?
Page 42	Coordinate Systems
Source cue	- Three main coordinate system exist / - Model coordinate system - model specified in own coordinate system / - World coordinate system - models arranged in common coordinate system / - Eye/camera coordinate system - model in relation to camera / - To transfer between coordinate systems,

Professor-style explanation	The slide 'Coordinate Systems' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Three main coordinate system exist / - Model coordinate system - model specified in own coordinate system / - World coordinate system - models arranged in common coordinate system / - Eye/camera coordinate system - model in relation to camera / - To transfer between coordinate systems,. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Coordinate Systems. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Three main coordinate system exist / - Model coordinate system - model specified in own coordinate system / - World coordinate system - models arranged in common coordinate system / - Eye/camera coordinate system - model in relation to camera / - To transfer between coordinate systems,. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Coordinate Systems' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Coordinate Systems' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Coordinate Systems'?
Page 43	Rendering
Source cue	- Rendering is the process to generate raster images from 3D models / - During rendering several steps are necessary / - Coordinate system transformation / - Visibility determination / - Illumination

Professor-style explanation	The slide 'Rendering' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Rendering is the process to generate raster images from 3D models / - During rendering several steps are necessary / - Coordinate system transformation / - Visibility determination / - Illumination. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Rendering. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Rendering is the process to generate raster images from 3D models / - During rendering several steps are necessary / - Coordinate system transformation / - Visibility determination / - Illumination. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Rendering' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Rendering' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Rendering'?
Page 44	1.5 Algorithmic Paradigms
Source cue	Frequently found algorithm strategies

Professor-style explanation	The slide '1.5 Algorithmic Paradigms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Frequently found algorithm strategies. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about 1.5 Algorithmic Paradigms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Frequently found algorithm strategies. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for '1.5 Algorithmic Paradigms' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer '1.5 Algorithmic Paradigms' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn '1.5 Algorithmic Paradigms' into a causal sentence instead of repeating the slide title?

Page 45	Photorealistic Rendering
Source cue	- Goal: simulate reality / [Embree Ray Tracing Website - http://embree.github.io/] [Maisch & Ropinski, Eurographics 2017] [Hendrik vann Jenssen]
Professor-style explanation	The slide 'Photorealistic Rendering' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Goal: simulate reality / [Embree Ray Tracing Website - http://embree.github.io/] [Maisch & Ropinski, Eurographics 2017] [Hendrik vann Jenssen]. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.

Technical commentary	This slide is about Photorealistic Rendering. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Goal: simulate reality / [Embree Ray Tracing Website - http://embree.github.io/] [Maisch & Ropinski, Eurographics 2017] [Hendrik vann Jenssen]. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Photorealistic Rendering' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Photorealistic Rendering' works per object, per image region, per ray, or per fragment?

Page 46	Non-Photorealistic Rendering (NPR)
Source cue	- Goal: abstract reality / [Suggestive contour line drawings, DeCarlo 2003]
Professor-style explanation	The slide 'Non-Photorealistic Rendering (NPR)' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Goal: abstract reality / [Suggestive contour line drawings, DeCarlo 2003]. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Non-Photorealistic Rendering (NPR). The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Goal: abstract reality / [Suggestive contour line drawings, DeCarlo 2003]. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Non-Photorealistic Rendering (NPR)' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.

Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Non-Photorealistic Rendering (NPR)'?
Page 47	Image-Based Algorithms
Source cue	- Number of computations direct proportional to number of pixels / for all Pixels do { / for all SceneObjects do { / ... / - Benefits
Professor-style explanation	The slide 'Image-Based Algorithms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Number of computations direct proportional to number of pixels / for all Pixels do { / for all SceneObjects do { / ... / - Benefits. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Image-Based Algorithms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Number of computations direct proportional to number of pixels / for all Pixels do { / for all SceneObjects do { / ... / - Benefits. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Image-Based Algorithms' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Image-Based Algorithms' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Image-Based Algorithms' into a causal sentence instead of repeating the slide title?
Page 48	Object-Based Algorithms
Source cue	- Number of computations direct proportional to number of 3D models / for all SceneObjects do { / for all Pixels do { / ... / - Benefits

Professor-style explanation	The slide 'Object-Based Algorithms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Number of computations direct proportional to number of 3D models / for all SceneObjects do { / for all Pixels do { / ... / - Benefits. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Object-Based Algorithms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Number of computations direct proportional to number of 3D models / for all SceneObjects do { / for all Pixels do { / ... / - Benefits. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Object-Based Algorithms' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Object-Based Algorithms' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Object-Based Algorithms' into a causal sentence instead of repeating the slide title?
Page 49	are assigned
Source cue	- Raster images are formed of pixels to which colors, transparencies and depth / are assigned / polygonal models / - Most frequent building blocks of polygonal models are triangles, / which are formed of vertices and edges

Professor-style explanation	The slide 'are assigned' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Raster images are formed of pixels to which colors, transparencies and depth / are assigned / polygonal models / - Most frequent building blocks of polygonal models are triangles, / which are formed of vertices and edges. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about are assigned. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Raster images are formed of pixels to which colors, transparencies and depth / are assigned / polygonal models / - Most frequent building blocks of polygonal models are triangles, / which are formed of vertices and edges. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'are assigned' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'are assigned'?
Page 50	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Literature and other sources used in this chapter' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Literature and other sources used in this chapter' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 51	Peters 2016. (Chapters 1, 3 & 21)
Source cue	- Text Books / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK / Peters 2016. (Chapters 1, 3 & 21) / - Crassin, Cyril, Fabrice Neyret, Miguel Sainz, Simon Green, and Elmar Eisemann. "Interactive / indirect illumination using voxel cone tracing." In Computer Graphics Forum, vol. 30, no. 7, pp.

Professor-style explanation	The slide 'Peters 2016. (Chapters 1, 3 & 21)' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: - Text Books / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK / Peters 2016. (Chapters 1, 3 & 21) / - Crassin, Cyril, Fabrice Neyret, Miguel Sainz, Simon Green, and Elmar Eisemann. "Interactive / indirect illumination using voxel cone tracing." In Computer Graphics Forum, vol. 30, no. 7, pp. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.
Technical commentary	This slide is about Peters 2016. (Chapters 1, 3 & 21). The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: - Text Books / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK / Peters 2016. (Chapters 1, 3 & 21) / - Crassin, Cyril, Fabrice Neyret, Miguel Sainz, Simon Green, and Elmar Eisemann. "Interactive / indirect illumination using voxel cone tracing." In Computer Graphics Forum, vol. 30, no. 7, pp. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Peters 2016. (Chapters 1, 3 & 21)' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Peters 2016. (Chapters 1, 3 & 21)' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Peters 2016. (Chapters 1, 3 & 21)' points to for deeper study?

01.1 Course Organization

Source location: Slides around 6-15

Commentary

The organizational slides matter because they explain how the theory and programming parts fit together. The exercises are not separate from the lecture; they are the practical version of the same pipeline ideas. When the course says that slides are generally not self-explanatory, that is a warning: the bullet points are anchors, not a textbook.

Treat every exercise sheet as an applied checkpoint. Rendering pipeline, primitive types, shaders, transformations, projection, clipping, and rasterization are introduced in the lectures and then turned into OpenGL tasks. If a topic appears both in a lecture and an exercise, it is more likely to be exam relevant.

Mental model

Lecture slides give the vocabulary; exercises force you to connect that vocabulary to code and visible output.

Check yourself

Can you list the five exercise themes and connect each one to a later lecture chapter?

01.2 Computer Graphics Overview

Source location: Section 1.2

Commentary

Computer graphics is about generating images from descriptions. The description can be pixel-based, object-based, physically motivated, artistic, or algorithmic. The common thread is that the computer must decide what color belongs at image samples.

Interactive graphics adds a time constraint. The image is not computed once and admired; it must respond to camera motion, user input, animation, and changing state. That is why real-time rendering often uses approximations and specialized GPU stages.

Mental model

Graphics is controlled image generation. Interactive graphics is controlled image generation under a strict time budget.

Check yourself

What changes when a renderer must be interactive rather than offline?

01.3 Pixel-Based Representations

Source location: Section 1.3

Commentary

A pixel-based representation stores an image directly as samples on a grid. It is easy to display and edit locally, but it does not know the underlying 3D scene. If you zoom into a raster image, you reveal the sampling grid, not more geometry.

This is the first place where sampling becomes important. Resolution, color depth, aliasing, and frame time are not cosmetic details. They decide what information can be represented and how much data must be processed per second.

Mental model

A raster image is already the answer. It contains color samples, not the geometric reasons behind them.

Check yourself

Why can a pixel image be easy to display but hard to reinterpret as a 3D scene?

01.4 3D Models

Source location: Section 1.4

Commentary

A 3D model is a structured description of objects before they become pixels. It may contain vertices, primitives, topology, normals, materials, textures, and transformations. Unlike a pixel image, it can be viewed from different camera positions.

The course mostly follows object-based rendering: start from model data, transform it, project it, rasterize it, shade it, test visibility, and finally update pixels. Later chapters explain these steps one by one.

Mental model

A model is not an image. It is a cause from which many possible images can be generated.

Check yourself

Which model attributes can influence final color besides vertex positions?

01.5 Algorithmic Paradigms

Source location: Section 1.5

Commentary

The lecture contrasts ways to generate images. Rasterization works from objects toward pixels and is the dominant real-time pipeline. Ray-based methods work from image samples into the scene and are powerful for visibility and lighting but can be more expensive.

Do not memorize these paradigms as names only. Ask what direction information flows. Does the algorithm start from geometry and find covered pixels, or from pixels/rays and find visible geometry? That direction explains the strengths and weaknesses.

Mental model

Rasterization asks, 'Which samples does this primitive cover?' Ray casting asks, 'What does this sample see?'

Check yourself

Which paradigm fits OpenGL's standard real-time pipeline most directly?

End-of-lecture summary

If you remember only one thing from Lecture 01, remember this: This lecture is the orientation layer for the whole course. It explains why interactive computer graphics is a pipeline problem: a scene must be represented, processed under time constraints, sampled into an image, and displayed fast enough for interaction.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 02 - Rendering Pipeline

This lecture turns the vague idea of rendering into a concrete sequence. It is the most important debugging map in the course: if an image is wrong, the pipeline tells you where the error could have entered.

Source chunk	course_text_parts/03_lectures/02_rendering-pipeline.txt
Extracted slide pages	94
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.

Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Object-based Rendering
Source cue	- Image generation done through rasterization / - Scene objects formed by 3D models (primitives) / are transformed through successive coordinate systems / - Primitives are rasterized into fragments, which are pixel predecessors / - A subset of the fragments becomes pixels
Professor-style explanation	The slide 'Object-based Rendering' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. The terms are successive representations in object-based rendering: - Image generation done through rasterization / - Scene objects formed by 3D models (primitives) / are transformed through successive coordinate systems / - Primitives are rasterized into fragments, which are pixel predecessors / - A subset of the fragments becomes pixels. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter. For examination purposes, the important content is the representation change at each stage: model data, vertices, primitives, fragments, fragment tests, and final framebuffer updates.
Technical commentary	This slide is about Object-based Rendering. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. The terms are successive representations in object-based rendering: - Image generation done through rasterization / - Scene objects formed by 3D models (primitives) / are transformed through successive coordinate systems / - Primitives are rasterized into fragments, which are pixel predecessors / - A subset of the fragments becomes pixels. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels.
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Exam-grade answer	A strong answer for 'Object-based Rendering' states what representation enters the stage, what representation leaves it, and which later stage depends on that output.
Common trap	Do not collapse 'Object-based Rendering' into 'the GPU draws it'; name the intermediate representations because that is where most errors and exam distinctions appear.
Check yourself	Can you name the input and output representation for 'Object-based Rendering' in the rendering pipeline?

Page 3	2.1 Rendering Process
Source cue	2.2 OpenGL Overview / 2.3 OpenGL Rendering / 2.4 OpenGL Rendering Pipeline / 2.5 Fragment Tests and Operations / 2.6 Framebuffer
Professor-style explanation	The outline '2.1 Rendering Process' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: 2.2 OpenGL Overview / 2.3 OpenGL Rendering / 2.4 OpenGL Rendering Pipeline / 2.5 Fragment Tests and Operations / 2.6 Framebuffer. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about 2.1 Rendering Process. The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: 2.2 OpenGL Overview / 2.3 OpenGL Rendering / 2.4 OpenGL Rendering Pipeline / 2.5 Fragment Tests and Operations / 2.6 Framebuffer. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for '2.1 Rendering Process' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize '2.1 Rendering Process' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where '2.1 Rendering Process' fits in the lecture order and what later section depends on it?
Page 4	2.1 Rendering Process
Source cue	From 3D models over fragments to pixels

Professor-style explanation	The slide '2.1 Rendering Process' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. The terms are successive representations in object-based rendering: From 3D models over fragments to pixels. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter. For examination purposes, the important content is the representation change at each stage: model data, vertices, primitives, fragments, fragment tests, and final framebuffer updates.
Technical commentary	This slide is about 2.1 Rendering Process. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. The terms are successive representations in object-based rendering: From 3D models over fragments to pixels. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels.
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Exam-grade answer	A strong answer for '2.1 Rendering Process' states what representation enters the stage, what representation leaves it, and which later stage depends on that output.
Common trap	Do not collapse '2.1 Rendering Process' into 'the GPU draws it'; name the intermediate representations because that is where most errors and exam distinctions appear.
Check yourself	Can you name the input and output representation for '2.1 Rendering Process' in the rendering pipeline?
Page 5	Geometry-based Rendering 1/2
Source cue	CPU Application GPU Rendering Display Hardware / Rendering / 3D Model Framebuffer / Process / Video

Professor-style explanation	The slide 'Geometry-based Rendering 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: CPU Application GPU Rendering Display Hardware / Rendering / 3D Model Framebuffer / Process / Video. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Geometry-based Rendering 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: CPU Application GPU Rendering Display Hardware / Rendering / 3D Model Framebuffer / Process / Video. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Geometry-based Rendering 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Geometry-based Rendering 1/2'?
Page 6	Geometry-based Rendering 2/2
Source cue	- The rendering process is realized through the rendering pipeline / - Renders 3D models to 2D images / - Conceptually, the rendering pipeline consists of two GPU stages / - Geometry stage - processes geometric primitives / - Rasterization stage - processes fragments and pixels

Professor-style explanation	The slide 'Geometry-based Rendering 2/2' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. The terms are successive representations in object-based rendering: - The rendering process is realized through the rendering pipeline / - Renders 3D models to 2D images / - Conceptually, the rendering pipeline consists of two GPU stages / - Geometry stage - processes geometric primitives / - Rasterization stage - processes fragments and pixels. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter. For examination purposes, the important content is the representation change at each stage: model data, vertices, primitives, fragments, fragment tests, and final framebuffer updates.
Technical commentary	This slide is about Geometry-based Rendering 2/2. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. The terms are successive representations in object-based rendering: - The rendering process is realized through the rendering pipeline / - Renders 3D models to 2D images / - Conceptually, the rendering pipeline consists of two GPU stages / - Geometry stage - processes geometric primitives / - Rasterization stage - processes fragments and pixels. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels.
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Exam-grade answer	A strong answer for 'Geometry-based Rendering 2/2' states what representation enters the stage, what representation leaves it, and which later stage depends on that output.
Common trap	Do not collapse 'Geometry-based Rendering 2/2' into 'the GPU draws it'; name the intermediate representations because that is where most errors and exam distinctions appear.
Check yourself	Can you name the input and output representation for 'Geometry-based Rendering 2/2' in the rendering pipeline?
Page 7	Application Stage
Source cue	- Specification of the model elements and the scene / as well as interaction capabilities / - Scene modeling: arrangement and attribution of the scene objects / - Calculation of animation paths / - Collision detection

Professor-style explanation	The slide 'Application Stage' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. The terms are successive representations in object-based rendering: - Specification of the model elements and the scene / as well as interaction capabilities - Scene modeling: arrangement and attribution of the scene objects - Calculation of animation paths - Collision detection. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter. For examination purposes, the important content is the representation change at each stage: model data, vertices, primitives, fragments, fragment tests, and final framebuffer updates.
Technical commentary	This slide is about Application Stage. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. The terms are successive representations in object-based rendering: - Specification of the model elements and the scene / as well as interaction capabilities - Scene modeling: arrangement and attribution of the scene objects - Calculation of animation paths - Collision detection. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels.
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Exam-grade answer	A strong answer for 'Application Stage' states what representation enters the stage, what representation leaves it, and which later stage depends on that output.
Common trap	Do not collapse 'Application Stage' into 'the GPU draws it'; name the intermediate representations because that is where most errors and exam distinctions appear.
Check yourself	Can you name the input and output representation for 'Application Stage' in the rendering pipeline?
Page 8	Geometry Stage
Source cue	- x x / - Geometric 3D transformation / Model Coordinate System World Coordinate System / - Alignment of individual scene objects in the overall scene - Basic transformations: translation, scaling, rotation

Professor-style explanation	The slide 'Geometry Stage' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: x / - Geometric 3D transformation / Model Coordinate System World Coordinate System / - Alignment of individual scene objects in the overall scene / - Basic transformations: translation, scaling, rotation. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Geometry Stage. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: x / - Geometric 3D transformation / Model Coordinate System World Coordinate System / - Alignment of individual scene objects in the overall scene / - Basic transformations: translation, scaling, rotation. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Geometry Stage' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Geometry Stage' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Geometry Stage'?
Page 9	Rasterization Stage
Source cue	- Conversion of primitives into fragments (rasterization) / - Discretization takes place in relation to a target grid / - Rasterization algorithms optimized for certain primitives / (e.g., lines, triangles) / - After rasterization per-fragment tests are performed

Professor-style explanation	The slide 'Rasterization Stage' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Conversion of primitives into fragments (rasterization) / - Discretization takes place in relation to a target grid / - Rasterization algorithms optimized for certain primitives / (e.g., lines, triangles) / - After rasterization per-fragment tests are performed. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Rasterization Stage. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Conversion of primitives into fragments (rasterization) / - Discretization takes place in relation to a target grid / - Rasterization algorithms optimized for certain primitives / (e.g., lines, triangles) / - After rasterization per-fragment tests are performed. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Rasterization Stage' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization Stage'?
Page 10	2.2 OpenGL Overview
Source cue	Library for real-time computer graphics

Professor-style explanation	The slide '2.2 OpenGL Overview' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Library for real-time computer graphics. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about 2.2 OpenGL Overview. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Library for real-time computer graphics. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for '2.2 OpenGL Overview' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '2.2 OpenGL Overview'?
Page 11	OpenGL History
Source cue	- Unified graphics software interface was missing / as knowledge channeled into proprietary software packages / - HP's starbase / - SGI's Graphics Library (GL) / - Some standardization efforts existed

Professor-style explanation	The slide 'OpenGL History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: - Unified graphics software interface was missing / as knowledge channeled into proprietary software packages / - HP's starbase / - SGI's Graphics Library (GL) / - Some standardization efforts existed. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input. For examination purposes, the important content is not the date alone but the reason the milestone matters: every historical item solved a limitation in representation, interaction, display, or computation.
Technical commentary	This slide is about OpenGL History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: - Unified graphics software interface was missing / as knowledge channeled into proprietary software packages / - HP's starbase / - SGI's Graphics Library (GL) / - Some standardization efforts existed. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Exam-grade answer	A strong answer for 'OpenGL History' states the milestone and the graphics limitation it helped overcome, such as limited interaction, limited displays, missing 3D representation, or slow rendering.
Common trap	Do not learn 'OpenGL History' as trivia only; connect each historical item to the technical capability it introduced or made practical.
Check yourself	Can you name the graphics capability or idea represented by 'OpenGL History' and why it mattered historically?
Page 12	OpenGL
Source cue	- OpenGL (Open Graphics Library) / - Platform independent 2D and 3D graphics API / - API specification is provided by the Khronos Group / - Technical implementation / - Rendering pipeline consisting of static and programmable pipeline stages

Professor-style explanation	The slide 'OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - OpenGL (Open Graphics Library) / - Platform independent 2D and 3D graphics API / - API specification is provided by the Khronos Group / - Technical implementation / - Rendering pipeline consisting of static and programmable pipeline stages. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - OpenGL (Open Graphics Library) / - Platform independent 2D and 3D graphics API / - API specification is provided by the Khronos Group / - Technical implementation / - Rendering pipeline consisting of static and programmable pipeline stages. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL'?
Page 13	OpenGL vs. Vulkan
Source cue	- How about Vulkan? / - More modern than OpenGL / - Designed for high performance and low-level GPU control / - Better suited for complex, large-scale rendering engines / - Vulkan also harder to learn and requires more setup/code

Professor-style explanation	The slide 'OpenGL vs. Vulkan' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - How about Vulkan? / - More modern than OpenGL / - Designed for high performance and low-level GPU control / - Better suited for complex, large-scale rendering engines / - Vulkan also harder to learn and requires more setup/code. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL vs. Vulkan. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - How about Vulkan? / - More modern than OpenGL / - Designed for high performance and low-level GPU control / - Better suited for complex, large-scale rendering engines / - Vulkan also harder to learn and requires more setup/code. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL vs. Vulkan' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL vs. Vulkan'?
Page 14	OpenGL Libraries
Source cue	- Different tastes of OpenGL / - OpenGL / - OpenGL ES - limited version for embedded systems / - WebGL - Javascript connection for Web Development / (functionality similar to OpenGL ES)

Professor-style explanation	The slide 'OpenGL Libraries' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Different tastes of OpenGL / - OpenGL / - OpenGL ES - limited version for embedded systems / - WebGL - Javascript connection for Web Development / (functionality similar to OpenGL ES). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Libraries. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Different tastes of OpenGL / - OpenGL / - OpenGL ES - limited version for embedded systems / - WebGL - Javascript connection for Web Development / (functionality similar to OpenGL ES). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Libraries' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Libraries'?
Page 15	OpenGL Functionality
Source cue	- Functional API / - Objects, scenes and images are described by a sequence of commands / (functional approach, low-level) / - Contrast: scene or object description (declarative approach, high-level) / - OpenGL context

Professor-style explanation	The slide 'OpenGL Functionality' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Functional API / - Objects, scenes and images are described by a sequence of commands / (functional approach, low-level) / - Contrast: scene or object description (declarative approach, high-level) / - OpenGL context. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Functionality. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Functional API / - Objects, scenes and images are described by a sequence of commands / (functional approach, low-level) / - Contrast: scene or object description (declarative approach, high-level) / - OpenGL context. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Functionality' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Functionality'?
Page 16	OpenGL Context
Source cue	- Context is created and modified by the application, / and contains all the state variables of the OpenGL engine / - Any number of contexts per application possible, / but at most one active context per application / - Context is generally tied to an output window

Professor-style explanation	The slide 'OpenGL Context' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Context is created and modified by the application, / and contains all the state variables of the OpenGL engine / - Any number of contexts per application possible, / but at most one active context per application / - Context is generally tied to an output window. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Context. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Context is created and modified by the application, / and contains all the state variables of the OpenGL engine / - Any number of contexts per application possible, / but at most one active context per application / - Context is generally tied to an output window. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Context' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Context'?
Page 17	OpenGL Rendering Process
Source cue	- Initiation from the developer's perspective / - Creation of the output window / - Creation of the context / - Specification of the scene geometry / - Uploading of the scene geometry to the GPU

Professor-style explanation	The slide 'OpenGL Rendering Process' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Initiation from the developer's perspective / - Creation of the output window / - Creation of the context / - Specification of the scene geometry / - Uploading of the scene geometry to the GPU. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Rendering Process. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Initiation from the developer's perspective / - Creation of the output window / - Creation of the context / - Specification of the scene geometry / - Uploading of the scene geometry to the GPU. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Rendering Process' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Rendering Process'?
Page 18	Window Creation with GLFW 1/2
Source cue	<pre>#include <GLFW/glfw3.h> / #include <stdlib.h> / #include <stdio.h> / static void error_callback(int error, const char* description) { / fputs(description, stderr);</pre>

Professor-style explanation	The slide 'Window Creation with GLFW 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: <code>#include <GLFW/glfw3.h> / #include <stdlib.h> / #include <stdio.h> / static void error_callback(int error, const char* description) { / fputs(description, stderr)</code> . The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Window Creation with GLFW 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: <code>#include <GLFW/glfw3.h> / #include <stdlib.h> / #include <stdio.h> / static void error_callback(int error, const char* description) { / fputs(description, stderr)</code> . The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Window Creation with GLFW 1/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Window Creation with GLFW 1/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Window Creation with GLFW 1/2' into a causal sentence instead of repeating the slide title?
Page 19	Window Creation with GLFW 2/2
Source cue	<pre>int main(void){ / GLFWwindow* window; / glfwSetErrorCallback(error_callback); / if (!glfwInit()) exit(EXIT_FAILURE); / window = glfwCreateWindow(640, 480, "Simple example", NULL, NULL);</pre>

Professor-style explanation	The slide 'Window Creation with GLFW 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: <code>int main(void){ / GLFWwindow* window; / glfwSetErrorCallback(error_callback); / if (!glfwInit()) exit(EXIT_FAILURE); / window = glfwCreateWindow(640, 480, "Simple example", NULL, NULL).</code> The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Window Creation with GLFW 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: <code>int main(void){ / GLFWwindow* window; / glfwSetErrorCallback(error_callback); / if (!glfwInit()) exit(EXIT_FAILURE); / window = glfwCreateWindow(640, 480, "Simple example", NULL, NULL).</code> The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Window Creation with GLFW 2/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Window Creation with GLFW 2/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Window Creation with GLFW 2/2' into a causal sentence instead of repeating the slide title?
Page 20	OpenGL Naming Conventions
Source cue	- Function-Prefix "gl" / - glClear, ... / - Constant-Prefix "GL" / - GL_POLYGON, GL_LINES, ... / - Parameter types are explicitly encoded in function names

Professor-style explanation	The slide 'OpenGL Naming Conventions' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Function-Prefix "gl" / - glClear, ... / - Constant-Prefix "GL" / - GL_POLYGON, GL_LINES, ... / - Parameter types are explicitly encoded in function names. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Naming Conventions. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Function-Prefix "gl" / - glClear, ... / - Constant-Prefix "GL" / - GL_POLYGON, GL_LINES, ... / - Parameter types are explicitly encoded in function names. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Naming Conventions' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Naming Conventions'?
Page 21	2.3 OpenGL Rendering
Source cue	Transfer of 3D models to the GPU

Professor-style explanation	The slide '2.3 OpenGL Rendering' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Transfer of 3D models to the GPU. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about 2.3 OpenGL Rendering. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Transfer of 3D models to the GPU. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for '2.3 OpenGL Rendering' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '2.3 OpenGL Rendering'?
Page 22	OpenGL Rendering (until V2.1)
Source cue	- Rendering is done through Immediate Mode / void renderScene() { / glColor3f(0.0f, 1.0f, 0.0f); // drawing color becomes green / glBegin(GL_POLYGON); // begin primitive / glVertex2f(-0.5f,-0.5f); // 1. vertex

Professor-style explanation	The slide 'OpenGL Rendering (until V2.1)' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Rendering is done through Immediate Mode / void renderScene() { / glColor3f(0.0f, 1.0f, 0.0f); // drawing color becomes green / glBegin(GL_POLYGON); // begin primitive / glVertex2f(-0.5f,-0.5f); // 1. vertex. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Rendering (until V2.1). The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Rendering is done through Immediate Mode / void renderScene() { / glColor3f(0.0f, 1.0f, 0.0f); // drawing color becomes green / glBegin(GL_POLYGON); // begin primitive / glVertex2f(-0.5f,-0.5f); // 1. vertex. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Rendering (until V2.1)' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Rendering (until V2.1)'?
Page 23	OpenGL Rendering (since V3.0)
Source cue	- Using Vertex Array Objects (VAOs) / - Vertex specification in CPU memory / - Vertex data transfer from CPU to GPU / - Memory layout description of the vertex data / - Number of vertices, number of components (x, y, z, ...)

Professor-style explanation	The slide 'OpenGL Rendering (since V3.0)' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Using Vertex Array Objects (VAOs) / - Vertex specification in CPU memory / - Vertex data transfer from CPU to GPU / - Memory layout description of the vertex data / - Number of vertices, number of components (x, y, z, ...). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Rendering (since V3.0). The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Using Vertex Array Objects (VAOs) / - Vertex specification in CPU memory / - Vertex data transfer from CPU to GPU / - Memory layout description of the vertex data / - Number of vertices, number of components (x, y, z, ...). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Rendering (since V3.0)' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Rendering (since V3.0)'?
Page 24	1. Vertex Specification
Source cue	- Vertex data is specified as a C array / // vertices of two triangles / GLfloat vertices[6][2] = { // 6 vertices with two components each (x,y) / {0.0, 1.0}, {1.0, 1.0}, {1.0, 0.0}, // 1. triangle / {0.5, 0.5}, {1.0, 0.3}, {0.7, 1.0} // 2. triangle

Professor-style explanation	The outline '1. Vertex Specification' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: - Vertex data is specified as a C array / // vertices of two triangles / GLfloat vertices[6][2] = { // 6 vertices with two components each (x,y) / {0.0, 1.0}, {1.0, 1.0}, {1.0, 0.0}, // 1. triangle / {0.5, 0.5}, {1.0, 0.3}, {0.7, 1.0} // 2. triangle. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about 1. Vertex Specification. The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: - Vertex data is specified as a C array / // vertices of two triangles / GLfloat vertices[6][2] = { // 6 vertices with two components each (x,y) / {0.0, 1.0}, {1.0, 1.0}, {1.0, 0.0}, // 1. triangle / {0.5, 0.5}, {1.0, 0.3}, {0.7, 1.0} // 2. triangle. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for '1. Vertex Specification' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize '1. Vertex Specification' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where '1. Vertex Specification' fits in the lecture order and what later section depends on it?
Page 25	2. Vertex Data Transfer
Source cue	- VBO allocation and data transfer via / - glBufferData(GL_ARRAY_BUFFER, size, data, GL_STATIC_DRAW); / - data - address in main memory (i.e., vertices or &vertices[0]) / - size - size of data in number of bytes / (i.e. sizeof(vertices) or numVertices*sizeof(GLfloat)*2)

Professor-style explanation	The slide '2. Vertex Data Transfer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - VBO allocation and data transfer via / - <code>glBufferData(GL_ARRAY_BUFFER, size, data, GL_STATIC_DRAW)</code>; / - data - address in main memory (i.e., vertices or <code>&vertices[0]</code>) / - size - size of data in number of bytes / (i.e. <code>sizeof(vertices)</code> or <code>numVertices*sizeof(GLfloat)*2</code>). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about 2. Vertex Data Transfer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - VBO allocation and data transfer via / - <code>glBufferData(GL_ARRAY_BUFFER, size, data, GL_STATIC_DRAW)</code>; / - data - address in main memory (i.e., vertices or <code>&vertices[0]</code>) / - size - size of data in number of bytes / (i.e. <code>sizeof(vertices)</code> or <code>numVertices*sizeof(GLfloat)*2</code>). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for '2. Vertex Data Transfer' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '2. Vertex Data Transfer'?
Page 26	3. Memory Layout Description
Source cue	- Memory layout is described by the following function / - <code>glVertexAttribPointer(attribIndex, size, type, normalized, stride, offset)</code>; / - attribIndex - attribute number (0 for position, later more ...) / - size - number of components per vertex (2 for xy, 3 for xyz) / - type - base type (typically <code>GL_FLOAT</code> or <code>GL_DOUBLE</code>)

Professor-style explanation	The slide '3. Memory Layout Description' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: - Memory layout is described by the following function / - glVertexAttribPointer(attribIndex, size, type, normalized, stride, offset); / - attribIndex - attribute number (0 for position, later more ...) / - size - number of components per vertex (2 for xy, 3 for xyz) / - type - base type (typically GL_FLOAT or GL_DOUBLE). Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations. For examination purposes, the important content is the chain width x height -> pixel count -> bits per pixel -> memory size; this chain also explains bandwidth and performance pressure.
Technical commentary	This slide is about 3. Memory Layout Description. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: - Memory layout is described by the following function / - glVertexAttribPointer(attribIndex, size, type, normalized, stride, offset); / - attribIndex - attribute number (0 for position, later more ...) / - size - number of components per vertex (2 for xy, 3 for xyz) / - type - base type (typically GL_FLOAT or GL_DOUBLE). Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Exam-grade answer	A strong answer for '3. Memory Layout Description' includes the formula relation width x height x bits per pixel, distinguishes bits from bytes, and links larger images to memory bandwidth.
Common trap	Do not confuse bits with bytes in '3. Memory Layout Description', and do not ignore that alpha or higher precision changes memory size.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in '3. Memory Layout Description'?
Page 27	4. VAO Drawing 1/2
Source cue	- Drawing is done with the command / - glDrawArrays(mode, first, count); / - mode - specifies the primitive type (e.g., GL_TRIANGLES or GL_LINES) / - From all vertices, the next count vertices are drawn from index first / - Actual vertex data no longer needs to be specified

Professor-style explanation	The slide '4. VAO Drawing 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Drawing is done with the command / - <code>glDrawArrays(mode, first, count);</code> / - mode - specifies the primitive type (e.g., <code>GL_TRIANGLES</code> or <code>GL_LINES</code>) / - From all vertices, the next count vertices are drawn from index first / - Actual vertex data no longer needs to be specified. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about 4. VAO Drawing 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Drawing is done with the command / - <code>glDrawArrays(mode, first, count);</code> / - mode - specifies the primitive type (e.g., <code>GL_TRIANGLES</code> or <code>GL_LINES</code>) / - From all vertices, the next count vertices are drawn from index first / - Actual vertex data no longer needs to be specified. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for '4. VAO Drawing 1/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by '4. VAO Drawing 1/2'?
Page 28	4. VAO Drawing 2/2
Source cue	- There is a small number of primitive types / - Points / - Lines, Line Strips, Line Loops / - Triangles, Triangle Strips, Triangle Fans, Patches / v2 v2 v2

Professor-style explanation	The slide '4. VAO Drawing 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - There is a small number of primitive types / - Points / - Lines, Line Strips, Line Loops / - Triangles, Triangle Strips, Triangle Fans, Patches / v2 v2 v2. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about 4. VAO Drawing 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - There is a small number of primitive types / - Points / - Lines, Line Strips, Line Loops / - Triangles, Triangle Strips, Triangle Fans, Patches / v2 v2 v2. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for '4. VAO Drawing 2/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by '4. VAO Drawing 2/2'?
Page 29	5. GPU Memory Release
Source cue	- After usage VAO and VBO should be deleted from GPU memory / - <code>glDeleteBuffers(...);</code> / - <code>glDeleteVertexArrays(...);</code>

Professor-style explanation	The slide '5. GPU Memory Release' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: - After usage VAO and VBO should be deleted from GPU memory / - <code>glDeleteBuffers(...)</code> ; / - <code>glDeleteVertexArrays(...)</code> . Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations. For examination purposes, the important content is the chain width x height -> pixel count -> bits per pixel -> memory size; this chain also explains bandwidth and performance pressure.
Technical commentary	This slide is about 5. GPU Memory Release. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: - After usage VAO and VBO should be deleted from GPU memory / - <code>glDeleteBuffers(...)</code> ; / - <code>glDeleteVertexArrays(...)</code> . Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Exam-grade answer	A strong answer for '5. GPU Memory Release' includes the formula relation width x height x bits per pixel, distinguishes bits from bytes, and links larger images to memory bandwidth.
Common trap	Do not confuse bits with bytes in '5. GPU Memory Release', and do not ignore that alpha or higher precision changes memory size.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in '5. GPU Memory Release'?
Page 30	Rendering Initialization 1/2
Source cue	<pre>static void init_render_scene(void) { / static const GLfloat vertices[6][2] = { / {0.0f, 1.0f}, {1.0f, 1.0f}, {1.0f, 0.0f}, /* 1st triangle */ / {0.5f, 0.5f}, {1.0f, 0.3f}, {0.7f, 1.0f} /* 2nd triangle */ / static const char* vs_src =</pre>

Professor-style explanation	The slide 'Rendering Initialization 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: <pre>static void init_render_scene(void) { / static const GLfloat vertices[6][2] = { / {0.0f, 1.0f}, {1.0f, 1.0f}, {1.0f, 0.0f}, /* 1st triangle */ / {0.5f, 0.5f}, {1.0f, 0.3f}, {0.7f, 1.0f} /* 2nd triangle */ / static const char* vs_src = .</pre> A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Rendering Initialization 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: <pre>static void init_render_scene(void) { / static const GLfloat vertices[6][2] = { / {0.0f, 1.0f}, {1.0f, 1.0f}, {1.0f, 0.0f}, /* 1st triangle */ / {0.5f, 0.5f}, {1.0f, 0.3f}, {0.7f, 1.0f} /* 2nd triangle */ / static const char* vs_src = .</pre> A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Rendering Initialization 1/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Rendering Initialization 1/2'?
Page 31	Rendering Initialization 2/2
Source cue	<pre>program = create_program(vs_src, fs_src); / glGenVertexArrays(1, &vao); / glGenBuffers(1, &vbo); / glBindVertexArray(vao); / glBindBuffer(GL_ARRAY_BUFFER, vbo);</pre>

Professor-style explanation	The slide 'Rendering Initialization 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: <code>program = create_program(vs_src, fs_src); / glGenVertexArrays(1, &vao); / glGenBuffers(1, &vbo); / glBindVertexArray(vao); / glBindBuffer(GL_ARRAY_BUFFER, vbo)</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Rendering Initialization 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: <code>program = create_program(vs_src, fs_src); / glGenVertexArrays(1, &vao); / glGenBuffers(1, &vbo); / glBindVertexArray(vao); / glBindBuffer(GL_ARRAY_BUFFER, vbo)</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Rendering Initialization 2/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Rendering Initialization 2/2'?
Page 32	Rendering
Source cue	<pre>static void render_scene(void) { / glClear(GL_COLOR_BUFFER_BIT); / glUseProgram(program); / glBindVertexArray(vao); / glDrawArrays(GL_TRIANGLES, 0, 6);</pre>

Professor-style explanation	The slide 'Rendering' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: static void render_scene(void) { / glClear(GL_COLOR_BUFFER_BIT); / glUseProgram(program); / glBindVertexArray(vao); / glDrawArrays(GL_TRIANGLES, 0, 6). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Rendering. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: static void render_scene(void) { / glClear(GL_COLOR_BUFFER_BIT); / glUseProgram(program); / glBindVertexArray(vao); / glDrawArrays(GL_TRIANGLES, 0, 6). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Rendering' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Rendering'?
Page 33	Additional Attributes
Source cue	- Additional attributes are necessary for drawing / - Color, normals, texture coordinates, ... / - There are two ways to assign these attributes to the vertex data / - Attributes are bound to the same VAO in separate VBO / - Attributes are stored interleaved with vertex data,

Professor-style explanation	The slide 'Additional Attributes' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Additional attributes are necessary for drawing / - Color, normals, texture coordinates, ... / - There are two ways to assign these attributes to the vertex data / - Attributes are bound to the same VAO in separate VBO / - Attributes are stored interleaved with vertex data,. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Additional Attributes. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Additional attributes are necessary for drawing / - Color, normals, texture coordinates, ... / - There are two ways to assign these attributes to the vertex data / - Attributes are bound to the same VAO in separate VBO / - Attributes are stored interleaved with vertex data,. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Additional Attributes' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Additional Attributes' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Additional Attributes'?
Page 34	2.4 OpenGL Rendering Pipeline
Source cue	Programmable and static stages

Professor-style explanation	The slide '2.4 OpenGL Rendering Pipeline' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Programmable and static stages. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about 2.4 OpenGL Rendering Pipeline. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Programmable and static stages. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for '2.4 OpenGL Rendering Pipeline' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '2.4 OpenGL Rendering Pipeline'?
Page 35	The Rendering Pipeline 1/2
Source cue	Scene Description Raster Image / Geometry Fragment / Rasterization / Processing Processing / Vertices Primitives Fragments Pixels

Professor-style explanation	The slide 'The Rendering Pipeline 1/2' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. The terms are successive representations in object-based rendering: Scene Description Raster Image / Geometry Fragment / Rasterization / Processing Processing / Vertices Primitives Fragments Pixels. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter. For examination purposes, the important content is the representation change at each stage: model data, vertices, primitives, fragments, fragment tests, and final framebuffer updates.
Technical commentary	This slide is about The Rendering Pipeline 1/2. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. The terms are successive representations in object-based rendering: Scene Description Raster Image / Geometry Fragment / Rasterization / Processing Processing / Vertices Primitives Fragments Pixels. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels.
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Exam-grade answer	A strong answer for 'The Rendering Pipeline 1/2' states what representation enters the stage, what representation leaves it, and which later stage depends on that output.
Common trap	Do not collapse 'The Rendering Pipeline 1/2' into 'the GPU draws it'; name the intermediate representations because that is where most errors and exam distinctions appear.
Check yourself	Can you name the input and output representation for 'The Rendering Pipeline 1/2' in the rendering pipeline?
Page 36	The Rendering Pipeline 2/2
Source cue	- Sequence of fixed and programmable stages Programmable / Vertex Processing / - Process is influenced by context and scene objects / Context Fixed Function / - Programmable levels are programmed

Professor-style explanation	The slide 'The Rendering Pipeline 2/2' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. The terms are successive representations in object-based rendering: - Sequence of fixed and programmable stages Programmable / Vertex Processing / - Process is influenced by context and scene objects / Context Fixed Function / - Programmable levels are programmed. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter. For examination purposes, the important content is the representation change at each stage: model data, vertices, primitives, fragments, fragment tests, and final framebuffer updates.
Technical commentary	This slide is about The Rendering Pipeline 2/2. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. The terms are successive representations in object-based rendering: - Sequence of fixed and programmable stages Programmable / Vertex Processing / - Process is influenced by context and scene objects / Context Fixed Function / - Programmable levels are programmed. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels.
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Exam-grade answer	A strong answer for 'The Rendering Pipeline 2/2' states what representation enters the stage, what representation leaves it, and which later stage depends on that output.
Common trap	Do not collapse 'The Rendering Pipeline 2/2' into 'the GPU draws it'; name the intermediate representations because that is where most errors and exam distinctions appear.
Check yourself	Can you name the input and output representation for 'The Rendering Pipeline 2/2' in the rendering pipeline?
Page 37	Shader Programming - History 1/2
Source cue	- There is special graphics hardware (dedicated) / and general graphics hardware (general purpose) / - Special graphics hardware / - Volume Rendering: VolumePro / - Game consoles: Custom NVIDIA chips in Xbox

Professor-style explanation	The slide 'Shader Programming - History 1/2' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: - There is special graphics hardware (dedicated) / and general graphics hardware (general purpose) / - Special graphics hardware / - Volume Rendering: VolumePro / - Game consoles: Custom NVIDIA chips in Xbox. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input. For examination purposes, the important content is not the date alone but the reason the milestone matters: every historical item solved a limitation in representation, interaction, display, or computation.
Technical commentary	This slide is about Shader Programming - History 1/2. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: - There is special graphics hardware (dedicated) / and general graphics hardware (general purpose) / - Special graphics hardware / - Volume Rendering: VolumePro / - Game consoles: Custom NVIDIA chips in Xbox. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Exam-grade answer	A strong answer for 'Shader Programming - History 1/2' states the milestone and the graphics limitation it helped overcome, such as limited interaction, limited displays, missing 3D representation, or slow rendering.
Common trap	Do not learn 'Shader Programming - History 1/2' as trivia only; connect each historical item to the technical capability it introduced or made practical.
Check yourself	Can you name the graphics capability or idea represented by 'Shader Programming - History 1/2' and why it mattered historically?
Page 38	Shader Programming - History 2/2
Source cue	- Various vendor-dependent extensions are introduced / to provide extensibility for general graphics hardware / - Texture shaders and register combiners allowed modification / through the application of textures / - Applicability is very limited by the possibilities of graphics hardware

Professor-style explanation	The slide 'Shader Programming - History 2/2' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. The names, dates, and examples form a timeline: - Various vendor-dependent extensions are introduced / to provide extensibility for general graphics hardware / - Texture shaders and register combiners allowed modification / through the application of textures / - Applicability is very limited by the possibilities of graphics hardware. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input. For examination purposes, the important content is not the date alone but the reason the milestone matters: every historical item solved a limitation in representation, interaction, display, or computation.
Technical commentary	This slide is about Shader Programming - History 2/2. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. The names, dates, and examples form a timeline: - Various vendor-dependent extensions are introduced / to provide extensibility for general graphics hardware / - Texture shaders and register combiners allowed modification / through the application of textures / - Applicability is very limited by the possibilities of graphics hardware. Each item marks a capability or use case that pushed graphics from static image construction toward interactive, programmable rendering.
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Exam-grade answer	A strong answer for 'Shader Programming - History 2/2' states the milestone and the graphics limitation it helped overcome, such as limited interaction, limited displays, missing 3D representation, or slow rendering.
Common trap	Do not learn 'Shader Programming - History 2/2' as trivia only; connect each historical item to the technical capability it introduced or made practical.
Check yourself	Can you name the graphics capability or idea represented by 'Shader Programming - History 2/2' and why it mattered historically?
Page 39	Assembler Shader
Source cue	- First OpenGL extensions support shader programming in assembler code / !! ARBvp1.0 / ATTRIB iPos = vertex.position ; / ATTRIB iNormal = vertex.normal ; / ...

Professor-style explanation	The slide 'Assembler Shader' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - First OpenGL extensions support shader programming in assembler code / !! ARBvp1.0 / ATTRIB iPos = vertex.position ; / ATTRIB iNormal = vertex.normal ; / . Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Assembler Shader. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - First OpenGL extensions support shader programming in assembler code / !! ARBvp1.0 / ATTRIB iPos = vertex.position ; / ATTRIB iNormal = vertex.normal ; / . Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Assembler Shader' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Assembler Shader'?
Page 40	High-Level Shading Languages 1/2
Source cue	- Assembler code is hard to read and hard to maintain / - High-level languages for shading are specified in C-like syntax / - Source code is translated by the compiler into appropriate ASM statements / - Using shaders after initial binding automatically when rendering geometry / - Interface to OpenGL by passing appropriate variables or changing the OpenGL

Professor-style explanation	The slide 'High-Level Shading Languages 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Assembler code is hard to read and hard to maintain / - High-level languages for shading are specified in C-like syntax / - Source code is translated by the compiler into appropriate ASM statements / - Using shaders after initial binding automatically when rendering geometry / - Interface to OpenGL by passing appropriate variables or changing the OpenGL. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about High-Level Shading Languages 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Assembler code is hard to read and hard to maintain / - High-level languages for shading are specified in C-like syntax / - Source code is translated by the compiler into appropriate ASM statements / - Using shaders after initial binding automatically when rendering geometry / - Interface to OpenGL by passing appropriate variables or changing the OpenGL. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'High-Level Shading Languages 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'High-Level Shading Languages 1/2'?
Page 41	High-Level Shading Languages 2/2
Source cue	- Existing high level languages for shader development / - CG - C for Graphics by NVIDIA / - CG was introduced in 2003 by NVIDIA as a platform independent solution / (in practice rather semi-platform independent) / - HLSL - High Level Shading Language from Microsoft

Professor-style explanation	The slide 'High-Level Shading Languages 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Existing high level languages for shader development / - CG - C for Graphics by NVIDIA / - CG was introduced in 2003 by NVIDIA as a platform independent solution / (in practice rather semi-platform independent) / - HLSL - High Level Shading Language from Microsoft. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about High-Level Shading Languages 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Existing high level languages for shader development / - CG - C for Graphics by NVIDIA / - CG was introduced in 2003 by NVIDIA as a platform independent solution / (in practice rather semi-platform independent) / - HLSL - High Level Shading Language from Microsoft. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'High-Level Shading Languages 2/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'High-Level Shading Languages 2/2'?
Page 42	Shader Types
Source cue	- Pipeline concept has been kept in place / and shaders are responsible for different stages / - First, geometry level programming was done by vertex shaders / - Vertex shaders allow modifiability of vertex transformations and lighting / - After that, there was an extension for image-based algorithms by fragment

Professor-style explanation	The slide 'Shader Types' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Pipeline concept has been kept in place / and shaders are responsible for different stages / - First, geometry level programming was done by vertex shaders / - Vertex shaders allow modifiability of vertex transformations and lighting / - After that, there was an extension for image-based algorithms by fragment. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Shader Types. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Pipeline concept has been kept in place / and shaders are responsible for different stages / - First, geometry level programming was done by vertex shaders / - Vertex shaders allow modifiability of vertex transformations and lighting / - After that, there was an extension for image-based algorithms by fragment. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Shader Types' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Types'?
Page 43	Vertex Shader Concepts
Source cue	- Vertex shaders enable the programmable modification of processed geometry / - Possibilities of a vertex shader / Programmable / - Geometric transformation of vertices and normals / Vertex Processing

Professor-style explanation	The slide 'Vertex Shader Concepts' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Vertex shaders enable the programmable modification of processed geometry / - Possibilities of a vertex shader / Programmable / - Geometric transformation of vertices and normals / Vertex Processing. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Vertex Shader Concepts. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Vertex shaders enable the programmable modification of processed geometry / - Possibilities of a vertex shader / Programmable / - Geometric transformation of vertices and normals / Vertex Processing. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Vertex Shader Concepts' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Concepts'?
Page 44	Vertex Shader Examples 1/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Vertex Shader Examples 1/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Vertex Shader Examples 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Vertex Shader Examples 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Examples 1/2'?

Page 45	Vertex Shader Examples 2/2
Source cue	- Morphing / - Shadow volume calculation for shadow effects / [ID Software's Doom III]
Professor-style explanation	The slide 'Vertex Shader Examples 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Morphing / - Shadow volume calculation for shadow effects / [ID Software's Doom III]. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.

Technical commentary	This slide is about Vertex Shader Examples 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Morphing / - Shadow volume calculation for shadow effects / [ID Software's Doom III]. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Vertex Shader Examples 2/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Examples 2/2'?

Page 46	Vertex Shader Coordinate Systems 1/2
Source cue	- Vertex shaders are executed for each vertex / - Vertex shaders calculate coordinates in clipping coordinates as output / - Vertex shaders can not / - Access adjacent vertices / - Change the number of vertices (i.e., generate or discard vertices)
Professor-style explanation	The slide 'Vertex Shader Coordinate Systems 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Vertex shaders are executed for each vertex / - Vertex shaders calculate coordinates in clipping coordinates as output / - Vertex shaders can not / - Access adjacent vertices / - Change the number of vertices (i.e., generate or discard vertices). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.

Technical commentary	This slide is about Vertex Shader Coordinate Systems 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Vertex shaders are executed for each vertex / - Vertex shaders calculate coordinates in clipping coordinates as output / - Vertex shaders can not / - Access adjacent vertices / - Change the number of vertices (i.e., generate or discard vertices). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Vertex Shader Coordinate Systems 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Coordinate Systems 1/2'?

Page 47	Vertex Shader Coordinate Systems 2/2
Source cue	Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform
Professor-style explanation	The slide 'Vertex Shader Coordinate Systems 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Vertex Shader Coordinate Systems 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.

Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Vertex Shader Coordinate Systems 2/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Coordinate Systems 2/2'?

Page 48	Local Space
Source cue	- Coordinates of your object relative to its local origin / - 3D vertex positions
Professor-style explanation	The slide 'Local Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Coordinates of your object relative to its local origin / - 3D vertex positions. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Local Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Coordinates of your object relative to its local origin / - 3D vertex positions. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Local Space' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Local Space' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Local Space'?

Page 49	World Space
---------	-------------

Source cue	- Coordinates in respect of a larger world / - Relative to some global origin of the world / - Together with many other objects
Professor-style explanation	The slide 'World Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Coordinates in respect of a larger world / - Relative to some global origin of the world / - Together with many other objects. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about World Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Coordinates in respect of a larger world / - Relative to some global origin of the world / - Together with many other objects. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'World Space' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'World Space' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'World Space'?
Page 50	View Space
Source cue	- Coordinates as seen from the camera

Professor-style explanation	The slide 'View Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Coordinates as seen from the camera. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about View Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Coordinates as seen from the camera. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'View Space' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'View Space' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'View Space'?
Page 51	Clip Space
Source cue	- Clip coordinates range from -1 to 1 / - Projection (Orthographic, Perspective)

Professor-style explanation	The slide 'Clip Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Clip coordinates range from -1 to 1 / - Projection (Orthographic, Perspective). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Clip Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Clip coordinates range from -1 to 1 / - Projection (Orthographic, Perspective). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Clip Space' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Clip Space' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Clip Space'?
Page 52	Screen Space
Source cue	- Transform to image coordinates with glViewport / - Results are sent to the rasterizer

Professor-style explanation	The slide 'Screen Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Transform to image coordinates with glViewport / - Results are sent to the rasterizer. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Screen Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Transform to image coordinates with glViewport / - Results are sent to the rasterizer. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Screen Space' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Screen Space' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Screen Space'?
Page 53	Coordinate Systems
Source cue	Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform

Professor-style explanation	The slide 'Coordinate Systems' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Coordinate Systems. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Coordinate Systems' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Coordinate Systems' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Coordinate Systems'?
Page 54	Vertex Shader Variables
Source cue	- Input types / - Uniforms: read only in VS (& FS) // Intrinsic vertex attributes / out gl_PerVertex { / - For example: light position or light color / vec4 gl_Position;

Professor-style explanation	The slide 'Vertex Shader Variables' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Input types / - Uniforms: read only in VS (& FS) // Intrinsic vertex attributes / out gl_PerVertex { / - For example: light position or light color / vec4 gl_Position. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Vertex Shader Variables. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Input types / - Uniforms: read only in VS (& FS) // Intrinsic vertex attributes / out gl_PerVertex { / - For example: light position or light color / vec4 gl_Position. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Vertex Shader Variables' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Variables'?
Page 55	Fixed Function Vertex Postprocessing
Source cue	- Clipping / - Discard invisible geometries / Programmable / - Perspective Division / Vertex Processing

Professor-style explanation	The slide 'Fixed Function Vertex Postprocessing' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Clipping / - Discard invisible geometries / Programmable / - Perspective Division / Vertex Processing. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Fixed Function Vertex Postprocessing. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Clipping / - Discard invisible geometries / Programmable / - Perspective Division / Vertex Processing. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Fixed Function Vertex Postprocessing' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Fixed Function Vertex Postprocessing'?
Page 56	Fixed Function Rasterization
Source cue	- Geometry is converted to raster coordinates / Programmable / Vertex Processing / Fixed Function / Primitives Fragments Vertex Postprocessing

Professor-style explanation	The slide 'Fixed Function Rasterization' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Geometry is converted to raster coordinates / Programmable / Vertex Processing / Fixed Function / Primitives Fragments Vertex Postprocessing. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Fixed Function Rasterization. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Geometry is converted to raster coordinates / Programmable / Vertex Processing / Fixed Function / Primitives Fragments Vertex Postprocessing. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Fixed Function Rasterization' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Fixed Function Rasterization' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Fixed Function Rasterization'?
Page 57	Fragment Shader Concept 1/2
Source cue	- Fragment shaders set the color of a pixel / - Possibilities of a fragment shader / Programmable / - Glare of textures / Vertex Processing

Professor-style explanation	The slide 'Fragment Shader Concept 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Fragment shaders set the color of a pixel / - Possibilities of a fragment shader / Programmable / - Glare of textures / Vertex Processing. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Fragment Shader Concept 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Fragment shaders set the color of a pixel / - Possibilities of a fragment shader / Programmable / - Glare of textures / Vertex Processing. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Fragment Shader Concept 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Fragment Shader Concept 1/2'?
Page 58	Fragment Shader Concept 2/2
Source cue	- Fragment shaders are executed for fragments (= pixel predecessors) / - Fragment shaders calculate the color and depth value of the current fragment / - No information about adjacent fragments available / - The number of fragments can be changed / - Fragment shaders can not generate fragments

Professor-style explanation	The slide 'Fragment Shader Concept 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Fragment shaders are executed for fragments (= pixel predecessors) / - Fragment shaders calculate the color and depth value of the current fragment / - No information about adjacent fragments available / - The number of fragments can be changed / - Fragment shaders can not generate fragments. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Fragment Shader Concept 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Fragment shaders are executed for fragments (= pixel predecessors) / - Fragment shaders calculate the color and depth value of the current fragment / - No information about adjacent fragments available / - The number of fragments can be changed / - Fragment shaders can not generate fragments. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Fragment Shader Concept 2/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Fragment Shader Concept 2/2'?
Page 59	Shader Example
Source cue	<pre>layout (std140) uniform Matrices { Vertex Shader / mat4 projModelViewMatrix; / mat3 normalMatrix; /** Fragment Shader / }; * Set fragment color to red. / in vec3 position;</pre>

Professor-style explanation	The slide 'Shader Example' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: layout (std140) uniform Matrices { Vertex Shader / mat4 projModelViewMatrix; / mat3 normalMatrix; /** Fragment Shader / }; * Set fragment color to red. / in vec3 position. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Shader Example. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: layout (std140) uniform Matrices { Vertex Shader / mat4 projModelViewMatrix; / mat3 normalMatrix; /** Fragment Shader / }; * Set fragment color to red. / in vec3 position. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Shader Example' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Example'?
Page 60	GLSL Shader Functions
Source cue	- Trigonometry functions / - radians, degrees, sin, cos, tan, asin, acos, atan / - Power calculations / - pow, exp2, log2, sqrt, inversesqrt / - General

Professor-style explanation	The slide 'GLSL Shader Functions' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Trigonometry functions / - radians, degrees, sin, cos, tan, asin, acos, atan / - Power calculations / - pow, exp2, log2, sqrt, inversesqrt / - General. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about GLSL Shader Functions. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Trigonometry functions / - radians, degrees, sin, cos, tan, asin, acos, atan / - Power calculations / - pow, exp2, log2, sqrt, inversesqrt / - General. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'GLSL Shader Functions' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'GLSL Shader Functions'?
Page 61	Shader Usage
Source cue	- One or multiple shaders are attached to a shader program / Vertex Shader / glCreateShader glCreateProgram / Fragment Shader / glShaderSource glAttachShader glCreateShader

Professor-style explanation	The slide 'Shader Usage' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - One or multiple shaders are attached to a shader program / Vertex Shader / <code>glCreateShader</code> <code>glCreateProgram</code> / Fragment Shader / <code>glShaderSource</code> <code>glAttachShader</code> <code>glCreateShader</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Shader Usage. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - One or multiple shaders are attached to a shader program / Vertex Shader / <code>glCreateShader</code> <code>glCreateProgram</code> / Fragment Shader / <code>glShaderSource</code> <code>glAttachShader</code> <code>glCreateShader</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Shader Usage' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Usage'?
Page 62	Binding Shaders
Source cue	<pre>v = glCreateShader(GL_VERTEX_SHADER); // vertex shader / char* vs = readTextFile("vertex-shader.vert"); / const char* vv = vs; / glShaderSource(v, 1, &vv, NULL); / free(vs);</pre>

Professor-style explanation	The slide 'Binding Shaders' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: <code>v = glCreateShader(GL_VERTEX_SHADER); // vertex shader / char* vs = readTextFile("vertex-shader.vert"); / const char* vv = vs; / glShaderSource(v, 1, &vv, NULL); / free(vs)</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Binding Shaders. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: <code>v = glCreateShader(GL_VERTEX_SHADER); // vertex shader / char* vs = readTextFile("vertex-shader.vert"); / const char* vv = vs; / glShaderSource(v, 1, &vv, NULL); / free(vs)</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Binding Shaders' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Binding Shaders'?
Page 63	Setting Uniforms
Source cue	- Uniforms are used to communicate between CPU and GPU / - Data is bound to uniform locations / - Must be updated after each linking process / <code>GLint loc0 = glGetUniformLocation(p, "isoValue_"); / glUniform1f(loc0, 0.45f);</code>

Professor-style explanation	The slide 'Setting Uniforms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Uniforms are used to communicate between CPU and GPU / - Data is bound to uniform locations / - Must be updated after each linking process / GLint loc0 = glGetUniformLocation(p, "isoValue_"); / glUniform1f(loc0, 0.45f). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Setting Uniforms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Uniforms are used to communicate between CPU and GPU / - Data is bound to uniform locations / - Must be updated after each linking process / GLint loc0 = glGetUniformLocation(p, "isoValue_"); / glUniform1f(loc0, 0.45f). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Setting Uniforms' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Setting Uniforms' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Setting Uniforms' into a causal sentence instead of repeating the slide title?
Page 64	Shader Performance
Source cue	- Guidelines for efficient shaders / - Since fragment shaders are executed for a large number of fragments, / they should contain as few instructions as possible / - There should be as little as possible a change of the active shaders, / because they have their own state and, accordingly, the pipeline may need

Professor-style explanation	The slide 'Shader Performance' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Guidelines for efficient shaders - Since fragment shaders are executed for a large number of fragments, / they should contain as few instructions as possible / - There should be as little as possible a change of the active shaders, / because they have their own state and, accordingly, the pipeline may need. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Shader Performance. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Guidelines for efficient shaders - Since fragment shaders are executed for a large number of fragments, / they should contain as few instructions as possible / - There should be as little as possible a change of the active shaders, / because they have their own state and, accordingly, the pipeline may need. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Shader Performance' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Performance'?
Page 65	Shader Development Environments 1/2
Source cue	- FX Composer, NVPerfHUD (NVIDIA) / - HLSL Shader IDE with performance analytics / www.developer.nvidia.com/page/tools.html / - RenderMonkey (ATI) / - HLSL, GLSL Shader IDE with performance analytics

Professor-style explanation	The slide 'Shader Development Environments 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - FX Composer, NVPerfHUD (NVIDIA) / - HLSL Shader IDE with performance analytics / www.developer.nvidia.com/page/tools.html / - RenderMonkey (ATI) / - HLSL, GLSL Shader IDE with performance analytics. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Shader Development Environments 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - FX Composer, NVPerfHUD (NVIDIA) / - HLSL Shader IDE with performance analytics / www.developer.nvidia.com/page/tools.html / - RenderMonkey (ATI) / - HLSL, GLSL Shader IDE with performance analytics. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Shader Development Environments 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Development Environments 1/2'?
Page 66	Shader Development Environments 2/2
Source cue	- Shdr online GLSL editor: https://shdr.bkcore.com/

Professor-style explanation	The slide 'Shader Development Environments 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Shdr online GLSL editor: https://shdr.bkcore.com/. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Shader Development Environments 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Shdr online GLSL editor: https://shdr.bkcore.com/. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Shader Development Environments 2/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Development Environments 2/2'?
Page 67	2.5 Fragment Tests and Operations
Source cue	Towards the finishing line to the pixel

Professor-style explanation	The slide '2.5 Fragment Tests and Operations' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: Towards the finishing line to the pixel. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about 2.5 Fragment Tests and Operations. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: Towards the finishing line to the pixel. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for '2.5 Fragment Tests and Operations' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by '2.5 Fragment Tests and Operations'?
Page 68	Fragment Tests and Operations 1/2
Source cue	- After fragments have been generated, they must go through the per-fragment / tests and operations / Programmable / - Only if all these tests are successful, / Vertex Processing

Professor-style explanation	The slide 'Fragment Tests and Operations 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - After fragments have been generated, they must go through the per-fragment / tests and operations / Programmable / - Only if all these tests are successful, / Vertex Processing. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Fragment Tests and Operations 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - After fragments have been generated, they must go through the per-fragment / tests and operations / Programmable / - Only if all these tests are successful, / Vertex Processing. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Fragment Tests and Operations 1/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Fragment Tests and Operations 1/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Fragment Tests and Operations 1/2' into a causal sentence instead of repeating the slide title?
Page 69	Fragment Tests and Operations 2/2
Source cue	Fragment Pixel Ownership Scissor Alpha / Data Test Test Test / Depth Stencil / Framebuffer Dithering Blending / Test Test

Professor-style explanation	The slide 'Fragment Tests and Operations 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Fragment Pixel Ownership Scissor Alpha / Data Test Test Test / Depth Stencil / Framebuffer Dithering Blending / Test Test. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Fragment Tests and Operations 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Fragment Pixel Ownership Scissor Alpha / Data Test Test Test / Depth Stencil / Framebuffer Dithering Blending / Test Test. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Fragment Tests and Operations 2/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Fragment Tests and Operations 2/2'?
Page 70	Alpha Test
Source cue	- If framebuffer is an RGBA buffer, the alpha test can be performed / - The test must first be activated / glEnable(GL_ALPHA_TEST); / - The comparison function indicates which fragments survive the test / glAlphaFunc(func, value);

Professor-style explanation	The slide 'Alpha Test' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - If framebuffer is an RGBA buffer, the alpha test can be performed / - The test must first be activated / <code>glEnable(GL_ALPHA_TEST)</code>; / - The comparison function indicates which fragments survive the test / <code>glAlphaFunc(func, value)</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Alpha Test. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - If framebuffer is an RGBA buffer, the alpha test can be performed / - The test must first be activated / <code>glEnable(GL_ALPHA_TEST)</code>; / - The comparison function indicates which fragments survive the test / <code>glAlphaFunc(func, value)</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Alpha Test' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Alpha Test'?
Page 71	Stencil Test
Source cue	- Stencil Buffer / - Is a frame buffer component / - Is (indirectly) filled by drawing operations / - No special stencil drawing operations / - Stencil operation

Professor-style explanation	The slide 'Stencil Test' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Stencil Buffer / - Is a frame buffer component / - Is (indirectly) filled by drawing operations / - No special stencil drawing operations / - Stencil operation. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Stencil Test. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Stencil Buffer / - Is a frame buffer component / - Is (indirectly) filled by drawing operations / - No special stencil drawing operations / - Stencil operation. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Stencil Test' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Stencil Test'?
Page 72	Depth Test
Source cue	- If the framebuffer has a depth buffer, the depth test can be performed / - The test must first be activated / <code>glEnable(GL_DEPTH_TEST)</code>; / - The comparison is made with the depth value already in the buffer / - The comparison function indicates which fragments survive the test

Professor-style explanation	The slide 'Depth Test' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - If the framebuffer has a depth buffer, the depth test can be performed / - The test must first be activated / <code>glEnable(GL_DEPTH_TEST)</code>; / - The comparison is made with the depth value already in the buffer / - The comparison function indicates which fragments survive the test. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Depth Test. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - If the framebuffer has a depth buffer, the depth test can be performed / - The test must first be activated / <code>glEnable(GL_DEPTH_TEST)</code>; / - The comparison is made with the depth value already in the buffer / - The comparison function indicates which fragments survive the test. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Depth Test' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Test'?
Page 73	Image Composition and Mixture
Source cue	- Alpha values of the pixels / - control the combination of pixel values / - are specified as color components of color pixels / - $P = [R, G, B, A]$, with $R, G, B, A \in [0,1]$ / - Applications are manifold

Professor-style explanation	The slide 'Image Composition and Mixture' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Alpha values of the pixels / - control the combination of pixel values / - are specified as color components of color pixels / - $P = [R, G, B, A]$, with $R, G, B, A \in [0,1]$ / - Applications are manifold. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Image Composition and Mixture. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Alpha values of the pixels / - control the combination of pixel values / - are specified as color components of color pixels / - $P = [R, G, B, A]$, with $R, G, B, A \in [0,1]$ / - Applications are manifold. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Image Composition and Mixture' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Image Composition and Mixture' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Image Composition and Mixture' into a causal sentence instead of repeating the slide title?

Page 74	Blending Application
Source cue	- Example application for transparency: drawing with semi-transparent brush
Professor-style explanation	The slide 'Blending Application' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Example application for transparency: drawing with semi-transparent brush. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Blending Application. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Example application for transparency: drawing with semi-transparent brush. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Blending Application' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Blending Application' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Blending Application' into a causal sentence instead of repeating the slide title?

Page 75	Blending Process 1/2
Source cue	- Blending is an operation called per fragment / - Fragments are modified after rasterization, / but before writing to the framebuffer with a blending function / - Combination of a source fragment to be written with the destination pixel / in the frame buffer by using a blending function
Professor-style explanation	The slide 'Blending Process 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Blending is an operation called per fragment / - Fragments are modified after rasterization, / but before writing to the framebuffer with a blending function / - Combination of a source fragment to be written with the destination pixel / in the frame buffer by using a blending function. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.

Technical commentary	This slide is about Blending Process 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Blending is an operation called per fragment / - Fragments are modified after rasterization, / but before writing to the framebuffer with a blending function / - Combination of a source fragment to be written with the destination pixel / in the frame buffer by using a blending function. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Blending Process 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Blending Process 1/2'?

Page 76	Blending Process 2/2
Source cue	- Source factor: F / - Destination factor: F / - Source fragment: [P , P , P , P] / s,r s,g s,b s,a / - Destination pixel: [P , P , P , P]
Professor-style explanation	The slide 'Blending Process 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Source factor: F / - Destination factor: F / - Source fragment: [P , P , P , P] / s,r s,g s,b s,a / - Destination pixel: [P , P , P , P]. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Blending Process 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Source factor: F / - Destination factor: F / - Source fragment: [P , P , P , P] / s,r s,g s,b s,a / - Destination pixel: [P , P , P , P]. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Blending Process 2/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.

Common trap	Do not answer 'Blending Process 2/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Blending Process 2/2' into a causal sentence instead of repeating the slide title?
Page 77	Applying Blending
Source cue	- Switch on (or switch off) blending / - glEnable(GL_BLEND); / - glDisable(GL_BLEND); / - Defining Blending Factors / - glBlendFunc(srcFac, destFac);
Professor-style explanation	The slide 'Applying Blending' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Switch on (or switch off) blending / - glEnable(GL_BLEND); / - glDisable(GL_BLEND); / - Defining Blending Factors / - glBlendFunc(srcFac, destFac). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Applying Blending. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Switch on (or switch off) blending / - glEnable(GL_BLEND); / - glDisable(GL_BLEND); / - Defining Blending Factors / - glBlendFunc(srcFac, destFac). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Applying Blending' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Applying Blending' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Applying Blending' into a causal sentence instead of repeating the slide title?
Page 78	Blending Examples
Source cue	- Example 1: Copy source pixels and overwrite destination pixels / - glBlendFunc(GL_ONE, GL_ZERO); / - P = [P , P , P , P] / blend s,r s,g s,b s,a / - Example 2: Alpha-based mixing of source pixels and destination pixels

Professor-style explanation	The slide 'Blending Examples' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Example 1: Copy source pixels and overwrite destination pixels / - <code>glBlendFunc(GL_ONE, GL_ZERO);</code> / - $P = [P, P, P, P] / \text{blend } s, r \text{ } s, g \text{ } s, b \text{ } s, a$ / - Example 2: Alpha-based mixing of source pixels and destination pixels. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Blending Examples. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Example 1: Copy source pixels and overwrite destination pixels / - <code>glBlendFunc(GL_ONE, GL_ZERO);</code> / - $P = [P, P, P, P] / \text{blend } s, r \text{ } s, g \text{ } s, b \text{ } s, a$ / - Example 2: Alpha-based mixing of source pixels and destination pixels. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Blending Examples' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Blending Examples' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Blending Examples' into a causal sentence instead of repeating the slide title?
Page 79	2.6 Framebuffer
Source cue	Contains everything that is (in)visible on the screen

Professor-style explanation	The slide '2.6 Framebuffer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Contains everything that is (in)visible on the screen. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about 2.6 Framebuffer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Contains everything that is (in)visible on the screen. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for '2.6 Framebuffer' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '2.6 Framebuffer'?
Page 80	Viewport
Source cue	- Rectangular area in pixel grid, set explicitly / - Rendering operations are limited to this area / - All primitives that are completely or partially outside / are clipped at the viewport area (clipping)

Professor-style explanation	The slide 'Viewport' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Rectangular area in pixel grid, set explicitly / - Rendering operations are limited to this area / - All primitives that are completely or partially outside / are clipped at the viewport area (clipping). A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Viewport. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Rectangular area in pixel grid, set explicitly / - Rendering operations are limited to this area / - All primitives that are completely or partially outside / are clipped at the viewport area (clipping). A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Viewport' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Viewport'?
Page 81	Framebuffer
Source cue	- Context-bound raster with a resolution and structure / specified by the application / - Physical realization on the GPU / - Logical subdivision / - Color memory

Professor-style explanation	The slide 'Framebuffer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Context-bound raster with a resolution and structure / specified by the application / - Physical realization on the GPU / - Logical subdivision / - Color memory. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Framebuffer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Context-bound raster with a resolution and structure / specified by the application / - Physical realization on the GPU / - Logical subdivision / - Color memory. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Framebuffer' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Framebuffer'?
Page 82	OpenGL Framebuffers 1/3
Source cue	- Typical OpenGL framebuffer configuration / 8bit Stencil Buffer / 32bit Depth Buffer / = 32bit Color Buffer

Professor-style explanation	The slide 'OpenGL Framebuffers 1/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Typical OpenGL framebuffer configuration / 8bit Stencil Buffer / 32bit Depth Buffer / = 32bit Color Buffer. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Framebuffers 1/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Typical OpenGL framebuffer configuration / 8bit Stencil Buffer / 32bit Depth Buffer / = 32bit Color Buffer. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Framebuffers 1/3' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Framebuffers 1/3'?
Page 83	OpenGL Framebuffers 2/3
Source cue	- All buffers of a frame buffer have the same grid size / - Number of bitplanes per buffer can be configured individually / - Individual buffers are directly supported by the hardware / - Buffer identifier / - GL_COLOR_BUFFER_BIT

Professor-style explanation	The slide 'OpenGL Framebuffers 2/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - All buffers of a frame buffer have the same grid size - Number of bitplanes per buffer can be configured individually - Individual buffers are directly supported by the hardware - Buffer identifier - GL_COLOR_BUFFER_BIT Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Framebuffers 2/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - All buffers of a frame buffer have the same grid size - Number of bitplanes per buffer can be configured individually - Individual buffers are directly supported by the hardware - Buffer identifier - GL_COLOR_BUFFER_BIT Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Framebuffers 2/3' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Framebuffers 2/3'?
Page 84	OpenGL Framebuffers 3/3
Source cue	- Delete a buffer / <code>glClear(GLbitfield mask);</code> - Set the initialization color for the color buffer / <code>glClearColor(1.0,1.0,1.0,1.0);</code> - <code>glClear(GL_COLOR_BUFFER_BIT);</code>

Professor-style explanation	The outline 'OpenGL Framebuffers 3/3' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: - Delete a buffer / <code>glClear(GLbitfield mask);</code> / - Set the initialization color for the color buffer / <code>glClearColor(1.0,1.0,1.0,1.0);</code> / <code>glClear(GL_COLOR_BUFFER_BIT)</code> . Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about OpenGL Framebuffers 3/3. The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: - Delete a buffer / <code>glClear(GLbitfield mask);</code> / - Set the initialization color for the color buffer / <code>glClearColor(1.0,1.0,1.0,1.0);</code> / <code>glClear(GL_COLOR_BUFFER_BIT)</code> . Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for 'OpenGL Framebuffers 3/3' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize 'OpenGL Framebuffers 3/3' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where 'OpenGL Framebuffers 3/3' fits in the lecture order and what later section depends on it?
Page 85	Double Buffering
Source cue	- Use of two color buffers to avoid flicker effects / - Image buildup takes place in the background (back buffer) / - Visible image is independent of this (front buffer) / Rendering / Process

Professor-style explanation	The slide 'Double Buffering' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Use of two color buffers to avoid flicker effects / - Image buildup takes place in the background (back buffer) / - Visible image is independent of this (front buffer) / Rendering / Process. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Double Buffering. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Use of two color buffers to avoid flicker effects / - Image buildup takes place in the background (back buffer) / - Visible image is independent of this (front buffer) / Rendering / Process. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Double Buffering' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Double Buffering'?
Page 86	2.7 Pixel-based Rendering
Source cue	Direct pixel processing without involving 3D models

Professor-style explanation	The slide '2.7 Pixel-based Rendering' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: Direct pixel processing without involving 3D models. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations. For examination purposes, the important content is the chain width x height -> pixel count -> bits per pixel -> memory size; this chain also explains bandwidth and performance pressure.
Technical commentary	This slide is about 2.7 Pixel-based Rendering. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: Direct pixel processing without involving 3D models. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Exam-grade answer	A strong answer for '2.7 Pixel-based Rendering' includes the formula relation width x height x bits per pixel, distinguishes bits from bytes, and links larger images to memory bandwidth.
Common trap	Do not confuse bits with bytes in '2.7 Pixel-based Rendering', and do not ignore that alpha or higher precision changes memory size.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in '2.7 Pixel-based Rendering'?
Page 87	Pixel-based Rendering Operations
Source cue	- Generally / - Read and write operations for various image data formats, / usually as an external library (e.g., DevIL) / - Conversions between image data types / - In OpenGL done with framebuffer objects

Professor-style explanation	The slide 'Pixel-based Rendering Operations' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: - Generally / - Read and write operations for various image data formats, / usually as an external library (e.g., DevIL) / - Conversions between image data types / - In OpenGL done with framebuffer objects. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations. For examination purposes, the important content is the chain width x height -> pixel count -> bits per pixel -> memory size; this chain also explains bandwidth and performance pressure.
Technical commentary	This slide is about Pixel-based Rendering Operations. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: - Generally / - Read and write operations for various image data formats, / usually as an external library (e.g., DevIL) / - Conversions between image data types / - In OpenGL done with framebuffer objects. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Exam-grade answer	A strong answer for 'Pixel-based Rendering Operations' includes the formula relation width x height x bits per pixel, distinguishes bits from bytes, and links larger images to memory bandwidth.
Common trap	Do not confuse bits with bytes in 'Pixel-based Rendering Operations', and do not ignore that alpha or higher precision changes memory size.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'Pixel-based Rendering Operations'?
Page 88	Image Operations
Source cue	- Rendering image data with OpenGL / Copy Pixel Data / Draw/Read / Pixel / Geometry

Professor-style explanation	The slide 'Image Operations' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Rendering image data with OpenGL / Copy Pixel Data / Draw/Read / Pixel / Geometry. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Image Operations. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Rendering image data with OpenGL / Copy Pixel Data / Draw/Read / Pixel / Geometry. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Image Operations' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Image Operations'?
Page 89	Write Pixels into Framebuffer
Source cue	- glDrawPixels(x, y, width, height, format, type, pixels); / - Example // 64x64 array with rgb components / GLubyte img[64][64][3]; / void draw() {

Professor-style explanation	The slide 'Write Pixels into Framebuffer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - <code>glDrawPixels(x, y, width, height, format, type, pixels);</code> / - Example <code>// 64x64 array with rgb components / GLubyte img[64][64][3]; / void draw() {</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Write Pixels into Framebuffer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - <code>glDrawPixels(x, y, width, height, format, type, pixels);</code> / - Example <code>// 64x64 array with rgb components / GLubyte img[64][64][3]; / void draw() {</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Write Pixels into Framebuffer' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Write Pixels into Framebuffer'?
Page 90	Read Pixels from Framebuffer
Source cue	- <code>glReadPixels(x, y, width, height, format, type, pixels);</code> / - Example <code>/ GLubyte* img = 0; / int width, height; // canvas size / void snapshot() {</code>

Professor-style explanation	The slide 'Read Pixels from Framebuffer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - glReadPixels(x, y, width, height, format, type, pixels); / - Example / GLubyte* img = 0; / int width, height; // canvas size / void snapshot() {. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Read Pixels from Framebuffer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - glReadPixels(x, y, width, height, format, type, pixels); / - Example / GLubyte* img = 0; / int width, height; // canvas size / void snapshot() {. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Read Pixels from Framebuffer' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Read Pixels from Framebuffer'?
Page 91	Formats & Data Types
Source cue	- Format information for pixel array operations / - GL_RGB - red-green-blue values of the frame buffer (FB) / - GL_RED - red values of the FB / - GL_ALPHA - transparency values of the FB / - GL_RGBA - red-Green-Blue-Alpha values of the FB

Professor-style explanation	The slide 'Formats & Data Types' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Format information for pixel array operations / - GL_RGB - red-green-blue values of the frame buffer (FB) / - GL_RED - red values of the FB / - GL_ALPHA - transparency values of the FB / - GL_RGBA - red-Green-Blue-Alpha values of the FB. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Formats & Data Types. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Format information for pixel array operations / - GL_RGB - red-green-blue values of the frame buffer (FB) / - GL_RED - red values of the FB / - GL_ALPHA - transparency values of the FB / - GL_RGBA - red-Green-Blue-Alpha values of the FB. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Formats & Data Types' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Formats & Data Types'?
Page 92	coordinate systems in the geometry stage
Source cue	- Rendering pipeline consist of geometry stage and rasterization stage / - Geometric primitives (mostly triangles) are transformed through different / coordinate systems in the geometry stage / - Rasterization results in fragments, which are pixel predecessors / - Fragments have to endure a sequence of tests in the rasterization stage, in

Professor-style explanation	The slide 'coordinate systems in the geometry stage' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. The terms are successive representations in object-based rendering: - Rendering pipeline consist of geometry stage and rasterization stage / - Geometric primitives (mostly triangles) are transformed through different / coordinate systems in the geometry stage / - Rasterization results in fragments, which are pixel predecessors / - Fragments have to endure a sequence of tests in the rasterization stage, in. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter. For examination purposes, the important content is the representation change at each stage: model data, vertices, primitives, fragments, fragment tests, and final framebuffer updates.
Technical commentary	This slide is about coordinate systems in the geometry stage. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. The terms are successive representations in object-based rendering: - Rendering pipeline consist of geometry stage and rasterization stage / - Geometric primitives (mostly triangles) are transformed through different / coordinate systems in the geometry stage / - Rasterization results in fragments, which are pixel predecessors / - Fragments have to endure a sequence of tests in the rasterization stage, in. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels.
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Exam-grade answer	A strong answer for 'coordinate systems in the geometry stage' states what representation enters the stage, what representation leaves it, and which later stage depends on that output.
Common trap	Do not collapse 'coordinate systems in the geometry stage' into 'the GPU draws it'; name the intermediate representations because that is where most errors and exam distinctions appear.
Check yourself	Can you name the input and output representation for 'coordinate systems in the geometry stage' in the rendering pipeline?
Page 93	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Literature and other sources used in this chapter' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Literature and other sources used in this chapter' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 94	Guide: The Official Guide to Learning OpenGL
Source cue	- Text Books / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer

Professor-style explanation	The slide 'Guide: The Official Guide to Learning OpenGL' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: - Text Books / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.
Technical commentary	This slide is about Guide: The Official Guide to Learning OpenGL. The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: - Text Books / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Guide: The Official Guide to Learning OpenGL' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Guide: The Official Guide to Learning OpenGL' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Guide: The Official Guide to Learning OpenGL' points to for deeper study?

02.1 Rendering Process

Source location: Section 2.1

Commentary

Object-based rendering starts with scene objects represented as primitives. The pipeline transforms vertices, assembles primitives, rasterizes them into fragments, and lets only some fragments become pixels. The important detail is that the representation changes at every stage.

The application stage prepares data and state. The geometry stage handles positions and primitives. The rasterization stage creates fragments and applies fragment operations. This separation helps you explain both theory questions and black-screen OpenGL bugs.

Mental model

Vertices become primitives, primitives become fragments, and fragments compete to become pixel updates.

Check yourself

What is the difference between a fragment and a final pixel?

02.2 OpenGL Overview

Source location: Section 2.2

Commentary

OpenGL is an API for controlling a graphics pipeline. It is not a renderer by itself in the sense of a single command that understands your scene. You create a context, provide buffers, set state, compile shaders, bind objects, and issue draw calls.

Modern OpenGL is stateful and shader-based. Many errors happen because the programmer assumes that a later call carries more information than it really does. The GPU uses the currently bound objects and current state at draw time.

Mental model

OpenGL draw calls consume the state machine as it exists right now.

Check yourself

Why can binding the wrong vertex array or shader program produce a valid draw call with wrong output?

02.3 OpenGL Rendering

Source location: Section 2.3

Commentary

Older OpenGL exposed a fixed-function style where many transformations and lighting choices were configured through predefined calls. Modern OpenGL expects you to write shader programs and explicitly manage data flow.

This is not just historical trivia. It explains why shader code, vertex attributes, uniforms, and buffer objects are central in the exercises. If the shader expects an attribute and the vertex layout does not provide it, the pipeline has no magic fallback.

Mental model

Modern OpenGL makes the data path explicit; that gives control but also creates responsibility.

Check yourself

Which data usually goes into vertex attributes, and which data usually goes into uniforms?

02.4 OpenGL Rendering Pipeline

Source location: Section 2.4

Commentary

The vertex shader runs per vertex and usually outputs clip-space position plus attributes for later interpolation. Primitive assembly groups vertices into points, lines, or triangles. Clipping handles geometry outside the view volume. Rasterization creates fragments.

The fragment shader computes candidate fragment output, often using interpolated attributes, uniforms, and textures. After that, tests and operations such as depth testing, stencil testing, blending, and masking decide what reaches the framebuffer.

Mental model

Programmable shaders compute values; fixed pipeline stages decide coverage, interpolation, tests, and output rules.

Check yourself

If your geometry appears in wireframe but colors are wrong, which stages become suspicious?

02.5 Fragment Tests, Framebuffer, and Pixel-Based Rendering

Source location: Sections 2.5-2.7

Commentary

Fragment tests are the gatekeepers after shading. A fragment can have a perfectly valid color and still fail because depth, stencil, scissor, masking, or blending state prevents a visible update. This is why final pixels are a subset of generated fragments.

The framebuffer stores the final render targets: color buffers, depth buffers, stencil buffers, or custom attachments. Pixel-based rendering starts from image data instead of geometric primitives, which is useful for image processing but conceptually different from the object-to-pixel pipeline.

Mental model

The framebuffer is the destination; fragment operations are the rules for writing into it.

Check yourself

Why can two fragments with different colors produce only one visible pixel color?

End-of-lecture summary

If you remember only one thing from Lecture 02, remember this: This lecture turns the vague idea of rendering into a concrete sequence. It is the most important debugging map in the course: if an image is wrong, the pipeline tells you where the error could have entered.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 03 - Geometric Transformations

This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Source chunk	course_text_parts/03_lectures/03_geometric-transformations.txt
Extracted slide pages	54
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.

Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Transforming 3D Models
Source cue	- 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through
Professor-style explanation	The slide 'Transforming 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Transforming 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Transforming 3D Models' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Transforming 3D Models' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Transforming 3D Models' before rendering begins?
Page 3	3.1 Mathematical Foundations

Source cue	3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL
Professor-style explanation	The outline '3.1 Mathematical Foundations' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: 3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about 3.1 Mathematical Foundations. The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: 3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for '3.1 Mathematical Foundations' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize '3.1 Mathematical Foundations' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where '3.1 Mathematical Foundations' fits in the lecture order and what later section depends on it?
Page 4	3.1 Mathematical Foundations
Source cue	Vector and Matrix Calculus

Professor-style explanation	The slide '3.1 Mathematical Foundations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Vector and Matrix Calculus. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about 3.1 Mathematical Foundations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Vector and Matrix Calculus. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for '3.1 Mathematical Foundations' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in '3.1 Mathematical Foundations' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after '3.1 Mathematical Foundations'?
Page 5	Vertices as Vectors
Source cue	- Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of $\mathbb{R}^2$, $\mathbb{R}^3$, $\mathbb{R}^4$ / - In computer graphics mostly $\mathbb{R}^3$ or $\mathbb{R}^4$ vectors used / - Vertices of a polygon are represented as column vectors / $x \ v$

Professor-style explanation	The slide 'Vertices as Vectors' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of R^2, R^3, R^4 / - In computer graphics mostly R^3 or R^4 vectors used / - Vertices of a polygon are represented as column vectors x or v. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Vertices as Vectors. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of R^2, R^3, R^4 / - In computer graphics mostly R^3 or R^4 vectors used / - Vertices of a polygon are represented as column vectors x or v. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Vertices as Vectors' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Vertices as Vectors' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Vertices as Vectors'?
Page 6	Transformations as Matrices
Source cue	- Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $a \ b \ c \ d \ e \ f \ g \ h \ M =$

Professor-style explanation	The slide 'Transformations as Matrices' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $\begin{bmatrix} a & b & c & d \\ e & f & g & h \end{bmatrix} / M =$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Transformations as Matrices. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $\begin{bmatrix} a & b & c & d \\ e & f & g & h \end{bmatrix} / M =$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Transformations as Matrices' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Transformations as Matrices' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Transformations as Matrices'?
Page 7	Matrix Vector Multiplication
Source cue	- Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $m \times n$ matrix A and R_m vector B / - Result: R_n vector C, whereby is B transformed through A / - Example for 4×4 matrix A and R_4 vector B:

Professor-style explanation	The slide 'Matrix Vector Multiplication' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $m \times n$ matrix A and R_m vector B / - Result: R_n vector C, whereby is B transformed through A / - Example for 4×4 matrix A and R_4 vector B. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Matrix Vector Multiplication. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $m \times n$ matrix A and R_m vector B / - Result: R_n vector C, whereby is B transformed through A / - Example for 4×4 matrix A and R_4 vector B. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Matrix Vector Multiplication' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Matrix Vector Multiplication' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Matrix Vector Multiplication'?
Page 8	3.2 Affine Transformations
Source cue	Translation, Rotation, Scaling, and more

Professor-style explanation	The slide '3.2 Affine Transformations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Translation, Rotation, Scaling, and more. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about 3.2 Affine Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Translation, Rotation, Scaling, and more. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for '3.2 Affine Transformations' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in '3.2 Affine Transformations' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after '3.2 Affine Transformations'?
Page 9	Translations - Moving 3D Models
Source cue	- Matrix representation of a translation / $\begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & d \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{bmatrix}$ / T_d , $d = [x \ y \ z]$

Professor-style explanation	The slide 'Translations - Moving 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Matrix representation of a translation / $\begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & d \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{bmatrix}$ / $T(d)$, $d = [x \ y \ z]$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Translations - Moving 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Matrix representation of a translation / $\begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & d \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{bmatrix}$ / $T(d)$, $d = [x \ y \ z]$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Translations - Moving 3D Models' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Translations - Moving 3D Models' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Translations - Moving 3D Models' before rendering begins?
Page 10	Translation Properties
Source cue	- Lengths and angles are preserved / - Identity: $T(0, 0, 0) = I$ (identity matrix) / - Changing order of translations: / - $T(d_1, d_1, d_1) \cdot T(d_2, d_2, d_2) = T(d_2, d_2, d_2) \cdot T(d_1, d_1, d_1)$ / $x \ y \ z \ x \ y \ z \ x \ y \ z \ x \ y \ z$

Professor-style explanation	The slide 'Translation Properties' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Lengths and angles are preserved / - Identity: $T(0, 0, 0) = I$ (identity matrix) / - Changing order of translations: $T(d_1, d_1, d_1) \cdot T(d_2, d_2, d_2) = T(d_2, d_2, d_2) \cdot T(d_1, d_1, d_1)$ / $x y z x y z x y z x y z$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Translation Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Lengths and angles are preserved / - Identity: $T(0, 0, 0) = I$ (identity matrix) / - Changing order of translations: $T(d_1, d_1, d_1) \cdot T(d_2, d_2, d_2) = T(d_2, d_2, d_2) \cdot T(d_1, d_1, d_1)$ / $x y z x y z x y z x y z$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Translation Properties' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Translation Properties' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Translation Properties'?
Page 11	Scaling - Resizing 3D Models
Source cue	- Matrix representation of a scaling / $s_0 0 0 / 0 s_0 0 / S s, s, s = / x y z$

Professor-style explanation	The slide 'Scaling - Resizing 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Matrix representation of a scaling $\begin{bmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, $s, s, s = \begin{bmatrix} x & y & z \end{bmatrix}$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Scaling - Resizing 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Matrix representation of a scaling $\begin{bmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & s & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, $s, s, s = \begin{bmatrix} x & y & z \end{bmatrix}$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Scaling - Resizing 3D Models' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Scaling - Resizing 3D Models' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Scaling - Resizing 3D Models' before rendering begins?
Page 12	Scaling Properties
Source cue	- Lengths are not preserved, angles only for uniform scalings / - Identity: $S(1, 1, 1) = I$ (identity matrix) / - Changing order of scalings: $S(s_1, s_1, s_1) \cdot S(s_2, s_2, s_2) = S(s_2, s_2, s_2) \cdot S(s_1, s_1, s_1)$ / $x y z x y z x y z x y z$

Professor-style explanation	The slide 'Scaling Properties' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Lengths are not preserved, angles only for uniform scalings / - Identity: $S(1, 1, 1) = I$ (identity matrix) / - Changing order of scalings: $S(s_1, s_1, s_1) \cdot S(s_2, s_2, s_2) = S(s_2, s_2, s_2) \cdot S(s_1, s_1, s_1)$ / $x y z x y z x y z x y z$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Scaling Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Lengths are not preserved, angles only for uniform scalings / - Identity: $S(1, 1, 1) = I$ (identity matrix) / - Changing order of scalings: $S(s_1, s_1, s_1) \cdot S(s_2, s_2, s_2) = S(s_2, s_2, s_2) \cdot S(s_1, s_1, s_1)$ / $x y z x y z x y z x y z$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Scaling Properties' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Scaling Properties' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Scaling Properties'?
Page 13	Rotation - Changing Orientation of 3D Models 1/2
Source cue	- Matrix representation of a rotation around x-, y- and z-axis / $1 \ 0 \ 0 \ 0 / 0 \ \cos \theta \ -\sin \theta \ 0 / R \ \theta = / 0 \ \sin \theta \ \cos \theta \ 0$

Professor-style explanation	The slide 'Rotation - Changing Orientation of 3D Models 1/2' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Matrix representation of a rotation around x-, y- and z-axis / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \end{bmatrix}$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Rotation - Changing Orientation of 3D Models 1/2. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Matrix representation of a rotation around x-, y- and z-axis / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \end{bmatrix}$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Rotation - Changing Orientation of 3D Models 1/2' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Rotation - Changing Orientation of 3D Models 1/2' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Rotation - Changing Orientation of 3D Models 1/2' before rendering begins?
Page 14	Rotation - Changing Orientation of 3D Models 2/2
Source cue	- Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x

Professor-style explanation	The slide 'Rotation - Changing Orientation of 3D Models 2/2' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Rotation - Changing Orientation of 3D Models 2/2. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Rotation - Changing Orientation of 3D Models 2/2' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Rotation - Changing Orientation of 3D Models 2/2' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Rotation - Changing Orientation of 3D Models 2/2' before rendering begins?
Page 15	Rotation Properties
Source cue	- Lengths and angles are preserved / - Identity: $R_0 = R_0 = R_0 = I$ (identity matrix) / - x y z / - Changing order of rotations around same axis: / - $R(\theta) \cdot R(\phi) = R(\phi) \cdot R(\theta)$

Professor-style explanation	The slide 'Rotation Properties' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Lengths and angles are preserved / - Identity: $R_0 = R_0 = R_0 = I$ (identity matrix) / x y z / - Changing order of rotations around same axis: / - $R(\theta) \cdot R(\phi) = R(\phi) \cdot R(\theta)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Rotation Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Lengths and angles are preserved / - Identity: $R_0 = R_0 = R_0 = I$ (identity matrix) / x y z / - Changing order of rotations around same axis: / - $R(\theta) \cdot R(\phi) = R(\phi) \cdot R(\theta)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Rotation Properties' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Rotation Properties' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Rotation Properties'?
Page 16	Mirroring - Reflecting 3D Models on a Plane
Source cue	- Matrix representation of mirroring along x-y plane / $1\ 0\ 0\ 0 / 0\ 1\ 0\ 0 / M = 0\ 0\ -1\ 0$

Professor-style explanation	The slide 'Mirroring - Reflecting 3D Models on a Plane' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Matrix representation of mirroring along x-y plane / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ / $M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Mirroring - Reflecting 3D Models on a Plane. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Matrix representation of mirroring along x-y plane / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ / $M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Mirroring - Reflecting 3D Models on a Plane' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Mirroring - Reflecting 3D Models on a Plane' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Mirroring - Reflecting 3D Models on a Plane' before rendering begins?
Page 17	Shearing - Skewing 3D Models
Source cue	- Matrix representation of a shearing / $\begin{bmatrix} 1 & h & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ / $\begin{bmatrix} 1 & h & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Professor-style explanation	The slide 'Shearing - Skewing 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Matrix representation of a shearing $\begin{bmatrix} 1 & h & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ / $xy\ xz\ / \ h\ 1\ h\ 0\ / \ yx\ yz$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Shearing - Skewing 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Matrix representation of a shearing $\begin{bmatrix} 1 & h & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ / $xy\ xz\ / \ h\ 1\ h\ 0\ / \ yx\ yz$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Shearing - Skewing 3D Models' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Shearing - Skewing 3D Models' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Shearing - Skewing 3D Models' before rendering begins?
Page 18	3.3 Composition of Transformations
Source cue	Concatenation of several transformation steps

Professor-style explanation	The slide '3.3 Composition of Transformations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Concatenation of several transformation steps. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about 3.3 Composition of Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Concatenation of several transformation steps. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for '3.3 Composition of Transformations' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in '3.3 Composition of Transformations' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after '3.3 Composition of Transformations'?
Page 19	Composition of Transformations
Source cue	- Composition of transformations T is done / by multiplying the respective transformation matrices / - $(T \cdot T \cdot \dots \cdot T \cdot T) \cdot v = (T \cdot (T \cdot (\dots \cdot (T \cdot (T \cdot v))))$ / 1 2 n-1 n 1 2 n-1 n / - Interpretation: apply first transformation T , then T , ..., and finally T

Professor-style explanation	The slide 'Composition of Transformations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Composition of transformations T is done / by multiplying the respective transformation matrices / - $(T \cdot T \cdot \dots \cdot T \cdot T) \cdot v = (T \cdot (T \cdot (\dots (T \cdot (T \cdot v)))))$ / 1 2 n-1 n 1 2 n-1 n / - Interpretation: apply first transformation T , then T , ..., and finally T. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Composition of Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Composition of transformations T is done / by multiplying the respective transformation matrices / - $(T \cdot T \cdot \dots \cdot T \cdot T) \cdot v = (T \cdot (T \cdot (\dots (T \cdot (T \cdot v)))))$ / 1 2 n-1 n 1 2 n-1 n / - Interpretation: apply first transformation T , then T , ..., and finally T. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Composition of Transformations' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Composition of Transformations' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Composition of Transformations'?
Page 20	Reference Points 1/2
Source cue	- Rotation and scaling happen in relation to origin (reference point)

Professor-style explanation	The slide 'Reference Points 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Rotation and scaling happen in relation to origin (reference point). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Reference Points 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Rotation and scaling happen in relation to origin (reference point). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Reference Points 1/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Reference Points 1/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Reference Points 1/2' into a causal sentence instead of repeating the slide title?
Page 21	Reference Points 2/2
Source cue	- In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point M to the origin O / - Rotate (or scale) the 3D model / - Move the 3D model back to M

Professor-style explanation	The slide 'Reference Points 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point M to the origin O / - Rotate (or scale) the 3D model / - Move the 3D model back to M. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Reference Points 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point M to the origin O / - Rotate (or scale) the 3D model / - Move the 3D model back to M. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Reference Points 2/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Reference Points 2/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Reference Points 2/2'?
Page 22	Rotation Around Arbitrary Axis
Source cue	- Rotation around any axis / - Axis given by center of rotation P and direction $U = (u_x, u_y, u_z)$ / x y z / - Rotation by θ degrees / - Construction of transformation as transformation sequence

Professor-style explanation	The slide 'Rotation Around Arbitrary Axis' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Rotation around any axis - Axis given by center of rotation P and direction $U = (u_x, u_y, u_z)$ - Rotation by θ degrees - Construction of transformation as transformation sequence. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Rotation Around Arbitrary Axis. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Rotation around any axis - Axis given by center of rotation P and direction $U = (u_x, u_y, u_z)$ - Rotation by θ degrees - Construction of transformation as transformation sequence. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Rotation Around Arbitrary Axis' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Rotation Around Arbitrary Axis' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Rotation Around Arbitrary Axis'?
Page 23	Reminder: Vector Calculus
Source cue	- Length of a vector is expressed as follows $v = \sqrt{v_x^2 + v_y^2 + v_z^2}$ - Vectors of length $v = 1$ are normalized and called unit vectors - Arbitrary vectors can be normalized as follows

Professor-style explanation	The slide 'Reminder: Vector Calculus' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Length of a vector is expressed as follows $v = \sqrt{v_x^2 + v_y^2 + v_z^2}$ - Vectors of length $v = 1$ are normalized and called unit vectors - Arbitrary vectors can be normalized as follows. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Reminder: Vector Calculus. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Length of a vector is expressed as follows $v = \sqrt{v_x^2 + v_y^2 + v_z^2}$ - Vectors of length $v = 1$ are normalized and called unit vectors - Arbitrary vectors can be normalized as follows. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Reminder: Vector Calculus' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Reminder: Vector Calculus'?
Page 24	Reminder: Cross Product (= vector product)
Source cue	- Given two vectors u and v in $\mathbb{R}^3$ - Cross product $u \times v$ of u and v is a vector defined as follows $u \times v = \begin{vmatrix} u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix} = \begin{pmatrix} u_y v_z - u_z v_y \\ u_z v_x - u_x v_z \\ u_x v_y - u_y v_x \end{pmatrix}$

Professor-style explanation	The slide 'Reminder: Cross Product (= vector product)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Given two vectors u and v in R^3 / - Cross product $u \times v$ of u and v is a vector defined as follows / $u \cdot v = u \cdot v - u \cdot v / x \times y \ z \ z \ y / u \times v = u \cdot v - u \cdot v$. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Reminder: Cross Product (= vector product). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Given two vectors u and v in R^3 / - Cross product $u \times v$ of u and v is a vector defined as follows / $u \cdot v = u \cdot v - u \cdot v / x \times y \ z \ z \ y / u \times v = u \cdot v - u \cdot v$. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Reminder: Cross Product (= vector product)' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Reminder: Cross Product (= vector product)' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Reminder: Cross Product (= vector product)' into a causal sentence instead of repeating the slide title?
Page 25	Example - Transforming Directed Line Segment 1/7
Source cue	- Example transformation task / - Given line segments $P_1 P_2$ and $P_1 P_3$ / 1 2 1 3 / - Transform it into the yz-plane so that $P_1 P_2$ lies on the z-axis / 1 2

Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 1/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Example transformation task / - Given line segments $P P$ and $P P / 1 \ 2 \ 1 \ 3$ / - Transform it into the yz-plane so that $P P$ lies on the z-axis / 1 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 1/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Example transformation task / - Given line segments $P P$ and $P P / 1 \ 2 \ 1 \ 3$ / - Transform it into the yz-plane so that $P P$ lies on the z-axis / 1 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example - Transforming Directed Line Segment 1/7' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example - Transforming Directed Line Segment 1/7' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 1/7'?
Page 26	Example - Transforming Directed Line Segment 2/7
Source cue	- Approach 1: Determine rotation angles for rotation sequence / - Move $P = x, y, z$ into the origin / 1 1 1 1 / - Rotate around the y-axis so that $P P$ lies in the yz-plane / 1 2

Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 2/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Approach 1: Determine rotation angles for rotation sequence / - Move $P = x, y, z$ into the origin / 1 1 1 1 / - Rotate around the y-axis so that P lies in the yz-plane / 1 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 2/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Approach 1: Determine rotation angles for rotation sequence / - Move $P = x, y, z$ into the origin / 1 1 1 1 / - Rotate around the y-axis so that P lies in the yz-plane / 1 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example - Transforming Directed Line Segment 2/7' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example - Transforming Directed Line Segment 2/7' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 2/7'?
Page 27	Example - Transforming Directed Line Segment 3/7
Source cue	- Step 2 - Rotation of P' around y-axis into yz-plane / 1 2 / - Rotation $R(\theta - 90^\circ)$ / - Rotation angle - $(90^\circ - \theta) = \theta - 90^\circ$ / - Matrix ingredients

Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 3/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Step 2 - Rotation of $P' P'$ around y-axis into yz-plane / 1 2 / - Rotation $R(\theta - 90^\circ)$ / - Rotation angle - $(90^\circ - \theta) = \theta - 90^\circ$ / - Matrix ingredients. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 3/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Step 2 - Rotation of $P' P'$ around y-axis into yz-plane / 1 2 / - Rotation $R(\theta - 90^\circ)$ / - Rotation angle - $(90^\circ - \theta) = \theta - 90^\circ$ / - Matrix ingredients. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example - Transforming Directed Line Segment 3/7' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example - Transforming Directed Line Segment 3/7' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 3/7'?
Page 28	Example - Transforming Directed Line Segment 4/7
Source cue	- Step 3 - Rotation of $P'' P''$ around x-axis onto z-axis / 1 2 / - Rotation $R(\phi)$ / - Matrix ingredients / z''

Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 4/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Step 3 - Rotation of $P'' P''$ around x-axis onto z-axis / 1 2 / - Rotation $R(\phi)$ / - Matrix ingredients / z'' . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 4/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Step 3 - Rotation of $P'' P''$ around x-axis onto z-axis / 1 2 / - Rotation $R(\phi)$ / - Matrix ingredients / z'' . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example - Transforming Directed Line Segment 4/7' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example - Transforming Directed Line Segment 4/7' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 4/7'?
Page 29	Example - Transforming Directed Line Segment 5/7
Source cue	- Step 4 - Rotation of $P''' P'''$ into yz-plane / 1 3 / - Rotation $R(\alpha)$ / $y \blacksquare \blacksquare \blacksquare$ / - Matrix ingredients

Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 5/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Step 4 - Rotation of P''' P''' into yz-plane / 1 3 / - Rotation $R(\alpha)$ / y_{axis} / - Matrix ingredients. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 5/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Step 4 - Rotation of P''' P''' into yz-plane / 1 3 / - Rotation $R(\alpha)$ / y_{axis} / - Matrix ingredients. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example - Transforming Directed Line Segment 5/7' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example - Transforming Directed Line Segment 5/7' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 5/7'?
Page 30	Example - Transforming Directed Line Segment 6/7
Source cue	- Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / (R on x-axis, R on y-axis, R on z-axis)

Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 6/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / (R on x-axis, R on y-axis, R on z-axis). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 6/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / (R on x-axis, R on y-axis, R on z-axis). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example - Transforming Directed Line Segment 6/7' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example - Transforming Directed Line Segment 6/7' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 6/7'?
Page 31	Example - Transforming Directed Line Segment 7/7
Source cue	- Consideration 2: Formulate R as unit vector / orthogonal to the plane spanned by P_1, P_2, and P_3 / $P_1 \times P_2 \times P_3$ / - $R = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & -1 \end{bmatrix}$

Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 7/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Consideration 2: Formulate R as unit vector / orthogonal to the plane spanned by P_1, P_2, and P_3 / $P_1 \times P_2$ / $R = \frac{1}{\ P_1 \times P_2\ } (P_1 \times P_2)$ / $R^T = \frac{1}{\ P_1 \times P_2\ } (P_2 \times P_1)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 7/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Consideration 2: Formulate R as unit vector / orthogonal to the plane spanned by P_1, P_2, and P_3 / $P_1 \times P_2$ / $R = \frac{1}{\ P_1 \times P_2\ } (P_1 \times P_2)$ / $R^T = \frac{1}{\ P_1 \times P_2\ } (P_2 \times P_1)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example - Transforming Directed Line Segment 7/7' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example - Transforming Directed Line Segment 7/7' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 7/7'?
Page 32	Example - Direction of Flight Transformation 1/3
Source cue	- Example transformation task / - Given 3D model centered in origin of x, y, z -coordinate system / p, p, p / - Transform it such that / - it is centered in arbitrary point P, and

Professor-style explanation	The slide 'Example - Direction of Flight Transformation 1/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Example transformation task / - Given 3D model centered in origin of x -, y -, z -coordinate system / p p p / - Transform it such that / - it is centered in arbitrary point P, and. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example - Direction of Flight Transformation 1/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Example transformation task / - Given 3D model centered in origin of x -, y -, z -coordinate system / p p p / - Transform it such that / - it is centered in arbitrary point P, and. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example - Direction of Flight Transformation 1/3' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example - Direction of Flight Transformation 1/3' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example - Direction of Flight Transformation 1/3'?
Page 33	Example - Direction of Flight Transformation 2/3
Source cue	$r \ r \ r / 1x \ 2x \ 3x /$ - Step 1 - create orthogonal rotation matrix $R = r \ r \ r / 1y \ 2y \ 3y / r \ r \ r$

Professor-style explanation	The slide 'Example - Direction of Flight Transformation 2/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: $r \ r \ r / 1x \ 2x \ 3x /$ - Step 1 - create orthogonal rotation matrix $R = r \ r \ r / 1y \ 2y \ 3y / r \ r \ r$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example - Direction of Flight Transformation 2/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: $r \ r \ r / 1x \ 2x \ 3x /$ - Step 1 - create orthogonal rotation matrix $R = r \ r \ r / 1y \ 2y \ 3y / r \ r \ r$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example - Direction of Flight Transformation 2/3' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example - Direction of Flight Transformation 2/3' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example - Direction of Flight Transformation 2/3'?
Page 34	Example - Direction of Flight Transformation 3/3
Source cue	- Transformation M is described by a translation and an orthogonal matrix $/ r \ r \ r \ 0 / 1x \ 2x \ 3x / r \ r \ r \ 0 / 1y \ 2y \ 3y$

Professor-style explanation	The slide 'Example - Direction of Flight Transformation 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Transformation M is described by a translation and an orthogonal matrix $\begin{bmatrix} r_x & r_y & r_z & 0 \\ 1x & 2x & 3x & r_x \\ r_y & 0 & 1y & 2y & 3y \end{bmatrix}$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example - Direction of Flight Transformation 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Transformation M is described by a translation and an orthogonal matrix $\begin{bmatrix} r_x & r_y & r_z & 0 \\ 1x & 2x & 3x & r_x \\ r_y & 0 & 1y & 2y & 3y \end{bmatrix}$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example - Direction of Flight Transformation 3/3' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example - Direction of Flight Transformation 3/3' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example - Direction of Flight Transformation 3/3'?
Page 35	3.4 Coordinate System Change
Source cue	Transformation as change of frame of reference

Professor-style explanation	The slide '3.4 Coordinate System Change' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Transformation as change of frame of reference. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about 3.4 Coordinate System Change. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Transformation as change of frame of reference. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for '3.4 Coordinate System Change' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in '3.4 Coordinate System Change' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after '3.4 Coordinate System Change'?
Page 36	Coordinate System Change
Source cue	- Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2

Professor-style explanation	The slide 'Coordinate System Change' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Coordinate System Change. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Coordinate System Change' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Coordinate System Change' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Coordinate System Change'?
Page 37	Local Coordinate Systems
Source cue	- Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS)

Professor-style explanation	The slide 'Local Coordinate Systems' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Local Coordinate Systems. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Local Coordinate Systems' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Local Coordinate Systems'?
Page 38	Example 1 - Coordinate System Change 1/2
Source cue	x(1) / - Example transformation task y y(5) / - Rotation and scaling of an object around its center M / y(1) / and subsequent translation to a point Q

Professor-style explanation	The slide 'Example 1 - Coordinate System Change 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: $x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center M / $y(1)$ / and subsequent translation to a point Q. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example 1 - Coordinate System Change 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: $x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center M / $y(1)$ / and subsequent translation to a point Q. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example 1 - Coordinate System Change 1/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example 1 - Coordinate System Change 1/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example 1 - Coordinate System Change 1/2'?
Page 39	Example 1 - Coordinate System Change 2/2
Source cue	$y(5)$ / $y(2)$ $y(3)$

Professor-style explanation	The slide 'Example 1 - Coordinate System Change 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: $y \ y \ y \ y / x \ x \ x \ x / 3D \ Model \ Transformation / y(5) / y(2) \ y(3)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example 1 - Coordinate System Change 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: $y \ y \ y \ y / x \ x \ x \ x / 3D \ Model \ Transformation / y(5) / y(2) \ y(3)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example 1 - Coordinate System Change 2/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example 1 - Coordinate System Change 2/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example 1 - Coordinate System Change 2/2'?
Page 40	Example 2 - Hierarchical Coordinate Systems 1/2
Source cue	- To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system

Professor-style explanation	The slide 'Example 2 - Hierarchical Coordinate Systems 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example 2 - Hierarchical Coordinate Systems 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example 2 - Hierarchical Coordinate Systems 1/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example 2 - Hierarchical Coordinate Systems 1/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example 2 - Hierarchical Coordinate Systems 1/2'?
Page 41	Example 2 - Hierarchical Coordinate Systems 1/2
Source cue	- Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation $R(\alpha)$ / y-tr

Professor-style explanation	The slide 'Example 2 - Hierarchical Coordinate Systems 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation $R(\alpha)$ / y-tr. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Example 2 - Hierarchical Coordinate Systems 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation $R(\alpha)$ / y-tr. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Example 2 - Hierarchical Coordinate Systems 1/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Example 2 - Hierarchical Coordinate Systems 1/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Example 2 - Hierarchical Coordinate Systems 1/2'?
Page 42	3.5 Transformations in OpenGL
Source cue	Implementation with GLM und GLSL

Professor-style explanation	The slide '3.5 Transformations in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Implementation with GLM und GLSL. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about 3.5 Transformations in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Implementation with GLM und GLSL. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for '3.5 Transformations in OpenGL' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '3.5 Transformations in OpenGL'?
Page 43	Geometric Transformations in OpenGL
Source cue	- Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements

Professor-style explanation	The slide 'Geometric Transformations in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Geometric Transformations in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Geometric Transformations in OpenGL' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Geometric Transformations in OpenGL'?
Page 44	OpenGL Transformation Pipeline
Source cue	- Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline

Professor-style explanation	The slide 'OpenGL Transformation Pipeline' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Transformation Pipeline. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Transformation Pipeline' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Transformation Pipeline'?
Page 45	Example - Translation and Scaling
Source cue	- Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized

Professor-style explanation	The slide 'Example - Translation and Scaling' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Example - Translation and Scaling. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Example - Translation and Scaling' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Example - Translation and Scaling' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Example - Translation and Scaling' into a causal sentence instead of repeating the slide title?
Page 46	Matrix Specification
Source cue	- Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] =;

Professor-style explanation	The slide 'Matrix Specification' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] = . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Matrix Specification. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] = . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Matrix Specification' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Matrix Specification' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Matrix Specification'?
Page 47	Example - Portico Modeling 1/5
Source cue	- Columned hall consists of floor space, $H \cdot 0.1$ / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is H,

Professor-style explanation	The slide 'Example - Portico Modeling 1/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Columned hall consists of floor space, $H \cdot 0.1$ / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is H,. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Example - Portico Modeling 1/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Columned hall consists of floor space, $H \cdot 0.1$ / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is H,. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Example - Portico Modeling 1/5' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Example - Portico Modeling 1/5' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 1/5' before rendering begins?
Page 48	Example - Portico Modeling 2/5
Source cue	- Drawing a cylinder closed at top and bottom / - Each column should "stand" in the y-axis / - Each column should have a lid surface at the top and bottom / - Disk primitive is embedded in the xy-plane / - Cylinder primitive consists only of lateral surface

Professor-style explanation	The slide 'Example - Portico Modeling 2/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Drawing a cylinder closed at top and bottom - Each column should "stand" in the y-axis - Each column should have a lid surface at the top and bottom - Disk primitive is embedded in the xy-plane - Cylinder primitive consists only of lateral surface. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Example - Portico Modeling 2/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Drawing a cylinder closed at top and bottom - Each column should "stand" in the y-axis - Each column should have a lid surface at the top and bottom - Disk primitive is embedded in the xy-plane - Cylinder primitive consists only of lateral surface. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Example - Portico Modeling 2/5' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Example - Portico Modeling 2/5' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 2/5' before rendering begins?
Page 49	Example - Portico Modeling 3/5
Source cue	- Drawing a single column - Composition of 5 cylinder objects - Rotation into y-axis to match orientation - Positioning of the parts by translation along the y-axis - Incremental translation around the partial column height

Professor-style explanation	The slide 'Example - Portico Modeling 3/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Drawing a single column / - Composition of 5 cylinder objects / - Rotation into y-axis to match orientation / - Positioning of the parts by translation along the y-axis / - Incremental translation around the partial column height. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Example - Portico Modeling 3/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Drawing a single column / - Composition of 5 cylinder objects / - Rotation into y-axis to match orientation / - Positioning of the parts by translation along the y-axis / - Incremental translation around the partial column height. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Example - Portico Modeling 3/5' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Example - Portico Modeling 3/5' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 3/5' before rendering begins?
Page 50	Example - Portico Modeling 4/5
Source cue	- Drawing the group of columns / - Columns are evenly distributed on a circle with radius R / - Number N, radius R of the columns as well as floor radius R adjustable

Professor-style explanation	The slide 'Example - Portico Modeling 4/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Drawing the group of columns / - Columns are evenly distributed on a circle with radius R / - Number N, radius R of the columns as well as floor radius R adjustable. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Example - Portico Modeling 4/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Drawing the group of columns / - Columns are evenly distributed on a circle with radius R / - Number N, radius R of the columns as well as floor radius R adjustable. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Example - Portico Modeling 4/5' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Example - Portico Modeling 4/5' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 4/5' before rendering begins?
Page 51	Example - Portico Modeling 5/5
Source cue	- Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof

Professor-style explanation	The slide 'Example - Portico Modeling 5/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Example - Portico Modeling 5/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Example - Portico Modeling 5/5' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Example - Portico Modeling 5/5' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 5/5' before rendering begins?
Page 52	by applying affine transformations to their vertices
Source cue	- 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication

Professor-style explanation	The slide 'by applying affine transformations to their vertices' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about by applying affine transformations to their vertices. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'by applying affine transformations to their vertices' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'by applying affine transformations to their vertices' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'by applying affine transformations to their vertices'?
Page 53	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Literature and other sources used in this chapter' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Literature and other sources used in this chapter' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 54	(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)
Source cue	- Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer

Professor-style explanation	The slide '(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: - Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.
Technical commentary	This slide is about (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11). The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: - Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for '(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip '(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic '(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)' points to for deeper study?

03.1 Mathematical Foundations

Source location: Section 3.1

Commentary

The lecture uses vectors and matrices to express geometric operations. A vector can represent a point position, a direction, an offset, or an attribute depending on context. A matrix expresses a linear map or, with homogeneous coordinates, an affine transformation.

The central exam habit is to name the coordinate space. A point in model coordinates and a light direction in view coordinates cannot be mixed safely. Many visual bugs are coordinate-space bugs.

Mental model

A numeric vector is incomplete until you know what coordinate system it belongs to.

Check yourself

Why is a normal vector not transformed exactly like a point under all transformations?

03.2 Affine Transformations

Source location: Section 3.2

Commentary

Affine transformations include translation, rotation, scaling, shearing, and combinations of these. Homogeneous coordinates allow translation to be represented in matrix form together with other transformations.

Translation moves positions but not pure directions. Rotation changes orientation while preserving lengths if it is a proper rotation. Scaling can change lengths and can also affect normals. These distinctions matter later in lighting.

Mental model

Use 4D homogeneous coordinates so that a single matrix pipeline can move 3D points through the scene.

Check yourself

What does the homogeneous w component let a transformation matrix express?

03.3 Composition of Transformations

Source location: Section 3.3

Commentary

Composition means applying several transformations in sequence. The order matters because matrix multiplication is generally not commutative. Rotating an object around its own origin and then translating it is different from translating it and then rotating around the world origin.

When you read a formula such as $\text{projection} * \text{view} * \text{model} * \text{position}$, read it from right to left for the point: first model, then view, then projection. The resulting matrix chain is compact, but the conceptual steps remain separate.

Mental model

Matrix chains are compressed stories of movement through coordinate spaces.

Check yourself

Why can swapping model and view matrices destroy the intended camera/object relation?

03.4 Coordinate System Change

Source location: Section 3.4

Commentary

A transformation can be read actively as moving an object, or passively as changing the coordinate frame used to describe it. Both interpretations are valid, but mixing them carelessly causes sign and order errors.

The view matrix is a classic example: conceptually you position a camera, but the pipeline usually transforms the world by the inverse of the camera transform so that the camera becomes the origin of view space.

Mental model

Moving the camera one way is equivalent to moving the world the opposite way for rendering.

Check yourself

Why is the view transform often related to the inverse camera transform?

03.5 Transformations in OpenGL

Source location: Section 3.5

Commentary

Modern OpenGL does not automatically manage the old matrix stacks. You usually compute model, view, and projection matrices in application code and pass them to shaders as uniforms.

The vertex shader is where the final clip-space position is normally computed. That means a wrong matrix uniform, wrong multiplication order, or wrong convention can make geometry vanish even though buffers and draw calls are correct.

Mental model

The shader is the place where abstract transformation math becomes GPU execution.

Check yourself

Which matrix would you inspect first if an object follows the camera instead of staying in the world?

End-of-lecture summary

If you remember only one thing from Lecture 03, remember this: This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 04 - Geometric Projection

This lecture explains how a 3D view becomes a 2D image. Projection is where camera geometry, homogeneous coordinates, clipping planes, depth precision, and viewport mapping meet.

Source chunk	course_text_parts/03_lectures/04_geometric-projection.txt
Extracted slide pages	61
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.

Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Projecting 3D Models
Source cue	- To compute the 2D representation of a 3D model on the screen, / it needs to be projected, whereby it loses its z-coordinates / - Depending on the projection type, the object might get skewed / (perspective shortening) / - Projection can also be expressed as a matrix multiplication,
Professor-style explanation	The slide 'Projecting 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - To compute the 2D representation of a 3D model on the screen, / it needs to be projected, whereby it loses its z-coordinates / - Depending on the projection type, the object might get skewed / (perspective shortening) / - Projection can also be expressed as a matrix multiplication,. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Projecting 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - To compute the 2D representation of a 3D model on the screen, / it needs to be projected, whereby it loses its z-coordinates / - Depending on the projection type, the object might get skewed / (perspective shortening) / - Projection can also be expressed as a matrix multiplication,. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Projecting 3D Models' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Projecting 3D Models' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Projecting 3D Models' before rendering begins?
Page 3	4.1 Linear Perspective

Source cue	4.2 Planar Projections / 4.3 Camera Modeling / 4.4 Specifying Projections in OpenGL / 4.5 Perspective Projection Derivation / 4.6 Orthographic Projection Derivation
Professor-style explanation	The outline '4.1 Linear Perspective' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: 4.2 Planar Projections / 4.3 Camera Modeling / 4.4 Specifying Projections in OpenGL / 4.5 Perspective Projection Derivation / 4.6 Orthographic Projection Derivation. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about 4.1 Linear Perspective. The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: 4.2 Planar Projections / 4.3 Camera Modeling / 4.4 Specifying Projections in OpenGL / 4.5 Perspective Projection Derivation / 4.6 Orthographic Projection Derivation. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for '4.1 Linear Perspective' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize '4.1 Linear Perspective' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where '4.1 Linear Perspective' fits in the lecture order and what later section depends on it?
Page 4	4.1 Linear Perspective
Source cue	Discovery of perspective distortion in art

Professor-style explanation	The slide '4.1 Linear Perspective' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: Discovery of perspective distortion in art. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about 4.1 Linear Perspective. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: Discovery of perspective distortion in art. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for '4.1 Linear Perspective' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how '4.1 Linear Perspective' changes positions before rasterization?
Page 5	Linear Perspective
Source cue	- What is wrong in this image? / William Hogarth: / False Perspective, 1754 / Satire on False Perspective / William Hogarth (1753)

Professor-style explanation	The slide 'Linear Perspective' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - What is wrong in this image? / William Hogarth: / False Perspective, 1754 / Satire on False Perspective / William Hogarth (1753). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Linear Perspective. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - What is wrong in this image? / William Hogarth: / False Perspective, 1754 / Satire on False Perspective / William Hogarth (1753). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Linear Perspective' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Linear Perspective' changes positions before rasterization?
Page 6	Drawing by Projecting
Source cue	- Projection: method for imaging objects using parallel or central beams / - Perspective: / "The science which teaches how to represent tri-dimensional objects / on a bi-dimensional surface, so that the perspective image coincides with / the one which is given by direct vision."

Professor-style explanation	The slide 'Drawing by Projecting' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Projection: method for imaging objects using parallel or central beams / - Perspective: / "The science which teaches how to represent tri-dimensional objects / on a bi-dimensional surface, so that the perspective image coincides with / the one which is given by direct vision.". View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Drawing by Projecting. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Projection: method for imaging objects using parallel or central beams / - Perspective: / "The science which teaches how to represent tri-dimensional objects / on a bi-dimensional surface, so that the perspective image coincides with / the one which is given by direct vision.". View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Drawing by Projecting' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Drawing by Projecting' changes positions before rasterization?
Page 7	Projection in the Arts 1/5
Source cue	- Before the Renaissance, pictures often had a religious / and symbolic character / - Perspective was not understood: / accordingly, they seem 2-dimensional / Fresco from an Egyptian grave

Professor-style explanation	The slide 'Projection in the Arts 1/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Before the Renaissance, pictures often had a religious / and symbolic character / - Perspective was not understood: / accordingly, they seem 2-dimensional / Fresco from an Egyptian grave. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection in the Arts 1/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Before the Renaissance, pictures often had a religious / and symbolic character / - Perspective was not understood: / accordingly, they seem 2-dimensional / Fresco from an Egyptian grave. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection in the Arts 1/5' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection in the Arts 1/5' changes positions before rasterization?
Page 8	Projection in the Arts 2/5
Source cue	- Techniques to simulate space: / - Shaded round volumetric shapes / - Spatial depth through converging lines / Giotto di Bondone / (1266-1337, Florence)

Professor-style explanation	The slide 'Projection in the Arts 2/5' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Techniques to simulate space: / - Shaded round volumetric shapes / - Spatial depth through converging lines / Giotto di Bondone / (1266-1337, Florence). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Projection in the Arts 2/5. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Techniques to simulate space: / - Shaded round volumetric shapes / - Spatial depth through converging lines / Giotto di Bondone / (1266-1337, Florence). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Projection in the Arts 2/5' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Projection in the Arts 2/5' works per object, per image region, per ray, or per fragment?
Page 9	Projection in the Arts 3/5
Source cue	- Renaissance (1400-1700) / - New emphasis on the importance of personal views and interpretation of / the world, power of observation, especially of nature / (astronomy, anatomy, botany, ...) / - Universe as a clockwork:

Professor-style explanation	The slide 'Projection in the Arts 3/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Renaissance (1400-1700) / - New emphasis on the importance of personal views and interpretation of / the world, power of observation, especially of nature / (astronomy, anatomy, botany, ...) / - Universe as a clockwork. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection in the Arts 3/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Renaissance (1400-1700) / - New emphasis on the importance of personal views and interpretation of / the world, power of observation, especially of nature / (astronomy, anatomy, botany, ...) / - Universe as a clockwork. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection in the Arts 3/5' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection in the Arts 3/5' changes positions before rasterization?
Page 10	Projection in the Arts 4/5
Source cue	- Examples from the Renaissance

Professor-style explanation	The slide 'Projection in the Arts 4/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Examples from the Renaissance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection in the Arts 4/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Examples from the Renaissance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection in the Arts 4/5' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection in the Arts 4/5' changes positions before rasterization?
Page 11	Projection in the Arts 4/5
Source cue	- Examples from the Renaissance / The Last Supper / Leonardo da Vinci (1498)

Professor-style explanation	The slide 'Projection in the Arts 4/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Examples from the Renaissance / The Last Supper / Leonardo da Vinci (1498). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection in the Arts 4/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Examples from the Renaissance / The Last Supper / Leonardo da Vinci (1498). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection in the Arts 4/5' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection in the Arts 4/5' changes positions before rasterization?
Page 12	Projection in the Arts 5/5
Source cue	- "The Invention of the Painting" (1830) by Karl Friedrich Schinkel (1781-1841): / - An image based on a legend by Pliny the Elder / - Tracing the shadow contours on a drawing surface

Professor-style explanation	The slide 'Projection in the Arts 5/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - "The Invention of the Painting" (1830) by Karl Friedrich Schinkel (1781-1841): / - An image based on a legend by Pliny the Elder / - Tracing the shadow contours on a drawing surface. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection in the Arts 5/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - "The Invention of the Painting" (1830) by Karl Friedrich Schinkel (1781-1841): / - An image based on a legend by Pliny the Elder / - Tracing the shadow contours on a drawing surface. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection in the Arts 5/5' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection in the Arts 5/5' changes positions before rasterization?
Page 13	Projection in the Arts 6/5
Source cue	- M. C. Escher (1898 - 1972)

Professor-style explanation	The slide 'Projection in the Arts 6/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - M. C. Escher (1898 - 1972). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection in the Arts 6/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - M. C. Escher (1898 - 1972). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection in the Arts 6/5' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection in the Arts 6/5' changes positions before rasterization?
Page 14	Projection Technics 1/3
Source cue	- Filippo Brunelleschi (1377-1446): / Systematic method for obtaining / a perspective projection

Professor-style explanation	The slide 'Projection Technics 1/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Filippo Brunelleschi (1377-1446): / Systematic method for obtaining / a perspective projection. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection Technics 1/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Filippo Brunelleschi (1377-1446): / Systematic method for obtaining / a perspective projection. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection Technics 1/3' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection Technics 1/3' changes positions before rasterization?
Page 15	Projection Technics 1/3
Source cue	The Delivery of the Keys / Perugino (1482)

Professor-style explanation	The slide 'Projection Technics 1/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: The Delivery of the Keys / Perugino (1482). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection Technics 1/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: The Delivery of the Keys / Perugino (1482). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection Technics 1/3' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection Technics 1/3' changes positions before rasterization?
Page 16	Projection Technics 2/3
Source cue	- Formal description of the perspective projection / - Leone Batista Alberti (1404-1472): / First essay on the subject of "La Pittura" (1435) / - "A painting [the projection plane] is the intersection of a visual pyramid [view / frustum] at a given distance, with a fixed center [center of projection] and a

Professor-style explanation	The slide 'Projection Technics 2/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Formal description of the perspective projection / - Leone Batista Alberti (1404-1472): / First essay on the subject of "La Pittura" (1435) / - "A painting [the projection plane] is the intersection of a visual pyramid [view / frustum] at a given distance, with a fixed center [center of projection] and a. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection Technics 2/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Formal description of the perspective projection / - Leone Batista Alberti (1404-1472): / First essay on the subject of "La Pittura" (1435) / - "A painting [the projection plane] is the intersection of a visual pyramid [view / frustum] at a given distance, with a fixed center [center of projection] and a. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection Technics 2/3' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection Technics 2/3' changes positions before rasterization?
Page 17	Projection Technics 3/3
Source cue	- Albrecht Dürer (1471-1528): / Geometric and mechanical / description of the concept / of similar triangles / - Construction of several

Professor-style explanation	The slide 'Projection Technics 3/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Albrecht Dürer (1471-1528): / Geometric and mechanical / description of the concept / of similar triangles / - Construction of several. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Projection Technics 3/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Albrecht Dürer (1471-1528): / Geometric and mechanical / description of the concept / of similar triangles / - Construction of several. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Projection Technics 3/3' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Projection Technics 3/3'?
Page 18	Projection Technics 3/3
Source cue	- Camera Obscura

Professor-style explanation	The slide 'Projection Technics 3/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Camera Obscura. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection Technics 3/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Camera Obscura. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection Technics 3/3' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection Technics 3/3' changes positions before rasterization?
Page 19	4.2 Planar Projections
Source cue	Perspective and parallel projections

Professor-style explanation	The slide '4.2 Planar Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: Perspective and parallel projections. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about 4.2 Planar Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: Perspective and parallel projections. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for '4.2 Planar Projections' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how '4.2 Planar Projections' changes positions before rasterization?
Page 20	Geometric Projections
Source cue	- Method of mapping objects onto a screen using projectors / - Planar geometric projections / - Projection beams are straight lines / - Projection surface is a plane / (image plane, picture plane,

Professor-style explanation	The slide 'Geometric Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Method of mapping objects onto a screen using projectors - Planar geometric projections - Projection beams are straight lines - Projection surface is a plane (image plane, picture plane, . View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Geometric Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Method of mapping objects onto a screen using projectors - Planar geometric projections - Projection beams are straight lines - Projection surface is a plane (image plane, picture plane, . View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Geometric Projections' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Geometric Projections' changes positions before rasterization?
Page 21	Projection Types 1/2
Source cue	- Perspective projection (= central projection) - Established by projection center / (COP, center of projection) - Perspective reduction: The size of the projection / of an object decreases with increasing distance

Professor-style explanation	The slide 'Projection Types 1/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Perspective projection (= central projection) / - Established by projection center / (COP, center of projection) / - Perspective reduction: The size of the projection / of an object decreases with increasing distance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection Types 1/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Perspective projection (= central projection) / - Established by projection center / (COP, center of projection) / - Perspective reduction: The size of the projection / of an object decreases with increasing distance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection Types 1/2' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection Types 1/2' changes positions before rasterization?
Page 22	Projection Types 2/2
Source cue	Planar geometric Projections / Parallel Perspective / Orthographic Oblique 1-Point / 2-Point / Top Axonometric

Professor-style explanation	The slide 'Projection Types 2/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: Planar geometric Projections / Parallel Perspective / Orthographic Oblique 1-Point / 2-Point / Top Axonometric. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection Types 2/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: Planar geometric Projections / Parallel Perspective / Orthographic Oblique 1-Point / 2-Point / Top Axonometric. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection Types 2/2' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection Types 2/2' changes positions before rasterization?
Page 23	Orthographic Projections
Source cue	- Orthographic projections often used for / construction drawings of machines and parts / - Main views: Front view, top view and side view / - Advantages / - Accurate length and angle measurements possible

Professor-style explanation	The slide 'Orthographic Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Orthographic projections often used for / construction drawings of machines and parts / - Main views: Front view, top view and side view / - Advantages / - Accurate length and angle measurements possible. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Orthographic Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Orthographic projections often used for / construction drawings of machines and parts / - Main views: Front view, top view and side view / - Advantages / - Accurate length and angle measurements possible. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Orthographic Projections' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Orthographic Projections' changes positions before rasterization?
Page 24	Axonometric Projections
Source cue	- Projection beams perpendicular to the projection plane / Isometric / - Projection plane not perpendicular to a coordinate axis / - Visible effects / - Several perpendicular surfaces visible

Professor-style explanation	The slide 'Axonometric Projections' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Projection beams perpendicular to the projection plane / Isometric / - Projection plane not perpendicular to a coordinate axis / - Visible effects / - Several perpendicular surfaces visible. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Axonometric Projections. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Projection beams perpendicular to the projection plane / Isometric / - Projection plane not perpendicular to a coordinate axis / - Visible effects / - Several perpendicular surfaces visible. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Axonometric Projections' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Axonometric Projections' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Axonometric Projections'?
Page 25	Isometric Projections
Source cue	- Applications in illustrations, furniture design, ... / - Advantages / - Communicate spatial impression in a single view / - Same length ratios along the main axes / - Disadvantages

Professor-style explanation	The slide 'Isometric Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Applications in illustrations, furniture design, ... / - Advantages / - Communicate spatial impression in a single view / - Same length ratios along the main axes / - Disadvantages. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Isometric Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Applications in illustrations, furniture design, ... / - Advantages / - Communicate spatial impression in a single view / - Same length ratios along the main axes / - Disadvantages. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Isometric Projections' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Isometric Projections' changes positions before rasterization?
Page 26	Oblique Projection 1/2
Source cue	- Projection rays not perpendicular to the projection plane / - Common: $\alpha = 30^\circ$ / - Cavalier projection $\alpha = 45^\circ$, i.e., $\tan(\alpha) = 1$ / a' projectors

Professor-style explanation	The slide 'Oblique Projection 1/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Projection rays not perpendicular to the projection plane / - Common: $\alpha = 30^\circ$ / - Cavalier projection $\alpha = 45^\circ$, i.e., $\tan(\alpha) = 1$ / a' projectors. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Oblique Projection 1/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Projection rays not perpendicular to the projection plane / - Common: $\alpha = 30^\circ$ / - Cavalier projection $\alpha = 45^\circ$, i.e., $\tan(\alpha) = 1$ / a' projectors. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Oblique Projection 1/2' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Oblique Projection 1/2' works per object, per image region, per ray, or per fragment?
Page 27	Oblique Projection 2/2
Source cue	- Advantages / - Can accurately represent the shape of a boundary surface / of an object for accurate measurements / - Missing foreshortening simplifies comparison of sizes / - Gives a good impression of the 3D appearance

Professor-style explanation	The slide 'Oblique Projection 2/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Advantages / - Can accurately represent the shape of a boundary surface / of an object for accurate measurements / - Missing foreshortening simplifies comparison of sizes / - Gives a good impression of the 3D appearance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Oblique Projection 2/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Advantages / - Can accurately represent the shape of a boundary surface / of an object for accurate measurements / - Missing foreshortening simplifies comparison of sizes / - Gives a good impression of the 3D appearance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Oblique Projection 2/2' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Oblique Projection 2/2' changes positions before rasterization?
Page 28	Properties of Parallel Projections
Source cue	- Assumption: Object surface of greatest interest lies parallel to a main plane / - Direction of projection: DOP (direction of projection) / - Normal of the projection plane: VPN (view plane normal) / - Top view (orthographic) / - VPN $\parallel$ Main axis, DOP $\parallel$ VPN

Professor-style explanation	The slide 'Properties of Parallel Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Assumption: Object surface of greatest interest lies parallel to a main plane - Direction of projection: DOP (direction of projection) - Normal of the projection plane: VPN (view plane normal) - Top view (orthographic) - $\text{VPN} \parallel \text{Main axis}$, $\text{DOP} \parallel \text{VPN}$ View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Properties of Parallel Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Assumption: Object surface of greatest interest lies parallel to a main plane - Direction of projection: DOP (direction of projection) - Normal of the projection plane: VPN (view plane normal) - Top view (orthographic) - $\text{VPN} \parallel \text{Main axis}$, $\text{DOP} \parallel \text{VPN}$ View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Properties of Parallel Projections' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Properties of Parallel Projections' changes positions before rasterization?
Page 29	Perspective Projections 1/3
Source cue	- Properties - Communicate realistic views of three-dimensional objects / and give a spatial impression - Distort objects in perspective view - Scaling of objects is lost

Professor-style explanation	The slide 'Perspective Projections 1/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Properties / - Communicate realistic views of three-dimensional objects / and give a spatial impression / - Distort objects in perspective view / - Scaling of objects is lost. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Perspective Projections 1/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Properties / - Communicate realistic views of three-dimensional objects / and give a spatial impression / - Distort objects in perspective view / - Scaling of objects is lost. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Perspective Projections 1/3' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Perspective Projections 1/3' changes positions before rasterization?
Page 30	Perspective Projections 2/3
Source cue	- Vanishing points / - For rectangular objects (e.g., axis parallel cubes) whose surface normals / are parallel to the x-, y-, or z-axis, the number of vanishing points is equal / to the number of coordinate axes intersected by the projection plane / 1-Point Perspective 2-Point Perspective 3-Point Perspective

Professor-style explanation	The slide 'Perspective Projections 2/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Vanishing points / - For rectangular objects (e.g., axis parallel cubes) whose surface normals / are parallel to the x-, y-, or z-axis, the number of vanishing points is equal / to the number of coordinate axes intersected by the projection plane / 1-Point Perspective 2-Point Perspective 3-Point Perspective. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Perspective Projections 2/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Vanishing points / - For rectangular objects (e.g., axis parallel cubes) whose surface normals / are parallel to the x-, y-, or z-axis, the number of vanishing points is equal / to the number of coordinate axes intersected by the projection plane / 1-Point Perspective 2-Point Perspective 3-Point Perspective. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Perspective Projections 2/3' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Perspective Projections 2/3' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Perspective Projections 2/3'?
Page 31	Perspective Projections 3/3
Source cue	- What happens if the cube is rotated so that its surface normals are not / parallel to the coordinate axes? / - Although the projection plane intersects only one coordinate axis, in this case / you get 3 vanishing points

Professor-style explanation	The slide 'Perspective Projections 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - What happens if the cube is rotated so that its surface normals are not / parallel to the coordinate axes? / - Although the projection plane intersects only one coordinate axis, in this case / you get 3 vanishing points. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Perspective Projections 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - What happens if the cube is rotated so that its surface normals are not / parallel to the coordinate axes? / - Although the projection plane intersects only one coordinate axis, in this case / you get 3 vanishing points. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Perspective Projections 3/3' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Perspective Projections 3/3' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Perspective Projections 3/3'?
Page 32	4.3 Camera Modeling
Source cue	Orientation of the camera and definition of the view volume

Professor-style explanation	The slide '4.3 Camera Modeling' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: Orientation of the camera and definition of the view volume. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about 4.3 Camera Modeling. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: Orientation of the camera and definition of the view volume. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for '4.3 Camera Modeling' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe '4.3 Camera Modeling' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by '4.3 Camera Modeling' before rendering begins?
Page 33	Camera Modeling
Source cue	- Application of planar geometric projections, represented by 4×4 matrices, / to project a 3D scene onto a projection screen, i.e., the canvas / - Basic camera parameters / - Camera position / - Camera orientation

Professor-style explanation	The slide 'Camera Modeling' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Application of planar geometric projections, represented by 4×4 matrices, / to project a 3D scene onto a projection screen, i.e., the canvas - Basic camera parameters - Camera position - Camera orientation. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Camera Modeling. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Application of planar geometric projections, represented by 4×4 matrices, / to project a 3D scene onto a projection screen, i.e., the canvas - Basic camera parameters - Camera position - Camera orientation. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Camera Modeling' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Camera Modeling' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Camera Modeling' before rendering begins?
Page 34	View Volume
Source cue	- Objects that are within the view volume are visible for the camera - Conical viewing volume - Clipping the scene objects against a conical viewing volume is expensive - (solving quadratic equations) - Frustum - rectangular viewing volume

Professor-style explanation	The slide 'View Volume' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Objects that are within the view volume are visible for the camera - Conical viewing volume - Clipping the scene objects against a conical viewing volume is expensive (solving quadratic equations) - Frustum - rectangular viewing volume A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about View Volume. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Objects that are within the view volume are visible for the camera - Conical viewing volume - Clipping the scene objects against a conical viewing volume is expensive (solving quadratic equations) - Frustum - rectangular viewing volume A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'View Volume' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'View Volume'?
Page 35	Projection Specification
Source cue	- Specified by Height Angle, Aspect Ratio, / Near Plane Distance and Far Plane Distance - Examples of Aspect Ratio - NTSC 4 : 3, HDTV 16 : 9 - Cinema 2.35 : 1, 2.39 : 1, 2.40 : 1, or 2.55 : 1

Professor-style explanation	The outline 'Projection Specification' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: - Specified by Height Angle, Aspect Ratio, / Near Plane Distance and Far Plane Distance / - Examples of Aspect Ratio / - NTSC 4 : 3, HDTV 16 : 9 / - Cinema 2.35 : 1, 2.39 : 1, 2.40 : 1, or 2.55 : 1. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about Projection Specification. The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: - Specified by Height Angle, Aspect Ratio, / Near Plane Distance and Far Plane Distance / - Examples of Aspect Ratio / - NTSC 4 : 3, HDTV 16 : 9 / - Cinema 2.35 : 1, 2.39 : 1, 2.40 : 1, or 2.55 : 1. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for 'Projection Specification' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize 'Projection Specification' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where 'Projection Specification' fits in the lecture order and what later section depends on it?
Page 36	View Angle
Source cue	- Parallel projection: no viewing angle as rays are parallel / - Perspective projection: / viewing angle determines perspective distortion in the image / - Specification / - Through the viewing angle in height (field of view in y, fovy)

Professor-style explanation	The slide 'View Angle' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Parallel projection: no viewing angle as rays are parallel / - Perspective projection: / viewing angle determines perspective distortion in the image / - Specification / - Through the viewing angle in height (field of view in y, fovy). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about View Angle. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Parallel projection: no viewing angle as rays are parallel / - Perspective projection: / viewing angle determines perspective distortion in the image / - Specification / - Through the viewing angle in height (field of view in y, fovy). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'View Angle' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'View Angle' works per object, per image region, per ray, or per fragment?
Page 37	Clipping Planes
Source cue	- Position of the planes is determined by the distance along the look vector: / near distance and far distance / - Near plane hides objects that are behind or too close to the camera / - Far plane hides objects that are too far away (and may be hard to spot) / $\Rightarrow$ Reduction of rendering overhead

Professor-style explanation	The slide 'Clipping Planes' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Position of the planes is determined by the distance along the look vector: / near distance and far distance / - Near plane hides objects that are behind or too close to the camera / - Far plane hides objects that are too far away (and may be hard to spot) / $\Rightarrow$ Reduction of rendering overhead. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Clipping Planes. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Position of the planes is determined by the distance along the look vector: / near distance and far distance / - Near plane hides objects that are behind or too close to the camera / - Far plane hides objects that are too far away (and may be hard to spot) / $\Rightarrow$ Reduction of rendering overhead. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Clipping Planes' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Clipping Planes'?
Page 38	4.4 Specifying Projections in OpenGL
Source cue	Functions to describe the view volume

Professor-style explanation	The slide '4.4 Specifying Projections in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Functions to describe the view volume. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about 4.4 Specifying Projections in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Functions to describe the view volume. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for '4.4 Specifying Projections in OpenGL' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '4.4 Specifying Projections in OpenGL'?
Page 39	Projection Matrix Generation 1/3
Source cue	- Convenient generation of perspective projection matrix with GLM / - glm::perspective(fovy, aspect, near, far) / h -n -f / aspect ■ / fovy fovy

Professor-style explanation	The slide 'Projection Matrix Generation 1/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Convenient generation of perspective projection matrix with GLM / - glm::perspective(fovy, aspect, near, far) / h -n -f / aspect ■ / fovy fovy. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Projection Matrix Generation 1/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Convenient generation of perspective projection matrix with GLM / - glm::perspective(fovy, aspect, near, far) / h -n -f / aspect ■ / fovy fovy. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Projection Matrix Generation 1/3' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Projection Matrix Generation 1/3' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Projection Matrix Generation 1/3'?
Page 40	Projection Matrix Generation 2/3
Source cue	- Alternative generation of perspective projection matrix with GLM / - glm::frustum(left, right, bottom, top, near, far) / (r, t, -n) / z = -f (far plane) / (l, b, -n)

Professor-style explanation	The slide 'Projection Matrix Generation 2/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Alternative generation of perspective projection matrix with GLM / - glm::frustum(left, right, bottom, top, near, far) / (r, t, -n) / z = -f (far plane) / (l, b, -n). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Projection Matrix Generation 2/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Alternative generation of perspective projection matrix with GLM / - glm::frustum(left, right, bottom, top, near, far) / (r, t, -n) / z = -f (far plane) / (l, b, -n). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Projection Matrix Generation 2/3' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Projection Matrix Generation 2/3' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Projection Matrix Generation 2/3'?
Page 41	Projection Matrix Generation 3/3
Source cue	- Implementation of glm::perspective / detail::tmat4x4<T> perspective(T const & fovy, T const & aspect, / T const & near, T const & far) { / T const & l, r, b, t; / t = n * tan(fovy * M_PI / 360.0);

Professor-style explanation	The slide 'Projection Matrix Generation 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Implementation of <code>glm::perspective / detail::tmat4x4<T> perspective(T const & fovy, T const & aspect, / T const & near, T const & far) { / T const & l, r, b, t; / t = n * tan(fovy * M_PI / 360.0). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.</code>
Technical commentary	This slide is about Projection Matrix Generation 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Implementation of <code>glm::perspective / detail::tmat4x4<T> perspective(T const & fovy, T const & aspect, / T const & near, T const & far) { / T const & l, r, b, t; / t = n * tan(fovy * M_PI / 360.0). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.</code>
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Projection Matrix Generation 3/3' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Projection Matrix Generation 3/3' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Projection Matrix Generation 3/3'?
Page 42	Orthographic Matrix Specification
Source cue	- Convenient generation of parallel projection matrix with GLM / - <code>glm::ortho(left, right, bottom, top, near, far) / (r, t, -f) / z = -f (far plane) / z = -n (near plane)</code>

Professor-style explanation	The slide 'Orthographic Matrix Specification' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Convenient generation of parallel projection matrix with GLM / - glm::ortho(left, right, bottom, top, near, far) / (r, t, -f) / z = -f (far plane) / z = -n (near plane). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Orthographic Matrix Specification. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Convenient generation of parallel projection matrix with GLM / - glm::ortho(left, right, bottom, top, near, far) / (r, t, -f) / z = -f (far plane) / z = -n (near plane). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Orthographic Matrix Specification' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Orthographic Matrix Specification' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Orthographic Matrix Specification'?
Page 43	4.5 Orthographic Projection Derivation
Source cue	Step-wise transformation of a cuboid into a cube

Professor-style explanation	The slide '4.5 Orthographic Projection Derivation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Step-wise transformation of a cuboid into a cube. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about 4.5 Orthographic Projection Derivation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Step-wise transformation of a cuboid into a cube. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for '4.5 Orthographic Projection Derivation' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in '4.5 Orthographic Projection Derivation' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after '4.5 Orthographic Projection Derivation'?
Page 44	Canonical View Volume
Source cue	- Canonical view volume - simplest possible view volume / - Cube centered at the origin / - Cube faces $x = \pm 1$, $y = \pm 1$, $z = \pm 1$ / - Defined in a left-handed coordinate system (LHS) / (1, 1, 1)

Professor-style explanation	The slide 'Canonical View Volume' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Canonical view volume - simplest possible view volume / - Cube centered at the origin / - Cube faces $x = \pm 1$, $y = \pm 1$, $z = \pm 1$ / - Defined in a left-handed coordinate system (LHS) / (1, 1, 1). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Canonical View Volume. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Canonical view volume - simplest possible view volume / - Cube centered at the origin / - Cube faces $x = \pm 1$, $y = \pm 1$, $z = \pm 1$ / - Defined in a left-handed coordinate system (LHS) / (1, 1, 1). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Canonical View Volume' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Canonical View Volume' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Canonical View Volume'?
Page 45	Orthographic Projection 1/4
Source cue	- Transform right-handed view volume specified as $[l, r] \times [b, t] \times [-n, -f]$ / into left-handed canonical view volume - 1, 1 $\times$ - 1, 1 $\times$ - 1, 1 / and then create the projected image by neglecting z-values / (r, t, -f) / (1, 1, 1)

Professor-style explanation	The slide 'Orthographic Projection 1/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Transform right-handed view volume specified as $[l, r] \times [b, t] \times [-n, -f]$ / into left-handed canonical view volume $-1, 1 \times -1, 1 \times -1, 1$ / and then create the projected image by neglecting z-values / $(r, t, -f) / (1, 1, 1)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Orthographic Projection 1/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Transform right-handed view volume specified as $[l, r] \times [b, t] \times [-n, -f]$ / into left-handed canonical view volume $-1, 1 \times -1, 1 \times -1, 1$ / and then create the projected image by neglecting z-values / $(r, t, -f) / (1, 1, 1)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Orthographic Projection 1/4' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Orthographic Projection 1/4' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Orthographic Projection 1/4'?
Page 46	Orthographic Projection 2/4
Source cue	- Let $p(x, y, z)$ be a model point and $q(x', y', z')$ its transformed point / - The desired pixel (x, y) results from orthographic projection of q / p / onto the plane $z = -1$: $x = x'$ and $y = y' / p$

Professor-style explanation	The slide 'Orthographic Projection 2/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Let $p(x, y, z)$ be a model point and $q(x', y', z')$ its transformed point / - The desired pixel (x, y) results from orthographic projection of q / p onto the plane $z = -1$: $x = x'$ and $y = y' / p_p$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Orthographic Projection 2/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Let $p(x, y, z)$ be a model point and $q(x', y', z')$ its transformed point / - The desired pixel (x, y) results from orthographic projection of q / p onto the plane $z = -1$: $x = x'$ and $y = y' / p_p$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Orthographic Projection 2/4' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Orthographic Projection 2/4' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Orthographic Projection 2/4'?
Page 47	Orthographic Projection 3/4
Source cue	- Step 2, scale cuboid from step 1 into cube / - Original cuboid bounds are $/ r - l r - l t - b t - b f - n f - n / - , x - , x - , / 2 2 2 2 2 2$

Professor-style explanation	The slide 'Orthographic Projection 3/4' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Step 2, scale cuboid from step 1 into cube / - Original cuboid bounds are $/r - l, r - l, t - b, t - b, f - n, f - n / - , x - , x - , / 2 \ 2 \ 2 \ 2 \ 2 \ 2$. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Orthographic Projection 3/4. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Step 2, scale cuboid from step 1 into cube / - Original cuboid bounds are $/r - l, r - l, t - b, t - b, f - n, f - n / - , x - , x - , / 2 \ 2 \ 2 \ 2 \ 2 \ 2$. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Orthographic Projection 3/4' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Orthographic Projection 3/4' changes positions before rasterization?
Page 48	Orthographic Projection 4/4
Source cue	- Multiplication of all orthographic projection matrices results in $/ 2 \ r + l / 0 \ 0 - / r - l, r - l / 2 \ t + b$

Professor-style explanation	The slide 'Orthographic Projection 4/4' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Multiplication of all orthographic projection matrices results in $\begin{bmatrix} 2r + l & 0 & 0 & -r - l \\ 0 & 2r + l & 0 & -r - l \\ 0 & 0 & 2t & -t \\ 0 & 0 & 0 & 1 \end{bmatrix}$. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Orthographic Projection 4/4. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Multiplication of all orthographic projection matrices results in $\begin{bmatrix} 2r + l & 0 & 0 & -r - l \\ 0 & 2r + l & 0 & -r - l \\ 0 & 0 & 2t & -t \\ 0 & 0 & 0 & 1 \end{bmatrix}$. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Orthographic Projection 4/4' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Orthographic Projection 4/4' changes positions before rasterization?
Page 49	4.6 Perspective Projection Derivation
Source cue	Step-wise transformation of a frustum into a cube

Professor-style explanation	The slide '4.6 Perspective Projection Derivation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Step-wise transformation of a frustum into a cube. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about 4.6 Perspective Projection Derivation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Step-wise transformation of a frustum into a cube. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for '4.6 Perspective Projection Derivation' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in '4.6 Perspective Projection Derivation' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after '4.6 Perspective Projection Derivation'?
Page 50	Perspective Projection
Source cue	- Transform view volume created by <code>glm::frustum</code> or <code>glm::perspective</code> into / canonical view volume / and then create the projected image by neglecting z-values / (1, 1, 1) / (-1,-1,-1)

Professor-style explanation	The slide 'Perspective Projection' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Transform view volume created by <code>glm::frustum</code> or <code>glm::perspective</code> into / canonical view volume / and then create the projected image by neglecting z-values / (1, 1, 1) / (-1,-1,-1). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Perspective Projection. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Transform view volume created by <code>glm::frustum</code> or <code>glm::perspective</code> into / canonical view volume / and then create the projected image by neglecting z-values / (1, 1, 1) / (-1,-1,-1). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Perspective Projection' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Perspective Projection' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Perspective Projection'?
Page 51	Transformation of z-Values 1/2
Source cue	- After the perspective projection, z-values shall be mapped as follows / - All z-values lying on the near plane shall have a value of -1 / - All z-values lying on the far plane shall have a value of 1 / - The following matrix P allows modification of the depth values / and assures the flip (left-handed to right-handed conversion)

Professor-style explanation	The slide 'Transformation of z-Values 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - After the perspective projection, z-values shall be mapped as follows / - All z-values lying on the near plane shall have a value of -1 / - All z-values lying on the far plane shall have a value of 1 / - The following matrix P allows modification of the depth values / and assures the flip (left-handed to right-handed conversion). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Transformation of z-Values 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - After the perspective projection, z-values shall be mapped as follows / - All z-values lying on the near plane shall have a value of -1 / - All z-values lying on the far plane shall have a value of 1 / - The following matrix P allows modification of the depth values / and assures the flip (left-handed to right-handed conversion). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Transformation of z-Values 1/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Transformation of z-Values 1/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Transformation of z-Values 1/2'?
Page 52	Transformation of z-Values 2/2
Source cue	- Choosing values for a and b in P / - a and b must fulfill the following / -$a + b / P \cdot 0 \ 0 \ -n \ 1 \ T = 0 \ 0 \ -a + b \ n \ T \ 0 \ 0 = 0 \ 0 \ -1 / \div w \ n$

Professor-style explanation	The slide 'Transformation of z-Values 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Choosing values for a and b in $P = a + b \cdot n$ / - a and b must fulfill the following $\begin{bmatrix} -a & b & 0 & 0 \\ 0 & 0 & -n & 1 \end{bmatrix} T = \begin{bmatrix} 0 & 0 & -a & b \\ 0 & 0 & 0 & -1 \end{bmatrix} / \div w \cdot n$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Transformation of z-Values 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Choosing values for a and b in $P = a + b \cdot n$ / - a and b must fulfill the following $\begin{bmatrix} -a & b & 0 & 0 \\ 0 & 0 & -n & 1 \end{bmatrix} T = \begin{bmatrix} 0 & 0 & -a & b \\ 0 & 0 & 0 & -1 \end{bmatrix} / \div w \cdot n$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Transformation of z-Values 2/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Transformation of z-Values 2/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Transformation of z-Values 2/2'?
Page 53	Transformation of x- and y-Values 1/2
Source cue	- Derivation so far leaves us with the following matrix / which only changes z-coordinates $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & f+n & 2nf \end{bmatrix}$ / - $P = f+n \cdot 2nf$

Professor-style explanation	The slide 'Transformation of x- and y-Values 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Derivation so far leaves us with the following matrix / which only changes z-coordinates / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$ / - $P = f+n \cdot 2nf$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Transformation of x- and y-Values 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Derivation so far leaves us with the following matrix / which only changes z-coordinates / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$ / - $P = f+n \cdot 2nf$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Transformation of x- and y-Values 1/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Transformation of x- and y-Values 1/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Transformation of x- and y-Values 1/2'?
Page 54	Transformation of x- and y-Values 2/2
Source cue	- To transform p to the coordinates (1,0,-1), the following equation must hold / $\alpha \tan / \alpha n \cdot \tan / n \cdot \tan 2 / 2 2 1$

Professor-style explanation	The slide 'Transformation of x- and y-Values 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - To transform p to the coordinates (1,0,-1), the following equation must hold $\frac{1}{n} \cdot \tan \frac{\alpha}{2} = \frac{p_x}{n} \cdot \tan \frac{\alpha}{2} + \frac{p_y}{n} \cdot \tan \frac{\alpha}{2} + 1$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Transformation of x- and y-Values 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - To transform p to the coordinates (1,0,-1), the following equation must hold $\frac{1}{n} \cdot \tan \frac{\alpha}{2} = \frac{p_x}{n} \cdot \tan \frac{\alpha}{2} + \frac{p_y}{n} \cdot \tan \frac{\alpha}{2} + 1$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Transformation of x- and y-Values 2/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Transformation of x- and y-Values 2/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Transformation of x- and y-Values 2/2'?
Page 55	4.7 Viewport Transformation
Source cue	Mapping the normalized device coordinates to the screen

Professor-style explanation	The slide '4.7 Viewport Transformation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Mapping the normalized device coordinates to the screen. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about 4.7 Viewport Transformation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Mapping the normalized device coordinates to the screen. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for '4.7 Viewport Transformation' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in '4.7 Viewport Transformation' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after '4.7 Viewport Transformation'?
Page 56	Screen Coordinate Mapping
Source cue	- Mapping of the viewing plane on a drawing surface / - Define the pixel rectangle to be mapped to / - Initially, this rectangle is the size of the entire canvas / - glViewport(GLint x, GLint y, GLsizei width, GLsizei height) / - When window size changes, adjust the aspect ratio to avoid distortion

Professor-style explanation	The slide 'Screen Coordinate Mapping' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Mapping of the viewing plane on a drawing surface - Define the pixel rectangle to be mapped to - Initially, this rectangle is the size of the entire canvas - <code>glViewport(GLint x, GLint y, GLsizei width, GLsizei height)</code> - When window size changes, adjust the aspect ratio to avoid distortion. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Screen Coordinate Mapping. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Mapping of the viewing plane on a drawing surface - Define the pixel rectangle to be mapped to - Initially, this rectangle is the size of the entire canvas - <code>glViewport(GLint x, GLint y, GLsizei width, GLsizei height)</code> - When window size changes, adjust the aspect ratio to avoid distortion. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Screen Coordinate Mapping' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Screen Coordinate Mapping' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Screen Coordinate Mapping'?
Page 57	Viewport Transformation 1/2
Source cue	- Specification of the viewport parameters by <code>glm::perspective</code> / <code>Projection</code> / <code>glm::frustum</code> / transformation - <code>glViewport(x, y, w, h)</code> <code>glm::ortho</code>

Professor-style explanation	The slide 'Viewport Transformation 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Specification of the viewport parameters by glm::perspective / Projection / glm::frustum / transformation / - glViewport(x, y, w, h) glm::ortho. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Viewport Transformation 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Specification of the viewport parameters by glm::perspective / Projection / glm::frustum / transformation / - glViewport(x, y, w, h) glm::ortho. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Viewport Transformation 1/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Viewport Transformation 1/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Viewport Transformation 1/2'?
Page 58	Viewport Transformation 2/2
Source cue	- Viewport transformation is a 2D transformation defined as $p = M \cdot p$ with $M = \begin{bmatrix} T_x & T_y & 0 & 0 \\ 0 & 0 & T_x & T_y \\ S_x & S_y & S_x & S_y \\ 0 & 0 & 0 & 0 \end{bmatrix}$ / - $M = \begin{bmatrix} T_x & T_y & 0 & 0 \\ 0 & 0 & T_x & T_y \\ S_x & S_y & S_x & S_y \\ 0 & 0 & 0 & 0 \end{bmatrix}$ / - $M = \begin{bmatrix} T_x & T_y & 0 & 0 \\ 0 & 0 & T_x & T_y \\ S_x & S_y & S_x & S_y \\ 0 & 0 & 0 & 0 \end{bmatrix}$

Professor-style explanation	The slide 'Viewport Transformation 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Viewport transformation is a 2D transformation defined as $p = M \cdot p$ with $/s \quad s \quad N \quad \Delta x \quad \Delta y / - M = T \quad x \quad y \cdot S \quad S \quad S \cdot T \quad -x \quad -y / s \quad S_{min} \quad S_{min} \quad N_{min} \quad N_{min}$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Viewport Transformation 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Viewport transformation is a 2D transformation defined as $p = M \cdot p$ with $/s \quad s \quad N \quad \Delta x \quad \Delta y / - M = T \quad x \quad y \cdot S \quad S \quad S \cdot T \quad -x \quad -y / s \quad S_{min} \quad S_{min} \quad N_{min} \quad N_{min}$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Viewport Transformation 2/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Viewport Transformation 2/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Viewport Transformation 2/2'?
Page 59	volume, we can differentiate different projection types
Source cue	- Projection has its origins in the arts / - Based on the deformation of the initial view volume to the canonical view / volume, we can differentiate different projection types / - In OpenGL two main projection types exist / - Perspective projection results in perspective shortening

Professor-style explanation	The slide 'volume, we can differentiate different projection types' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Projection has its origins in the arts / - Based on the deformation of the initial view volume to the canonical view / volume, we can differentiate different projection types / - In OpenGL two main projection types exist / - Perspective projection results in perspective shortening. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about volume, we can differentiate different projection types. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Projection has its origins in the arts / - Based on the deformation of the initial view volume to the canonical view / volume, we can differentiate different projection types / - In OpenGL two main projection types exist / - Perspective projection results in perspective shortening. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'volume, we can differentiate different projection types' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'volume, we can differentiate different projection types'?
Page 60	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Literature and other sources used in this chapter' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Literature and other sources used in this chapter' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?

Page 61	- Text Books
Source cue	- J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice (3rd / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics
Professor-style explanation	The slide '- Text Books' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice (3rd / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.

Technical commentary	This slide is about - Text Books. The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice (3rd / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for '- Text Books' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip '- Text Books' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic '- Text Books' points to for deeper study?

04.1 Linear Perspective and Planar Projections

Source location: Sections 4.1-4.2

Commentary

Perspective projection models the visual effect that farther objects appear smaller. Orthographic projection removes this distance-based size change and keeps parallel lines parallel. Both are useful, but they communicate different spatial relationships.

Planar projection means mapping points onto an image plane. The lecture's art examples are not decoration; they show that projection is a geometric rule for constructing an image from a viewpoint.

Mental model

Projection is the rule that turns 3D positions into image-plane positions.

Check yourself

What visual cue does perspective projection add that orthographic projection lacks?

04.2 Camera Modeling

Source location: Section 4.3

Commentary

A virtual camera is defined by position, orientation, projection type, viewing volume, and image/viewport settings. The view transform places the world relative to the camera. The projection transform maps that view into clip coordinates.

Near and far clipping planes are part of the camera model. They decide what range of depths is represented and strongly affect depth buffer precision. A careless near/far setup can create z-fighting even if the scene geometry is correct.

Mental model

A camera is not only an eye position; it is also a volume and a mapping rule.

Check yourself

Why can changing the near plane affect depth artifacts?

04.3 Specifying Projections in OpenGL

Source location: Section 4.4

Commentary

In OpenGL, projection is usually encoded in a projection matrix sent to a shader. The matrix maps view-space coordinates to clip space. Clipping and the perspective divide then lead toward normalized device coordinates.

Aspect ratio and field of view must match the intended viewport. If the aspect ratio is wrong, objects appear stretched. If the field of view is extreme, the image can look distorted even though the math is functioning.

Mental model

The projection matrix is the camera lens encoded as linear algebra in homogeneous coordinates.

Check yourself

What symptom suggests that the projection aspect ratio does not match the window?

04.4 Orthographic and Perspective Derivations

Source location: Sections 4.5-4.6

Commentary

The derivations are there to explain where the matrix entries come from. Orthographic projection maps a box-shaped view volume into normalized coordinates. Perspective projection maps a frustum and uses the homogeneous component for the later divide.

You do not need to treat these matrices as random formulas. Each part maps a coordinate range into a standard range. The perspective matrix additionally arranges w so that the perspective divide creates foreshortening.

Mental model

Projection matrices normalize a viewing volume; perspective also prepares the divide by w .

Check yourself

What does the perspective divide do to x and y coordinates?

04.5 Viewport Transformation

Source location: Section 4.7

Commentary

After normalized device coordinates, the viewport transform maps the normalized range to actual window coordinates. This is the last geometric mapping before rasterization uses the target grid.

Separating projection from viewport mapping helps debugging. A wrong projection can produce wrong normalized coordinates; a wrong viewport can map otherwise valid coordinates into the wrong part of the window.

Mental model

Projection decides normalized position; viewport decides where that position lands on the screen.

Check yourself

Why is resizing a window related to both viewport and projection settings?

End-of-lecture summary

If you remember only one thing from Lecture 04, remember this: This lecture explains how a 3D view becomes a 2D image. Projection is where camera geometry, homogeneous coordinates, clipping planes, depth precision, and viewport mapping meet.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 05 - Clipping

This lecture focuses on algorithms that remove or trim geometry against boundaries. Clipping is a geometric operation before rasterization, not a visibility test between overlapping objects.

Source chunk	course_text_parts/03_lectures/05_clipping.txt
Extracted slide pages	56
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.

Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Clipping Projected Geometry 1/2
Source cue	- Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized
Professor-style explanation	The slide 'Clipping Projected Geometry 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Clipping Projected Geometry 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Clipping Projected Geometry 1/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Clipping Projected Geometry 1/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Clipping Projected Geometry 1/2'?

Page 3	Clipping Projected Geometry 2/2
Source cue	- Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping
Professor-style explanation	The slide 'Clipping Projected Geometry 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Clipping Projected Geometry 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Clipping Projected Geometry 2/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Clipping Projected Geometry 2/2'?
Page 4	5.1 Cohen-Sutherland Line Clipping
Source cue	5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping

Professor-style explanation	The outline '5.1 Cohen-Sutherland Line Clipping' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: 5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about 5.1 Cohen-Sutherland Line Clipping. The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: 5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for '5.1 Cohen-Sutherland Line Clipping' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize '5.1 Cohen-Sutherland Line Clipping' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where '5.1 Cohen-Sutherland Line Clipping' fits in the lecture order and what later section depends on it?
Page 5	5.1 Cohen-Sutherland Line Clipping
Source cue	Clip lines against rectangle

Professor-style explanation	The slide '5.1 Cohen-Sutherland Line Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: Clip lines against rectangle. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about 5.1 Cohen-Sutherland Line Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: Clip lines against rectangle. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for '5.1 Cohen-Sutherland Line Clipping' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.1 Cohen-Sutherland Line Clipping'?
Page 6	Brute-Force Line Clipping 1/2
Source cue	- Check each edge of the rectangle for intersection with line / - Intersection calculation for the line defined by the edge, and the line / - If intersection lies within edge and within line, clipping necessary

Professor-style explanation	The slide 'Brute-Force Line Clipping 1/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Check each edge of the rectangle for intersection with line / - Intersection calculation for the line defined by the edge, and the line / - If intersection lies within edge and within line, clipping necessary. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Brute-Force Line Clipping 1/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Check each edge of the rectangle for intersection with line / - Intersection calculation for the line defined by the edge, and the line / - If intersection lies within edge and within line, clipping necessary. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Brute-Force Line Clipping 1/2' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Brute-Force Line Clipping 1/2'?
Page 7	Brute-Force Line Clipping 2/2
Source cue	- Using the parameter representation for lines $x - x_0 = t(x_1 - x_0)$ / $y - y_0 = t(y_1 - y_0)$ / $p \cdot x + q \cdot y = r$ / $p \cdot x_0 + q \cdot y_0 = r_0$ / $p \cdot x_1 + q \cdot y_1 = r_1$

Professor-style explanation	The slide 'Brute-Force Line Clipping 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Using the parameter representation for lines $x = x_0 + t(x_1 - x_0)$, $y = y_0 + t(y_1 - y_0)$, $0 \leq t \leq 1$. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Brute-Force Line Clipping 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Using the parameter representation for lines $x = x_0 + t(x_1 - x_0)$, $y = y_0 + t(y_1 - y_0)$, $0 \leq t \leq 1$. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Brute-Force Line Clipping 2/2' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Brute-Force Line Clipping 2/2'?
Page 8	Endpoint Analysis 1/2
Source cue	- Idea: Use end point analysis to create new lines / - Clipping against a rectangle yields a line / - Clipping against a convex polygon results in a line / - Endpoint analysis / - Line segment $s = (p_0, p_1)$, $p_0 = (x_0, y_0)$ and $p_1 = (x_1, y_1)$

Professor-style explanation	The slide 'Endpoint Analysis 1/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Idea: Use end point analysis to create new lines / - Clipping against a rectangle yields a line / - Clipping against a convex polygon results in a line / - Endpoint analysis / - Line segment $s = (p, p)$, $p = (x, y)$ and $p = (x, y)$. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Endpoint Analysis 1/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Idea: Use end point analysis to create new lines / - Clipping against a rectangle yields a line / - Clipping against a convex polygon results in a line / - Endpoint analysis / - Line segment $s = (p, p)$, $p = (x, y)$ and $p = (x, y)$. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Endpoint Analysis 1/2' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Endpoint Analysis 1/2'?
Page 9	Endpoint Analysis 2/2
Source cue	- Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of x / min max / and above y and below y , they are inside / min max

Professor-style explanation	The slide 'Endpoint Analysis 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of $x / \min \max$ / and above y and below y , they are inside / $\min \max$. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Endpoint Analysis 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of $x / \min \max$ / and above y and below y , they are inside / $\min \max$. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Endpoint Analysis 2/2' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Endpoint Analysis 2/2'?
Page 10	Binary Coding of Half-Planes 1/3
Source cue	- Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode

Professor-style explanation	The slide 'Binary Coding of Half-Planes 1/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Binary Coding of Half-Planes 1/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Binary Coding of Half-Planes 1/3' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Binary Coding of Half-Planes 1/3' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Binary Coding of Half-Planes 1/3' into a causal sentence instead of repeating the slide title?
Page 11	Binary Coding of Half-Planes 2/3
Source cue	- Endpoint classification / - Line segment $s = (p, p) / 01$ / - Outcodes: / 1001 1000 1010

Professor-style explanation	The slide 'Binary Coding of Half-Planes 2/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Endpoint classification / - Line segment $s = (p, p) / 0 1$ / - Outcodes: / 1001 1000 1010. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Binary Coding of Half-Planes 2/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Endpoint classification / - Line segment $s = (p, p) / 0 1$ / - Outcodes: / 1001 1000 1010. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Binary Coding of Half-Planes 2/3' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Binary Coding of Half-Planes 2/3'?
Page 12	Binary Coding of Half-Planes 3/3
Source cue	- Clipping, if $(oc0 \ \& \ oc1) = 0$: / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip s at this clip edge

Professor-style explanation	The slide 'Binary Coding of Half-Planes 3/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Clipping, if $(oc0 \ \& \ oc1) = 0$: / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip s at this clip edge. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Binary Coding of Half-Planes 3/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Clipping, if $(oc0 \ \& \ oc1) = 0$: / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip s at this clip edge. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Binary Coding of Half-Planes 3/3' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Binary Coding of Half-Planes 3/3'?
Page 13	Pseudo Code
Source cue	<pre>void cohensutherlandLineClipper(Line s, Rectangle r) { int oc0 = outcode(s.p0, r); int oc1 = outcode(s.p1, r); bool done = false; while (!done) { int outcode(Point p, Rectangle r) {</pre>

Professor-style explanation	The slide 'Pseudo Code' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: void cohensutherlandLineClipper(Line s, Rectangle r) { / int oc0 = outcode(s.p0, r); / int oc1 = outcode(s.p1, r); / bool done = false; / while (!done) { int outcode(Point p, Rectangle r) { . A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Pseudo Code. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: void cohensutherlandLineClipper(Line s, Rectangle r) { / int oc0 = outcode(s.p0, r); / int oc1 = outcode(s.p1, r); / bool done = false; / while (!done) { int outcode(Point p, Rectangle r) { . A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Pseudo Code' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pseudo Code'?

Page 14	Example
Source cue	1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110
Professor-style explanation	The slide 'Example' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: 1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Example. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: 1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Example' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Example' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Example' into a causal sentence instead of repeating the slide title?

Page 15	Evaluation
Source cue	- Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside
Professor-style explanation	The slide 'Evaluation' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Evaluation. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.

Exam-grade answer	A strong answer for 'Evaluation' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Evaluation'?

Page 16	5.2 Cyrus-Beck Line Clipping
Source cue	Clip lines against convex polygon
Professor-style explanation	The slide '5.2 Cyrus-Beck Line Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: Clip lines against convex polygon. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about 5.2 Cyrus-Beck Line Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: Clip lines against convex polygon. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for '5.2 Cyrus-Beck Line Clipping' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.2 Cyrus-Beck Line Clipping'?

Page 17	Cyrus-Beck Line Clipping
Source cue	- Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky

Professor-style explanation	The slide 'Cyrus-Beck Line Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Cyrus-Beck Line Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Cyrus-Beck Line Clipping' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Cyrus-Beck Line Clipping'?
Page 18	Parametric Line Representation
Source cue	- Parametric line representation written as follows / $p + t \cdot p - p$, $0 \leq t \leq 1$ / 0 1 0 / - Proceeding / - Find the 4 t's for the 4 clip edges

Professor-style explanation	The slide 'Parametric Line Representation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Parametric line representation written as follows $p(t) = p + t \cdot p - p$, $0 \leq t \leq 1$ - Proceeding - Find the 4 t's for the 4 clip edges. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Parametric Line Representation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Parametric line representation written as follows $p(t) = p + t \cdot p - p$, $0 \leq t \leq 1$ - Proceeding - Find the 4 t's for the 4 clip edges. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Parametric Line Representation' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Parametric Line Representation'?
Page 19	Intersection Point Calculation 1/2
Source cue	- Consider edge E and its outward pointing edge normal N - Through dot product $N \cdot (p(t) - p)$ it can be decided in which half-space a line point p(t) lies - $N \cdot (p(t) - p) < 0$ / E

Professor-style explanation	The slide 'Intersection Point Calculation 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Consider edge E and its outward pointing edge normal N / - Through dot product $N \cdot (p(t) - p)$ it can be decided / in which half-space a line point $p(t)$ lies / $N \cdot (p(t) - p) < 0$ / E E. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Intersection Point Calculation 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Consider edge E and its outward pointing edge normal N / - Through dot product $N \cdot (p(t) - p)$ it can be decided / in which half-space a line point $p(t)$ lies / $N \cdot (p(t) - p) < 0$ / E E. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Intersection Point Calculation 1/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Intersection Point Calculation 1/2'?
Page 20	Intersection Point Calculation 2/2
Source cue	- Determine intersection by calculating t for $N \cdot p \cdot t - p = 0$ / - Substitution of $p \cdot t$ and regrouping gives $N \cdot p - p / 0$ E

Professor-style explanation	The slide 'Intersection Point Calculation 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Determine intersection by calculating t for $N \cdot p \cdot t - p = 0$ / - Substitution of $p \cdot t$ and regrouping gives $N \cdot p - p / 0$ E. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Intersection Point Calculation 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Determine intersection by calculating t for $N \cdot p \cdot t - p = 0$ / - Substitution of $p \cdot t$ and regrouping gives $N \cdot p - p / 0$ E. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Intersection Point Calculation 2/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Intersection Point Calculation 2/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Intersection Point Calculation 2/2' into a causal sentence instead of repeating the slide title?

Page 21	Determine Relevant t-Values
Source cue	- Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / s PE / PL PL PL / p s
Professor-style explanation	The slide 'Determine Relevant t-Values' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / s PE / PL PL PL / p s. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Determine Relevant t-Values. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / s PE / PL PL PL / p s. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Determine Relevant t-Values' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Determine Relevant t-Values' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Determine Relevant t-Values' into a causal sentence instead of repeating the slide title?

Page 22	Pseudo Code
Source cue	<code>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</code>
Professor-style explanation	The slide 'Pseudo Code' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: <code>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</code> . A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Pseudo Code. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: <code>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</code> . A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Pseudo Code' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.

Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pseudo Code'?
Page 23	Axis-Parallel Clipping Region
Source cue	- When clipping region is axis parallel and given by $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$, / min max min max / following values occur / $N \cdot p - p / 0 E$
Professor-style explanation	The slide 'Axis-Parallel Clipping Region' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - When clipping region is axis parallel and given by $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$, / min max min max / following values occur / $N \cdot p - p / 0 E$. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Axis-Parallel Clipping Region. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - When clipping region is axis parallel and given by $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$, / min max min max / following values occur / $N \cdot p - p / 0 E$. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Axis-Parallel Clipping Region' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Axis-Parallel Clipping Region'?
Page 24	Comparison with Cohen-Sutherland
Source cue	- Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected

Professor-style explanation	The slide 'Comparison with Cohen-Sutherland' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Comparison with Cohen-Sutherland. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Comparison with Cohen-Sutherland' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Comparison with Cohen-Sutherland'?
Page 25	5.3 Sutherland-Hodgman Polygon Clipping
Source cue	A pipelined version of the scalar product algorithm extended to clip polygons

Professor-style explanation	The slide '5.3 Sutherland-Hodgman Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: A pipelined version of the scalar product algorithm extended to clip polygons. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about 5.3 Sutherland-Hodgman Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: A pipelined version of the scalar product algorithm extended to clip polygons. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for '5.3 Sutherland-Hodgman Polygon Clipping' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.3 Sutherland-Hodgman Polygon Clipping'?
Page 26	Sutherland-Hodgman Polygon Clipping 1/2
Source cue	- Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping

Professor-style explanation	The slide 'Sutherland-Hodgman Polygon Clipping 1/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Sutherland-Hodgman Polygon Clipping 1/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Sutherland-Hodgman Polygon Clipping 1/2' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Sutherland-Hodgman Polygon Clipping 1/2'?
Page 27	Sutherland-Hodgman Polygon Clipping 2/2
Source cue	- Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region

Professor-style explanation	The slide 'Sutherland-Hodgman Polygon Clipping 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Sutherland-Hodgman Polygon Clipping 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Sutherland-Hodgman Polygon Clipping 2/2' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Sutherland-Hodgman Polygon Clipping 2/2'?
Page 28	Four Cases
Source cue	- Depending on which case occurs, following points added to output list / - Case 1: p / - Case 2: Intersection point i / - Case 3: -- / - Case 4: Intersection point i and p

Professor-style explanation	The slide 'Four Cases' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Depending on which case occurs, following points added to output list / - Case 1: p / - Case 2: Intersection point i / - Case 3: -- / - Case 4: Intersection point i and p. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Four Cases. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Depending on which case occurs, following points added to output list / - Case 1: p / - Case 2: Intersection point i / - Case 3: -- / - Case 4: Intersection point i and p. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Four Cases' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Four Cases' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Four Cases' into a causal sentence instead of repeating the slide title?
Page 29	Proceeding
Source cue	- Polygon to be clipped is defined by / - Vertices: $[v_1, v_2, \dots, v_n]$ / 1 2 n / - Contour: $[v_1, v_2, \dots, v_n, v_1]$ / 1 2 n 1
Professor-style explanation	The slide 'Proceeding' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Polygon to be clipped is defined by / - Vertices: $[v_1, v_2, \dots, v_n]$ / 1 2 n / - Contour: $[v_1, v_2, \dots, v_n, v_1]$ / 1 2 n 1. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Proceeding. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Polygon to be clipped is defined by / - Vertices: $[v_1, v_2, \dots, v_n]$ / - Contour: $[v_1, v_2, \dots, v_n]$ / - 1. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Proceeding' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Proceeding' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?

Page 30	Pipeline Version
Source cue	- Speed can be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware
Professor-style explanation	The slide 'Pipeline Version' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Speed can be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Pipeline Version. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Speed can be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.

Exam-grade answer	A strong answer for 'Pipeline Version' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pipeline Version'?
Page 31	Pseudo Code
Source cue	<pre>public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) {</pre>
Professor-style explanation	The slide 'Pseudo Code' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) { . The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Pseudo Code. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) { . The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Pseudo Code' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Pseudo Code' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Pseudo Code' into a causal sentence instead of repeating the slide title?
Page 32	Inside Test

Source cue	- Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top
Professor-style explanation	The slide 'Inside Test' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Inside Test. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Inside Test' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Inside Test' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Inside Test' into a causal sentence instead of repeating the slide title?
Page 33	Intersection Computation
Source cue	- For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y;

Professor-style explanation	The slide 'Intersection Computation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Intersection Computation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Intersection Computation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Intersection Computation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Intersection Computation' into a causal sentence instead of repeating the slide title?
Page 34	5.4 Weiler-Atherton Polygon Clipping
Source cue	Clip arbitrary polygons against arbitrary clip polygons
Professor-style explanation	The slide '5.4 Weiler-Atherton Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: Clip arbitrary polygons against arbitrary clip polygons. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.

Technical commentary	This slide is about 5.4 Weiler-Atherton Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: Clip arbitrary polygons against arbitrary clip polygons. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for '5.4 Weiler-Atherton Polygon Clipping' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.4 Weiler-Atherton Polygon Clipping'?

Page 35	Weiler-Atherton Polygon Clipping
Source cue	- Basic idea: edges of the clip polygon A and the polygon B to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray)
Professor-style explanation	The slide 'Weiler-Atherton Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Basic idea: edges of the clip polygon A and the polygon B to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray). A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Weiler-Atherton Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Basic idea: edges of the clip polygon A and the polygon B to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray). A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.

Exam-grade answer	A strong answer for 'Weiler-Atherton Polygon Clipping' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Polygon Clipping'?
Page 36	Example Run
Source cue	- Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of A and B / - Double all edge segments, and label each segment / according to the adjacent region
Professor-style explanation	The slide 'Example Run' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of A and B / - Double all edge segments, and label each segment / according to the adjacent region. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Example Run. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of A and B / - Double all edge segments, and label each segment / according to the adjacent region. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Example Run' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Example Run' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Example Run' into a causal sentence instead of repeating the slide title?
Page 37	Intersection Types

Source cue	- There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal)
Professor-style explanation	The slide 'Intersection Types' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Intersection Types. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Intersection Types' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Intersection Types' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Intersection Types' into a causal sentence instead of repeating the slide title?
Page 38	Contour Processing
Source cue	- In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of A / and insert the contour edges of B one after the other

Professor-style explanation	The slide 'Contour Processing' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of A / and insert the contour edges of B one after the other. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Contour Processing. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of A / and insert the contour edges of B one after the other. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Contour Processing' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Contour Processing' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Contour Processing' into a causal sentence instead of repeating the slide title?
Page 39	Evaluation
Source cue	- Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion

Professor-style explanation	The slide 'Evaluation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Evaluation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Evaluation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Evaluation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Evaluation' into a causal sentence instead of repeating the slide title?
Page 40	5.5 Greiner-Hormann Polygon Clipping
Source cue	Exploitation of the winding number to clip arbitrary polygons
Professor-style explanation	The slide '5.5 Greiner-Hormann Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: Exploitation of the winding number to clip arbitrary polygons. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.

Technical commentary	This slide is about 5.5 Greiner-Hormann Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: Exploitation of the winding number to clip arbitrary polygons. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for '5.5 Greiner-Hormann Polygon Clipping' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.5 Greiner-Hormann Polygon Clipping'?

Page 41	Greiner-Hormann
Source cue	- Basic Idea: clipped region is made up of all edges of S that lie in C, / combined with all edges of C that lie in S / [Greiner & Hormann, TOG 1998]
Professor-style explanation	The slide 'Greiner-Hormann' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Basic Idea: clipped region is made up of all edges of S that lie in C, / combined with all edges of C that lie in S / [Greiner & Hormann, TOG 1998]. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Greiner-Hormann. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Basic Idea: clipped region is made up of all edges of S that lie in C, / combined with all edges of C that lie in S / [Greiner & Hormann, TOG 1998]. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Greiner-Hormann' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.

Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Greiner-Hormann'?
Page 42	Chalk Wagon Metaphor
Source cue	- Using a chalk wagon metaphor / - Drive along all edges of A with a chalk wagon / that is only open when the current position is within B / - Drive along all edges of B with a chalk wagon / that is only open when the current position is within A
Professor-style explanation	The slide 'Chalk Wagon Metaphor' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Using a chalk wagon metaphor / - Drive along all edges of A with a chalk wagon / that is only open when the current position is within B / - Drive along all edges of B with a chalk wagon / that is only open when the current position is within A. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Chalk Wagon Metaphor. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Using a chalk wagon metaphor / - Drive along all edges of A with a chalk wagon / that is only open when the current position is within B / - Drive along all edges of B with a chalk wagon / that is only open when the current position is within A. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Chalk Wagon Metaphor' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Chalk Wagon Metaphor' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Chalk Wagon Metaphor' into a causal sentence instead of repeating the slide title?
Page 43	Contour Detection
Source cue	- The edges of polygon A within polygon B can thus be found as follows / - Insert in A all intersection points with B / - Start at any vertex v of A / - If v is within B, the edge flag is turned on, otherwise not / - Traverse edges of A and invert edge marker flag when intersecting with B

Professor-style explanation	The slide 'Contour Detection' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - The edges of polygon A within polygon B can thus be found as follows / - Insert in A all intersection points with B / - Start at any vertex v of A / - If v is within B, the edge flag is turned on, otherwise not / - Traverse edges of A and invert edge marker flag when intersecting with B. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Contour Detection. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - The edges of polygon A within polygon B can thus be found as follows / - Insert in A all intersection points with B / - Start at any vertex v of A / - If v is within B, the edge flag is turned on, otherwise not / - Traverse edges of A and invert edge marker flag when intersecting with B. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Contour Detection' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Contour Detection' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Contour Detection' into a causal sentence instead of repeating the slide title?
Page 44	Set Operations
Source cue	[Greiner & Hormann, TOG 1998]

Professor-style explanation	The slide 'Set Operations' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: [Greiner & Hormann, TOG 1998]. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Set Operations. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: [Greiner & Hormann, TOG 1998]. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Set Operations' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Set Operations'?

Page 45	Data Structure 1/2
Source cue	- Polygons exist as a closed, double-linked list / $P_1, P_2, P_3, \dots, P_n$ / $P_n \rightarrow P_1$ / - The following segments are included / $P_1P_2, P_2P_3, \dots, P_{n-1}P_n$ / P_nP_1
Professor-style explanation	The slide 'Data Structure 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Polygons exist as a closed, double-linked list / $P_1, P_2, P_3, \dots, P_n$ / $P_n \rightarrow P_1$ / - The following segments are included / $P_1P_2, P_2P_3, \dots, P_{n-1}P_n$ / P_nP_1 . The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Data Structure 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Polygons exist as a closed, double-linked list / $P, P, P, \dots, P = P / 0 \ 1 \ 2 \ n \ 0 /$ - The following segments are included / $P \ P, P \ P, \dots, P \ P = P \ P$. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Data Structure 1/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Data Structure 1/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Data Structure 1/2' into a causal sentence instead of repeating the slide title?

Page 46	Data Structure 2/2
Source cue	[Greiner & Hormann, TOG 1998]
Professor-style explanation	The slide 'Data Structure 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: [Greiner & Hormann, TOG 1998]. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Data Structure 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: [Greiner & Hormann, TOG 1998]. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Data Structure 2/2' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Data Structure 2/2'?

Page 47	Proceeding
Source cue	- Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule)
Professor-style explanation	The slide 'Proceeding' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Proceeding. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Proceeding' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Proceeding' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?
Page 48	Pseudo Code
Source cue	while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current)

Professor-style explanation	The slide 'Pseudo Code' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Pseudo Code. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Pseudo Code' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Pseudo Code' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Pseudo Code' into a causal sentence instead of repeating the slide title?
Page 49	Winding Number 1/3
Source cue	- Winding number of a point A and a closed curve γ / indicates the number of times a ray that traverses the entire curve from A / winds around A / $\omega \gamma$, $A = d\phi$ / - Counterclockwise: +1

Professor-style explanation	The slide 'Winding Number 1/3' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Winding number of a point A and a closed curve γ / indicates the number of times a ray that traverses the entire curve from A / winds around A / $\omega \gamma$, $A = d\phi$ / - Counterclockwise: +1. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Winding Number 1/3. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Winding number of a point A and a closed curve γ / indicates the number of times a ray that traverses the entire curve from A / winds around A / $\omega \gamma$, $A = d\phi$ / - Counterclockwise: +1. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Winding Number 1/3' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Winding Number 1/3' works per object, per image region, per ray, or per fragment?
Page 50	Winding Number 2/3
Source cue	- The winding number of a point A has the following useful properties / 1. If A or the curve γ is continuously changed so that A maintains a / positive distance to γ, the winding number will not change / For curve γ, the winding number is constant in each region formed by γ / 2. When A moves and γ crosses once, winding number changes by exactly 1

Professor-style explanation	The slide 'Winding Number 2/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - The winding number of a point A has the following useful properties / 1. If A or the curve γ is continuously changed so that A maintains a / positive distance to γ , the winding number will not change / For curve γ , the winding number is constant in each region formed by γ / 2. When A moves and γ crosses once, winding number changes by exactly 1. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Winding Number 2/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - The winding number of a point A has the following useful properties / 1. If A or the curve γ is continuously changed so that A maintains a / positive distance to γ , the winding number will not change / For curve γ , the winding number is constant in each region formed by γ / 2. When A moves and γ crosses once, winding number changes by exactly 1. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Winding Number 2/3' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Winding Number 2/3' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Winding Number 2/3' into a causal sentence instead of repeating the slide title?
Page 51	Winding Number 3/3
Source cue	- From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside / - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]

Professor-style explanation	The slide 'Winding Number 3/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Winding Number 3/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Winding Number 3/3' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Winding Number 3/3'?
Page 52	Special Cases
Source cue	- Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation - One polygon is contained in the other polygon - Polygons have no overlaps - Problem: One point lies on one edge

Professor-style explanation	The slide 'Special Cases' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation / - One polygon is contained in the other polygon / - Polygons have no overlaps / - Problem: One point lies on one edge. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Special Cases. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation / - One polygon is contained in the other polygon / - Polygons have no overlaps / - Problem: One point lies on one edge. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Special Cases' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Special Cases' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Special Cases' into a causal sentence instead of repeating the slide title?
Page 53	Evaluation
Source cue	- Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,

Professor-style explanation	The slide 'Evaluation' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Evaluation. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Evaluation' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Evaluation'?
Page 54	by exploiting binary outcodes
Source cue	- Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon

Professor-style explanation	The slide 'by exploiting binary outcodes' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about by exploiting binary outcodes. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'by exploiting binary outcodes' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'by exploiting binary outcodes'?
Page 55	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Literature and other sources used in this chapter' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Literature and other sources used in this chapter' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 56	Addison-Wesley 2013.
Source cue	- Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016.

Professor-style explanation	The slide 'Addison-Wesley 2013.' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: - Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.
Technical commentary	This slide is about Addison-Wesley 2013.. The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: - Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Addison-Wesley 2013.' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Addison-Wesley 2013.' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Addison-Wesley 2013.' points to for deeper study?

05.1 Cohen-Sutherland Line Clipping

Source location: Section 5.1

Commentary

Cohen-Sutherland assigns outcodes to line endpoints based on which side of the clipping window they lie on. These codes allow quick accept, quick reject, or partial clipping. The algorithm is efficient because many cases are decided before computing intersections.

The exam-friendly idea is the logic of region codes: if both endpoints are inside, keep the segment. If the bitwise AND of the outcodes is nonzero, both endpoints share an outside region and the segment can be rejected. Otherwise, compute an intersection and continue.

Mental model

Outcodes are a cheap classification before doing intersection math.

Check yourself

What does a nonzero bitwise AND of two endpoint outcodes imply?

05.2 Cyrus-Beck Line Clipping

Source location: Section 5.2

Commentary

Cyrus-Beck treats the line parametrically and clips against convex boundaries. Instead of repeatedly using region codes, it finds entering and leaving parameter values along the line.

The key is to reason about the line parameter t . A clipped line segment is not a new unrelated line; it is a restricted interval of the original parametric line. Boundary tests shrink that interval.

Mental model

Clipping a parametric line means narrowing the valid t interval.

Check yourself

Why does Cyrus-Beck naturally require convex clipping regions?

05.3 Sutherland-Hodgman Polygon Clipping

Source location: Section 5.3

Commentary

Sutherland-Hodgman clips a polygon against one boundary at a time. For each edge of the polygon, it decides whether vertices are inside or outside and emits zero, one, or two vertices depending on transitions across the boundary.

This algorithm is easiest to understand as a stream processor. Feed in a polygon, clip it against the left boundary, feed the result into the right boundary, and so on. Each boundary can add intersection vertices.

Mental model

Polygon clipping is repeated edge-by-edge filtering plus intersection insertion.

Check yourself

When can polygon clipping increase the number of vertices?

05.4 Weiler-Atherton and Greiner-Hormann

Source location: Sections 5.4-5.5

Commentary

These algorithms address more complex polygon clipping scenarios, especially when polygon relationships are not as simple as convex-window clipping. They organize intersections and traversal rules to produce correct output boundaries.

For preparation, focus on why simpler algorithms are not enough. Complex polygon intersection can require following alternating boundaries of subject and clipping polygons. That is a topology problem, not only a line-intersection problem.

Mental model

Complex polygon clipping is about traversing boundary networks after intersections are known.

Check yourself

Why are intersection ordering and traversal rules important for polygon clipping?

End-of-lecture summary

If you remember only one thing from Lecture 05, remember this: This lecture focuses on algorithms that remove or trim geometry against boundaries. Clipping is a geometric operation before rasterization, not a visibility test between overlapping objects.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 06 - Rasterization

This lecture explains how continuous primitives become discrete fragments on a grid. Rasterization is the bridge from geometric descriptions to sample-based image generation.

Source chunk	course_text_parts/03_lectures/06_rasterization.txt
Extracted slide pages	61
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.

Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Rasterization 1/2
Source cue	- Pixels ... / - ... are picture elements A Pixel Is NotA Little Square, / A Pixel Is NotA Little Square, / - ... are the smallest elements of a raster image / A Pixel Is NotA Little Square!
Professor-style explanation	The slide 'Rasterization 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Pixels ... / - ... are picture elements A Pixel Is NotA Little Square, / A Pixel Is NotA Little Square, / - ... are the smallest elements of a raster image / A Pixel Is NotA Little Square!. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Rasterization 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Pixels ... / - ... are picture elements A Pixel Is NotA Little Square, / A Pixel Is NotA Little Square, / - ... are the smallest elements of a raster image / A Pixel Is NotA Little Square!. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Rasterization 1/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization 1/2'?
Page 3	Rasterization 2/2
Source cue	- Pixel-based displays require discretization of primitives into pixels / - Discretization is done by rasterization algorithms / - Rasterization is time-critical / - Avoid expensive operations / - Optimize for hardware implementation

Professor-style explanation	The slide 'Rasterization 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Pixel-based displays require discretization of primitives into pixels / - Discretization is done by rasterization algorithms / - Rasterization is time-critical / - Avoid expensive operations / - Optimize for hardware implementation. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Rasterization 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Pixel-based displays require discretization of primitives into pixels / - Discretization is done by rasterization algorithms / - Rasterization is time-critical / - Avoid expensive operations / - Optimize for hardware implementation. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Rasterization 2/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization 2/2'?
Page 4	6.1 Line Rasterization
Source cue	6.2 Triangle Edge Rasterization / 6.3 Region Filling / 6.4 Scanline-Based Triangle Rasterization / 6.5 Block-Based Triangle Rasterization

Professor-style explanation	The outline '6.1 Line Rasterization' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: 6.2 Triangle Edge Rasterization / 6.3 Region Filling / 6.4 Scanline-Based Triangle Rasterization / 6.5 Block-Based Triangle Rasterization. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about 6.1 Line Rasterization. The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: 6.2 Triangle Edge Rasterization / 6.3 Region Filling / 6.4 Scanline-Based Triangle Rasterization / 6.5 Block-Based Triangle Rasterization. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for '6.1 Line Rasterization' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize '6.1 Line Rasterization' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where '6.1 Line Rasterization' fits in the lecture order and what later section depends on it?
Page 5	6.1 Line Rasterization
Source cue	Drawing a one pixel thick line

Professor-style explanation	The slide '6.1 Line Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: Drawing a one pixel thick line. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about 6.1 Line Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: Drawing a one pixel thick line. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for '6.1 Line Rasterization' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by '6.1 Line Rasterization'?
Page 6	Line Rasterization
Source cue	- Given two 2D endpoints in integer coordinates, / determine pixels depicting 1-pixel wide line segment between endpoints / - Trivial cases / - Horizontal line segments / - Vertical line segments

Professor-style explanation	The slide 'Line Rasterization' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Given two 2D endpoints in integer coordinates, / determine pixels depicting 1-pixel wide line segment between endpoints / - Trivial cases / - Horizontal line segments / - Vertical line segments. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Line Rasterization. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Given two 2D endpoints in integer coordinates, / determine pixels depicting 1-pixel wide line segment between endpoints / - Trivial cases / - Horizontal line segments / - Vertical line segments. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Line Rasterization' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Line Rasterization' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Line Rasterization'?
Page 7	Quality Criteria
Source cue	- For all line segments with a slope $m \leq 1$, / one pixel should be drawn per column / - For all line segments with a slope $m \geq 1$, / one pixel per line should be drawn / $m > 1$

Professor-style explanation	The slide 'Quality Criteria' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - For all line segments with a slope $m \leq 1$, / one pixel should be drawn per column / - For all line segments with a slope $m \geq 1$, / one pixel per line should be drawn / $m > 1$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Quality Criteria. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - For all line segments with a slope $m \leq 1$, / one pixel should be drawn per column / - For all line segments with a slope $m \geq 1$, / one pixel per line should be drawn / $m > 1$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Quality Criteria' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Quality Criteria'?
Page 8	Naïve Algorithm 1/2
Source cue	- Utilization of the Cartesian normal form of the straight line equation / $y = m \cdot x + B$ / - Assumptions (without restriction of generality) / - Line segments lie between (x, y) and (x, y) / 0 0 1 1

Professor-style explanation	The slide 'Naïve Algorithm 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Utilization of the Cartesian normal form of the straight line equation $y = m \cdot x + B$ / - Assumptions (without restriction of generality) / - Line segments lie between (x, y) and $(x, y) / 0 \leq x \leq 1$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Naïve Algorithm 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Utilization of the Cartesian normal form of the straight line equation $y = m \cdot x + B$ / - Assumptions (without restriction of generality) / - Line segments lie between (x, y) and $(x, y) / 0 \leq x \leq 1$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Naïve Algorithm 1/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Naïve Algorithm 1/2'?
Page 9	Naïve Algorithm 2/2
Source cue	- Downsides / - Does not work for $x = x / 0 \leq x \leq 1$ / - Does not work for $m > 1$, since gaps arise / (solution: swap the roles of x and y for $m > 1$)

Professor-style explanation	The slide 'Naïve Algorithm 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Downsides / - Does not work for $x = x / 0$ / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Naïve Algorithm 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Downsides / - Does not work for $x = x / 0$ / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Naïve Algorithm 2/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Naïve Algorithm 2/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Naïve Algorithm 2/2' into a causal sentence instead of repeating the slide title?
Page 10	Digital Difference Analyzer (DDA) 1/3
Source cue	- Reuse the last computed y to avoid multiplications in the loop / $x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + m)$ / <code>void drawLine(int x0, int y0, int x1, int y1) { / float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope / float y = y0; // Start y at y0</code>

Professor-style explanation	The slide 'Digital Difference Analyzer (DDA) 1/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Reuse the last computed y to avoid multiplications in the loop / $x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + m)$ / void drawLine(int x0, int y0, int x1, int y1) { / float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope / float y = y0; // Start y at y0. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Digital Difference Analyzer (DDA) 1/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Reuse the last computed y to avoid multiplications in the loop / $x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + m)$ / void drawLine(int x0, int y0, int x1, int y1) { / float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope / float y = y0; // Start y at y0. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Digital Difference Analyzer (DDA) 1/3' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Digital Difference Analyzer (DDA) 1/3' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 1/3' into a causal sentence instead of repeating the slide title?
Page 11	Digital Difference Analyzer (DDA) 2/3
Source cue	- Algorithm can be extended to relax endpoint order / void drawLine(int x0, int y0, int x1, int y1) { / int sx = (x0 < x1 ? 1 : -1); // Determine x direction / float sm = sx * float(y1 - y0) / float(x1 - x0); // Calculate slope multiplied by step / float y = y0; // Start y at y0

Professor-style explanation	The slide 'Digital Difference Analyzer (DDA) 2/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Algorithm can be extended to relax endpoint order / void drawLine(int x0, int y0, int x1, int y1) { / int sx = (x0 < x1 ? 1 : -1); // Determine x direction / float sm = sx * float(y1 - y0) / float(x1 - x0); // Calculate slope multiplied by step / float y = y0; // Start y at y0. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Digital Difference Analyzer (DDA) 2/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Algorithm can be extended to relax endpoint order / void drawLine(int x0, int y0, int x1, int y1) { / int sx = (x0 < x1 ? 1 : -1); // Determine x direction / float sm = sx * float(y1 - y0) / float(x1 - x0); // Calculate slope multiplied by step / float y = y0; // Start y at y0. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Digital Difference Analyzer (DDA) 2/3' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Digital Difference Analyzer (DDA) 2/3' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 2/3' into a causal sentence instead of repeating the slide title?
Page 12	Digital Difference Analyzer (DDA) 3/3
Source cue	- Downsides / - Does not work for $x = x_0 \pm 1$ / - Does not work for $m > 1$, since gaps arise / (solution: swap the roles of x and y for $m > 1$)

Professor-style explanation	The slide 'Digital Difference Analyzer (DDA) 3/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Downsides / - Does not work for $x = x / 0 \ 1$ / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Digital Difference Analyzer (DDA) 3/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Downsides / - Does not work for $x = x / 0 \ 1$ / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Digital Difference Analyzer (DDA) 3/3' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Digital Difference Analyzer (DDA) 3/3' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 3/3' into a causal sentence instead of repeating the slide title?
Page 13	Midpoint-Line Algorithm 1/2
Source cue	- Assumption 1: $0 \leq m \leq 1$ / (without loss of generality, as otherwise mirroring on axes/diagonals) / - Lower left end point $P = (x, y)$, / $0 \ 0 \ 0$ / upper right end point $P = (x, y)$

Professor-style explanation	The slide 'Midpoint-Line Algorithm 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Assumption 1: $0 \leq m \leq 1$ / (without loss of generality, as otherwise mirroring on axes/diagonals) / - Lower left end point $P = (x, y)$, / $0 \ 0 \ 0$ / upper right end point $P = (x, y)$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Midpoint-Line Algorithm 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Assumption 1: $0 \leq m \leq 1$ / (without loss of generality, as otherwise mirroring on axes/diagonals) / - Lower left end point $P = (x, y)$, / $0 \ 0 \ 0$ / upper right end point $P = (x, y)$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Midpoint-Line Algorithm 1/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Midpoint-Line Algorithm 1/2'?
Page 14	Midpoint-Line Algorithm 2/2
Source cue	- Observation / - E is closer to the line if M is above the line / - NE is closer to the line if M is below the line / - Vertical distance between pixel to chose / and the actual line segment is always < 0.5

Professor-style explanation	The slide 'Midpoint-Line Algorithm 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Observation / - E is closer to the line if M is above the line / - NE is closer to the line if M is below the line / - Vertical distance between pixel to chose / and the actual line segment is always < 0.5. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Midpoint-Line Algorithm 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Observation / - E is closer to the line if M is above the line / - NE is closer to the line if M is below the line / - Vertical distance between pixel to chose / and the actual line segment is always < 0.5. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Midpoint-Line Algorithm 2/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Midpoint-Line Algorithm 2/2'?
Page 15	Implicit Function
Source cue	- Line equation usually given as explicit function $f(x) / f x = y = m \cdot x + B = \cdot x + B$, with $dx = x - x$ and $dy = y - y / 1 \text{ o } 1 \text{ o } /$ - Line equation can also be written as implicit function $F(x, y) / F x, y = a \cdot x + b \cdot y + c = 0$

Professor-style explanation	The slide 'Implicit Function' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Line equation usually given as explicit function $f(x) / f x = y = m \cdot x + B = \cdot x + B$, with $dx = x - x$ and $dy = y - y / 1 \text{ o } 1 \text{ o } /$ - Line equation can also be written as implicit function $F(x, y) / F x, y = a \cdot x + b \cdot y + c = 0$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Implicit Function. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Line equation usually given as explicit function $f(x) / f x = y = m \cdot x + B = \cdot x + B$, with $dx = x - x$ and $dy = y - y / 1 \text{ o } 1 \text{ o } /$ - Line equation can also be written as implicit function $F(x, y) / F x, y = a \cdot x + b \cdot y + c = 0$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Implicit Function' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Implicit Function'?
Page 16	Decision Variable d
Source cue	- Sign of $F(M)$ is sufficient to decide which point is chosen / - Define decision variable d as $d = F(M) = F(x + 1, y + 1) / p p /$ - If $d > 0$, select NE (M below the line) / - If $d < 0$, select E (M above the line)

Professor-style explanation	The slide 'Decision Variable d' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Sign of $F(M)$ is sufficient to decide which point is chosen / - Define decision variable d as $d = F M = F(x + 1, y +) / p p /$ - If $d > 0$, select NE (M below the line) / - If $d < 0$, select E (M above the line). The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Decision Variable d . The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Sign of $F(M)$ is sufficient to decide which point is chosen / - Define decision variable d as $d = F M = F(x + 1, y +) / p p /$ - If $d > 0$, select NE (M below the line) / - If $d < 0$, select E (M above the line). The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Decision Variable d ' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Decision Variable d ' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Decision Variable d ' into a causal sentence instead of repeating the slide title?
Page 17	Update of d for E
Source cue	- Previous decision variable was given by / 1 1 / $d = F x + 1, y + = a \cdot x + 1 + b \cdot y + + c /$ old $p p p p / 2 2$
Professor-style explanation	The slide 'Update of d for E' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Previous decision variable was given by / 1 1 / $d = F x + 1, y + = a \cdot x + 1 + b \cdot y + + c /$ old $p p p p / 2 2$. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Update of d for E. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Previous decision variable was given by $1/d = Fx + 1, y = a \cdot x + 1 + b \cdot y + c$ / old p p p / 2 2. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Update of d for E' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Update of d for E' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Update of d for E' into a causal sentence instead of repeating the slide title?

Page 18	Update of d for NE
Source cue	- Previous decision variable was given by $1/d = Fx + 1, y = a \cdot x + 1 + b \cdot y + c$ / old p p p / 2 2
Professor-style explanation	The slide 'Update of d for NE' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Previous decision variable was given by $1/d = Fx + 1, y = a \cdot x + 1 + b \cdot y + c$ / old p p p / 2 2. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Update of d for NE. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Previous decision variable was given by $1/d = Fx + 1, y = a \cdot x + 1 + b \cdot y + c$ / old p p p / 2 2. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Update of d for NE' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Update of d for NE' by repeating the title; name what changes, why it changes, and what the next rendering step receives.

Check yourself	Can you turn 'Update of d for NE' into a causal sentence instead of repeating the slide title?
Page 19	Proceeding
Source cue	- In each step, the algorithm selects between two pixels NE and E / based on sign of decision variable d / - Depending on the choice, decision variable is updated / by adding difference ΔNE or ΔE
Professor-style explanation	The slide 'Proceeding' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - In each step, the algorithm selects between two pixels NE and E / based on sign of decision variable d / - Depending on the choice, decision variable is updated / by adding difference ΔNE or ΔE . The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Proceeding. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - In each step, the algorithm selects between two pixels NE and E / based on sign of decision variable d / - Depending on the choice, decision variable is updated / by adding difference ΔNE or ΔE . The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Proceeding' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Proceeding' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?
Page 20	Initializing d
Source cue	- First midpoint M is located at $(x + 1, y +) / 0 0 / 1 1 b / F x + 1, y + = a \cdot x + 1 + b \cdot y + + c = a \cdot x + b \cdot y + c + a + / 0 0 0 0 0 0$

Professor-style explanation	The slide 'Initializing d' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - First midpoint M is located at $(x + 1, y + 1) / 0 \ 0 \ 1 \ 1 \ b / F \ x + 1, y + 1 = a \cdot x + 1 + b \cdot y + 1 + c = a \cdot x + b \cdot y + c + a + 1 / 0 \ 0 \ 0 \ 0 \ 0$. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Initializing d. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - First midpoint M is located at $(x + 1, y + 1) / 0 \ 0 \ 1 \ 1 \ b / F \ x + 1, y + 1 = a \cdot x + 1 + b \cdot y + 1 + c = a \cdot x + b \cdot y + c + a + 1 / 0 \ 0 \ 0 \ 0 \ 0$. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Initializing d' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Initializing d' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Initializing d' into a causal sentence instead of repeating the slide title?
Page 21	Using d
Source cue	- Midpoint line algorithms uses d as follows $d = 2 \cdot dy - dx / \text{start} / \Delta E = 2 \cdot dy - dx / \Delta E = 2 \cdot dy$

Professor-style explanation	The slide 'Using d' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Midpoint line algorithms uses d as follows $d = 2 \cdot dy - dx$ / start / $\Delta NE = 2 \cdot dy - dx$ / $\Delta E = 2 \cdot dy$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Using d. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Midpoint line algorithms uses d as follows $d = 2 \cdot dy - dx$ / start / $\Delta NE = 2 \cdot dy - dx$ / $\Delta E = 2 \cdot dy$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Using d' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Using d'?
Page 22	Pseudo Code - Simple Version
Source cue	<pre>void midPointLine(int x0, int y0, int x1, int y1) { / int dx = x1 - x0; // Calculate delta x / int dy = y1 - y0; // Calculate delta y / int d = 2 * dy - dx; // Initial decision variable / int dE = 2 * dy; // Increment used for move to E</pre>

Professor-style explanation	The slide 'Pseudo Code - Simple Version' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: <code>void midPointLine(int x0, int y0, int x1, int y1) { / int dx = x1 - x0; // Calculate delta x / int dy = y1 - y0; // Calculate delta y / int d = 2 * dy - dx; // Initial decision variable / int dE = 2 * dy; // Increment used for move to E. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.</code>
Technical commentary	This slide is about Pseudo Code - Simple Version. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: <code>void midPointLine(int x0, int y0, int x1, int y1) { / int dx = x1 - x0; // Calculate delta x / int dy = y1 - y0; // Calculate delta y / int d = 2 * dy - dx; // Initial decision variable / int dE = 2 * dy; // Increment used for move to E. The slide groups them because one object or quantity is processed into another result used later in rendering.</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Pseudo Code - Simple Version' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Pseudo Code - Simple Version' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Pseudo Code - Simple Version' into a causal sentence instead of repeating the slide title?
Page 23	Example
Source cue	- Draw line from (5, 8) to (9, 11) / - Order of values for d / - 2, 0, 6, 4 / - Order of chosen pixels / - NE, E, NE, NE

Professor-style explanation	The slide 'Example' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Draw line from (5, 8) to (9, 11) / - Order of values for d / - 2, 0, 6, 4 / - Order of chosen pixels / - NE, E, NE, NE. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Example. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Draw line from (5, 8) to (9, 11) / - Order of values for d / - 2, 0, 6, 4 / - Order of chosen pixels / - NE, E, NE, NE. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Example' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Example'?
Page 24	Evaluation
Source cue	- Necessary arithmetic operations / - As only integer values are used, only integer operations are necessary / - Additions/subtractions only and multiplication by 2 / (left shift by 1 bit position) / - No multiplications in the loop

Professor-style explanation	The slide 'Evaluation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Necessary arithmetic operations / - As only integer values are used, only integer operations are necessary / - Additions/subtractions only and multiplication by 2 / (left shift by 1 bit position) / - No multiplications in the loop. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Evaluation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Necessary arithmetic operations / - As only integer values are used, only integer operations are necessary / - Additions/subtractions only and multiplication by 2 / (left shift by 1 bit position) / - No multiplications in the loop. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Evaluation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Evaluation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Evaluation' into a causal sentence instead of repeating the slide title?
Page 25	Pseudo Code - Reversible
Source cue	<pre>void drawLine(int x0, int y0, int x1, int y1) { / int dx = abs(x1 - x0); // Calculate absolute delta x / int sx = (x0 < x1 ? 1 : -1); // Set step x based on direction / int dy = abs(y1 - y0); // Calculate absolute delta y / int sy = (y0 < y1 ? 1 : -1); // Set step y based on direction</pre>

Professor-style explanation	The slide 'Pseudo Code - Reversible' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: <code>void drawLine(int x0, int y0, int x1, int y1) { / int dx = abs(x1 - x0); // Calculate absolute delta x / int sx = (x0 < x1 ? 1 : -1); // Set step x based on direction / int dy = abs(y1 - y0); // Calculate absolute delta y / int sy = (y0 < y1 ? 1 : -1); // Set step y based on direction. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.</code>
Technical commentary	This slide is about Pseudo Code - Reversible. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: <code>void drawLine(int x0, int y0, int x1, int y1) { / int dx = abs(x1 - x0); // Calculate absolute delta x / int sx = (x0 < x1 ? 1 : -1); // Set step x based on direction / int dy = abs(y1 - y0); // Calculate absolute delta y / int sy = (y0 < y1 ? 1 : -1); // Set step y based on direction. The slide groups them because one object or quantity is processed into another result used later in rendering.</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Pseudo Code - Reversible' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Pseudo Code - Reversible' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Pseudo Code - Reversible' into a causal sentence instead of repeating the slide title?
Page 26	6.2 Triangle Edge Rasterization
Source cue	Drawing a one pixel thick triangle edge

Professor-style explanation	The slide '6.2 Triangle Edge Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: Drawing a one pixel thick triangle edge. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about 6.2 Triangle Edge Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: Drawing a one pixel thick triangle edge. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for '6.2 Triangle Edge Rasterization' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by '6.2 Triangle Edge Rasterization'?
Page 27	Triangle Edges 1/2
Source cue	- Using midpoint line algorithm for triangle edge rasterization / - Provides pixels closest to triangle edge / - Does not consider if pixel lies inside or outside triangle / $\Rightarrow$ Edge pixels can penetrate into interior of adjacent triangles

Professor-style explanation	The slide 'Triangle Edges 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Using midpoint line algorithm for triangle edge rasterization / - Provides pixels closest to triangle edge / - Does not consider if pixel lies inside or outside triangle / $\Rightarrow$ Edge pixels can penetrate into interior of adjacent triangles. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Triangle Edges 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Using midpoint line algorithm for triangle edge rasterization / - Provides pixels closest to triangle edge / - Does not consider if pixel lies inside or outside triangle / $\Rightarrow$ Edge pixels can penetrate into interior of adjacent triangles. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Triangle Edges 1/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Edges 1/2'?
Page 28	Triangle Edges 2/2
Source cue	- Midpoint line algorithm must consider inside vs. outside pixels / - Adaptation of the midpoint line algorithm / - Restriction to pixels strictly inside triangle / - Must be applied to line rasterization also for consistency reasons / - Definition of triangle inside:

Professor-style explanation	The slide 'Triangle Edges 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Midpoint line algorithm must consider inside vs. outside pixels - Adaptation of the midpoint line algorithm - Restriction to pixels strictly inside triangle - Must be applied to line rasterization also for consistency reasons - Definition of triangle inside. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Triangle Edges 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Midpoint line algorithm must consider inside vs. outside pixels - Adaptation of the midpoint line algorithm - Restriction to pixels strictly inside triangle - Must be applied to line rasterization also for consistency reasons - Definition of triangle inside. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Triangle Edges 2/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Edges 2/2'?
Page 29	Edge Rasterization 1/3
Source cue	- Triangle edge rasterization can exploit edge coherence - Two edges of triangle intersect a scanline - Same edges intersected by scanline i likely also intersect scanline $i + 1$ - Due to coherence, intersections can be calculated incrementally - For each increase of $y \rightarrow y + 1$

Professor-style explanation	The slide 'Edge Rasterization 1/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Triangle edge rasterization can exploit edge coherence - Two edges of triangle intersect a scanline - Same edges intersected by scanline i likely also intersect scanline $i + 1$ - Due to coherence, intersections can be calculated incrementally - For each increase of $y \rightarrow y + 1$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Edge Rasterization 1/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Triangle edge rasterization can exploit edge coherence - Two edges of triangle intersect a scanline - Same edges intersected by scanline i likely also intersect scanline $i + 1$ - Due to coherence, intersections can be calculated incrementally - For each increase of $y \rightarrow y + 1$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Edge Rasterization 1/3' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 1/3'?
Page 30	Edge Rasterization 2/3
Source cue	- Integer arithmetic can be used for (left) edges with slopes > 1 - Consider $dx/dy = (x_{\max} - x_{\min}) / (y_{\max} - y_{\min})$ - x is only changed by $dx/dy \Rightarrow x$ can be represented as a fraction with denominator $(y_{\max} - y_{\min})$ - Let a be the integer part

Professor-style explanation	The slide 'Edge Rasterization 2/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Integer arithmetic can be used for (left) edges with slopes > 1 / - Consider $dx dy = (x_{max} - x_{min}) / (y_{max} - y_{min})$ / - x is only changed by $dx dy$ / $\Rightarrow x$ can be represented as a fraction with denominator $(y_{max} - y_{min})$ / - Let a be the integer part. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Edge Rasterization 2/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Integer arithmetic can be used for (left) edges with slopes > 1 / - Consider $dx dy = (x_{max} - x_{min}) / (y_{max} - y_{min})$ / - x is only changed by $dx dy$ / $\Rightarrow x$ can be represented as a fraction with denominator $(y_{max} - y_{min})$ / - Let a be the integer part. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Edge Rasterization 2/3' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 2/3'?
Page 31	Edge Rasterization 3/3
Source cue	<pre>void leftEdgeScan(int x0, int y0, int x1, int y1) { // slope > 1 / int d = y1 - y0; // denominator (Nenner) / int n = x1 - x0; // numerator (Zähler) / int a = x0; // integer part of x / int b = d; // fraction part of x plus d</pre>

Professor-style explanation	The slide 'Edge Rasterization 3/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: <code>void leftEdgeScan(int x0, int y0, int x1, int y1) { // slope > 1 / int d = y1 - y0; // denominator (Nenner) / int n = x1 - x0; // numerator (Zähler) / int a = x0; // integer part of x / int b = d; // fraction part of x plus d</code>. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Edge Rasterization 3/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: <code>void leftEdgeScan(int x0, int y0, int x1, int y1) { // slope > 1 / int d = y1 - y0; // denominator (Nenner) / int n = x1 - x0; // numerator (Zähler) / int a = x0; // integer part of x / int b = d; // fraction part of x plus d</code>. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Edge Rasterization 3/3' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 3/3'?
Page 32	6.3 Region Filling
Source cue	Homogeneous coloring of regions

Professor-style explanation	The slide '6.3 Region Filling' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: Homogeneous coloring of regions. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about 6.3 Region Filling. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: Homogeneous coloring of regions. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for '6.3 Region Filling' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by '6.3 Region Filling'?
Page 33	Region Types 1/2
Source cue	- 4-connected / - All pixels of region can be reached by following movements / - Left, right, bottom, top / - 8-connected / - All pixels of region can be reached by following movements

Professor-style explanation	The slide 'Region Types 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - 4-connected / - All pixels of region can be reached by following movements / - Left, right, bottom, top / - 8-connected / - All pixels of region can be reached by following movements. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Region Types 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - 4-connected / - All pixels of region can be reached by following movements / - Left, right, bottom, top / - 8-connected / - All pixels of region can be reached by following movements. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Region Types 1/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Region Types 1/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Region Types 1/2' into a causal sentence instead of repeating the slide title?

Page 34	Region Types 2/2
Source cue	- Examples / 4-connected 8-connected 8-connected
Professor-style explanation	The slide 'Region Types 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Examples / 4-connected 8-connected 8-connected. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Region Types 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Examples / 4-connected 8-connected 8-connected. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Region Types 2/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Region Types 2/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Region Types 2/2' into a causal sentence instead of repeating the slide title?

Page 35	Region Spezifikation
Source cue	- Color-based / - Starting at pixel P, determine largest contiguous region / which pixels have a specific color / - Color assignment done by flood fill algorithms / - Border-based
Professor-style explanation	The slide 'Region Spezifikation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Color-based / - Starting at pixel P, determine largest contiguous region / which pixels have a specific color / - Color assignment done by flood fill algorithms / - Border-based. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Region Spezifikation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Color-based / - Starting at pixel P, determine largest contiguous region / which pixels have a specific color / - Color assignment done by flood fill algorithms / - Border-based. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Region Spezifikation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Region Spezifikation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.

Check yourself	Can you turn 'Region Spezifikation' into a causal sentence instead of repeating the slide title?
Page 36	Flood Fill Algorithm
Source cue	<pre>void floodFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / if (oldC == readPixel(x, y)) { // Check if the current color matches oldC / writePixel(x, y, newC); // Change the color of the pixel</pre>
Professor-style explanation	The slide 'Flood Fill Algorithm' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: void floodFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / if (oldC == readPixel(x, y)) { // Check if the current color matches oldC / writePixel(x, y, newC); // Change the color of the pixel. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Flood Fill Algorithm. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: void floodFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / if (oldC == readPixel(x, y)) { // Check if the current color matches oldC / writePixel(x, y, newC); // Change the color of the pixel. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Flood Fill Algorithm' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Flood Fill Algorithm' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Flood Fill Algorithm' into a causal sentence instead of repeating the slide title?
Page 37	Boundary Fill Algorithm
Source cue	<pre>void boundaryFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / Color c = readPixel(x, y); / if (c == oldC) return; // Return if pixel is a boundary pixel</pre>

Professor-style explanation	The slide 'Boundary Fill Algorithm' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: <code>void boundaryFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / Color c = readPixel(x, y); / if (c == oldC) return; // Return if pixel is a boundary pixel. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.</code>
Technical commentary	This slide is about Boundary Fill Algorithm. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: <code>void boundaryFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / Color c = readPixel(x, y); / if (c == oldC) return; // Return if pixel is a boundary pixel. The slide groups them because one object or quantity is processed into another result used later in rendering.</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Boundary Fill Algorithm' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Boundary Fill Algorithm' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Boundary Fill Algorithm' into a causal sentence instead of repeating the slide title?
Page 38	Fill Algorithms
Source cue	- High number of recursions can lead to stack overflow / - Use smarter data structures, i.e., queues and stacks / Queue Stack

Professor-style explanation	The slide 'Fill Algorithms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - High number of recursions can lead to stack overflow / - Use smarter data structures, i.e., queues and stacks / Queue Stack. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Fill Algorithms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - High number of recursions can lead to stack overflow / - Use smarter data structures, i.e., queues and stacks / Queue Stack. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Fill Algorithms' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Fill Algorithms' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Fill Algorithms' into a causal sentence instead of repeating the slide title?
Page 39	6.4 Scanline-Based Triangle Rasterization
Source cue	Segment-based rasterization of the simplest type of polygon

Professor-style explanation	The slide '6.4 Scanline-Based Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: Segment-based rasterization of the simplest type of polygon. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about 6.4 Scanline-Based Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: Segment-based rasterization of the simplest type of polygon. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for '6.4 Scanline-Based Triangle Rasterization' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by '6.4 Scanline-Based Triangle Rasterization'?
Page 40	Triangle Rasterization
Source cue	- Triangle rasterization / - Rasterize filled triangles is main task in real-time rendering / - Realization through graphics hardware desired / - General polygons split into triangles through tessellation / - Rasterization requires three steps

Professor-style explanation	The slide 'Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Triangle rasterization - Rasterize filled triangles is main task in real-time rendering - Realization through graphics hardware desired - General polygons split into triangles through tessellation Rasterization requires three steps. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Triangle rasterization - Rasterize filled triangles is main task in real-time rendering - Realization through graphics hardware desired - General polygons split into triangles through tessellation Rasterization requires three steps. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Triangle Rasterization' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Rasterization'?
Page 41	Scanline-Based Rasterization
Source cue	- Rasterization through scanline conversion exploits spatial coherence - Pixels of a scanline are colored similarly - Adjacent scanlines span similar pixels - Adjacent scanlines are colored similarly - Scanline segments can be calculated

Professor-style explanation	The slide 'Scanline-Based Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Rasterization through scanline conversion exploits spatial coherence / - Pixels of a scanline are colored similarly / - Adjacent scanlines span similar pixels / - Adjacent scanlines are colored similarly / - Scanline segments can be calculated. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Scanline-Based Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Rasterization through scanline conversion exploits spatial coherence / - Pixels of a scanline are colored similarly / - Adjacent scanlines span similar pixels / - Adjacent scanlines are colored similarly / - Scanline segments can be calculated. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Scanline-Based Rasterization' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Scanline-Based Rasterization'?
Page 42	Algorithm Overview
Source cue	- Observation: triangle can be split into top and bottom half / - For each half two edge pixels and one segment exist / - Segments differ only slightly, depending on edge slope / - Procedure / - Sort triangle vertices based on y-coordinate

Professor-style explanation	The slide 'Algorithm Overview' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Observation: triangle can be split into top and bottom half - For each half two edge pixels and one segment exist - Segments differ only slightly, depending on edge slope - Procedure - Sort triangle vertices based on y-coordinate. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Algorithm Overview. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Observation: triangle can be split into top and bottom half - For each half two edge pixels and one segment exist - Segments differ only slightly, depending on edge slope - Procedure - Sort triangle vertices based on y-coordinate. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Algorithm Overview' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Algorithm Overview' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Algorithm Overview'?
Page 43	Slope Calculation
Source cue	- Calculate slope for each triangle edge: $1/A = dx/dy$ - $\Delta = (x - x_0)/(y - y_0)$ - $\Delta = (x_1 - x_0)/(y_1 - y_0)$

Professor-style explanation	The slide 'Slope Calculation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Calculate slope for each triangle edge: $1/A = dx/dy$ / $-\Delta = (x - x_0)/(y - y_0)$ / $0.2 \ 0.2 \ 0.2$ / $-\Delta = (x - x_0)/(y - y_0)$ / $0.1 \ 1.0 \ 1.0$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Slope Calculation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Calculate slope for each triangle edge: $1/A = dx/dy$ / $-\Delta = (x - x_0)/(y - y_0)$ / $0.2 \ 0.2 \ 0.2$ / $-\Delta = (x - x_0)/(y - y_0)$ / $0.1 \ 1.0 \ 1.0$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Slope Calculation' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Slope Calculation'?
Page 44	Pseudo Code
Source cue	<pre>void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) { float XLeft = x0; // Start position for left x / float XRight = x0; // Start position for right x / int Y = y0; // Start position for y / float DeltaLeft = delta02; // Delta for left x for bottom half</pre>

Professor-style explanation	The slide 'Pseudo Code' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: <pre>void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) { / float XLeft = x0; // Start position for left x / float XRight = x0; // Start position for right x / int Y = y0; // Start position for y / float DeltaLeft = delta02; // Delta for left x for bottom half. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.</pre>
Technical commentary	This slide is about Pseudo Code. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: <pre>void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) { / float XLeft = x0; // Start position for left x / float XRight = x0; // Start position for right x / int Y = y0; // Start position for y / float DeltaLeft = delta02; // Delta for left x for bottom half. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.</pre>
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Pseudo Code' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Pseudo Code'?
Page 45	Segment Rasterization
Source cue	- Each segment generates fragments / - Rendering pipeline processes generated fragments / (x , y) / (e.g., depth testing, alpha blending) right s / (x , y)

Professor-style explanation	The slide 'Segment Rasterization' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. The terms are successive representations in object-based rendering: - Each segment generates fragments - Rendering pipeline processes generated fragments (x, y) (e.g., depth testing, alpha blending) right s (x, y). Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter. For examination purposes, the important content is the representation change at each stage: model data, vertices, primitives, fragments, fragment tests, and final framebuffer updates.
Technical commentary	This slide is about Segment Rasterization. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. The terms are successive representations in object-based rendering: - Each segment generates fragments - Rendering pipeline processes generated fragments (x, y) (e.g., depth testing, alpha blending) right s (x, y). Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels.
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Exam-grade answer	A strong answer for 'Segment Rasterization' states what representation enters the stage, what representation leaves it, and which later stage depends on that output.
Common trap	Do not collapse 'Segment Rasterization' into 'the GPU draws it'; name the intermediate representations because that is where most errors and exam distinctions appear.
Check yourself	Can you name the input and output representation for 'Segment Rasterization' in the rendering pipeline?
Page 46	Vertex Attribute Interpolation
Source cue	- During rasterization vertex attributes (e.g., color, depth) (x, y, z) must be interpolated over triangle (x, y, z) right s right - Bilinear interpolation can be used

Professor-style explanation	The slide 'Vertex Attribute Interpolation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - During rasterization vertex attributes (e.g., color, depth) / must be interpolated over triangle / (x , y , z) / right s right / - Bilinear interpolation can be used. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Vertex Attribute Interpolation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - During rasterization vertex attributes (e.g., color, depth) / must be interpolated over triangle / (x , y , z) / right s right / - Bilinear interpolation can be used. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Vertex Attribute Interpolation' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Vertex Attribute Interpolation'?
Page 47	Incremental Interpolation 1/3
Source cue	- Spatial coherence allows for an incremental attribute interpolation / - Example: consider triangle's plane equation for calculating z increments / - $Ax + By + Cz + D = 0$ / - With plane coefficients A, B, C: $A, B, C = V - V \times (V - V) / 2 \ 0 \ 1 \ 2$

Professor-style explanation	The slide 'Incremental Interpolation 1/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Spatial coherence allows for an incremental attribute interpolation - Example: consider triangle's plane equation for calculating z increments - $Ax + By + Cz + D = 0$ - With plane coefficients A, B, C: $A, B, C = V - V \times (V - V) / 2 \ 0 \ 1 \ 2$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Incremental Interpolation 1/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Spatial coherence allows for an incremental attribute interpolation - Example: consider triangle's plane equation for calculating z increments - $Ax + By + Cz + D = 0$ - With plane coefficients A, B, C: $A, B, C = V - V \times (V - V) / 2 \ 0 \ 1 \ 2$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Incremental Interpolation 1/3' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Incremental Interpolation 1/3'?
Page 48	Incremental Interpolation 2/3
Source cue	- Increments $dzdx$ and $dzdy$ are used to incrementally modify given z value, / i.e., when moving from (x, y) to $(x + 1, y)$ or $(x, y + 1)$ - Case 1: next fragment in scanline (horizontal change by +1) / - $x \rightarrow x \Rightarrow z \blacksquare z + dzdx / i \ i+1 \ i$

Professor-style explanation	The slide 'Incremental Interpolation 2/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Increments $dzdx$ and $dzdy$ are used to incrementally modify given z value, / i.e., when moving from (x, y) to $(x + 1, y)$ or $(x, y + 1)$ / - Case 1: next fragment in scanline (horizontal change by +1) / - $x \rightarrow x \Rightarrow z \blacksquare z + dzdx / i + 1$ i. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Incremental Interpolation 2/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Increments $dzdx$ and $dzdy$ are used to incrementally modify given z value, / i.e., when moving from (x, y) to $(x + 1, y)$ or $(x, y + 1)$ / - Case 1: next fragment in scanline (horizontal change by +1) / - $x \rightarrow x \Rightarrow z \blacksquare z + dzdx / i + 1$ i. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Incremental Interpolation 2/3' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Incremental Interpolation 2/3'?
Page 49	Incremental Interpolation 3/3
Source cue	- All vertex attributes can be interpolated incrementally analogous to depth / - Colors (r, g, b, a) / - Texture coordinates (u, v) / - ... / - For each attribute θ define

Professor-style explanation	The slide 'Incremental Interpolation 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - All vertex attributes can be interpolated incrementally analogous to depth / - Colors (r, g, b, a) / - Texture coordinates (u, v) / - ... / - For each attribute θ define. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Incremental Interpolation 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - All vertex attributes can be interpolated incrementally analogous to depth / - Colors (r, g, b, a) / - Texture coordinates (u, v) / - ... / - For each attribute θ define. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Incremental Interpolation 3/3' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Incremental Interpolation 3/3' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Incremental Interpolation 3/3'?
Page 50	6.5 Tile-Based Triangle Rasterization
Source cue	Parallel tile-based triangle rasterization

Professor-style explanation	The slide '6.5 Tile-Based Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: Parallel tile-based triangle rasterization. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about 6.5 Tile-Based Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: Parallel tile-based triangle rasterization. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for '6.5 Tile-Based Triangle Rasterization' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by '6.5 Tile-Based Triangle Rasterization'?
Page 51	Tile-Based Triangle Rasterization
Source cue	- Modern GPUs have a block-wise structure, / where one processor is responsible for one screen tile of adjacent pixels / - Segment-based triangled rasterization is difficult to parallelize / - Tile-based rasterization is a better fit with modern GPUs / - Advantage

Professor-style explanation	The slide 'Tile-Based Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Modern GPUs have a block-wise structure, / where one processor is responsible for one screen tile of adjacent pixels / - Segment-based triangled rasterization is difficult to parallelize / - Tile-based rasterization is a better fit with modern GPUs / - Advantage. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Tile-Based Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Modern GPUs have a block-wise structure, / where one processor is responsible for one screen tile of adjacent pixels / - Segment-based triangled rasterization is difficult to parallelize / - Tile-based rasterization is a better fit with modern GPUs / - Advantage. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Tile-Based Triangle Rasterization' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Tile-Based Triangle Rasterization'?
Page 52	Edge-Based Triangle Representation 1/2
Source cue	- Edge-based area representation is possible through implicit edge functions / - Implicit edge function can be defined based on two edge vertices / $V = (x, y)$ and $V = (x, y) / 0 \ 0 \ 0 \ 1 \ 1 \ 1$ / - For a given coordinate (x, y), the implicit edge function is defined as

Professor-style explanation	The slide 'Edge-Based Triangle Representation 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Edge-based area representation is possible through implicit edge functions / - Implicit edge function can be defined based on two edge vertices / $V = (x, y)$ and $V = (x, y) / 0 \ 0 \ 0 \ 1 \ 1 \ 1$ / - For a given coordinate (x, y), the implicit edge function is defined as. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Edge-Based Triangle Representation 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Edge-based area representation is possible through implicit edge functions / - Implicit edge function can be defined based on two edge vertices / $V = (x, y)$ and $V = (x, y) / 0 \ 0 \ 0 \ 1 \ 1 \ 1$ / - For a given coordinate (x, y), the implicit edge function is defined as. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Edge-Based Triangle Representation 1/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Edge-Based Triangle Representation 1/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Edge-Based Triangle Representation 1/2'?
Page 53	Edge-Based Triangle Representation 2/2
Source cue	- To represent a triangle, three implicit edge functions are combined / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0$ / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 1 \ 2 \ 2 \ 1 \ 2 \ 1 \ 1 \ 2 \ 2 \ 1$

Professor-style explanation	The slide 'Edge-Based Triangle Representation 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - To represent a triangle, three implicit edge functions are combined / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 01\ 1\ 0\ 1\ 0\ 01\ 1\ 0$ / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 12\ 2\ 1\ 2\ 1\ 12\ 2\ 1$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Edge-Based Triangle Representation 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - To represent a triangle, three implicit edge functions are combined / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 01\ 1\ 0\ 1\ 0\ 01\ 1\ 0$ / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 12\ 2\ 1\ 2\ 1\ 12\ 2\ 1$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Edge-Based Triangle Representation 2/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Edge-Based Triangle Representation 2/2'?
Page 54	Rasterization Procedure
Source cue	1. Determine tiles which are (partially) covering triangle / (e.g., easiest approximation would be bounding box) / 2. For each covering tile / - For each pixel / - If all implicit edge functions evaluate to < 0 2

Professor-style explanation	The slide 'Rasterization Procedure' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: 1. Determine tiles which are (partially) covering triangle / (e.g., easiest approximation would be bounding box) / 2. For each covering tile / - For each pixel / - If all implicit edge functions evaluate to < 0 2. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Rasterization Procedure. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: 1. Determine tiles which are (partially) covering triangle / (e.g., easiest approximation would be bounding box) / 2. For each covering tile / - For each pixel / - If all implicit edge functions evaluate to < 0 2. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Rasterization Procedure' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization Procedure'?
Page 55	between adjacent triangles
Source cue	- Pixel-based displays require the rasterization of primitives / - Rasterization fidelity is critical for image quality / - Rasterization is time-critical and should be optimized for hardware / - Focus on triangles due to speed and ease of tessellation of other polygons / - Segment-based triangle rasterization avoids overlap

Professor-style explanation	The slide 'between adjacent triangles' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Pixel-based displays require the rasterization of primitives / - Rasterization fidelity is critical for image quality / - Rasterization is time-critical and should be optimized for hardware / - Focus on triangles due to speed and ease of tessellation of other polygons / - Segment-based triangle rasterization avoids overlap. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about between adjacent triangles. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Pixel-based displays require the rasterization of primitives / - Rasterization fidelity is critical for image quality / - Rasterization is time-critical and should be optimized for hardware / - Focus on triangles due to speed and ease of tessellation of other polygons / - Segment-based triangle rasterization avoids overlap. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'between adjacent triangles' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'between adjacent triangles'?
Page 56	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Literature and other sources used in this chapter' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Literature and other sources used in this chapter' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 57	Practice, Addison-Wesley.
Source cue	- Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Research Publications / - Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter.

Professor-style explanation	The slide 'Practice, Addison-Wesley.' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Research Publications / - Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.
Technical commentary	This slide is about Practice, Addison-Wesley.. The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Research Publications / - Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Practice, Addison-Wesley.' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Practice, Addison-Wesley.' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Practice, Addison-Wesley.' points to for deeper study?
Page 58	Slide for Future: Sustainability in CG (@ EG25)
Source cue	also @ IEEE VR
Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: also @ IEEE VR. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: also @ IEEE VR. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Slide for Future: Sustainability in CG (@ EG25)' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Slide for Future: Sustainability in CG (@ EG25)' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?

Page 59	Slide for Future: Sustainability in CG (@ EG25)
Source cue	Unoptimized Optimized Unoptimized Optimized / Inverse Simulation of Radiative Thermal Transport / Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn
Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Unoptimized Optimized Unoptimized Optimized / Inverse Simulation of Radiative Thermal Transport / Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Unoptimized Optimized Unoptimized Optimized / Inverse Simulation of Radiative Thermal Transport / Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Slide for Future: Sustainability in CG (@ EG25)' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.

Common trap	Do not answer 'Slide for Future: Sustainability in CG (@ EG25)' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?
Page 60	Slide for Future: Sustainability in CG (@ EG25)
Source cue	Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints / Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi
Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints / Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints / Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Slide for Future: Sustainability in CG (@ EG25)' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Slide for Future: Sustainability in CG (@ EG25)' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?
Page 61	Slide for Future: Sustainability in CG (@ EG25)
Source cue	Stress-Aligned Hexahedral Lattice Structures / Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann

Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Stress-Aligned Hexahedral Lattice Structures / Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Stress-Aligned Hexahedral Lattice Structures / Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Slide for Future: Sustainability in CG (@ EG25)' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Slide for Future: Sustainability in CG (@ EG25)' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?

06.1 Line Rasterization

Source location: Section 6.1

Commentary

A mathematical line has infinitely many points, but a raster display has a finite grid. Line rasterization decides which grid cells or samples best approximate the ideal line. The algorithm must balance accuracy, speed, and consistency.

Incremental algorithms avoid recomputing expensive formulas for every pixel. The important idea is to step along one axis and update an error term that decides when to step along the other axis.

Mental model

Rasterizing a line means approximating a continuous path by grid decisions.

Check yourself

Why are incremental error updates useful in line rasterization?

06.2 Triangle Edge Rasterization

Source location: Section 6.2

Commentary

Triangles are the core primitive in real-time rendering. Edge functions or half-space tests decide whether a sample lies inside the triangle. This allows the rasterizer to test coverage systematically.

Coverage is not final visibility. A covered sample becomes a fragment candidate. Depth testing, stencil testing, blending, and other operations still decide whether the framebuffer changes.

Mental model

Triangle rasterization answers the question: which samples are inside this projected triangle?

Check yourself

What later stage can reject a fragment after triangle coverage succeeds?

06.3 Region Filling

Source location: Section 6.3

Commentary

Region filling generalizes the idea of determining interior samples. The algorithm must identify connected areas or spans that should receive color. This connects rasterization to scan conversion and image-space reasoning.

The practical issue is avoiding gaps, double fills, or inconsistent boundary handling. Small choices at edges can become visible artifacts when many primitives meet.

Mental model

Filling is about turning boundary descriptions into consistent interior samples.

Check yourself

Why can inconsistent edge rules create cracks between adjacent primitives?

06.4 Scanline-Based Triangle Rasterization

Source location: Section 6.4

Commentary

Scanline rasterization processes horizontal rows of the image. For each row that crosses a triangle, the algorithm finds the covered interval and fills the span. Attribute interpolation can be updated along edges and across each span.

The method is intuitive because it follows the memory layout of many images. It also demonstrates why interpolation is tied to rasterization: once you know a sample location inside the primitive, you can interpolate depth, color, normals, or texture coordinates.

Mental model

Each scanline asks: where does this triangle enter and leave this row?

Check yourself

Which attributes might be interpolated while filling triangle spans?

06.5 Tile-Based Triangle Rasterization

Source location: Section 6.5

Commentary

Tile- or block-based approaches group pixels into small regions. This can improve locality and parallel work distribution. Modern GPUs often reason in blocks or tiles internally because rendering is massively parallel.

For the exam, connect this to efficiency. The mathematical goal is still coverage, but the implementation is organized to use hardware resources well.

Mental model

Block-based rasterization keeps the same coverage problem but changes the work organization.

Check yourself

Why is block organization attractive for parallel graphics hardware?

End-of-lecture summary

If you remember only one thing from Lecture 06, remember this: This lecture explains how continuous primitives become discrete fragments on a grid. Rasterization is the bridge from geometric descriptions to sample-based image generation.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 07 - Visibility Determination

This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones should affect the image.

Source chunk	course_text_parts/03_lectures/07_visibility-determination.txt
Extracted slide pages	69
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.

Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Visibility Determination
Source cue	- Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects
Professor-style explanation	The slide 'Visibility Determination' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Visibility Determination. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Visibility Determination' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Visibility Determination' works per object, per image region, per ray, or per fragment?
Page 3	Object-Based Algorithms
Source cue	- Geometry-based analysis of scene objects before rasterization / - Compare each scene object against all other scene objects / - Determine visible parts based on virtual camera / - Rasterize the visible scene parts / For each object in sceneObjects

Professor-style explanation	The slide 'Object-Based Algorithms' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Geometry-based analysis of scene objects before rasterization - Compare each scene object against all other scene objects - Determine visible parts based on virtual camera - Rasterize the visible scene parts For each object in sceneObjects. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Object-Based Algorithms. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Geometry-based analysis of scene objects before rasterization - Compare each scene object against all other scene objects - Determine visible parts based on virtual camera - Rasterize the visible scene parts For each object in sceneObjects. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Object-Based Algorithms' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Object-Based Algorithms'?
Page 4	Image-Based Algorithms
Source cue	- Pixel-based analysis of scene objects after rasterization - Rasterize scene objects based on image resolution - Investigate for each pixel which scene object is visible For each pixel in outputRaster // Determine which scene object is visible at this pixel

Professor-style explanation	The slide 'Image-Based Algorithms' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Pixel-based analysis of scene objects after rasterization - Rasterize scene objects based on image resolution - Investigate for each pixel which scene object is visible / For each pixel in outputRaster // Determine which scene object is visible at this pixel. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Image-Based Algorithms. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Pixel-based analysis of scene objects after rasterization - Rasterize scene objects based on image resolution - Investigate for each pixel which scene object is visible / For each pixel in outputRaster // Determine which scene object is visible at this pixel. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Image-Based Algorithms' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Image-Based Algorithms'?
Page 5	7.1 Object-Based Algorithms
Source cue	7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting

Professor-style explanation	The slide '7.1 Object-Based Algorithms' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: 7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about 7.1 Object-Based Algorithms. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: 7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for '7.1 Object-Based Algorithms' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether '7.1 Object-Based Algorithms' works per object, per image region, per ray, or per fragment?
Page 6	7.1 Object-Based Algorithms
Source cue	Exploiting sorting and clipping
Professor-style explanation	The slide '7.1 Object-Based Algorithms' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: Exploiting sorting and clipping. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.

Technical commentary	This slide is about 7.1 Object-Based Algorithms. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: Exploiting sorting and clipping. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for '7.1 Object-Based Algorithms' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '7.1 Object-Based Algorithms'?

Page 7	Painter's Algorithm 1/3
Source cue	Bob Ross - Grandeur of Summer
Professor-style explanation	The slide 'Painter's Algorithm 1/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Bob Ross - Grandeur of Summer. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Painter's Algorithm 1/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Bob Ross - Grandeur of Summer. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Painter's Algorithm 1/3' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Painter's Algorithm 1/3' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Painter's Algorithm 1/3' into a causal sentence instead of repeating the slide title?

Page 8	Painter's Algorithm 2/3
--------	-------------------------

Source cue	- First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order
Professor-style explanation	The slide 'Painter's Algorithm 2/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Painter's Algorithm 2/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Painter's Algorithm 2/3' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Painter's Algorithm 2/3' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Painter's Algorithm 2/3' into a causal sentence instead of repeating the slide title?
Page 9	Painter's Algorithm 3/3
Source cue	- Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects

Professor-style explanation	The slide 'Painter's Algorithm 3/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Painter's Algorithm 3/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Painter's Algorithm 3/3' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Painter's Algorithm 3/3' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Painter's Algorithm 3/3' into a causal sentence instead of repeating the slide title?
Page 10	Weiler-Atherton Algorithm 1/3
Source cue	- Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list L in front-to-back order

Professor-style explanation	The slide 'Weiler-Atherton Algorithm 1/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list L in front-to-back order. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Weiler-Atherton Algorithm 1/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list L in front-to-back order. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Weiler-Atherton Algorithm 1/3' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 1/3'?
Page 11	Weiler-Atherton Algorithm 2/3
Source cue	- Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts

Professor-style explanation	The slide 'Weiler-Atherton Algorithm 2/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Weiler-Atherton Algorithm 2/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Weiler-Atherton Algorithm 2/3' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 2/3'?
Page 12	Weiler-Atherton Algorithm 3/3
Source cue	- Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles

Professor-style explanation	The slide 'Weiler-Atherton Algorithm 3/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.
Technical commentary	This slide is about Weiler-Atherton Algorithm 3/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'Weiler-Atherton Algorithm 3/3' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 3/3'?
Page 13	Back Face Culling 1/2
Source cue	- Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object

Professor-style explanation	The slide 'Back Face Culling 1/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Back Face Culling 1/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Back Face Culling 1/2' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Back Face Culling 1/2' works per object, per image region, per ray, or per fragment?
Page 14	Back Face Culling 2/2
Source cue	- Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process

Professor-style explanation	The slide 'Back Face Culling 2/2' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. The terms are successive representations in object-based rendering: - Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter. For examination purposes, the important content is the representation change at each stage: model data, vertices, primitives, fragments, fragment tests, and final framebuffer updates.
Technical commentary	This slide is about Back Face Culling 2/2. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. The terms are successive representations in object-based rendering: - Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process. Scene data becomes vertices, vertices become primitives, primitives become fragments, and only passing fragments update pixels.
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Exam-grade answer	A strong answer for 'Back Face Culling 2/2' states what representation enters the stage, what representation leaves it, and which later stage depends on that output.
Common trap	Do not collapse 'Back Face Culling 2/2' into 'the GPU draws it'; name the intermediate representations because that is where most errors and exam distinctions appear.
Check yourself	Can you name the input and output representation for 'Back Face Culling 2/2' in the rendering pipeline?
Page 15	Robert's Algorithm
Source cue	- First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling

Professor-style explanation	The slide 'Robert's Algorithm' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Robert's Algorithm. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Robert's Algorithm' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Robert's Algorithm'?
Page 16	Assessment Robert's Algorithm
Source cue	- Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings

Professor-style explanation	The slide 'Assessment Robert's Algorithm' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Assessment Robert's Algorithm. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Assessment Robert's Algorithm' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Assessment Robert's Algorithm'?
Page 17	7.2 Binary Space Partitioning
Source cue	Hierarchical scene bisection

Professor-style explanation	The slide '7.2 Binary Space Partitioning' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Hierarchical scene bisection. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about 7.2 Binary Space Partitioning. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Hierarchical scene bisection. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for '7.2 Binary Space Partitioning' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer '7.2 Binary Space Partitioning' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn '7.2 Binary Space Partitioning' into a causal sentence instead of repeating the slide title?

Page 18	Spatial Data Structures
Source cue	- Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes
Professor-style explanation	The slide 'Spatial Data Structures' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Spatial Data Structures. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Spatial Data Structures' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Spatial Data Structures' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Spatial Data Structures' into a causal sentence instead of repeating the slide title?
Page 19	BSP Trees 1/2
Source cue	- BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility
Professor-style explanation	The slide 'BSP Trees 1/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.

Technical commentary	This slide is about BSP Trees 1/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'BSP Trees 1/2' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'BSP Trees 1/2' works per object, per image region, per ray, or per fragment?

Page 20	BSP Trees 2/2
Source cue	- Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C
Professor-style explanation	The slide 'BSP Trees 2/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about BSP Trees 2/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.

Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'BSP Trees 2/2' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'BSP Trees 2/2' works per object, per image region, per ray, or per fragment?

Page 21	Node Data Structure
Source cue	- Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane
Professor-style explanation	The slide 'Node Data Structure' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Node Data Structure. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Node Data Structure' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Node Data Structure' works per object, per image region, per ray, or per fragment?

Page 22	BSP Tree Construction
---------	-----------------------

Source cue	- Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node
Professor-style explanation	The slide 'BSP Tree Construction' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about BSP Tree Construction. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'BSP Tree Construction' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'BSP Tree Construction' works per object, per image region, per ray, or per fragment?
Page 23	BSP Tree Construction - Example
Source cue	1 2 / 3 4 / A B E / 3 1

Professor-style explanation	The slide 'BSP Tree Construction - Example' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: 1 2 / 3 4 / A B E / 3 1. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about BSP Tree Construction - Example. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: 1 2 / 3 4 / A B E / 3 1. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'BSP Tree Construction - Example' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'BSP Tree Construction - Example' works per object, per image region, per ray, or per fragment?
Page 24	BSP Tree Construction - Pseudo Code
Source cue	Function buildBSPTree(BSPNode node, List of Polygons list) // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry

Professor-style explanation	The slide 'BSP Tree Construction - Pseudo Code' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: Function buildBSPTree(BSPNode node, List of Polygons list) / // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about BSP Tree Construction - Pseudo Code. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: Function buildBSPTree(BSPNode node, List of Polygons list) / // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'BSP Tree Construction - Pseudo Code' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'BSP Tree Construction - Pseudo Code' works per object, per image region, per ray, or per fragment?
Page 25	Visibility Determination
Source cue	- Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order"

Professor-style explanation	The slide 'Visibility Determination' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order". Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Visibility Determination. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order". Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Visibility Determination' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Visibility Determination' works per object, per image region, per ray, or per fragment?
Page 26	Visibility Determination - Pseudo Code
Source cue	<pre>Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then // Eye-point is on the back side of the plane</pre>

Professor-style explanation	The slide 'Visibility Determination - Pseudo Code' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) / // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then / // Eye-point is on the back side of the plane. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Visibility Determination - Pseudo Code. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) / // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then / // Eye-point is on the back side of the plane. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Visibility Determination - Pseudo Code' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Visibility Determination - Pseudo Code' works per object, per image region, per ray, or per fragment?
Page 27	BSP Tree Traversal - Example
Source cue	B F / 4 1 2 / F B F B / 3 4 / F B F B

Professor-style explanation	The slide 'BSP Tree Traversal - Example' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: B F / 4 1 2 / F B F B / 3 4 / F B F B. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about BSP Tree Traversal - Example. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: B F / 4 1 2 / F B F B / 3 4 / F B F B. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'BSP Tree Traversal - Example' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'BSP Tree Traversal - Example' works per object, per image region, per ray, or per fragment?
Page 28	Splitting Planes Selection
Source cue	- Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics

Professor-style explanation	The slide 'Splitting Planes Selection' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Splitting Planes Selection. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Splitting Planes Selection' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Splitting Planes Selection' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Splitting Planes Selection' into a causal sentence instead of repeating the slide title?
Page 29	Traversal Optimization
Source cue	- Sub-trees can be skipped during traversal if content is outside view frustum / - Testing is done by checking 8 corner points of 3D view frustum / - If there are two points on different sides, then the subtree is traversed / - If all points are in one subspace, then the other subspace does not have / to be traversed (view-frustum culling)

Professor-style explanation	The slide 'Traversal Optimization' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Sub-trees can be skipped during traversal if content is outside view frustum - Testing is done by checking 8 corner points of 3D view frustum - If there are two points on different sides, then the subtree is traversed - If all points are in one subspace, then the other subspace does not have to be traversed (view-frustum culling). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Traversal Optimization. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Sub-trees can be skipped during traversal if content is outside view frustum - Testing is done by checking 8 corner points of 3D view frustum - If there are two points on different sides, then the subtree is traversed - If all points are in one subspace, then the other subspace does not have to be traversed (view-frustum culling). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Traversal Optimization' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Traversal Optimization' works per object, per image region, per ray, or per fragment?
Page 30	Assessment BSP Trees
Source cue	- Advantages / - Simple implementation / - Do not need a depth buffer / - Applicable to static and dynamic scenes / (static scene geometry precalculated, dynamic objects inserted)

Professor-style explanation	The slide 'Assessment BSP Trees' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Advantages / - Simple implementation / - Do not need a depth buffer / - Applicable to static and dynamic scenes / (static scene geometry precalculated, dynamic objects inserted). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Assessment BSP Trees. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Advantages / - Simple implementation / - Do not need a depth buffer / - Applicable to static and dynamic scenes / (static scene geometry precalculated, dynamic objects inserted). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Assessment BSP Trees' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Assessment BSP Trees'?
Page 31	7.3 Warnock Algorithm
Source cue	Image-based divide-and-conquer strategy

Professor-style explanation	The slide '7.3 Warnock Algorithm' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: Image-based divide-and-conquer strategy. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about 7.3 Warnock Algorithm. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: Image-based divide-and-conquer strategy. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for '7.3 Warnock Algorithm' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether '7.3 Warnock Algorithm' works per object, per image region, per ray, or per fragment?
Page 32	Warnock Algorithm
Source cue	- Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions

Professor-style explanation	The slide 'Warnock Algorithm' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Warnock Algorithm. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Warnock Algorithm' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Warnock Algorithm' works per object, per image region, per ray, or per fragment?
Page 33	Region-Based Visibility Determination
Source cue	$R \cap P$ / Case 1 / - Given viewport region R and set of polygons P, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in P are outside of R $R \cap P = \emptyset$

Professor-style explanation	The slide 'Region-Based Visibility Determination' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: $R \cap P \neq \emptyset$ / Case 1 / - Given viewport region R and set of polygons P, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in P are outside of R $R \cap P = \emptyset$. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Region-Based Visibility Determination. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: $R \cap P \neq \emptyset$ / Case 1 / - Given viewport region R and set of polygons P, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in P are outside of R $R \cap P = \emptyset$. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Region-Based Visibility Determination' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Region-Based Visibility Determination' works per object, per image region, per ray, or per fragment?
Page 34	Procedure
Source cue	- Start with entire viewport region R / - Subdivide R into four equal subregions / - Terminate if subregion size matches pixel size / - Classify subregions based on trivial cases / - If subregion matches trivial cases: resolve

Professor-style explanation	The slide 'Procedure' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Start with entire viewport region R - Subdivide R into four equal subregions - Terminate if subregion size matches pixel size - Classify subregions based on trivial cases - If subregion matches trivial cases: resolve. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Procedure. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Start with entire viewport region R - Subdivide R into four equal subregions - Terminate if subregion size matches pixel size - Classify subregions based on trivial cases - If subregion matches trivial cases: resolve. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Procedure' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Procedure' changes positions before rasterization?
Page 35	Pseudo Code
Source cue	<pre> Procedure warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then // Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else </pre>

Professor-style explanation	The slide 'Pseudo Code' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: Procedure <pre>warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then / //</pre> Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Pseudo Code. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: Procedure <pre>warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then / //</pre> Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Pseudo Code' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Pseudo Code' works per object, per image region, per ray, or per fragment?
Page 36	Assessment Warnock Algorithm
Source cue	- Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives

Professor-style explanation	The slide 'Assessment Warnock Algorithm' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Assessment Warnock Algorithm. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Assessment Warnock Algorithm' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Assessment Warnock Algorithm' works per object, per image region, per ray, or per fragment?
Page 37	7.4 Depth Buffer Algorithm
Source cue	Exploiting image storage for visibility determination

Professor-style explanation	The slide '7.4 Depth Buffer Algorithm' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Exploiting image storage for visibility determination. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about 7.4 Depth Buffer Algorithm. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Exploiting image storage for visibility determination. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for '7.4 Depth Buffer Algorithm' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '7.4 Depth Buffer Algorithm'?
Page 38	Depth Buffer Algorithm 1/3
Source cue	- Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization

Professor-style explanation	The slide 'Depth Buffer Algorithm 1/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Depth Buffer Algorithm 1/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Depth Buffer Algorithm 1/3' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 1/3'?
Page 39	Depth Buffer Algorithm 2/3
Source cue	- Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an <code>OcclusionRepresentation OR</code> / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by <code>OR</code> Then

Professor-style explanation	The slide 'Depth Buffer Algorithm 2/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an OcclusionRepresentation OR / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by OR Then. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Depth Buffer Algorithm 2/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an OcclusionRepresentation OR / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by OR Then. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Depth Buffer Algorithm 2/3' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 2/3'?
Page 40	Depth Buffer Algorithm 3/3
Source cue	- Depth buffer algorithm performs depth testing for each fragment / - Depth buffer is initialized with background value (e.g., $z = 1$) / - For every scene object / - Rasterize scene object / - For each fragment

Professor-style explanation	The slide 'Depth Buffer Algorithm 3/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Depth buffer algorithm performs depth testing for each fragment - Depth buffer is initialized with background value (e.g., $z = 1$) - For every scene object - Rasterize scene object - For each fragment. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Depth Buffer Algorithm 3/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Depth buffer algorithm performs depth testing for each fragment - Depth buffer is initialized with background value (e.g., $z = 1$) - For every scene object - Rasterize scene object - For each fragment. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Depth Buffer Algorithm 3/3' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 3/3'?
Page 41	Pseudo Code - Depth Buffer Test
Source cue	<pre>Function zBufferHSR(List of Geometry list) // Initialize z-buffer and color buffer for each pixel on the screen / For x from 0 to Width / For y from 0 to Height / Set depthbuffer[x][y] to 1.0 // Maximum depth value</pre>

Professor-style explanation	The slide 'Pseudo Code - Depth Buffer Test' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Function <code>zBufferHSR(List of Geometry list)</code> / // Initialize z-buffer and color buffer for each pixel on the screen / For x from 0 to Width / For y from 0 to Height / Set <code>depthbuffer[x][y]</code> to 1.0 // Maximum depth value. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Pseudo Code - Depth Buffer Test. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Function <code>zBufferHSR(List of Geometry list)</code> / // Initialize z-buffer and color buffer for each pixel on the screen / For x from 0 to Width / For y from 0 to Height / Set <code>depthbuffer[x][y]</code> to 1.0 // Maximum depth value. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Pseudo Code - Depth Buffer Test' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Pseudo Code - Depth Buffer Test'?
Page 42	OpenGL Depth Buffer
Source cue	- OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code>

Professor-style explanation	The slide 'OpenGL Depth Buffer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Depth Buffer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Depth Buffer' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Depth Buffer'?
Page 43	Depth Buffer Precision
Source cue	- Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog

Professor-style explanation	The slide 'Depth Buffer Precision' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Depth Buffer Precision. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Depth Buffer Precision' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Precision'?
Page 44	Z-Fighting 1/2
Source cue	- Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene

Professor-style explanation	The slide 'Z-Fighting 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Z-Fighting 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Z-Fighting 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Z-Fighting 1/2'?
Page 45	Z-Fighting 2/2
Source cue	- Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties

Professor-style explanation	The slide 'Z-Fighting 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Z-Fighting 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Z-Fighting 2/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Z-Fighting 2/2'?
Page 46	Assessment Depth Buffering
Source cue	- Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries

Professor-style explanation	The slide 'Assessment Depth Buffering' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Assessment Depth Buffering. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Assessment Depth Buffering' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Assessment Depth Buffering'?
Page 47	7.5 Depth Buffer Extensions
Source cue	How to render semi-transparent scene objects, and how to generate halos

Professor-style explanation	The slide '7.5 Depth Buffer Extensions' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: How to render semi-transparent scene objects, and how to generate halos. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about 7.5 Depth Buffer Extensions. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: How to render semi-transparent scene objects, and how to generate halos. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for '7.5 Depth Buffer Extensions' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '7.5 Depth Buffer Extensions'?
Page 48	Semi-Transparent Objects 1/2
Source cue	- Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function draw(Vector scene) / // Clear the framebuffer to prepare for new drawing

Professor-style explanation	The slide 'Semi-Transparent Objects 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function draw(Vector scene) / // Clear the framebuffer to prepare for new drawing. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Semi-Transparent Objects 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function draw(Vector scene) / // Clear the framebuffer to prepare for new drawing. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Semi-Transparent Objects 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Semi-Transparent Objects 1/2'?
Page 49	Semi-Transparent Objects 2/2
Source cue	- Issue: correct transparency without presorting of scene objects not possible

Professor-style explanation	The slide 'Semi-Transparent Objects 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Issue: correct transparency without presorting of scene objects not possible. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Semi-Transparent Objects 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Issue: correct transparency without presorting of scene objects not possible. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Semi-Transparent Objects 2/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Semi-Transparent Objects 2/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Semi-Transparent Objects 2/2' into a causal sentence instead of repeating the slide title?
Page 50	Order-Independent Transparency 1/4
Source cue	- Order-independent transparency exploits depth peeling to avoid sorting / - Depth peeling peels away depth layers one-by-one in front-to-back order / - Extract nearest fragments with smallest z values in first rendering pass / - Extract next further fragments in each subsequent rendering pass / - The nth rendering pass delivers the fragments to the nth depth plane

Professor-style explanation	The slide 'Order-Independent Transparency 1/4' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Order-independent transparency exploits depth peeling to avoid sorting - Depth peeling peels away depth layers one-by-one in front-to-back order - Extract nearest fragments with smallest z values in first rendering pass - Extract next further fragments in each subsequent rendering pass - The nth rendering pass delivers the fragments to the nth depth plane. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Order-Independent Transparency 1/4. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Order-independent transparency exploits depth peeling to avoid sorting - Depth peeling peels away depth layers one-by-one in front-to-back order - Extract nearest fragments with smallest z values in first rendering pass - Extract next further fragments in each subsequent rendering pass - The nth rendering pass delivers the fragments to the nth depth plane. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Order-Independent Transparency 1/4' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Order-Independent Transparency 1/4' works per object, per image region, per ray, or per fragment?
Page 51	Order-Independent Transparency 2/4
Source cue	- Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1

Professor-style explanation	The slide 'Order-Independent Transparency 2/4' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Order-Independent Transparency 2/4. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Order-Independent Transparency 2/4' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Order-Independent Transparency 2/4' works per object, per image region, per ray, or per fragment?
Page 52	Order-Independent Transparency 3/4
Source cue	- 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass

Professor-style explanation	The slide 'Order-Independent Transparency 3/4' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Order-Independent Transparency 3/4. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Order-Independent Transparency 3/4' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Order-Independent Transparency 3/4'?
Page 53	Order-Independent Transparency 4/4
Source cue	1 layer 2 layers / 3 layers 4 layers

Professor-style explanation	The slide 'Order-Independent Transparency 4/4' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: 1 layer 2 layers / 3 layers 4 layers. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Order-Independent Transparency 4/4. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: 1 layer 2 layers / 3 layers 4 layers. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Order-Independent Transparency 4/4' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Order-Independent Transparency 4/4' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Order-Independent Transparency 4/4' into a causal sentence instead of repeating the slide title?
Page 54	Generating Haloes 1/2
Source cue	- Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines

Professor-style explanation	The slide 'Generating Haloes 1/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Generating Haloes 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Generating Haloes 1/2' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Generating Haloes 1/2'?
Page 55	Generating Haloes 2/2
Source cue	- Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin"

Professor-style explanation	The slide 'Generating Haloes 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin". Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Generating Haloes 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin". Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Generating Haloes 2/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Generating Haloes 2/2'?
Page 56	7.6 Ray Casting
Source cue	Shooting rays to determine visibility

Professor-style explanation	The slide '7.6 Ray Casting' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: Shooting rays to determine visibility. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about 7.6 Ray Casting. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: Shooting rays to determine visibility. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for '7.6 Ray Casting' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether '7.6 Ray Casting' works per object, per image region, per ray, or per fragment?
Page 57	Ray Casting Principle
Source cue	- Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection)

Professor-style explanation	The slide 'Ray Casting Principle' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Ray Casting Principle. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Ray Casting Principle' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Ray Casting Principle' works per object, per image region, per ray, or per fragment?
Page 58	Ray Initialization 1/3
Source cue	- Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $t / s - e / - p t = e + t s / s - e$

Professor-style explanation	The slide 'Ray Initialization 1/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $t / s - e / - p \ t = e + t \ s / s - e$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Ray Initialization 1/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $t / s - e / - p \ t = e + t \ s / s - e$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Ray Initialization 1/3' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Ray Initialization 1/3'?
Page 59	Ray Initialization 2/3
Source cue	- How can we find s? / - Assumption: we have a simplified camera model / - Positioned in e / - Orthonormal base $\{u, v, w\}$ / - Orthonormal: orthogonal + normalized

Professor-style explanation	The slide 'Ray Initialization 2/3' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - How can we find s? / - Assumption: we have a simplified camera model / - Positioned in e / - Orthonormal base $\{u, v, w\}$ / - Orthonormal: orthogonal + normalized. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Ray Initialization 2/3. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - How can we find s? / - Assumption: we have a simplified camera model / - Positioned in e / - Orthonormal base $\{u, v, w\}$ / - Orthonormal: orthogonal + normalized. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Ray Initialization 2/3' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Ray Initialization 2/3' works per object, per image region, per ray, or per fragment?
Page 60	Ray Initialization 3/3
Source cue	- Definition of the pixel grid with $n \times n$ pixels / $x\ y$ / - Size of the pixel raster is $r - l \times t - b$ / - Pixel spacing is / - x-coordinate: $(r - l)/n$

Professor-style explanation	The slide 'Ray Initialization 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Definition of the pixel grid with $n \times n$ pixels / $x\ y$ / - Size of the pixel raster is $r - l \times t - b$ / - Pixel spacing is / - x-coordinate: $(r - l)/n$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Ray Initialization 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Definition of the pixel grid with $n \times n$ pixels / $x\ y$ / - Size of the pixel raster is $r - l \times t - b$ / - Pixel spacing is / - x-coordinate: $(r - l)/n$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Ray Initialization 3/3' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Ray Initialization 3/3' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Ray Initialization 3/3'?
Page 61	Intersection Calculation
Source cue	- Find smallest positive t for which $p(t)$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest t

Professor-style explanation	The slide 'Intersection Calculation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Find smallest positive t for which $p(t)$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest t . The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Intersection Calculation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Find smallest positive t for which $p(t)$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest t . The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Intersection Calculation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Intersection Calculation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Intersection Calculation' into a causal sentence instead of repeating the slide title?

Page 62	Implicit Surface Intersection Calculation
Source cue	- Find set of all points with $f(x, y, z) = 0$ / - Examples: plane, sphere, quadrics, ... / $n = (A, B, C)$ / $c = (x, y, z)$ / $c \cdot c$
Professor-style explanation	The slide 'Implicit Surface Intersection Calculation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Find set of all points with $f(x, y, z) = 0$ / - Examples: plane, sphere, quadrics, ... / $n = (A, B, C)$ / $c = (x, y, z)$ / $c \cdot c$. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Implicit Surface Intersection Calculation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Find set of all points with $f(x, y, z) = 0$ / - Examples: plane, sphere, quadrics, ... / $n = (A, B, C)$ / $c = (x, y, z)$ / $c \cdot c$. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Implicit Surface Intersection Calculation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Implicit Surface Intersection Calculation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Implicit Surface Intersection Calculation' into a causal sentence instead of repeating the slide title?

Page 63	Intersection Calculation - Triangle
Source cue	- Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates
Professor-style explanation	The slide 'Intersection Calculation - Triangle' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.

Technical commentary	This slide is about Intersection Calculation - Triangle. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Intersection Calculation - Triangle' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Intersection Calculation - Triangle' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Intersection Calculation - Triangle'?

Page 64	Pseudo Code - Analytic Intersection Computation
Source cue	Function castRay(List sceneObjects, Window win, Camera cam) // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y)
Professor-style explanation	The slide 'Pseudo Code - Analytic Intersection Computation' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: Function castRay(List sceneObjects, Window win, Camera cam) // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.

Technical commentary	This slide is about Pseudo Code - Analytic Intersection Computation. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: Function castRay(List sceneObjects, Window win, Camera cam) / // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Pseudo Code - Analytic Intersection Computation' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Pseudo Code - Analytic Intersection Computation' works per object, per image region, per ray, or per fragment?

Page 65	Pseudo Code - Iterative Intersection Calculation (Ray Marching)
Source cue	- Ray march in ϵ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter / // Loop to incrementally trace the ray through the scene
Professor-style explanation	The slide 'Pseudo Code - Iterative Intersection Calculation (Ray Marching)' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Ray march in ϵ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter / // Loop to incrementally trace the ray through the scene. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.

Technical commentary	This slide is about Pseudo Code - Iterative Intersection Calculation (Ray Marching). The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Ray march in ϵ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter // Loop to incrementally trace the ray through the scene. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Pseudo Code - Iterative Intersection Calculation (Ray Marching)' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Pseudo Code - Iterative Intersection Calculation (Ray Marching)' works per object, per image region, per ray, or per fragment?

Page 66	Assessment Ray Casting
Source cue	- Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages
Professor-style explanation	The slide 'Assessment Ray Casting' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.

Technical commentary	This slide is about Assessment Ray Casting. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Assessment Ray Casting' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Assessment Ray Casting' works per object, per image region, per ray, or per fragment?

Page 67	but are crucial for accurate visibility determination
Source cue	- Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable
Professor-style explanation	The slide 'but are crucial for accurate visibility determination' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion. For examination purposes, the important content is classification: inside, outside, crossing, intersection point, and the newly produced primitive segment or polygon.

Technical commentary	This slide is about but are crucial for accurate visibility determination. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Exam-grade answer	A strong answer for 'but are crucial for accurate visibility determination' classifies geometry relative to the boundary, explains intersection creation, and names the geometry passed to rasterization.
Common trap	Do not describe clipping as only deletion; crossing primitives can be cut and replaced by new vertices or primitive pieces.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'but are crucial for accurate visibility determination'?

Page 68	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.

Exam-grade answer	A strong answer for 'Literature and other sources used in this chapter' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Literature and other sources used in this chapter' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?

Page 69	Practice, Addison-Wesley.
Source cue	- Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,
Professor-style explanation	The slide 'Practice, Addison-Wesley.' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.
Technical commentary	This slide is about Practice, Addison-Wesley.. The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Practice, Addison-Wesley.' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Practice, Addison-Wesley.' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Practice, Addison-Wesley.' points to for deeper study?

07.1 Object-Based Algorithms

Source location: Section 7.1

Commentary

Object-based visibility algorithms reason about geometry before or while comparing surfaces. They can sort, split, or reject objects based on spatial relations. This is different from simply letting every fragment fight in the depth buffer.

The value of object-based reasoning is reducing work or establishing correct order. The difficulty is that geometry can overlap in complicated ways, so simple global ordering is not always possible.

Mental model

Object-based visibility tries to solve visibility with geometry before final pixels are written.

Check yourself

Why can intersecting polygons make simple depth sorting fail?

07.2 Binary Space Partitioning

Source location: Section 7.2

Commentary

A BSP tree recursively divides space with planes. Once built, it can help traverse geometry in a view-dependent order. This is useful for visibility and ordering because the tree encodes spatial relationships.

The cost is preprocessing and possible splitting of geometry. BSP is a good example of trading memory and setup work for faster or more structured visibility decisions later.

Mental model

A BSP tree stores a recursive answer to 'which side of this plane is the geometry on?'

Check yourself

Why can building a BSP tree require splitting polygons?

07.3 Warnock Algorithm

Source location: Section 7.3

Commentary

Warnock's algorithm works in image space by subdividing regions until visibility is simple enough to decide. If a region is ambiguous, split it. If it is simple, fill it.

This illustrates a general graphics strategy: recursively reduce a hard global problem into smaller local problems. The algorithm is less central in modern OpenGL practice than the depth buffer, but it helps contrast image-space and object-space approaches.

Mental model

When a region is too complicated, subdivide until the answer becomes simple.

Check yourself

What makes Warnock's algorithm image-space rather than object-space?

07.4 Depth Buffer Algorithm

Source location: Section 7.4

Commentary

The depth buffer stores the closest accepted depth at each sample or pixel. For each fragment, compare its depth to the stored depth. If it passes, update the color and depth; otherwise discard it.

The strength of the depth buffer is simplicity and hardware efficiency. The weakness is precision: depth values are finite, and projection distributes precision unevenly. Bad near/far settings can create z-fighting.

Mental model

The depth buffer is a per-sample competition for closest visible fragment.

Check yourself

Why should the near plane not be unnecessarily close to the camera?

07.5 Depth Extensions and Ray Casting

Source location: Sections 7.5-7.6

Commentary

Depth buffer extensions refine or adapt depth-based visibility, for example by handling precision, transparency, or multiple layers more carefully. Standard depth testing alone is not a complete solution for every visibility situation.

Ray casting takes the opposite direction from rasterization: for each image sample, cast a ray into the scene and find the closest intersection. This naturally solves primary visibility but requires intersection work instead of raster coverage work.

Mental model

Rasterization pushes primitives to pixels; ray casting pulls visibility from pixels into the scene.

Check yourself

Why is transparency harder than opaque nearest-surface visibility?

End-of-lecture summary

If you remember only one thing from Lecture 07, remember this: This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones should affect the image.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 08 - Local Illumination

This lecture explains how visible surface points receive color from local light, material, normal, and view information. It is about shading a point, not solving all global light transport.

Source chunk	course_text_parts/03_lectures/08_local-illumination.txt
Extracted slide pages	62
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.

Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Local Illumination
Source cue	- Illumination is the process of simulating light interactions in a virtual scene / - Requires light and material properties / - Achieved through illumination model / - Essential for scene perception / without illumination with illumination
Professor-style explanation	The slide 'Local Illumination' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Illumination is the process of simulating light interactions in a virtual scene / - Requires light and material properties / - Achieved through illumination model / - Essential for scene perception / without illumination with illumination. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Local Illumination. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Illumination is the process of simulating light interactions in a virtual scene / - Requires light and material properties / - Achieved through illumination model / - Essential for scene perception / without illumination with illumination. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Local Illumination' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Local Illumination'?
Page 3	8.1 Physics of Light
Source cue	8.2 Light Sources / 8.3 Material Models / 8.4 Phong Illumination Model / 8.5 Shading

Professor-style explanation	The outline '8.1 Physics of Light' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: 8.2 Light Sources / 8.3 Material Models / 8.4 Phong Illumination Model / 8.5 Shading. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about 8.1 Physics of Light. The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: 8.2 Light Sources / 8.3 Material Models / 8.4 Phong Illumination Model / 8.5 Shading. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for '8.1 Physics of Light' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize '8.1 Physics of Light' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where '8.1 Physics of Light' fits in the lecture order and what later section depends on it?
Page 4	8.1 Physics of Light
Source cue	Understanding light in the real world

Professor-style explanation	The slide '8.1 Physics of Light' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: Understanding light in the real world. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about 8.1 Physics of Light. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: Understanding light in the real world. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for '8.1 Physics of Light' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '8.1 Physics of Light'?
Page 5	Electromagnetic Radiation
Source cue	- Electromagnetic radiation is defined by a spectrum of wavelengths / Wave length λ / - Visible light is subset of electromagnetic spectrum between 400 and 700nm, / which humans perceive as colors

Professor-style explanation	The slide 'Electromagnetic Radiation' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Electromagnetic radiation is defined by a spectrum of wavelengths / Wave length λ / - Visible light is subset of electromagnetic spectrum between 400 and 700nm, / which humans perceive as colors. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Electromagnetic Radiation. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Electromagnetic radiation is defined by a spectrum of wavelengths / Wave length λ / - Visible light is subset of electromagnetic spectrum between 400 and 700nm, / which humans perceive as colors. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Electromagnetic Radiation' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Electromagnetic Radiation'?
Page 6	Visible Light
Source cue	1000 nm / Infrared / Radio / 700 nm / TV Red

Professor-style explanation	The slide 'Visible Light' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: 1000 nm / Infrared / Radio / 700 nm / TV Red. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Visible Light. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: 1000 nm / Infrared / Radio / 700 nm / TV Red. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Visible Light' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Visible Light'?

Page 7	Visible Light
Source cue	1000 nm / Infrared / Radio / 700 nm / TV Red
Professor-style explanation	The slide 'Visible Light' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: 1000 nm / Infrared / Radio / 700 nm / TV Red. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.

Technical commentary	This slide is about Visible Light. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: 1000 nm / Infrared / Radio / 700 nm / TV Red. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Visible Light' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Visible Light'?

Page 8	Light Spectra
Source cue	- Different light sources have different light spectra / - Intensity differs based on wavelength in visible spectrum / [http://www.micro.magnet.fsu.edu/optics/lightandcolor/sources.html]
Professor-style explanation	The slide 'Light Spectra' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Different light sources have different light spectra / - Intensity differs based on wavelength in visible spectrum / [http://www.micro.magnet.fsu.edu/optics/lightandcolor/sources.html]. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Light Spectra. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Different light sources have different light spectra / - Intensity differs based on wavelength in visible spectrum / [http://www.micro.magnet.fsu.edu/optics/lightandcolor/sources.html]. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.

Exam-grade answer	A strong answer for 'Light Spectra' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Spectra'?

Page 9	Black-Body Radiation
Source cue	- A continuous spectrum emitted by a black body at a certain temperature / Steel / > 1570 K / 820 K / Stars
Professor-style explanation	The slide 'Black-Body Radiation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - A continuous spectrum emitted by a black body at a certain temperature / Steel / > 1570 K / 820 K / Stars. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Black-Body Radiation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - A continuous spectrum emitted by a black body at a certain temperature / Steel / > 1570 K / 820 K / Stars. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Black-Body Radiation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Black-Body Radiation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Black-Body Radiation' into a causal sentence instead of repeating the slide title?

Page 10	Black-Body Radiation
Source cue	Stars / https://www.tec-science.com/thermodynamics/temperature/black-body-radiation/ 23/06/2026 10

Professor-style explanation	The slide 'Black-Body Radiation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Stars / https://www.tec-science.com/thermodynamics/temperature/black-body-radiation/ 23/06/2026 10. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Black-Body Radiation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Stars / https://www.tec-science.com/thermodynamics/temperature/black-body-radiation/ 23/06/2026 10. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Black-Body Radiation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Black-Body Radiation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Black-Body Radiation' into a causal sentence instead of repeating the slide title?
Page 11	Spectral Density Function (SDF)
Source cue	- Light spectra are represented as spectral density functions / - Spectral density functions provide the light intensity (=energy) per wavelength / 400 Wavelength λ (nm) 700 / ygrenE / 400 Wavelength λ (nm) 700

Professor-style explanation	The slide 'Spectral Density Function (SDF)' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Light spectra are represented as spectral density functions / - Spectral density functions provide the light intensity (=energy) per wavelength / 400 Wavelength λ (nm) 700 / ygrenE / 400 Wavelength λ (nm) 700. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Spectral Density Function (SDF). The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Light spectra are represented as spectral density functions / - Spectral density functions provide the light intensity (=energy) per wavelength / 400 Wavelength λ (nm) 700 / ygrenE / 400 Wavelength λ (nm) 700. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Spectral Density Function (SDF)' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Spectral Density Function (SDF)'?
Page 12	Light Perception
Source cue	- Humans perceive light spectra / as differently colored lights / - Often described using three / main properties (HSL/HSV model) / - Hue - primary color tone

Professor-style explanation	The slide 'Light Perception' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Humans perceive light spectra / as differently colored lights / - Often described using three / main properties (HSL/HSV model) / - Hue - primary color tone. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Light Perception. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Humans perceive light spectra / as differently colored lights / - Often described using three / main properties (HSL/HSV model) / - Hue - primary color tone. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Light Perception' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Perception'?
Page 13	Interpreting SDFs
Source cue	- Shape of an SDF depicts how humans perceive represented light / - Hue: dominant wavelength / - Saturation: % of energy in dom. wavelength / - Luminance: total energy (SDF integral) / 400 Wavelength λ (nm) 700

Professor-style explanation	The slide 'Interpreting SDFs' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Shape of an SDF depicts how humans perceive represented light / - Hue: dominant wavelength / - Saturation: % of energy in dom. wavelength / - Luminance: total energy (SDF integral) / 400 Wavelength λ (nm) 700. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Interpreting SDFs. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Shape of an SDF depicts how humans perceive represented light / - Hue: dominant wavelength / - Saturation: % of energy in dom. wavelength / - Luminance: total energy (SDF integral) / 400 Wavelength λ (nm) 700. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Interpreting SDFs' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Interpreting SDFs'?
Page 14	Representing SDFs
Source cue	- In spectral rendering SDFs are discretized as arrays / - Cell number represents differentiable wavelength ranges / - Cell values represent energy per wavelength range / - Much more frequently SDFs are represented as three-channel color / - HSL values are derived from SDF

Professor-style explanation	The slide 'Representing SDFs' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - In spectral rendering SDFs are discretized as arrays / - Cell number represents differentiable wavelength ranges / - Cell values represent energy per wavelength range / - Much more frequently SDFs are represented as three-channel color / - HSL values are derived from SDF. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Representing SDFs. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - In spectral rendering SDFs are discretized as arrays / - Cell number represents differentiable wavelength ranges / - Cell values represent energy per wavelength range / - Much more frequently SDFs are represented as three-channel color / - HSL values are derived from SDF. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Representing SDFs' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Representing SDFs' works per object, per image region, per ray, or per fragment?
Page 15	8.2 Light Sources
Source cue	How to represent light sources in computer graphics

Professor-style explanation	The slide '8.2 Light Sources' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: How to represent light sources in computer graphics. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about 8.2 Light Sources. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: How to represent light sources in computer graphics. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for '8.2 Light Sources' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '8.2 Light Sources'?
Page 16	Light Sources
Source cue	- All light sources have a RGB color property representing their SDF / - Light sources are also defined through their location, orientation and extent / within a virtual scene / - Different light source types omit different spatial properties / - Light sources have no associated geometry

Professor-style explanation	The slide 'Light Sources' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - All light sources have a RGB color property representing their SDF - Light sources are also defined through their location, orientation and extent / within a virtual scene - Different light source types omit different spatial properties - Light sources have no associated geometry. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Light Sources. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - All light sources have a RGB color property representing their SDF - Light sources are also defined through their location, orientation and extent / within a virtual scene - Different light source types omit different spatial properties - Light sources have no associated geometry. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Light Sources' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Sources'?
Page 17	Point Light Sources
Source cue	- Point light sources $L = P, C$ have a geometric center $P = (x, y, z)$ / and radiate light with color $C = (R, G, B)$ evenly in all directions

Professor-style explanation	The slide 'Point Light Sources' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Point light sources $L = P, C$ have a geometric center $P = (x, y, z)$ / and radiate light with color $C = (R, G, B)$ evenly in all directions. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Point Light Sources. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Point light sources $L = P, C$ have a geometric center $P = (x, y, z)$ / and radiate light with color $C = (R, G, B)$ evenly in all directions. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Point Light Sources' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Point Light Sources'?
Page 18	Cone Light Sources
Source cue	- Special point light source with geometric center / - Radiation of light restricted to a cone-shaped area with its tip in the center / - Specified by cut-off angle and exponents for the decrease / of brightness with distance to the center / (spot light exponent)

Professor-style explanation	The slide 'Cone Light Sources' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Special point light source with geometric center / - Radiation of light restricted to a cone-shaped area with its tip in the center / - Specified by cut-off angle and exponents for the decrease / of brightness with distance to the center / (spot light exponent). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Cone Light Sources. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Special point light source with geometric center / - Radiation of light restricted to a cone-shaped area with its tip in the center / - Specified by cut-off angle and exponents for the decrease / of brightness with distance to the center / (spot light exponent). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Cone Light Sources' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Cone Light Sources'?
Page 19	Light Attenuation
Source cue	- Light attenuation is inversely proportional to the square / of the distance d between the light source and the object / - Attenuation factor / $f = \min, 1 / att$ $c + c \cdot d + c \cdot d^2$

Professor-style explanation	The slide 'Light Attenuation' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Light attenuation is inversely proportional to the square of the distance d between the light source and the object / - Attenuation factor $f = \min(1, 1 / (c_a + c_d \cdot d + c_s \cdot d^2))$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Light Attenuation. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Light attenuation is inversely proportional to the square of the distance d between the light source and the object / - Attenuation factor $f = \min(1, 1 / (c_a + c_d \cdot d + c_s \cdot d^2))$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Light Attenuation' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Attenuation'?
Page 20	Area Light Sources
Source cue	- Light source with geometric center and geometric extension / - Mostly of planar shape / - Transmission of light over the entire surface analogous to a point light source

Professor-style explanation	The slide 'Area Light Sources' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Light source with geometric center and geometric extension / - Mostly of planar shape / - Transmission of light over the entire surface analogous to a point light source. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Area Light Sources. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Light source with geometric center and geometric extension / - Mostly of planar shape / - Transmission of light over the entire surface analogous to a point light source. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Area Light Sources' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Area Light Sources'?
Page 21	Directed Light
Source cue	- Infinity-far away imaginary light source / - Light rays run parallel and evenly throughout the scene, / direction is specified as a vector / - Light intensity does not diminish / with increasing distance

Professor-style explanation	The slide 'Directed Light' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Infinity-far away imaginary light source / - Light rays run parallel and evenly throughout the scene, / direction is specified as a vector / - Light intensity does not diminish / with increasing distance. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Directed Light. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Infinity-far away imaginary light source / - Light rays run parallel and evenly throughout the scene, / direction is specified as a vector / - Light intensity does not diminish / with increasing distance. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Directed Light' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Directed Light' works per object, per image region, per ray, or per fragment?
Page 22	Local vs. Global Illumination
Source cue	- Simple case: Local lighting / - Consideration of direct illumination by light sources / (without obstacles or reflection) / - Simplified light models approximate / real lighting effect

Professor-style explanation	The slide 'Local vs. Global Illumination' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Simple case: Local lighting / - Consideration of direct illumination by light sources / (without obstacles or reflection) / - Simplified light models approximate / real lighting effect. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Local vs. Global Illumination. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Simple case: Local lighting / - Consideration of direct illumination by light sources / (without obstacles or reflection) / - Simplified light models approximate / real lighting effect. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Local vs. Global Illumination' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Local vs. Global Illumination'?
Page 23	Ambient Light
Source cue	- Undirected light rays, evenly distributed / - Light intensity constant, regardless of distance / - Simulates general room brightness (basic brightness)

Professor-style explanation	The slide 'Ambient Light' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Undirected light rays, evenly distributed / - Light intensity constant, regardless of distance / - Simulates general room brightness (basic brightness). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Ambient Light. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Undirected light rays, evenly distributed / - Light intensity constant, regardless of distance / - Simulates general room brightness (basic brightness). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Ambient Light' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Ambient Light' works per object, per image region, per ray, or per fragment?
Page 24	8.3 Material Models
Source cue	How to represent materials in computer graphics

Professor-style explanation	The slide '8.3 Material Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: How to represent materials in computer graphics. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about 8.3 Material Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: How to represent materials in computer graphics. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for '8.3 Material Models' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe '8.3 Material Models' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by '8.3 Material Models' before rendering begins?
Page 25	Material Models
Source cue	- Material models capture the different properties of materials / - Materials can have a wide variety of appearances / - Different groups of materials can be classified / based on appearance / - Metal - shiny and specular appearance

Professor-style explanation	The slide 'Material Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Material models capture the different properties of materials - Materials can have a wide variety of appearances - Different groups of materials can be classified / based on appearance - Metal - shiny and specular appearance. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Material Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Material models capture the different properties of materials - Materials can have a wide variety of appearances - Different groups of materials can be classified / based on appearance - Metal - shiny and specular appearance. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Material Models' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Material Models' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Material Models' before rendering begins?
Page 26	Materials vs. Textures
Source cue	- Material models specify the material properties for an infinitesimal surface - No material variance with respect to location is supported / (i.e., structures such as wood grain cannot be represented) - Textures are able to support material variance with respect to location

Professor-style explanation	The slide 'Materials vs. Textures' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Material models specify the material properties for an infinitesimal surface - No material variance with respect to location is supported (i.e., structures such as wood grain cannot be represented) - Textures are able to support material variance with respect to location. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Materials vs. Textures. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Material models specify the material properties for an infinitesimal surface - No material variance with respect to location is supported (i.e., structures such as wood grain cannot be represented) - Textures are able to support material variance with respect to location. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Materials vs. Textures' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Materials vs. Textures'?
Page 27	Light Material Interaction 1/3
Source cue	- Material appearance is influenced by three wavelength-dependent effects - Reflection - light energy bounces off - Absorption - light energy is reduced - Transmission - light passes through the object (refraction occurs) - R = reflected energy

Professor-style explanation	The slide 'Light Material Interaction 1/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Material appearance is influenced by three wavelength-dependent effects / - Reflection - light energy bounces off / - Absorption - light energy is reduced / - Transmission - light passes through the object (refraction occurs) / R = reflected energy. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Light Material Interaction 1/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Material appearance is influenced by three wavelength-dependent effects / - Reflection - light energy bounces off / - Absorption - light energy is reduced / - Transmission - light passes through the object (refraction occurs) / R = reflected energy. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Light Material Interaction 1/3' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Material Interaction 1/3'?
Page 28	Light Material Interaction 2/3
Source cue	- Material properties determine weight of the individual effects / opaque colored material opaque black material / $A > 0$, $T = 0$, $R > 0$ $A = 1$, $T = 0$, $R = 0$ / perfect mirror material transparent material / $A = 0$, $T = 0$, $R = 1$ $A \sim 0$, $R \sim 0$, $T \sim 1$

Professor-style explanation	The slide 'Light Material Interaction 2/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Material properties determine weight of the individual effects / opaque colored material opaque black material / $A > 0$, $T = 0$, $R > 0$ $A = I$, $T = 0$, $R = 0$ / perfect mirror material transparent material / $A = 0$, $T = 0$, $R = I$ $A \approx 0$, $R \approx 0$, $T \approx I$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Light Material Interaction 2/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Material properties determine weight of the individual effects / opaque colored material opaque black material / $A > 0$, $T = 0$, $R > 0$ $A = I$, $T = 0$, $R = 0$ / perfect mirror material transparent material / $A = 0$, $T = 0$, $R = I$ $A \approx 0$, $R \approx 0$, $T \approx I$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Light Material Interaction 2/3' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Material Interaction 2/3'?
Page 29	Light Material Interaction 3/3
Source cue	- Light-material interaction described by the Spectral Response Function (SRF) / 400 Wavelength λ (nm) 700 / ygrenE / 100% / All pass Filter (ideal, real)

Professor-style explanation	The slide 'Light Material Interaction 3/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Light-material interaction described by the Spectral Response Function (SRF) / 400 Wavelength λ (nm) 700 / ygrenE / 100% / All pass Filter (ideal, real). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Light Material Interaction 3/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Light-material interaction described by the Spectral Response Function (SRF) / 400 Wavelength λ (nm) 700 / ygrenE / 100% / All pass Filter (ideal, real). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Light Material Interaction 3/3' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Material Interaction 3/3'?
Page 30	Reflection = SDF x SRF
Source cue	- Light-material interaction results are the product of the SDF and SRF / 100% $\times$ 100% / SDF SRF / 0% 0% / 400 Wavelength λ (nm) 700 400 Wavelength λ (nm) 700

Professor-style explanation	The slide 'Reflection = SDF x SRF' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Light-material interaction results are the product of the SDF and SRF / $100\% \times 100\%$ / SDF SRF / $0\% \ 0\%$ / 400 Wavelength λ (nm) 700 400 Wavelength λ (nm) 700. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Reflection = SDF x SRF. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Light-material interaction results are the product of the SDF and SRF / $100\% \times 100\%$ / SDF SRF / $0\% \ 0\%$ / 400 Wavelength λ (nm) 700 400 Wavelength λ (nm) 700. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Reflection = SDF x SRF' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Reflection = SDF x SRF'?
Page 31	Reflection Modeling 1/2
Source cue	- All light sources can be described by their Spectral Density Functions / - Natural light: sun, fire, ... / - Artificial light: light bulb, laser, neon tube, LED, ... / - All material properties can be described by Spectral Response Functions / - Glass, Water: SRF for transmission

Professor-style explanation	The slide 'Reflection Modeling 1/2' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - All light sources can be described by their Spectral Density Functions / - Natural light: sun, fire, ... / - Artificial light: light bulb, laser, neon tube, LED, ... / - All material properties can be described by Spectral Response Functions / - Glass, Water: SRF for transmission. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Reflection Modeling 1/2. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - All light sources can be described by their Spectral Density Functions / - Natural light: sun, fire, ... / - Artificial light: light bulb, laser, neon tube, LED, ... / - All material properties can be described by Spectral Response Functions / - Glass, Water: SRF for transmission. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Reflection Modeling 1/2' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Reflection Modeling 1/2' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Reflection Modeling 1/2' before rendering begins?
Page 32	Reflection Modeling 2/2
Source cue	- Synthetic and natural objects are in general not self-emitting / - Light is reflected at its surfaces, i.e., re-emitted into the scene / - Reflection is dependent on surface properties / - Rough surface: scattering of light / - Smooth surface: reflection of light

Professor-style explanation	The slide 'Reflection Modeling 2/2' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Synthetic and natural objects are in general not self-emitting / - Light is reflected at its surfaces, i.e., re-emitted into the scene / - Reflection is dependent on surface properties / - Rough surface: scattering of light / - Smooth surface: reflection of light. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Reflection Modeling 2/2. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Synthetic and natural objects are in general not self-emitting / - Light is reflected at its surfaces, i.e., re-emitted into the scene / - Reflection is dependent on surface properties / - Rough surface: scattering of light / - Smooth surface: reflection of light. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Reflection Modeling 2/2' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Reflection Modeling 2/2' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Reflection Modeling 2/2' before rendering begins?
Page 33	8.4 Phong Illumination Model
Source cue	Determining the color of vertices

Professor-style explanation	The slide '8.4 Phong Illumination Model' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: Determining the color of vertices. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about 8.4 Phong Illumination Model. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: Determining the color of vertices. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for '8.4 Phong Illumination Model' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '8.4 Phong Illumination Model'?
Page 34	Empirical Observations
Source cue	- Light reflections are / - Dependent on the position of the observer / "... position of the observer..." / - Dependent on the material properties of the surface / "... the specular properties of the object." Photo

Professor-style explanation	The slide 'Empirical Observations' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Light reflections are / - Dependent on the position of the observer / "... position of the observer..." / - Dependent on the material properties of the surface / "... the specular properties of the object." Photo. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Empirical Observations. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Light reflections are / - Dependent on the position of the observer / "... position of the observer..." / - Dependent on the material properties of the surface / "... the specular properties of the object." Photo. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Empirical Observations' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Empirical Observations'?
Page 35	Illumination Models
Source cue	- Illumination models calculate the color of scene objects / based on the following properties / - Location - Position $P = (x, y, z)$ of the selected point P / - Orientation - normal direction $N = (n_x, n_y, n_z)$ of surface in $P / x y z$

Professor-style explanation	The slide 'Illumination Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Illumination models calculate the color of scene objects / based on the following properties / - Location - Position $P = (x, y, z)$ of the selected point P / - Orientation - normal direction $N = (n_x, n_y, n_z)$ of surface in $P / x y z$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors. For examination purposes, the important content is that models are structured causes of images, not images themselves; positions, topology, attributes, and materials become input for later rendering stages.
Technical commentary	This slide is about Illumination Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Illumination models calculate the color of scene objects / based on the following properties / - Location - Position $P = (x, y, z)$ of the selected point P / - Orientation - normal direction $N = (n_x, n_y, n_z)$ of surface in $P / x y z$. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Exam-grade answer	A strong answer for 'Illumination Models' distinguishes model data from rendered image data and names the geometric or attribute data that later pipeline stages consume.
Common trap	Do not describe 'Illumination Models' as if the model were already a pixel image; a model is data that still needs transformation, visibility, and shading.
Check yourself	Can you name the geometric objects and attributes represented by 'Illumination Models' before rendering begins?
Page 36	yaR
Source cue	thgiL / Diffuse Intensity 1/3 / - Simulation of light reflections on diffuse surfaces / - Model assumption / - Diffuse light reflections are non-directional (scattering)

Professor-style explanation	The slide 'yaR' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: $\text{thgiL} / \text{Diffuse Intensity } 1/3 / \text{- Simulation of light reflections on diffuse surfaces} / \text{- Model assumption} / \text{- Diffuse light reflections are non-directional (scattering)}$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about yaR. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: $\text{thgiL} / \text{Diffuse Intensity } 1/3 / \text{- Simulation of light reflections on diffuse surfaces} / \text{- Model assumption} / \text{- Diffuse light reflections are non-directional (scattering)}$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'yaR' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'yaR'?
Page 37	Diffuse Intensity 2/3
Source cue	- Equation $I = I_k \cdot n \cdot L \cdot \max(0, \dots)$ / diffuse $d d$ / - k diffuse material properties (surface dependent) / - L diffuse light intensity (light source dependent)

Professor-style explanation	The slide 'Diffuse Intensity 2/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Equation $I = I_k \cdot n \cdot L \cdot \max(0, \dots)$ / diffuse d / - k diffuse material properties (surface dependent) / - L diffuse light intensity (light source dependent). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Diffuse Intensity 2/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Equation $I = I_k \cdot n \cdot L \cdot \max(0, \dots)$ / diffuse d / - k diffuse material properties (surface dependent) / - L diffuse light intensity (light source dependent). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Diffuse Intensity 2/3' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Diffuse Intensity 2/3'?
Page 38	Diffuse Intensity 3/3
Source cue	$I = k \cdot L \cdot \max(0, \mathbf{l} \cdot \mathbf{n})$ / diffuse d / https://bit.ly/2XFhbcM / Extrahierte Tabellen: / [Tabelle 1]

Professor-style explanation	The slide 'Diffuse Intensity 3/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: $I = k \cdot L \cdot \max(0, \mathbf{l} \cdot \mathbf{n}) / \text{diffuse } d$ / https://bit.ly/2XFhbcM / Extrahierte Tabellen: / [Tabelle 1]. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Diffuse Intensity 3/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: $I = k \cdot L \cdot \max(0, \mathbf{l} \cdot \mathbf{n}) / \text{diffuse } d$ / https://bit.ly/2XFhbcM / Extrahierte Tabellen: / [Tabelle 1]. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Diffuse Intensity 3/3' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Diffuse Intensity 3/3' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Diffuse Intensity 3/3' into a causal sentence instead of repeating the slide title?
Page 39	Ambient Intensity 1/2
Source cue	- Simulation of the general room brightness / - Indirect light incidence or multiple reflections / - Model assumptions / - Constant influence on all scene objects / - No influence of position or alignment

Professor-style explanation	The slide 'Ambient Intensity 1/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Simulation of the general room brightness / - Indirect light incidence or multiple reflections / - Model assumptions / - Constant influence on all scene objects / - No influence of position or alignment. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Ambient Intensity 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Simulation of the general room brightness / - Indirect light incidence or multiple reflections / - Model assumptions / - Constant influence on all scene objects / - No influence of position or alignment. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Ambient Intensity 1/2' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Ambient Intensity 1/2'?
Page 40	Ambient Intensity 2/2
Source cue	$I = k \cdot L / \text{ambient } a$ / https://bit.ly/2XGn61n / Extrahierte Tabellen: / [Tabelle 1]

Professor-style explanation	The slide 'Ambient Intensity 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: $I = k \cdot L$ / ambient a / https://bit.ly/2XGn61n / Extrahierte Tabellen: / [Tabelle 1]. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Ambient Intensity 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: $I = k \cdot L$ / ambient a / https://bit.ly/2XGn61n / Extrahierte Tabellen: / [Tabelle 1]. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Ambient Intensity 2/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Ambient Intensity 2/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Ambient Intensity 2/2' into a causal sentence instead of repeating the slide title?
Page 41	Specular Intensity 1/3
Source cue	- Simulation of light reflections on specular surfaces / - Model assumption / - Reflection of light are directed (reflection) / - Observations / - Reflections of light are observer-dependent

Professor-style explanation	The slide 'Specular Intensity 1/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Simulation of light reflections on specular surfaces / - Model assumption / - Reflection of light are directed (reflection) / - Observations / - Reflections of light are observer-dependent. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Specular Intensity 1/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Simulation of light reflections on specular surfaces / - Model assumption / - Reflection of light are directed (reflection) / - Observations / - Reflections of light are observer-dependent. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Specular Intensity 1/3' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Specular Intensity 1/3'?
Page 42	Reflection Vector $\mathbf{r}$
Source cue	- Calculation of the reflection vector / - Incidence angle corresponds to exit angle / - Equation / $\mathbf{r} = 2\mathbf{n} \cdot \mathbf{n} - \mathbf{l}$

Professor-style explanation	The slide 'Reflection Vector $\mathbf{r}$ ' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - Calculation of the reflection vector / - Incidence angle corresponds to exit angle / - Equation $\mathbf{r} = 2\mathbf{n} \cdot \mathbf{n} \cdot \mathbf{l} - \mathbf{l}$. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Reflection Vector $\mathbf{r}$. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - Calculation of the reflection vector / - Incidence angle corresponds to exit angle / - Equation $\mathbf{r} = 2\mathbf{n} \cdot \mathbf{n} \cdot \mathbf{l} - \mathbf{l}$. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Reflection Vector $\mathbf{r}$ ' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Reflection Vector $\mathbf{r}$ ' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Reflection Vector $\mathbf{r}$ ' into a causal sentence instead of repeating the slide title?
Page 43	Specular Intensity 2/3
Source cue	- Equation $I = k \mathbf{r} \cdot \mathbf{v} L \cdot \max(0,)$ / specular s / - k specular material properties (surface dependant) / - L specular light intensity (light source dependent)

Professor-style explanation	The slide 'Specular Intensity 2/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Equation $I = k_r \mathbf{r} \cdot \mathbf{v} + L \cdot \max(0, \mathbf{n} \cdot \mathbf{l})^p$ / specular s s - k specular material properties (surface dependant) / - L specular light intensity (light source dependent). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Specular Intensity 2/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Equation $I = k_r \mathbf{r} \cdot \mathbf{v} + L \cdot \max(0, \mathbf{n} \cdot \mathbf{l})^p$ / specular s s - k specular material properties (surface dependant) / - L specular light intensity (light source dependent). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Specular Intensity 2/3' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Specular Intensity 2/3'?
Page 44	Specular Exponent p
Source cue	- p influences fall off of specular highlight ($p \in [1, M]$ mit $M \approx 100$) / $p = 1 / \max(0, \mathbf{r} \cdot \mathbf{v})$ / $p = 5 / \max(0, \mathbf{r} \cdot \mathbf{v})^5$

Professor-style explanation	The slide 'Specular Exponent p' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - p influences fall off of specular highlight ($p \in [1, M]$ mit $M \approx 100$) / $p = 1 / \max(0, \mathbf{r} \cdot \mathbf{v})$ / $p = 5 / \max(0, \mathbf{r} \cdot \mathbf{v})^5$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Specular Exponent p. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - p influences fall off of specular highlight ($p \in [1, M]$ mit $M \approx 100$) / $p = 1 / \max(0, \mathbf{r} \cdot \mathbf{v})$ / $p = 5 / \max(0, \mathbf{r} \cdot \mathbf{v})^5$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Specular Exponent p' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Specular Exponent p'?
Page 45	Specular Intensity 3/3
Source cue	$r_s = 2 \cdot n \cdot n \cdot l - l / \sqrt{2} \cdot \max(0, \mathbf{r} \cdot \mathbf{v})^p$ Extrahierte Tabellen: / [Tabelle 1]

Professor-style explanation	The slide 'Specular Intensity 3/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: $r_{\text{specular}} = 2 \cdot n \cdot n \cdot I - I_r$ / https://bit.ly/2XbPc7s $I = k \cdot L \cdot \max(0, r_{\text{specular}} \cdot v_{\text{specular}})^p$ / specular s s / Extrahierte Tabellen: / [Tabelle 1]. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Specular Intensity 3/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: $r_{\text{specular}} = 2 \cdot n \cdot n \cdot I - I_r$ / https://bit.ly/2XbPc7s $I = k \cdot L \cdot \max(0, r_{\text{specular}} \cdot v_{\text{specular}})^p$ / specular s s / Extrahierte Tabellen: / [Tabelle 1]. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Specular Intensity 3/3' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Specular Intensity 3/3' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Specular Intensity 3/3' into a causal sentence instead of repeating the slide title?
Page 46	Blinn-Phong Illumination Model 1/2
Source cue	- Alternative simulation of specular reflections / - Replace mirror direction r_{specular} with Halfway-Vector h / - $h = (I + v_{\text{specular}}) / 2$ / - h is halfway between I and v_{specular} / - Equation

Professor-style explanation	The slide 'Blinn-Phong Illumination Model 1/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Alternative simulation of specular reflections / - Replace mirror direction r with Halfway-Vector h / - $h = l + v$ / - h is halfway between l and v / - Equation. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Blinn-Phong Illumination Model 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Alternative simulation of specular reflections / - Replace mirror direction r with Halfway-Vector h / - $h = l + v$ / - h is halfway between l and v / - Equation. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Blinn-Phong Illumination Model 1/2' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Blinn-Phong Illumination Model 1/2'?
Page 47	Blinn-Phong Illumination Model 2/2
Source cue	$h = \frac{l + v}{ l + v }$ / https://bit.ly/2ICDwBD $I = k \cdot L \cdot \max(0, h \cdot n)^p$ / specular s / Extrahierte Tabellen: / [Tabelle 1]

Professor-style explanation	The slide 'Blinn-Phong Illumination Model 2/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: $h = \frac{L + v}{2}$ / https://bit.ly/2ICDwBD $I = k \cdot L \cdot \max(0, h \cdot n)^p$ / specular $s s$ / Extrahierte Tabellen: / [Tabelle 1]. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Blinn-Phong Illumination Model 2/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: $h = \frac{L + v}{2}$ / https://bit.ly/2ICDwBD $I = k \cdot L \cdot \max(0, h \cdot n)^p$ / specular $s s$ / Extrahierte Tabellen: / [Tabelle 1]. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Blinn-Phong Illumination Model 2/2' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Blinn-Phong Illumination Model 2/2'?
Page 48	Phong Illumination Model
Source cue	- Additive combination of all factors / $I = k \cdot L$ / ambient $a a$ / $I = k \cdot L \cdot \max(0, n \cdot l)$ / diffuse $d d$

Professor-style explanation	The slide 'Phong Illumination Model' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Additive combination of all factors $I = k \cdot L / \text{ambient } a / I = k \cdot L \cdot \max 0, n \cdot l / \text{diffuse } d$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Phong Illumination Model. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Additive combination of all factors $I = k \cdot L / \text{ambient } a / I = k \cdot L \cdot \max 0, n \cdot l / \text{diffuse } d$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Phong Illumination Model' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Phong Illumination Model'?
Page 49	Phong Illumination Model
Source cue	- Additive combination of all factors $I = k \cdot L / \text{ambient } a / I = k \cdot L \cdot \max 0, n \cdot l / \text{diffuse } d$

Professor-style explanation	The slide 'Phong Illumination Model' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Additive combination of all factors $I = k \cdot L / \text{ambient } a / I = k \cdot L \cdot \max(0, n \cdot l) / \text{diffuse } d$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Phong Illumination Model. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Additive combination of all factors $I = k \cdot L / \text{ambient } a / I = k \cdot L \cdot \max(0, n \cdot l) / \text{diffuse } d$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Phong Illumination Model' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Phong Illumination Model'?
Page 50	8.5 Shading
Source cue	Coloring triangles when using vertex-based illumination

Professor-style explanation	The slide '8.5 Shading' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: Coloring triangles when using vertex-based illumination. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about 8.5 Shading. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: Coloring triangles when using vertex-based illumination. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for '8.5 Shading' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by '8.5 Shading'?
Page 51	Shading
Source cue	- Illumination model usually applied in the fragment stage / - Vertex-based illumination also possible to save compute / - Requires to spread the illumination information of the vertices / over the triangle / Flat Gouroud Phong

Professor-style explanation	The slide 'Shading' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Illumination model usually applied in the fragment stage / - Vertex-based illumination also possible to save compute / - Requires to spread the illumination information of the vertices / over the triangle / Flat Gouroud Phong. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Shading. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Illumination model usually applied in the fragment stage / - Vertex-based illumination also possible to save compute / - Requires to spread the illumination information of the vertices / over the triangle / Flat Gouroud Phong. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Shading' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Shading'?
Page 52	Shading Computation
Source cue	- Tasks of shading procedures / - Colorize surfaces based on colors calculated by illumination model / - Shading is closely linked to the rasterization process and integrated into it / - Distribute lighting and shading / - Calculate light intensity for surface points

Professor-style explanation	The slide 'Shading Computation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Tasks of shading procedures / - Colorize surfaces based on colors calculated by illumination model / - Shading is closely linked to the rasterization process and integrated into it / - Distribute lighting and shading / - Calculate light intensity for surface points. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Shading Computation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Tasks of shading procedures / - Colorize surfaces based on colors calculated by illumination model / - Shading is closely linked to the rasterization process and integrated into it / - Distribute lighting and shading / - Calculate light intensity for surface points. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Shading Computation' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Shading Computation'?
Page 53	Flat Shading
Source cue	- Operation / - Calling the lighting model for a single point of a surface / (e.g., polygon center) / - Shading of the entire polygon in the determined intensity / - Useful if the following assumptions apply

Professor-style explanation	The slide 'Flat Shading' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Operation / - Calling the lighting model for a single point of a surface / (e.g., polygon center) / - Shading of the entire polygon in the determined intensity / - Useful if the following assumptions apply. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Flat Shading. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Operation / - Calling the lighting model for a single point of a surface / (e.g., polygon center) / - Shading of the entire polygon in the determined intensity / - Useful if the following assumptions apply. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Flat Shading' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Flat Shading'?
Page 54	Gouraud Shading 1/5
Source cue	- Shading method for polygons based on the interpolation / of intensity values at the polygon vertices / - Requirements / - Area decomposition in polygons (usually triangles) / - Normals for polygon corners (vertex normals)

Professor-style explanation	The slide 'Gouraud Shading 1/5' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Shading method for polygons based on the interpolation / of intensity values at the polygon vertices / - Requirements / - Area decomposition in polygons (usually triangles) / - Normals for polygon corners (vertex normals). A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Gouraud Shading 1/5. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Shading method for polygons based on the interpolation / of intensity values at the polygon vertices / - Requirements / - Area decomposition in polygons (usually triangles) / - Normals for polygon corners (vertex normals). A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Gouraud Shading 1/5' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Gouraud Shading 1/5'?
Page 55	Gouraud Shading 2/5
Source cue	- Proceeding / - Compute intensity values I for polygon corners v_i / - Linear interpolation of the intensity values along the edges / - Linear interpolation of the intensity values along the scanlines

Professor-style explanation	The slide 'Gouraud Shading 2/5' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Proceeding / - Compute intensity values I for polygon corners v_i / - Linear interpolation of the intensity values along the edges / - Linear interpolation of the intensity values along the scanlines. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Gouraud Shading 2/5. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Proceeding / - Compute intensity values I for polygon corners v_i / - Linear interpolation of the intensity values along the edges / - Linear interpolation of the intensity values along the scanlines. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Gouraud Shading 2/5' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Gouraud Shading 2/5'?
Page 56	Gouraud Shading 3/5
Source cue	- Shading at conceptually smooth transitions / - Geometrically "hard" edges instead of conceptually "soft" edges / - Gouraud shading can create visually smooth transitions / through smooth shading / - Example: P / P smooth transition, P / P hard transition

Professor-style explanation	The slide 'Gouraud Shading 3/5' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Shading at conceptually smooth transitions / - Geometrically "hard" edges instead of conceptually "soft" edges / - Gouraud shading can create visually smooth transitions / through smooth shading / - Example: P / P smooth transition, P / P hard transition. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Gouraud Shading 3/5. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Shading at conceptually smooth transitions / - Geometrically "hard" edges instead of conceptually "soft" edges / - Gouraud shading can create visually smooth transitions / through smooth shading / - Example: P / P smooth transition, P / P hard transition. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Gouraud Shading 3/5' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Gouraud Shading 3/5'?
Page 57	Gouraud Shading 4/5
Source cue	- interpolation problems / - The shading inside a triangle results from interpolation / - Intensity fluctuations in the interior are not taken into account / - Intensity fluctuations can only be considered by finer tessellation / - Example: Spotlight shines into the interior of a triangle

Professor-style explanation	The slide 'Gouraud Shading 4/5' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - interpolation problems / - The shading inside a triangle results from interpolation / - Intensity fluctuations in the interior are not taken into account / - Intensity fluctuations can only be considered by finer tessellation / - Example: Spotlight shines into the interior of a triangle. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Gouraud Shading 4/5. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - interpolation problems / - The shading inside a triangle results from interpolation / - Intensity fluctuations in the interior are not taken into account / - Intensity fluctuations can only be considered by finer tessellation / - Example: Spotlight shines into the interior of a triangle. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Gouraud Shading 4/5' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Gouraud Shading 4/5'?
Page 58	Gouraud Shading 5/5
Source cue	- Interpolation problems / - Contiguous polygons that do not fully share the same edges / will be shaded differently

Professor-style explanation	The slide 'Gouraud Shading 5/5' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: - Interpolation problems / - Contiguous polygons that do not fully share the same edges / will be shaded differently. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Gouraud Shading 5/5. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: - Interpolation problems / - Contiguous polygons that do not fully share the same edges / will be shaded differently. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Gouraud Shading 5/5' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Gouraud Shading 5/5'?
Page 59	Phong Shading 1/2
Source cue	- Interpolation of the corner normals instead of the corner intensities / - Proceeding / - Calculation of the corner normals / - Interpolation of the normals between the corners / - Interpolation between the endpoints of a scanline

Professor-style explanation	The slide 'Phong Shading 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Interpolation of the corner normals instead of the corner intensities / - Proceeding / - Calculation of the corner normals / - Interpolation of the normals between the corners / - Interpolation between the endpoints of a scanline. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Phong Shading 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Interpolation of the corner normals instead of the corner intensities / - Proceeding / - Calculation of the corner normals / - Interpolation of the normals between the corners / - Interpolation between the endpoints of a scanline. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Phong Shading 1/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Phong Shading 1/2'?
Page 60	Phong Shading 2/2
Source cue	- Advantages / - Pixel-based (i.e., image-precise) evaluation of the light sources / - Highly focused highlights at corners are not interpolated over the edge / - Highlights inside a triangle can be displayed / - Disadvantage

Professor-style explanation	The slide 'Phong Shading 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Advantages / - Pixel-based (i.e., image-precise) evaluation of the light sources / - Highly focused highlights at corners are not interpolated over the edge / - Highlights inside a triangle can be displayed / - Disadvantage. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Phong Shading 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Advantages / - Pixel-based (i.e., image-precise) evaluation of the light sources / - Highly focused highlights at corners are not interpolated over the edge / - Highlights inside a triangle can be displayed / - Disadvantage. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Phong Shading 2/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Phong Shading 2/2'?
Page 61	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Literature and other sources used in this chapter' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Literature and other sources used in this chapter' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 62	Lesen und Ausprobieren
Source cue	- Zum Lesen - Zum Ausprobieren / Marschner, Steve und Peter Shirley: / - Simulation von drei / Fundamentals of Computer Graphics, / ausgewählten Materialien
Professor-style explanation	The slide 'Lesen und Ausprobieren' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: - Zum Lesen - Zum Ausprobieren / Marschner, Steve und Peter Shirley: / - Simulation von drei / Fundamentals of Computer Graphics, / ausgewählten Materialien. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.

Technical commentary	This slide is about Lesen und Ausprobieren. The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: - Zum Lesen - Zum Ausprobieren / Marschner, Steve und Peter Shirley: / - Simulation von drei / Fundamentals of Computer Graphics, / ausgewählten Materialien. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Lesen und Ausprobieren' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Lesen und Ausprobieren' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Lesen und Ausprobieren' points to for deeper study?

08.1 Physics of Light

Source location: Section 8.1

Commentary

The physics slides provide intuition for light as energy interacting with surfaces. For this course, the goal is not full physical simulation but a usable model for real-time rendering.

Important terms are reflection, absorption, wavelength/color, intensity, and direction. These terms become shader inputs or material parameters later. The simplified model must still preserve the relation between light direction, surface orientation, and observed brightness.

Mental model

Shading is an approximation of light-surface interaction that is cheap enough for rendering.

Check yourself

Why does surface orientation affect diffuse brightness?

08.2 Light Sources

Source location: Section 8.2

Commentary

Light source models define where light comes from and how its direction and intensity are computed. Directional lights approximate distant sources with parallel rays. Point lights emit from a position and often include attenuation. Spotlights add a directional cone.

In shader code, the type of light determines how you compute the light vector. A directional light can use a fixed direction; a point light requires subtracting the surface position from the light position.

Mental model

Different light types mainly change how the light direction and intensity are computed at the surface point.

Check yourself

What is the difference between a directional light vector and a point light vector?

08.3 Material Models

Source location: Section 8.3

Commentary

A material model tells the shader how a surface responds to light. Diffuse material scatters light broadly; specular material creates view-dependent highlights; ambient terms approximate background illumination.

Material coefficients are not arbitrary decoration. They scale the contribution of each lighting term. If the specular coefficient is zero, the surface should not show a specular highlight under that model.

Mental model

Light asks what arrives; material answers how the surface reacts.

Check yourself

Which material parameter would you adjust to reduce shiny highlights?

08.4 Phong Illumination Model

Source location: Section 8.4

Commentary

The Phong illumination model combines ambient, diffuse, and specular terms. Diffuse lighting uses the angle between normal and light direction. Specular lighting also depends on the viewer because highlights move with the view direction.

The formula is less important than the decomposition. Ambient is a rough constant approximation. Diffuse is orientation-dependent matte reflection. Specular is view-dependent shininess. A correct answer should name the vectors and normalize them.

Mental model

Phong-style lighting is a sum of simple terms, each modeling a different visual effect.

Check yourself

Why does the specular term depend on the view direction?

08.5 Shading

Source location: Section 8.5

Commentary

Shading decides where the illumination calculation is evaluated and how results are interpolated. Flat shading uses one value per primitive. Gouraud shading evaluates at vertices and interpolates colors. Phong shading

interpolates normals and evaluates lighting per fragment.

Per-fragment shading is more expensive but can represent highlights more accurately. This connects directly to the pipeline: interpolation during rasterization provides the data used by the fragment shader.

Mental model

The shading method decides what is computed at vertices and what is computed at fragments.

Check yourself

Why can Gouraud shading miss a small specular highlight?

End-of-lecture summary

If you remember only one thing from Lecture 08, remember this: This lecture explains how visible surface points receive color from local light, material, normal, and view information. It is about shading a point, not solving all global light transport.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 09 - Texturing

This lecture treats textures as sampled data for shading, not merely images glued onto objects. It connects UV coordinates, sampling, filtering, mipmaps, OpenGL texture state, and advanced texture uses.

Source chunk	course_text_parts/03_lectures/09_texturing.txt
Extracted slide pages	133
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.

Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Textures: Sampled Data for Shading
Source cue	■ Textures are sampled data fields used during rendering / ■ Most common case: 2D image data mapped onto 3D geometry / ■ Texture values can represent color, transparency, normals, roughness, height, masks, or arbitrary / lookup data
Professor-style explanation	The slide 'Textures: Sampled Data for Shading' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Textures are sampled data fields used during rendering / ■ Most common case: 2D image data mapped onto 3D geometry / ■ Texture values can represent color, transparency, normals, roughness, height, masks, or arbitrary / lookup data. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Textures: Sampled Data for Shading. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Textures are sampled data fields used during rendering / ■ Most common case: 2D image data mapped onto 3D geometry / ■ Texture values can represent color, transparency, normals, roughness, height, masks, or arbitrary / lookup data. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Textures: Sampled Data for Shading' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Textures: Sampled Data for Shading'?
Page 3	Texturing Example
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Texturing Example' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Texturing Example. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Texturing Example' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Texturing Example' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Texturing Example' into a causal sentence instead of repeating the slide title?
Page 4	Texture Mapping
Source cue	■ Texture mapping / • Application of texture defined in multidimensional texture space to object defined in 3D space / • Realized through per-fragment operations / • A texture lookup turns texture coordinates into data used by the shader

Professor-style explanation	The slide 'Texture Mapping' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Texture mapping / • Application of texture defined in multidimensional texture space to object defined in 3D space / • Realized through per-fragment operations / • A texture lookup turns texture coordinates into data used by the shader. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Texture Mapping. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Texture mapping / • Application of texture defined in multidimensional texture space to object defined in 3D space / • Realized through per-fragment operations / • A texture lookup turns texture coordinates into data used by the shader. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Texture Mapping' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texture Mapping'?
Page 5	Why Textures Matter
Source cue	■ Geometry defines shape / ■ Textures define appearance detail / ■ This separates visual complexity from geometric complexity / ■ Instead of modeling every scratch, brick, pore, label, or color variation, we store it in texture data / ■ Textures are efficient

Professor-style explanation	The slide 'Why Textures Matter' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Geometry defines shape / ■ Textures define appearance detail / ■ This separates visual complexity from geometric complexity / ■ Instead of modeling every scratch, brick, pore, label, or color variation, we store it in texture data / ■ Textures are efficient. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Why Textures Matter. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Geometry defines shape / ■ Textures define appearance detail / ■ This separates visual complexity from geometric complexity / ■ Instead of modeling every scratch, brick, pore, label, or color variation, we store it in texture data / ■ Textures are efficient. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Why Textures Matter' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Why Textures Matter'?
Page 6	Textures Are More Than Color Images
Source cue	■ Modern rendering usually uses several texture maps per object / ■ Each map controls one part of the material or shading model / ■ Textures are therefore general-purpose GPU data sources / ■ Common texture maps / • Base color / albedo

Professor-style explanation	The slide 'Textures Are More Than Color Images' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Modern rendering usually uses several texture maps per object / ■ Each map controls one part of the material or shading model / ■ Textures are therefore general-purpose GPU data sources / ■ Common texture maps / • Base color / albedo. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Textures Are More Than Color Images. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Modern rendering usually uses several texture maps per object / ■ Each map controls one part of the material or shading model / ■ Textures are therefore general-purpose GPU data sources / ■ Common texture maps / • Base color / albedo. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Textures Are More Than Color Images' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Textures Are More Than Color Images'?
Page 7	Texturing Demo
Source cue	Preset Plain Object / Rotate / Base color / albedo / Normal / Roughness

Professor-style explanation	The slide 'Texturing Demo' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: Preset Plain Object / Rotate / Base color / albedo / Normal / Roughness. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Texturing Demo. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: Preset Plain Object / Rotate / Base color / albedo / Normal / Roughness. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Texturing Demo' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Texturing Demo'?
Page 8	Overview
Source cue	9.1 Texture Objects, Texels, Formats, and Color Space / 9.2 Texture Coordinates, UV Mapping, and Wrapping / 9.3 Sampling, Filtering, Mipmaps, and Anisotropy / 9.4 Modern OpenGL Texture Pipeline / 9.5 Normal Mapping

Professor-style explanation	The outline 'Overview' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: 9.1 Texture Objects, Texels, Formats, and Color Space / 9.2 Texture Coordinates, UV Mapping, and Wrapping / 9.3 Sampling, Filtering, Mipmaps, and Anisotropy / 9.4 Modern OpenGL Texture Pipeline / 9.5 Normal Mapping. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about Overview. The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: 9.1 Texture Objects, Texels, Formats, and Color Space / 9.2 Texture Coordinates, UV Mapping, and Wrapping / 9.3 Sampling, Filtering, Mipmaps, and Anisotropy / 9.4 Modern OpenGL Texture Pipeline / 9.5 Normal Mapping. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for 'Overview' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize 'Overview' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where 'Overview' fits in the lecture order and what later section depends on it?
Page 9	9.1 Texture Objects, Texels, Formats, and Color Space
Source cue	What texture data stores and how the GPU interprets it

Professor-style explanation	The slide '9.1 Texture Objects, Texels, Formats, and Color Space' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: What texture data stores and how the GPU interprets it. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about 9.1 Texture Objects, Texels, Formats, and Color Space. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: What texture data stores and how the GPU interprets it. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for '9.1 Texture Objects, Texels, Formats, and Color Space' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in '9.1 Texture Objects, Texels, Formats, and Color Space'?
Page 10	From Image File to GPU Texture Object
Source cue	■ Texture data starts as image file or generated array / ■ During loading, data is uploaded to GPU memory / ■ On the GPU, data becomes a texture object / ■ The shader accesses it through a sampler / ■ Pipeline

Professor-style explanation	The slide 'From Image File to GPU Texture Object' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Texture data starts as image file or generated array / ■ During loading, data is uploaded to GPU memory / ■ On the GPU, data becomes a texture object / ■ The shader accesses it through a sampler / ■ Pipeline. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about From Image File to GPU Texture Object. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Texture data starts as image file or generated array / ■ During loading, data is uploaded to GPU memory / ■ On the GPU, data becomes a texture object / ■ The shader accesses it through a sampler / ■ Pipeline. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'From Image File to GPU Texture Object' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'From Image File to GPU Texture Object'?
Page 11	Texels, Channels, and Meaning
Source cue	■ Texel: smallest sample element of a texture / ■ In 2D, a texel is analogous to a pixel / ■ Texels store numeric channel values / ■ Common channel layouts / • R, RG, RGB, RGBA

Professor-style explanation	The slide 'Texels, Channels, and Meaning' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Texel: smallest sample element of a texture / ■ In 2D, a texel is analogous to a pixel / ■ Texels store numeric channel values / ■ Common channel layouts / • R, RG, RGB, RGBA. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Texels, Channels, and Meaning. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Texel: smallest sample element of a texture / ■ In 2D, a texel is analogous to a pixel / ■ Texels store numeric channel values / ■ Common channel layouts / • R, RG, RGB, RGBA. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Texels, Channels, and Meaning' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Texels, Channels, and Meaning'?
Page 12	Texture Targets and Dimensionality
Source cue	■ Texture target describes the sampled data domain / ■ Different targets use different coordinate types / ■ 1D texture / • Coordinate / • Lookup tables or transfer functions

Professor-style explanation	The slide 'Texture Targets and Dimensionality' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Texture target describes the sampled data domain / ■ Different targets use different coordinate types / ■ 1D texture / • Coordinate / • Lookup tables or transfer functions. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Texture Targets and Dimensionality. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Texture target describes the sampled data domain / ■ Different targets use different coordinate types / ■ 1D texture / • Coordinate / • Lookup tables or transfer functions. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Texture Targets and Dimensionality' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Texture Targets and Dimensionality' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Texture Targets and Dimensionality'?
Page 13	Internal Texture Formats
Source cue	■ Internal format defines GPU texel storage / ■ It specifies channels, precision, and representation / ■ Examples / • GL_R8: one 8-bit normalized channel / • GL_RGBA8: four 8-bit normalized channels

Professor-style explanation	The slide 'Internal Texture Formats' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Internal format defines GPU texel storage / ■ It specifies channels, precision, and representation / ■ Examples / • GL_R8: one 8-bit normalized channel / • GL_RGBA8: four 8-bit normalized channels. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Internal Texture Formats. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Internal format defines GPU texel storage / ■ It specifies channels, precision, and representation / ■ Examples / • GL_R8: one 8-bit normalized channel / • GL_RGBA8: four 8-bit normalized channels. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Internal Texture Formats' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Internal Texture Formats'?
Page 14	Numeric Texture Representations
Source cue	■ Texel values can be interpreted differently / ■ Normalized integer formats / • Stored as integers / • Sampled as normalized floats / 255 → 1.0

Professor-style explanation	The slide 'Numeric Texture Representations' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Texel values can be interpreted differently / ■ Normalized integer formats / • Stored as integers / • Sampled as normalized floats / 255 → 1.0. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Numeric Texture Representations. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Texel values can be interpreted differently / ■ Normalized integer formats / • Stored as integers / • Sampled as normalized floats / 255 → 1.0. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Numeric Texture Representations' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Numeric Texture Representations'?
Page 15	Color Textures vs. Data Textures
Source cue	■ Some textures store perceptual color / ■ Other textures store numeric shader data / ■ Color textures / • Base color / albedo / • Emissive color

Professor-style explanation	The slide 'Color Textures vs. Data Textures' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Some textures store perceptual color / ■ Other textures store numeric shader data / ■ Color textures / • Base color / albedo / • Emissive color. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Color Textures vs. Data Textures. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Some textures store perceptual color / ■ Other textures store numeric shader data / ■ Color textures / • Base color / albedo / • Emissive color. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Color Textures vs. Data Textures' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Color Textures vs. Data Textures'?
Page 16	2D Texture Access
Source cue	■ For each fragment, texturecoordinates are interpolated / ■ During texture access texel values are bilinearly interpolated

Professor-style explanation	The slide '2D Texture Access' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ For each fragment, texturecoordinates are interpolated / ■ During texture access texel values are bilinearly interpolated. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about 2D Texture Access. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ For each fragment, texturecoordinates are interpolated / ■ During texture access texel values are bilinearly interpolated. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for '2D Texture Access' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in '2D Texture Access' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after '2D Texture Access'?
Page 17	Bilinear Interpolation
Source cue	■ Example: linear interpolation in 2D (=bilinear) / $f = f(s, t)$ / ■ Interpolation requires three computations (Notation: i_j) / $s = s / f = (1 - \alpha) \cdot f + \alpha \cdot f_{\alpha} =$

Professor-style explanation	The slide 'Bilinear Interpolation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: ■ Example: linear interpolation in 2D (=bilinear) / $f = f(s, t)$ / ■ Interpolation requires three computations (Notation: $ij\ ij\ ij$) / $s-s\ / f = (1 - \alpha) \cdot f + \alpha \cdot f\ \alpha =$. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Bilinear Interpolation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: ■ Example: linear interpolation in 2D (=bilinear) / $f = f(s, t)$ / ■ Interpolation requires three computations (Notation: $ij\ ij\ ij$) / $s-s\ / f = (1 - \alpha) \cdot f + \alpha \cdot f\ \alpha =$. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Bilinear Interpolation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Bilinear Interpolation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Bilinear Interpolation' into a causal sentence instead of repeating the slide title?
Page 18	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.

Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?

Page 19	Texture Memory Footprint
Source cue	■ Texture memory depends on resolution and format / $\text{memory} = \text{width} \cdot \text{height} \cdot \text{bytes per texel}$ / ■ Example memory costs / • RGBA8: 4 MiB / • RGBA8: 16 MiB
Professor-style explanation	The slide 'Texture Memory Footprint' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: ■ Texture memory depends on resolution and format / $\text{memory} = \text{width} \cdot \text{height} \cdot \text{bytes per texel}$ / ■ Example memory costs / • RGBA8: 4 MiB / • RGBA8: 16 MiB. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations. For examination purposes, the important content is the chain width x height -> pixel count -> bits per pixel -> memory size; this chain also explains bandwidth and performance pressure.
Technical commentary	This slide is about Texture Memory Footprint. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: ■ Texture memory depends on resolution and format / $\text{memory} = \text{width} \cdot \text{height} \cdot \text{bytes per texel}$ / ■ Example memory costs / • RGBA8: 4 MiB / • RGBA8: 16 MiB. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.

Exam-grade answer	A strong answer for 'Texture Memory Footprint' includes the formula relation width x height x bits per pixel, distinguishes bits from bytes, and links larger images to memory bandwidth.
Common trap	Do not confuse bits with bytes in 'Texture Memory Footprint', and do not ignore that alpha or higher precision changes memory size.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'Texture Memory Footprint'?
Page 20	Texture Compression and Asset Pipelines
Source cue	■ File formats and GPU formats are different concepts / ■ File formats / • PNG, JPEG, OpenEXR / • Used for storage and authoring / ■ GPU formats
Professor-style explanation	The slide 'Texture Compression and Asset Pipelines' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ File formats and GPU formats are different concepts / ■ File formats / • PNG, JPEG, OpenEXR / • Used for storage and authoring / ■ GPU formats. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Texture Compression and Asset Pipelines. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ File formats and GPU formats are different concepts / ■ File formats / • PNG, JPEG, OpenEXR / • Used for storage and authoring / ■ GPU formats. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Texture Compression and Asset Pipelines' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Texture Compression and Asset Pipelines'?
Page 21	Steps Involved in Texturing

Source cue	■ Provide textures (e.g., load from file) / ■ Configure texture mapping(e.g., border handling, texture coordinates, texture filtering) / ■ Rendering / • Fetch texture values (usually by exploiting interpolation) / • Modify fragment color based on fetched texel value
Professor-style explanation	The slide 'Steps Involved in Texturing' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Provide textures (e.g., load from file) / ■ Configure texture mapping(e.g., border handling, texture coordinates, texture filtering) / ■ Rendering / • Fetch texture values (usually by exploiting interpolation) / • Modify fragment color based on fetched texel value. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Steps Involved in Texturing. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Provide textures (e.g., load from file) / ■ Configure texture mapping(e.g., border handling, texture coordinates, texture filtering) / ■ Rendering / • Fetch texture values (usually by exploiting interpolation) / • Modify fragment color based on fetched texel value. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Steps Involved in Texturing' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Steps Involved in Texturing' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Steps Involved in Texturing'?
Page 22	9.2 Texture Coordinates, UV Mapping, and Wrapping
Source cue	Aligning texture space with geometry

Professor-style explanation	The slide '9.2 Texture Coordinates, UV Mapping, and Wrapping' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Aligning texture space with geometry. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about 9.2 Texture Coordinates, UV Mapping, and Wrapping. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Aligning texture space with geometry. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for '9.2 Texture Coordinates, UV Mapping, and Wrapping' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in '9.2 Texture Coordinates, UV Mapping, and Wrapping' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after '9.2 Texture Coordinates, UV Mapping, and Wrapping'?
Page 23	Texture Coordinate System 1/2
Source cue	■ Object space: 3D vertices in model coordinate system / (x, y, z) / • Coordinate representation: / ■ Texture space: 1D, 2D, 3D space of texture data / (s, t, r, q)

Professor-style explanation	The slide 'Texture Coordinate System 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Object space: 3D vertices in model coordinate system / (x, y, z) / • Coordinate representation: / ■ Texture space: 1D, 2D, 3D space of texture data / (s, t, r, q). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Texture Coordinate System 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Object space: 3D vertices in model coordinate system / (x, y, z) / • Coordinate representation: / ■ Texture space: 1D, 2D, 3D space of texture data / (s, t, r, q). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Texture Coordinate System 1/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Texture Coordinate System 1/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate System 1/2'?
Page 24	Texture Coordinate System 2/2
Source cue	[0, 1]² / ■ Texture coordinate system normalized to for 2D textures (analog for 1D and 3D textures) / s, t, r, q s, t, r, q ∈ [0, 1] / • Coordinate axes denoted by with / (s)

Professor-style explanation	The slide 'Texture Coordinate System 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: $[0, 1]^2$ / ■ Texture coordinate system normalized to for 2D textures (analog for 1D and 3D textures) / $s, t, r, q \in [0, 1]$ / • Coordinate axes denoted by s, t, r, q (s). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Texture Coordinate System 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: $[0, 1]^2$ / ■ Texture coordinate system normalized to for 2D textures (analog for 1D and 3D textures) / $s, t, r, q \in [0, 1]$ / • Coordinate axes denoted by s, t, r, q (s). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Texture Coordinate System 2/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Texture Coordinate System 2/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate System 2/2'?
Page 25	Texture Coordinate Specification
Source cue	■ 2D texturing / $p = (x, y, z)$ / $p = (s, t)$ / • For every vertex g of a polygon, texture coordinates t are specified / • Texture coordinates t are obtained from parametrization algorithms / • Texture coordinates t are linearly interpolated over a polygon (analogous to Gouraud shading)

Professor-style explanation	The slide 'Texture Coordinate Specification' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ 2D texturing / $p = (x, y, z)$ $p = (s, t)$ / • For every vertex g of a polygon, texture coordinates t are specified / • Texture coordinates t are obtained from parametrization algorithms / • Texture coordinates t are linearly interpolated over a polygon (analogous to Gouraud shading). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Texture Coordinate Specification. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ 2D texturing / $p = (x, y, z)$ $p = (s, t)$ / • For every vertex g of a polygon, texture coordinates t are specified / • Texture coordinates t are obtained from parametrization algorithms / • Texture coordinates t are linearly interpolated over a polygon (analogous to Gouraud shading). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Texture Coordinate Specification' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Texture Coordinate Specification' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate Specification'?
Page 26	Disc Parametrization
Source cue	r xy / ■ Circular disc with radius r, centered in the plane / $p = (x, y, z) = (a \cdot \cos(\theta), a \cdot \sin(\theta), 0)$ $\theta \in [0, 2\pi]$ $a \in [0, r]$ / • g with and / $p = (s, t) \in [0, 1]^2$ $s = \theta$ $t = a$

Professor-style explanation	The slide 'Disc Parametrization' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: $r \times y / \blacksquare$ Circular disc with radius r, centered in the plane $p = (x, y, z) = (a \cdot \cos(\theta), a \cdot \sin(\theta), 0)$ $\theta \in [0, 2\pi]$ $a \in [0, r]$ $\bullet$ g with and $p = (s, t) \in [0, 1]^2$ $s = \theta$ $t = a$. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Disc Parametrization. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: $r \times y / \blacksquare$ Circular disc with radius r, centered in the plane $p = (x, y, z) = (a \cdot \cos(\theta), a \cdot \sin(\theta), 0)$ $\theta \in [0, 2\pi]$ $a \in [0, r]$ $\bullet$ g with and $p = (s, t) \in [0, 1]^2$ $s = \theta$ $t = a$. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Disc Parametrization' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Disc Parametrization' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Disc Parametrization' into a causal sentence instead of repeating the slide title?
Page 27	Cylinder Parametrization
Source cue	$x y r = 1 h z / \blacksquare$ Cylinder based in plane with radius and height along axis (surface is topologically / two-dimensional and overlap-free) $p = (x, y, z) = (r \cdot \cos(\theta), r \cdot \sin(\theta), z)$ $\theta \in [0, 2\pi]$ $z \in [0, h]$ $\bullet$ g with and
Professor-style explanation	The slide 'Cylinder Parametrization' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: $x y r = 1 h z / \blacksquare$ Cylinder based in plane with radius and height along axis (surface is topologically / two-dimensional and overlap-free) $p = (x, y, z) = (r \cdot \cos(\theta), r \cdot \sin(\theta), z)$ $\theta \in [0, 2\pi]$ $z \in [0, h]$ $\bullet$ g with and. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Cylinder Parametrization. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: $xy \ r = 1 \ h \ z$ / ■ Cylinder based in plane with radius and height along axis (surface is topologically / two-dimensional and overlap-free) / $p = (x, y, z) = (r \cdot \cos(\theta), r \cdot \sin(\theta), z) \ \theta \in [0, 2\pi] \ z \in [0, h]$ / • g with and. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Cylinder Parametrization' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Cylinder Parametrization' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Cylinder Parametrization' into a causal sentence instead of repeating the slide title?
Page 28	Two-Part Mapping 1/2
Source cue	■ Texture coordinates for complex geometries are derived from intermediate objects (i.e., geometries / with inherent parametrization) / ■ Proceeding / • Calculate texture coordinates for intermediate object / $p = (x, y, z) \ p = (s, t) \ p$
Professor-style explanation	The slide 'Two-Part Mapping 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Texture coordinates for complex geometries are derived from intermediate objects (i.e., geometries / with inherent parametrization) / ■ Proceeding / • Calculate texture coordinates for intermediate object / $p = (x, y, z) \ p = (s, t) \ p$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.

Technical commentary	This slide is about Two-Part Mapping 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Texture coordinates for complex geometries are derived from intermediate objects (i.e., geometries / with inherent parametrization) / ■ Proceeding / • Calculate texture coordinates for intermediate object / $p = (x, y, z)$ $p = (s, t)$ p . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Two-Part Mapping 1/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Two-Part Mapping 1/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Two-Part Mapping 1/2'?

Page 29	Two-Part Mapping 2/2
Source cue	■ Two-part mapping has no restrictions wrt. objects to be textured / ■ Intermediate object must be appropriate for object to be textured
Professor-style explanation	The slide 'Two-Part Mapping 2/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Two-part mapping has no restrictions wrt. objects to be textured / ■ Intermediate object must be appropriate for object to be textured. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Two-Part Mapping 2/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Two-part mapping has no restrictions wrt. objects to be textured / ■ Intermediate object must be appropriate for object to be textured. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.

Exam-grade answer	A strong answer for 'Two-Part Mapping 2/2' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Two-Part Mapping 2/2'?
Page 30	Two-Part Mapping Example
Source cue	■ Object: non-convex polyhedron / ■ Intermediate object: cylinder / • 1 : Cast rays from object center through vertices to intersect cylinder (projection) / • 2 : Cylinder parametrization / $F(p) = F(F(p)) = (s, t)$
Professor-style explanation	The slide 'Two-Part Mapping Example' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ Object: non-convex polyhedron / ■ Intermediate object: cylinder / • 1 : Cast rays from object center through vertices to intersect cylinder (projection) / • 2 : Cylinder parametrization / $F(p) = F(F(p)) = (s, t)$. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Two-Part Mapping Example. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ Object: non-convex polyhedron / ■ Intermediate object: cylinder / • 1 : Cast rays from object center through vertices to intersect cylinder (projection) / • 2 : Cylinder parametrization / $F(p) = F(F(p)) = (s, t)$. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Two-Part Mapping Example' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Two-Part Mapping Example' works per object, per image region, per ray, or per fragment?
Page 31	Projection Calculation Variations

Source cue	O O / ■ G center through vertices to I / O O / ■ G along normal to I / O O
Professor-style explanation	The slide 'Projection Calculation Variations' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: O O / ■ G center through vertices to I / O O / ■ G along normal to I / O O. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Projection Calculation Variations. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: O O / ■ G center through vertices to I / O O / ■ G along normal to I / O O. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Projection Calculation Variations' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Projection Calculation Variations' changes positions before rasterization?
Page 32	Texture Coordinate Transformation
Source cue	■ Texture coordinates can be transformed through transformation matrix / • No restriction regarding matrix coefficients (4x4 matrix) / ■ Applications / • Shifting texture coordinates / • Scaling texture coordinates

Professor-style explanation	The slide 'Texture Coordinate Transformation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Texture coordinates can be transformed through transformation matrix / • No restriction regarding matrix coefficients (4x4 matrix) / ■ Applications / • Shifting texture coordinates / • Scaling texture coordinates. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Texture Coordinate Transformation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Texture coordinates can be transformed through transformation matrix / • No restriction regarding matrix coefficients (4x4 matrix) / ■ Applications / • Shifting texture coordinates / • Scaling texture coordinates. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Texture Coordinate Transformation' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Texture Coordinate Transformation' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate Transformation'?
Page 33	Demo - Texture Coordinate Transformation
Source cue	Stage Transformed Texture / scaleS 1 / scaleT 1 / offsetS 0 / offsetT 0

Professor-style explanation	The slide 'Demo - Texture Coordinate Transformation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Stage Transformed Texture / scaleS 1 / scaleT 1 / offsetS 0 / offsetT 0. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Demo - Texture Coordinate Transformation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Stage Transformed Texture / scaleS 1 / scaleT 1 / offsetS 0 / offsetT 0. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Demo - Texture Coordinate Transformation' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Demo - Texture Coordinate Transformation' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Demo - Texture Coordinate Transformation'?
Page 34	Height Field Texturing Example
Source cue	■ Height field represented as triangular mesh / (x, y) (s, t) / • Use normalized coordinates as texture coordinates (can be used to map aerial / image) / z r

Professor-style explanation	The slide 'Height Field Texturing Example' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Height field represented as triangular mesh / (x, y) (s, t) / • Use normalized coordinates as texture coordinates (can be used to map aerial / image) / z r. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Height Field Texturing Example. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Height field represented as triangular mesh / (x, y) (s, t) / • Use normalized coordinates as texture coordinates (can be used to map aerial / image) / z r. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Height Field Texturing Example' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Height Field Texturing Example' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Height Field Texturing Example'?
Page 35	9.3 Sampling, Filtering, Mipmaps, and Anisotropy
Source cue	Reconstructing texture values under magnification and minification

Professor-style explanation	The slide '9.3 Sampling, Filtering, Mipmaps, and Anisotropy' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: Reconstructing texture values under magnification and minification. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about 9.3 Sampling, Filtering, Mipmaps, and Anisotropy. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: Reconstructing texture values under magnification and minification. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for '9.3 Sampling, Filtering, Mipmaps, and Anisotropy' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in '9.3 Sampling, Filtering, Mipmaps, and Anisotropy'?
Page 36	Forward vs. Inverse Mapping
Source cue	■ Forward mapping - from texture to screen space / • Object's surface parameterization / • Projection transformation / ■ Inverse mapping - from screen to texture space / • Find corresponding pre-image/footprint of each pixel in texture

Professor-style explanation	The slide 'Forward vs. Inverse Mapping' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Forward mapping - from texture to screen space / • Object's surface parameterization / • Projection transformation / ■ Inverse mapping - from screen to texture space / • Find corresponding pre-image/footprint of each pixel in texture. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Forward vs. Inverse Mapping. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Forward mapping - from texture to screen space / • Object's surface parameterization / • Projection transformation / ■ Inverse mapping - from screen to texture space / • Find corresponding pre-image/footprint of each pixel in texture. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Forward vs. Inverse Mapping' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Forward vs. Inverse Mapping' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Forward vs. Inverse Mapping'?
Page 37	Forward Mapping
Source cue	■ Maps each texel to screen space / • Uniform sampling of texture does not guarantee uniform sampling in screen space / ■ Forward mapping used for following cases / • Texture-to-screen mapping difficult to invert / • Texture image does not fit into memory

Professor-style explanation	The slide 'Forward Mapping' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Maps each texel to screen space / • Uniform sampling of texture does not guarantee uniform sampling in screen space / ■ Forward mapping used for following cases / • Texture-to-screen mapping difficult to invert / • Texture image does not fit into memory. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Forward Mapping. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Maps each texel to screen space / • Uniform sampling of texture does not guarantee uniform sampling in screen space / ■ Forward mapping used for following cases / • Texture-to-screen mapping difficult to invert / • Texture image does not fit into memory. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Forward Mapping' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Forward Mapping'?
Page 38	Inverse Mapping 1/2
Source cue	■ For each pixel pre-image in texture space is found and its area is integrated over / ■ Inverse mapping can be used when / • Texture image fits into memory / • Forward mapping can be inverted / 1 for y in range(height): # screen 'height'

Professor-style explanation	The slide 'Inverse Mapping 1/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ For each pixel pre-image in texture space is found and its area is integrated over / ■ Inverse mapping can be used when / • Texture image fits into memory / • Forward mapping can be inverted / 1 for y in range(height): # screen 'height'. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Inverse Mapping 1/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ For each pixel pre-image in texture space is found and its area is integrated over / ■ Inverse mapping can be used when / • Texture image fits into memory / • Forward mapping can be inverted / 1 for y in range(height): # screen 'height'. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Inverse Mapping 1/2' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Inverse Mapping 1/2'?
Page 39	Inverse Mapping 2/2
Source cue	■ Pre-image of square screen pixel intersecting curved surface is curvilinear quadrilateral in texture / space / ■ Curvilinear quadrilateral often approximated / • By general quadrilateral / • By parallelogram

Professor-style explanation	The slide 'Inverse Mapping 2/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Pre-image of square screen pixel intersecting curved surface is curvilinear quadrilateral in texture / space / ■ Curvilinear quadrilateral often approximated / • By general quadrilateral / • By parallelogram. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Inverse Mapping 2/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Pre-image of square screen pixel intersecting curved surface is curvilinear quadrilateral in texture / space / ■ Curvilinear quadrilateral often approximated / • By general quadrilateral / • By parallelogram. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Inverse Mapping 2/2' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Inverse Mapping 2/2'?
Page 40	Magnification vs. Minification
Source cue	■ Magnification / • Map few texels onto many pixels / • Two filtering approaches / • Nearest: Take the nearest texel / • Bilinear interpolation: Interpolation between 4 nearest texels

Professor-style explanation	The slide 'Magnification vs. Minification' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Magnification / • Map few texels onto many pixels • Two filtering approaches • Nearest: Take the nearest texel • Bilinear interpolation: Interpolation between 4 nearest texels. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Magnification vs. Minification. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Magnification / • Map few texels onto many pixels • Two filtering approaches • Nearest: Take the nearest texel • Bilinear interpolation: Interpolation between 4 nearest texels. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Magnification vs. Minification' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Magnification vs. Minification'?
Page 41	Nearest vs. Linear Interpolation
Source cue	■ Nearest neighbor interpolation / (s, t) / • For coordinates select nearest integer texel / $(\text{int}(\text{round}(s \cdot \text{texdim})), \text{int}(\text{round}(t \cdot \text{texdim}))) / x y$

Professor-style explanation	The slide 'Nearest vs. Linear Interpolation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Nearest neighbor interpolation / (s, t) / • For coordinates select nearest integer texel / (int(round(s · texdim)), int(round(t · texdim))) / x y. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Nearest vs. Linear Interpolation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Nearest neighbor interpolation / (s, t) / • For coordinates select nearest integer texel / (int(round(s · texdim)), int(round(t · texdim))) / x y. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Nearest vs. Linear Interpolation' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Nearest vs. Linear Interpolation' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Nearest vs. Linear Interpolation'?
Page 42	Minification Filtering Requirements
Source cue	■ Preimage integration with insufficient filtering results in aliasing / artifacts

Professor-style explanation	The slide 'Minification Filtering Requirements' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Preimage integration with insufficient filtering results in aliasing / artifacts. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Minification Filtering Requirements. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Preimage integration with insufficient filtering results in aliasing / artifacts. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Minification Filtering Requirements' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Minification Filtering Requirements'?
Page 43	Demo - Magnification Filtering: Nearest vs Linear
Source cue	Stage Split Comparison / magFilter GL_NEAREST / zoom 12 / textureResolution 8x8 / showTexelGrid

Professor-style explanation	The slide 'Demo - Magnification Filtering: Nearest vs Linear' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: Stage Split Comparison / magFilter GL_NEAREST / zoom 12 / textureResolution 8x8 / showTexelGrid. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Demo - Magnification Filtering: Nearest vs Linear. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: Stage Split Comparison / magFilter GL_NEAREST / zoom 12 / textureResolution 8x8 / showTexelGrid. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Demo - Magnification Filtering: Nearest vs Linear' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Magnification Filtering: Nearest vs Linear'?
Page 44	MIP Mapping (multum in parvo = 'much in little')
Source cue	■ Efficient scheme for sampling texture values / ■ Representation of a texture at different resolution levels(levels of Detail, LoDs) / ■ Exploiting a texture pyramid (here: 2D, analog for 1D and 3D) / • Each pyramid level contains filtered texture of half resolution of previous level / • Each layer contains only quarter of texels of previous layer

Professor-style explanation	The slide 'MIP Mapping (multum in parvo = 'much in little')' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Efficient scheme for sampling texture values / ■ Representation of a texture at different resolution levels(levels of Detail, LoDs) / ■ Exploiting a texture pyramid (here: 2D, analog for 1D and 3D) / • Each pyramid level contains filtered texture of half resolution of previous level / • Each layer contains only quarter of texels of previous layer. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about MIP Mapping (multum in parvo = 'much in little'). The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Efficient scheme for sampling texture values / ■ Representation of a texture at different resolution levels(levels of Detail, LoDs) / ■ Exploiting a texture pyramid (here: 2D, analog for 1D and 3D) / • Each pyramid level contains filtered texture of half resolution of previous level / • Each layer contains only quarter of texels of previous layer. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'MIP Mapping (multum in parvo = 'much in little')' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'MIP Mapping (multum in parvo = 'much in little')'?
Page 45	MIP Map Pyramid 1/2
Source cue	■ MIP Map-Level 0 : Texture in original size / $2N-1$ / ■ MIP Map-Level 1: Texture size / ■ ... / $N/20$

Professor-style explanation	The slide 'MIP Map Pyramid 1/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ MIP Map-Level 0 : Texture in original size / $2N-1$ / ■ MIP Map-Level 1: Texture size / ■ ... / $N/20$. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about MIP Map Pyramid 1/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ MIP Map-Level 0 : Texture in original size / $2N-1$ / ■ MIP Map-Level 1: Texture size / ■ ... / $N/20$. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'MIP Map Pyramid 1/2' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'MIP Map Pyramid 1/2'?
Page 46	MIP Map Pyramid 2/2
Source cue	■ Example MIP Map / ■ Self-Contained MIP Map for efficient storage
Professor-style explanation	The slide 'MIP Map Pyramid 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: ■ Example MIP Map / ■ Self-Contained MIP Map for efficient storage. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about MIP Map Pyramid 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: ■ Example MIP Map / ■ Self-Contained MIP Map for efficient storage. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'MIP Map Pyramid 2/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'MIP Map Pyramid 2/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'MIP Map Pyramid 2/2' into a causal sentence instead of repeating the slide title?

Page 47	MIP Mapping Memory
Source cue	■ 33% more memory is needed
Professor-style explanation	The slide 'MIP Mapping Memory' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: ■ 33% more memory is needed. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations. For examination purposes, the important content is the chain width x height -> pixel count -> bits per pixel -> memory size; this chain also explains bandwidth and performance pressure.
Technical commentary	This slide is about MIP Mapping Memory. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: ■ 33% more memory is needed. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Exam-grade answer	A strong answer for 'MIP Mapping Memory' includes the formula relation width x height x bits per pixel, distinguishes bits from bytes, and links larger images to memory bandwidth.
Common trap	Do not confuse bits with bytes in 'MIP Mapping Memory', and do not ignore that alpha or higher precision changes memory size.

Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'MIP Mapping Memory'?
Page 48	MIP Mapping Procedure
Source cue	$\lambda \in [0, N]$ / ■ Selection of the resolution levels based on continuous LoD level for a given fragment / ■ Procedure / • LoD level calculation / • Intra-level interpolation: interpolation within two levels closest to
Professor-style explanation	The slide 'MIP Mapping Procedure' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: $\lambda \in [0, N]$ / ■ Selection of the resolution levels based on continuous LoD level for a given fragment / ■ Procedure / • LoD level calculation / • Intra-level interpolation: interpolation within two levels closest to. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about MIP Mapping Procedure. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: $\lambda \in [0, N]$ / ■ Selection of the resolution levels based on continuous LoD level for a given fragment / ■ Procedure / • LoD level calculation / • Intra-level interpolation: interpolation within two levels closest to. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'MIP Mapping Procedure' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'MIP Mapping Procedure' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'MIP Mapping Procedure' into a causal sentence instead of repeating the slide title?
Page 49	LoD Calculation 1/2
Source cue	■ Assumptions / $[0, 1]2$ / • 2D texture is parametrized with texture coordinates / $M \cdot M = 2N$ / • 2D texture has texels with (power-of-two texture)

Professor-style explanation	The slide 'LoD Calculation 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Assumptions / $[0, 1]^2$ / • 2D texture is parametrized with texture coordinates / $M \cdot M = 2^N$ / • 2D texture has texels with (power-of-two texture). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about LoD Calculation 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Assumptions / $[0, 1]^2$ / • 2D texture is parametrized with texture coordinates / $M \cdot M = 2^N$ / • 2D texture has texels with (power-of-two texture). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'LoD Calculation 1/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'LoD Calculation 1/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'LoD Calculation 1/2'?
Page 50	LoD Calculation 2/2
Source cue	■ Approximation of the preimage by square / ■ Calculation of the MIP map level / • Area of approximating square is / $b = A$ / • Length of square's side is

Professor-style explanation	The slide 'LoD Calculation 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: ■ Approximation of the preimage by square / ■ Calculation of the MIP map level / • Area of approximating square is / $b = A /$ • Length of square's side is. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about LoD Calculation 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: ■ Approximation of the preimage by square / ■ Calculation of the MIP map level / • Area of approximating square is / $b = A /$ • Length of square's side is. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'LoD Calculation 2/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'LoD Calculation 2/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'LoD Calculation 2/2' into a causal sentence instead of repeating the slide title?
Page 51	Demo - Minification, Mipmaps, and Mipmap Level Visualization
Source cue	Stage Minified Floor / minFilter GL_LINEAR_MIPMAP_LINEAR / mipmapsEnabled / lodBias 0 / visualizeMipLevels

Professor-style explanation	The slide 'Demo - Minification, Mipmaps, and Mipmap Level Visualization' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: Stage Minified Floor / minFilter GL_LINEAR_MIPMAP_LINEAR / mipmapsEnabled / lodBias 0 / visualizeMipLevels. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Demo - Minification, Mipmaps, and Mipmap Level Visualization. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: Stage Minified Floor / minFilter GL_LINEAR_MIPMAP_LINEAR / mipmapsEnabled / lodBias 0 / visualizeMipLevels. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Demo - Minification, Mipmaps, and Mipmap Level Visualization' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Minification, Mipmaps, and Mipmap Level Visualization'?
Page 52	Anisotropic Filtering
Source cue	■ Approximation of preimage through several squares

Professor-style explanation	The slide 'Anisotropic Filtering' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Approximation of preimage through several squares. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Anisotropic Filtering. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Approximation of preimage through several squares. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Anisotropic Filtering' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Anisotropic Filtering'?
Page 53	Demo - Anisotropic Filtering
Source cue	Stage 1x vs Selected Anisotropy / anisotropy 16x / minFilter GL_LINEAR_MIPMAP_LINEAR / textureFrequency 34 / cameraAngle 10

Professor-style explanation	The slide 'Demo - Anisotropic Filtering' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: Stage 1x vs Selected Anisotropy / anisotropy 16x / minFilter GL_LINEAR_MIPMAP_LINEAR / textureFrequency 34 / cameraAngle 10. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Demo - Anisotropic Filtering. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: Stage 1x vs Selected Anisotropy / anisotropy 16x / minFilter GL_LINEAR_MIPMAP_LINEAR / textureFrequency 34 / cameraAngle 10. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Demo - Anisotropic Filtering' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Demo - Anisotropic Filtering' changes positions before rasterization?
Page 54	9.4 Modern OpenGL Texture Pipeline
Source cue	Texture objects, sampler uniforms, and shader-based access

Professor-style explanation	The slide '9.4 Modern OpenGL Texture Pipeline' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Texture objects, sampler uniforms, and shader-based access. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about 9.4 Modern OpenGL Texture Pipeline. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Texture objects, sampler uniforms, and shader-based access. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for '9.4 Modern OpenGL Texture Pipeline' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '9.4 Modern OpenGL Texture Pipeline'?
Page 55	OpenGL Texturing
Source cue	■ Texturing procedure / • Fetch interpolated texel value from texture / • Use texel value to modulate fragment color or depth / ■ Configuration / • Per texture

Professor-style explanation	The slide 'OpenGL Texturing' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Texturing procedure / • Fetch interpolated texel value from texture / • Use texel value to modulate fragment color or depth / ■ Configuration / • Per texture. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Texturing. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Texturing procedure / • Fetch interpolated texel value from texture / • Use texel value to modulate fragment color or depth / ■ Configuration / • Per texture. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Texturing' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Texturing'?
Page 56	OpenGL Minification & Magnification 1/3
Source cue	■ OpenGL exposes two texture filtering parameters: / • GL_TEXTURE_MIN_FILTER / • GL_TEXTURE_MAG_FILTER / ■ Texture minification (GL_TEXTURE_MIN_FILTER) options / • GL_NEAREST - nearest neighbor interpolation

Professor-style explanation	The slide 'OpenGL Minification & Magnification 1/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ OpenGL exposes two texture filtering parameters: / • GL_TEXTURE_MIN_FILTER / • GL_TEXTURE_MAG_FILTER / ■ Texture minification (GL_TEXTURE_MIN_FILTER) options / • GL_NEAREST - nearest neighbor interpolation. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Minification & Magnification 1/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ OpenGL exposes two texture filtering parameters: / • GL_TEXTURE_MIN_FILTER / • GL_TEXTURE_MAG_FILTER / ■ Texture minification (GL_TEXTURE_MIN_FILTER) options / • GL_NEAREST - nearest neighbor interpolation. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Minification & Magnification 1/3' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 1/3'?
Page 57	OpenGL Minification & Magnification 2/3
Source cue	■ OpenGL bilinear filtering example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...

Professor-style explanation	The slide 'OpenGL Minification & Magnification 2/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ OpenGL bilinear filtering example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 . Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Minification & Magnification 2/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ OpenGL bilinear filtering example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 . Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Minification & Magnification 2/3' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 2/3'?
Page 58	OpenGL Minification & Magnification 3/3
Source cue	■ OpenGL mipmapping example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...

Professor-style explanation	The slide 'OpenGL Minification & Magnification 3/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ OpenGL mipmapping example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 . Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about OpenGL Minification & Magnification 3/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ OpenGL mipmapping example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 . Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'OpenGL Minification & Magnification 3/3' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 3/3'?
Page 59	Demo - Texture Units and Sampler Uniforms
Source cue	Stage Combined Sampling / textureUnit0 Checker / textureUnit1 Brick / textureUnit2 UV Test / baseSamplerUnit 0

Professor-style explanation	The slide 'Demo - Texture Units and Sampler Uniforms' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: Stage Combined Sampling / textureUnit0 Checker / textureUnit1 Brick / textureUnit2 UV Test / baseSamplerUnit 0. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Demo - Texture Units and Sampler Uniforms. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: Stage Combined Sampling / textureUnit0 Checker / textureUnit1 Brick / textureUnit2 UV Test / baseSamplerUnit 0. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Demo - Texture Units and Sampler Uniforms' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Texture Units and Sampler Uniforms'?
Page 60	Texture Wrapping 1/4
Source cue	■ Texture wrapping specifies border handling behavior when accessing texture outside the $[0, 1]^D$ / normalized texture value range / ■ Texture wrapping mode is part of texture specification, and specified separately for each texture / dimension

Professor-style explanation	The slide 'Texture Wrapping 1/4' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Texture wrapping specifies border handling behavior when accessing texture outside the $[0, 1]$D / normalized texture value range / ■ Texture wrapping mode is part of texture specification, and specified separately for each texture / dimension. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Texture Wrapping 1/4. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Texture wrapping specifies border handling behavior when accessing texture outside the $[0, 1]$D / normalized texture value range / ■ Texture wrapping mode is part of texture specification, and specified separately for each texture / dimension. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Texture Wrapping 1/4' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Texture Wrapping 1/4'?
Page 61	Texture Wrapping 2/4
Source cue	■ GL_CLAMP_TO_EDGE / • Texture coordinates below 0 become 0 / • Texture coordinates above 1 become 1 / • Results in constant continuation of texture / ■ GL_CLAMP_TO_BORDER

Professor-style explanation	The slide 'Texture Wrapping 2/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ GL_CLAMP_TO_EDGE / • Texture coordinates below 0 become 0 / • Texture coordinates above 1 become 1 / • Results in constant continuation of texture / ■ GL_CLAMP_TO_BORDER. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Texture Wrapping 2/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ GL_CLAMP_TO_EDGE / • Texture coordinates below 0 become 0 / • Texture coordinates above 1 become 1 / • Results in constant continuation of texture / ■ GL_CLAMP_TO_BORDER. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Texture Wrapping 2/4' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Texture Wrapping 2/4' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Texture Wrapping 2/4'?
Page 62	Texture Wrapping 3/4
Source cue	■ GL_REPEAT / • Texture coordinates ignore digits before decimal point, and use digits after decimal point as / texture coordinates / • Results in periodic continuation of normalized texture space / ■ GL_MIRRORED_REPEAT

Professor-style explanation	The slide 'Texture Wrapping 3/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ GL_REPEAT / • Texture coordinates ignore digits before decimal point, and use digits after decimal point as / texture coordinates / • Results in periodic continuation of normalized texture space / ■ GL_MIRRORED_REPEAT. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Texture Wrapping 3/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ GL_REPEAT / • Texture coordinates ignore digits before decimal point, and use digits after decimal point as / texture coordinates / • Results in periodic continuation of normalized texture space / ■ GL_MIRRORED_REPEAT. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Texture Wrapping 3/4' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Texture Wrapping 3/4' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Texture Wrapping 3/4'?
Page 63	Texture Wrapping 4/4
Source cue	■ OpenGL code snippets / 1 glTexParameteri(textureID, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT); / 2 glTexParameteri(textureID, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT); / 1 const GLfloat borderColor[] = { 1.0f, 1.0f, 0.0f, 1.0f }; / 3 glTexParameteri(textureID, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER);

Professor-style explanation	The slide 'Texture Wrapping 4/4' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ OpenGL code snippets / 1 <code>glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT);</code> / 2 <code>glTextureParameteri(textureID, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT);</code> / 1 <code>const GLfloat borderColor[] = { 1.0f, 1.0f, 0.0f, 1.0f };</code> / 3 <code>glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER)</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Texture Wrapping 4/4. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ OpenGL code snippets / 1 <code>glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT);</code> / 2 <code>glTextureParameteri(textureID, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT);</code> / 1 <code>const GLfloat borderColor[] = { 1.0f, 1.0f, 0.0f, 1.0f };</code> / 3 <code>glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER)</code>. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Texture Wrapping 4/4' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texture Wrapping 4/4'?
Page 64	Texel Value Usage
Source cue	■ Older OpenGL versions support several modes for modulating fragment colors by texel colors / (today still relevant to reimplement in shader) - is current fragment with calculated color values / C A T C A / F and alpha values F - is current texture with color values T and alpha values T (or / L C A C

Professor-style explanation	The slide 'Texel Value Usage' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Older OpenGL versions support several modes for modulating fragment colors by texel colors / (today still relevant to reimplement in shader) - is current fragment with calculated color values / C A T C A / F and alpha values F - is current texture with color values T and alpha values T (or / L C A C. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Texel Value Usage. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Older OpenGL versions support several modes for modulating fragment colors by texel colors / (today still relevant to reimplement in shader) - is current fragment with calculated color values / C A T C A / F and alpha values F - is current texture with color values T and alpha values T (or / L C A C. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Texel Value Usage' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texel Value Usage'?
Page 65	Texturing in GLSL Shaders 1/3
Source cue	■ Textures are bound to texture units / ■ Sampler uniforms select the texture unit used by the shader / 1 glBindTextureUnit(0, texture2D); / 2 glBindTextureUnit(1, texture3D); / 4 glUseProgram(p);

Professor-style explanation	The slide 'Texturing in GLSL Shaders 1/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Textures are bound to texture units / ■ Sampler uniforms select the texture unit used by the shader / 1 glBindTextureUnit(0, texture2D); / 2 glBindTextureUnit(1, texture3D); / 4 glUseProgram(p). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Texturing in GLSL Shaders 1/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Textures are bound to texture units / ■ Sampler uniforms select the texture unit used by the shader / 1 glBindTextureUnit(0, texture2D); / 2 glBindTextureUnit(1, texture3D); / 4 glUseProgram(p). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Texturing in GLSL Shaders 1/3' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 1/3'?
Page 66	Texturing in GLSL Shaders 2/3
Source cue	■ Vertex Shader / 1 layout(location = 0) in vec3 position; / 2 layout(location = 1) in vec3 texCoord; / 4 out vec3 vTexCoord; / 6 uniform mat4 modelViewProjectionMatrix;

Professor-style explanation	The slide 'Texturing in GLSL Shaders 2/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Vertex Shader / 1 layout(location = 0) in vec3 position; / 2 layout(location = 1) in vec3 texCoord; / 4 out vec3 vTexCoord; / 6 uniform mat4 modelViewProjectionMatrix. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Texturing in GLSL Shaders 2/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Vertex Shader / 1 layout(location = 0) in vec3 position; / 2 layout(location = 1) in vec3 texCoord; / 4 out vec3 vTexCoord; / 6 uniform mat4 modelViewProjectionMatrix. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Texturing in GLSL Shaders 2/3' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 2/3'?
Page 67	Texturing in GLSL Shaders 3/3
Source cue	■ Fragment Shader / 1 in vec3 vTexCoord; / 3 out vec4 fragmentColor; / 5 uniform sampler2D texture2D_; / 6 uniform sampler3D texture3D_;

Professor-style explanation	The slide 'Texturing in GLSL Shaders 3/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Fragment Shader / 1 in vec3 vTexCoord; / 3 out vec4 fragmentColor; / 5 uniform sampler2D texture2D_; / 6 uniform sampler3D texture3D_. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Texturing in GLSL Shaders 3/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Fragment Shader / 1 in vec3 vTexCoord; / 3 out vec4 fragmentColor; / 5 uniform sampler2D texture2D_; / 6 uniform sampler3D texture3D_. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Texturing in GLSL Shaders 3/3' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 3/3'?
Page 68	9.5 Normal Mapping
Source cue	Encoding surface detail in texture space

Professor-style explanation	The slide '9.5 Normal Mapping' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: Encoding surface detail in texture space. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about 9.5 Normal Mapping. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: Encoding surface detail in texture space. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for '9.5 Normal Mapping' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '9.5 Normal Mapping'?
Page 69	Motivation
Source cue	■ Render content dichotomy / • Shape: Triangles are used to generate geometric details / k k k / • Appearance: Textures are used to generate material variations (a , d , s) / ■ Consequences

Professor-style explanation	The slide 'Motivation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: ■ Render content dichotomy / • Shape: Triangles are used to generate geometric details / k k k / • Appearance: Textures are used to generate material variations (a , d , s) / ■ Consequences. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Motivation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: ■ Render content dichotomy / • Shape: Triangles are used to generate geometric details / k k k / • Appearance: Textures are used to generate material variations (a , d , s) / ■ Consequences. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Motivation' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Motivation'?

Page 70	Mesh Compression
Source cue	[source]
Professor-style explanation	The slide 'Mesh Compression' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: [source]. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Mesh Compression. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: [source]. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Mesh Compression' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Mesh Compression' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Mesh Compression' into a causal sentence instead of repeating the slide title?

Page 71	Normal Mapping 1/2
Source cue	■ Breaks dichotomy by representing geometric details in textures, as texels represent variations in / surface normals / ■ Normal maps accompany diffuse maps with same resolution Material details and geometric / details match
Professor-style explanation	The slide 'Normal Mapping 1/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Breaks dichotomy by representing geometric details in textures, as texels represent variations in / surface normals / ■ Normal maps accompany diffuse maps with same resolution Material details and geometric / details match. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Normal Mapping 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Breaks dichotomy by representing geometric details in textures, as texels represent variations in / surface normals / ■ Normal maps accompany diffuse maps with same resolution Material details and geometric / details match. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.

Exam-grade answer	A strong answer for 'Normal Mapping 1/2' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Mapping 1/2'?
Page 72	Normal Mapping 2/2
Source cue	$c = k_d \frac{n \cdot l}{ n } + k_s \frac{v \cdot r}{ v } + a$ During illumination computation d is fetched from the diffuse map, while i is fetched from the / normal map / $i = k \cdot L \cdot k \cdot L \cdot \max(0, n \cdot l) + k \cdot L \cdot \max(0, v \cdot r)$ $p / a a + d d s s$
Professor-style explanation	The slide 'Normal Mapping 2/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: $k_d \frac{n \cdot l}{ n } + k_s \frac{v \cdot r}{ v } + a$ $p / a a + d d s s$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Normal Mapping 2/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: $k_d \frac{n \cdot l}{ n } + k_s \frac{v \cdot r}{ v } + a$ $p / a a + d d s s$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Normal Mapping 2/2' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Mapping 2/2'?
Page 73	Normal Maps

Source cue	■ Normal vector components can be positive or negative Components need to be normalized to $[0, 1]$ $[-1, 1]$ / before storage, and rescaled to after texture fetch $n' := (n) + 0.5$ / Before storage:
Professor-style explanation	The slide 'Normal Maps' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Normal vector components can be positive or negative Components need to be normalized to $[0, 1]$ $[-1, 1]$ / before storage, and rescaled to after texture fetch $n' := (n) + 0.5$ / Before storage. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Normal Maps. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Normal vector components can be positive or negative Components need to be normalized to $[0, 1]$ $[-1, 1]$ / before storage, and rescaled to after texture fetch $n' := (n) + 0.5$ / Before storage. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Normal Maps' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Maps'?
Page 74	Tangent Space
Source cue	■ Normal Map vectors align along axis and need to be transformed into view coordinate system / before illumination computation / ■ Tangent coordinate system spanned by t / s

Professor-style explanation	The slide 'Tangent Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Normal Map vectors align along axis and need to be transformed into view coordinate system / before illumination computation / ■ Tangent coordinate system spanned by / Normal $t / t s$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Tangent Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Normal Map vectors align along axis and need to be transformed into view coordinate system / before illumination computation / ■ Tangent coordinate system spanned by / Normal $t / t s$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Tangent Space' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Tangent Space' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Tangent Space'?
Page 75	TBN Matrix
Source cue	■ TBN matrix (Tangent, Bitangent, and Normal) is an orthonormal matrix, which transforms tangent / space normal to view space normal / ■ TBN matrix construction (similar to camera transformation) / • TBN matrix contains three perpendicular vectors / • First column vector: tangent

Professor-style explanation	The slide 'TBN Matrix' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ TBN matrix (Tangent, Bitangent, and Normal) is an orthonormal matrix, which transforms tangent / space normal to view space normal / ■ TBN matrix construction (similar to camera transformation) / • TBN matrix contains three perpendicular vectors / • First column vector: tangent. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about TBN Matrix. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ TBN matrix (Tangent, Bitangent, and Normal) is an orthonormal matrix, which transforms tangent / space normal to view space normal / ■ TBN matrix construction (similar to camera transformation) / • TBN matrix contains three perpendicular vectors / • First column vector: tangent. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'TBN Matrix' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'TBN Matrix' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'TBN Matrix'?
Page 76	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?

Page 77	Tangent Vector Derivation 1/2
Source cue	■ ■ / t b / Observation: triangle edges can be expressed based on basis given by and / ■ ■ / e ■ = $\Delta u \cdot t + \Delta v \cdot b$
Professor-style explanation	The slide 'Tangent Vector Derivation 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: ■ ■ / t b / Observation: triangle edges can be expressed based on basis given by and / ■ ■ / e ■ = $\Delta u \cdot t + \Delta v \cdot b$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.

Technical commentary	This slide is about Tangent Vector Derivation 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: $\mathbf{p} = \mathbf{t} \cdot \mathbf{b} + \mathbf{e}$ / Observation: triangle edges can be expressed based on basis given by $\mathbf{u}$ and $\mathbf{v}$ / $\mathbf{e} = \Delta \mathbf{u} \cdot \mathbf{t} + \Delta \mathbf{v} \cdot \mathbf{b}$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Tangent Vector Derivation 1/2' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Tangent Vector Derivation 1/2'?

Page 78	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.

Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 79	Tangent Vector Derivation 2/2
Source cue	■ ■ / t b / We can solve this matrix for and / t t t Δu Δv e e e / x y z 0 0 0x 0y 0z
Professor-style explanation	The slide 'Tangent Vector Derivation 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ ■ / t b / We can solve this matrix for and / t t t Δu Δv e e e / x y z 0 0 0x 0y 0z. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Tangent Vector Derivation 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ ■ / t b / We can solve this matrix for and / t t t Δu Δv e e e / x y z 0 0 0x 0y 0z. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Tangent Vector Derivation 2/2' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Tangent Vector Derivation 2/2' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Tangent Vector Derivation 2/2'?
Page 80	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?

Page 81	Observations
Source cue	$\mathbf{t} \perp \mathbf{b} \Rightarrow \mathbf{t} \perp \mathbf{b}$ / and are perpendicular to each other as well as to $\mathbf{t}$ only one, or , needs to be / computed, while the other can be derived through cross product / $\mathbf{t} \perp \mathbf{b}$
Professor-style explanation	The slide 'Observations' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: $\mathbf{t} \perp \mathbf{b} \Rightarrow \mathbf{t} \perp \mathbf{b}$ / and are perpendicular to each other as well as to $\mathbf{t}$ only one, or , needs to be / computed, while the other can be derived through cross product / $\mathbf{t} \perp \mathbf{b}$. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about Observations. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: $\mathbf{n} \perp \mathbf{t} \perp \mathbf{b}$ / $\mathbf{t} \perp \mathbf{n} \Rightarrow \mathbf{t} \perp \mathbf{b}$ / and are perpendicular to each other as well as to $\mathbf{t}$ only one, or $\mathbf{n}$, needs to be computed, while the other can be derived through cross product / $\mathbf{n} \times \mathbf{t} = \mathbf{b}$. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Observations' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Observations' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Observations' into a causal sentence instead of repeating the slide title?

Page 82	Variations
Source cue	■ Different variations of normal maps exist • Bump maps - specify displacement of surface points in normal direction • Vector offset bump maps - offset vector added to normal • Vector rotation bump maps - normal rotated given angle
Professor-style explanation	The slide 'Variations' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Different variations of normal maps exist / • Bump maps - specify displacement of surface points in normal direction / • Vector offset bump maps - offset vector added to normal / • Vector rotation bump maps - normal rotated given angle. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Variations. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Different variations of normal maps exist / • Bump maps - specify displacement of surface points in normal direction / • Vector offset bump maps - offset vector added to normal / • Vector rotation bump maps - normal rotated given angle. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.

Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Variations' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Variations'?

Page 83	9.6 Environment Mapping
Source cue	Sampling directional environment data for reflections and lighting
Professor-style explanation	The slide '9.6 Environment Mapping' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: Sampling directional environment data for reflections and lighting. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about 9.6 Environment Mapping. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: Sampling directional environment data for reflections and lighting. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for '9.6 Environment Mapping' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '9.6 Environment Mapping'?

Page 84	Environment Mapping Motivation
Source cue	■ Many reflective and glossy materials show light from their surroundings / ■ Explicitly tracing all reflection rays is expensive in interactive rendering / ■ Environment maps provide a compact representation of incident radiance / ■ Typical use cases • mirror-like reflections

Professor-style explanation	The slide 'Environment Mapping Motivation' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ Many reflective and glossy materials show light from their surroundings / ■ Explicitly tracing all reflection rays is expensive in interactive rendering / ■ Environment maps provide a compact representation of incident radiance / ■ Typical use cases / • mirror-like reflections. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Environment Mapping Motivation. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ Many reflective and glossy materials show light from their surroundings / ■ Explicitly tracing all reflection rays is expensive in interactive rendering / ■ Environment maps provide a compact representation of incident radiance / ■ Typical use cases / • mirror-like reflections. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Environment Mapping Motivation' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Environment Mapping Motivation' works per object, per image region, per ray, or per fragment?
Page 85	Environment Mapping Process
Source cue	■ Environment mapping simulates reflections by sampling an image of the surrounding scene / ■ The environment is treated as if it were infinitely far away / ■ The reflected view direction determines which part of the environment is visible / [Need for Speed no Limits]

Professor-style explanation	The slide 'Environment Mapping Process' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Environment mapping simulates reflections by sampling an image of the surrounding scene / ■ The environment is treated as if it were infinitely far away / ■ The reflected view direction determines which part of the environment is visible / [Need for Speed no Limits]. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Environment Mapping Process. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Environment mapping simulates reflections by sampling an image of the surrounding scene / ■ The environment is treated as if it were infinitely far away / ■ The reflected view direction determines which part of the environment is visible / [Need for Speed no Limits]. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Environment Mapping Process' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Process'?
Page 86	Environment Mapping Assumptions
Source cue	■ The environment is assumed to be far away / • translation of the object does not change the sampled environment / ■ The reflected object does not reflect itself / ■ Nearby dynamic objects are usually missing / ■ The approximation works best for

Professor-style explanation	The slide 'Environment Mapping Assumptions' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ The environment is assumed to be far away / • translation of the object does not change the sampled environment / ■ The reflected object does not reflect itself / ■ Nearby dynamic objects are usually missing / ■ The approximation works best for. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Environment Mapping Assumptions. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ The environment is assumed to be far away / • translation of the object does not change the sampled environment / ■ The reflected object does not reflect itself / ■ Nearby dynamic objects are usually missing / ■ The approximation works best for. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Environment Mapping Assumptions' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Assumptions'?
Page 87	Environment Mapping 2/2
Source cue	$V \cdot N$ / ■ At the shaded surface point, the view vector is reflected about the normal / ■ The resulting reflection vector is used to access the environment map / ■ is calculated analog to the Phong illumination model / $R = 2 \cdot (V \cdot N) \cdot N - V$

Professor-style explanation	The slide 'Environment Mapping 2/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: $V \cdot N$ / ■ At the shaded surface point, the view vector is reflected about the normal / ■ The resulting reflection vector is used to access the environment map / ■ is calculated analog to the Phong illumination model $R = 2 \cdot (V \cdot N) \cdot N - V$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Environment Mapping 2/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: $V \cdot N$ / ■ At the shaded surface point, the view vector is reflected about the normal / ■ The resulting reflection vector is used to access the environment map / ■ is calculated analog to the Phong illumination model $R = 2 \cdot (V \cdot N) \cdot N - V$. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Environment Mapping 2/2' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Environment Mapping 2/2'?
Page 88	Demo - Cube-map Reflection Vector
Source cue	Stage Reflection Vector / object Sphere / showSkybox / reflectionMode Reflection / reflectivity 1

Professor-style explanation	The slide 'Demo - Cube-map Reflection Vector' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: Stage Reflection Vector / object Sphere / showSkybox / reflectionMode Reflection / reflectivity 1. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Demo - Cube-map Reflection Vector. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: Stage Reflection Vector / object Sphere / showSkybox / reflectionMode Reflection / reflectivity 1. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Demo - Cube-map Reflection Vector' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Demo - Cube-map Reflection Vector' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Demo - Cube-map Reflection Vector' into a causal sentence instead of repeating the slide title?
Page 89	Environment Mapping Types 1/2
Source cue	■ Different parameterizations store directional data in different ways / ■ They differ with respect to / • distortion • filtering behavior • hardware support

Professor-style explanation	The slide 'Environment Mapping Types 1/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Different parameterizations store directional data in different ways / ■ They differ with respect to / • distortion / • filtering behavior / • hardware support. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Environment Mapping Types 1/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Different parameterizations store directional data in different ways / ■ They differ with respect to / • distortion / • filtering behavior / • hardware support. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Environment Mapping Types 1/2' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Types 1/2'?
Page 90	Environment Mapping Types 2/2
Source cue	■ Spherical environment maps / • compact representation in one 2D texture / • strong distortion near the border / • difficult filtering and interpolation behavior / ■ Dual-paraboloid maps

Professor-style explanation	The slide 'Environment Mapping Types 2/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Spherical environment maps / • compact representation in one 2D texture / • strong distortion near the border / • difficult filtering and interpolation behavior / ■ Dual-paraboloid maps. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Environment Mapping Types 2/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Spherical environment maps / • compact representation in one 2D texture / • strong distortion near the border / • difficult filtering and interpolation behavior / ■ Dual-paraboloid maps. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Environment Mapping Types 2/2' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Types 2/2'?
Page 91	Spherical Mapping 1/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Spherical Mapping 1/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Spherical Mapping 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Spherical Mapping 1/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Spherical Mapping 1/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Spherical Mapping 1/2' into a causal sentence instead of repeating the slide title?
Page 92	Spherical Mapping 2/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Spherical Mapping 2/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.

Technical commentary	This slide is about Spherical Mapping 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Spherical Mapping 2/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Spherical Mapping 2/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Spherical Mapping 2/2' into a causal sentence instead of repeating the slide title?

Page 93	Spherical Environment Mapping
Source cue	■ A sphere map stores the environment as seen in a perfectly reflective sphere / ■ It was useful before cube maps became widely supported in graphics hardware / ■ It is mainly relevant today as historical context / ■ Important limitations / • non-uniform distortion
Professor-style explanation	The slide 'Spherical Environment Mapping' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ A sphere map stores the environment as seen in a perfectly reflective sphere / ■ It was useful before cube maps became widely supported in graphics hardware / ■ It is mainly relevant today as historical context / ■ Important limitations / • non-uniform distortion. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.

Technical commentary	This slide is about Spherical Environment Mapping. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ A sphere map stores the environment as seen in a perfectly reflective sphere / ■ It was useful before cube maps became widely supported in graphics hardware / ■ It is mainly relevant today as historical context / ■ Important limitations / • non-uniform distortion. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Spherical Environment Mapping' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping'?

Page 94	Sphere Map Distortion
Source cue	■ Sphere-map texels represent differently sized regions of the environment / ■ Distortion is especially problematic in the outer region of the map / ■ Hardware performs linear texture filtering, not spherical interpolation / ■ For modern real-time graphics, this is one reason cube maps are preferred
Professor-style explanation	The slide 'Sphere Map Distortion' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Sphere-map texels represent differently sized regions of the environment / ■ Distortion is especially problematic in the outer region of the map / ■ Hardware performs linear texture filtering, not spherical interpolation / ■ For modern real-time graphics, this is one reason cube maps are preferred. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.

Technical commentary	This slide is about Sphere Map Distortion. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Sphere-map texels represent differently sized regions of the environment / ■ Distortion is especially problematic in the outer region of the map / ■ Hardware performs linear texture filtering, not spherical interpolation / ■ For modern real-time graphics, this is one reason cube maps are preferred. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Sphere Map Distortion' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Sphere Map Distortion'?

Page 95	Demo - Environment Map Distortion
Source cue	Stage Debug Distortion / object Sphere / environmentContent Debug / mapType Spherical map / mappingType Reflection
Professor-style explanation	The slide 'Demo - Environment Map Distortion' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: Stage Debug Distortion / object Sphere / environmentContent Debug / mapType Spherical map / mappingType Reflection. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Demo - Environment Map Distortion. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: Stage Debug Distortion / object Sphere / environmentContent Debug / mapType Spherical map / mappingType Reflection. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.

Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Demo - Environment Map Distortion' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Environment Map Distortion'?

Page 96	Sphere Map Interpolation
Source cue	■ Sphere maps represent non-linear images / ■ Two sphere map texels should be interpolated along an arc / • Graphics hardware does not support spherical interpolation / • Interpolation in the inner area is little distorted / • Interpolation in the outer area is highly distorted
Professor-style explanation	The slide 'Sphere Map Interpolation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: ■ Sphere maps represent non-linear images / ■ Two sphere map texels should be interpolated along an arc / • Graphics hardware does not support spherical interpolation / • Interpolation in the inner area is little distorted / • Interpolation in the outer area is highly distorted. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Sphere Map Interpolation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: ■ Sphere maps represent non-linear images / ■ Two sphere map texels should be interpolated along an arc / • Graphics hardware does not support spherical interpolation / • Interpolation in the inner area is little distorted / • Interpolation in the outer area is highly distorted. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Sphere Map Interpolation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Sphere Map Interpolation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Sphere Map Interpolation' into a causal sentence instead of repeating the slide title?

Page 97	Sphere Map Coordinates
Source cue	■ Sphere map texture coordinates are calculated in camera coordinates / $(0, 0, 0) / \bullet$ Camera in origin / $V -z +x +y / \bullet$ View direction along the -axis (axis to the right, axis up)
Professor-style explanation	The slide 'Sphere Map Coordinates' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Sphere map texture coordinates are calculated in camera coordinates / $(0, 0, 0) / \bullet$ Camera in origin / $V -z +x +y / \bullet$ View direction along the -axis (axis to the right, axis up). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Sphere Map Coordinates. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Sphere map texture coordinates are calculated in camera coordinates / $(0, 0, 0) / \bullet$ Camera in origin / $V -z +x +y / \bullet$ View direction along the -axis (axis to the right, axis up). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Sphere Map Coordinates' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Sphere Map Coordinates' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Sphere Map Coordinates'?
Page 98	Cube Environment Mapping
Source cue	■ Cube maps are represented by 6 square textures forming the sides of a cube / ■ The cube is centered around the shading point or camera / ■ Each face stores the environment as seen through one side of the cube / ■ A 3D direction vector is used to select the face and lookup position

Professor-style explanation	The slide 'Cube Environment Mapping' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: ■ Cube maps are represented by 6 square textures forming the sides of a cube ■ The cube is centered around the shading point or camera ■ Each face stores the environment as seen through one side of the cube ■ A 3D direction vector is used to select the face and lookup position. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Cube Environment Mapping. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: ■ Cube maps are represented by 6 square textures forming the sides of a cube ■ The cube is centered around the shading point or camera ■ Each face stores the environment as seen through one side of the cube ■ A 3D direction vector is used to select the face and lookup position. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Cube Environment Mapping' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Cube Environment Mapping' changes positions before rasterization?
Page 99	Cube Mapping Example 1/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Cube Mapping Example 1/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Cube Mapping Example 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Cube Mapping Example 1/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Cube Mapping Example 1/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Cube Mapping Example 1/2' into a causal sentence instead of repeating the slide title?
Page 100	Cube Mapping Example 2/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Cube Mapping Example 2/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.

Technical commentary	This slide is about Cube Mapping Example 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Cube Mapping Example 2/2' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Cube Mapping Example 2/2' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Cube Mapping Example 2/2' into a causal sentence instead of repeating the slide title?

Page 101	Cube Mapping Distortion
Source cue	■ Cube maps still distort the environment / ■ The distortion is distributed more uniformly than in sphere maps / ■ There are no pole singularities / ■ The main artifacts occur at cube-face boundaries
Professor-style explanation	The slide 'Cube Mapping Distortion' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Cube maps still distort the environment / ■ The distortion is distributed more uniformly than in sphere maps / ■ There are no pole singularities / ■ The main artifacts occur at cube-face boundaries. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Cube Mapping Distortion. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Cube maps still distort the environment / ■ The distortion is distributed more uniformly than in sphere maps / ■ There are no pole singularities / ■ The main artifacts occur at cube-face boundaries. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.

Exam-grade answer	A strong answer for 'Cube Mapping Distortion' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Cube Mapping Distortion'?
Page 102	Cube Map Texture Coordinates
Source cue	$R = (R_x, R_y, R_z)$ / ■ The reflection vector $x\ y\ z$ is used as a 3D lookup direction / ■ The cube-map face is selected by the major axis of $ R_x , R_y , R_z $ / • largest absolute component determines the face / ■ The remaining two components determine the 2D coordinates on that face
Professor-style explanation	The slide 'Cube Map Texture Coordinates' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: $R = (R_x, R_y, R_z)$ / ■ The reflection vector $x\ y\ z$ is used as a 3D lookup direction / ■ The cube-map face is selected by the major axis of $ R_x , R_y , R_z $ / • largest absolute component determines the face / ■ The remaining two components determine the 2D coordinates on that face. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Cube Map Texture Coordinates. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: $R = (R_x, R_y, R_z)$ / ■ The reflection vector $x\ y\ z$ is used as a 3D lookup direction / ■ The cube-map face is selected by the major axis of $ R_x , R_y , R_z $ / • largest absolute component determines the face / ■ The remaining two components determine the 2D coordinates on that face. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Cube Map Texture Coordinates' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Cube Map Texture Coordinates' without naming coordinate spaces and order of operations.

Check yourself	Can you state the coordinate space before and after 'Cube Map Texture Coordinates'?
Page 103	Environment Mapping in OpenGL
Source cue	■ In modern OpenGL, reflections are usually stored in a cube map / • six square images form the faces of a virtual cube / • reflection vector is used as a 3D texture coordinate / ■ Typical use cases / • reflective objects
Professor-style explanation	The slide 'Environment Mapping in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ In modern OpenGL, reflections are usually stored in a cube map / • six square images form the faces of a virtual cube / • reflection vector is used as a 3D texture coordinate / ■ Typical use cases / • reflective objects. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Environment Mapping in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ In modern OpenGL, reflections are usually stored in a cube map / • six square images form the faces of a virtual cube / • reflection vector is used as a 3D texture coordinate / ■ Typical use cases / • reflective objects. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Environment Mapping in OpenGL' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Environment Mapping in OpenGL'?
Page 104	Cube Map Texture Object

Source cue	■ Cube maps are OpenGL texture objects with target GL_TEXTURE_CUBE_MAP / ■ Each face is addressed by a separate cube-map face target / ■ Use immutable storage for fixed size, format, and mipmap count / 1 GLuint envMap; / 2 glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &envMap);
Professor-style explanation	The slide 'Cube Map Texture Object' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Cube maps are OpenGL texture objects with target GL_TEXTURE_CUBE_MAP / ■ Each face is addressed by a separate cube-map face target / ■ Use immutable storage for fixed size, format, and mipmap count / 1 GLuint envMap; / 2 glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &envMap). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Cube Map Texture Object. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Cube maps are OpenGL texture objects with target GL_TEXTURE_CUBE_MAP / ■ Each face is addressed by a separate cube-map face target / ■ Use immutable storage for fixed size, format, and mipmap count / 1 GLuint envMap; / 2 glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &envMap). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Cube Map Texture Object' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Cube Map Texture Object'?
Page 105	Sampling Direction

Source cue	■ A cube map is sampled with a 3D direction vector / ■ The largest absolute component of the direction selects the cube face / ■ The remaining two components determine the 2D position on that face / ■ For reflection mapping, use the reflected view vector / 1 vec3 V = normalize(cameraPosition - worldPosition);
Professor-style explanation	The slide 'Sampling Direction' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: ■ A cube map is sampled with a 3D direction vector / ■ The largest absolute component of the direction selects the cube face / ■ The remaining two components determine the 2D position on that face / ■ For reflection mapping, use the reflected view vector / 1 vec3 V = normalize(cameraPosition - worldPosition). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Sampling Direction. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: ■ A cube map is sampled with a 3D direction vector / ■ The largest absolute component of the direction selects the cube face / ■ The remaining two components determine the 2D position on that face / ■ For reflection mapping, use the reflected view vector / 1 vec3 V = normalize(cameraPosition - worldPosition). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Sampling Direction' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Sampling Direction' changes positions before rasterization?
Page 106	Environment Mapping in GLSL 1/2

Source cue	■ Use samplerCube instead of sampler2D / ■ Store positions and normals in world space / ■ Keep the cube map fixed in world space / • otherwise the reflection appears glued to the camera / 1 // Fragment shader
Professor-style explanation	The slide 'Environment Mapping in GLSL 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Use samplerCube instead of sampler2D / ■ Store positions and normals in world space / ■ Keep the cube map fixed in world space / • otherwise the reflection appears glued to the camera / 1 // Fragment shader. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Environment Mapping in GLSL 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Use samplerCube instead of sampler2D / ■ Store positions and normals in world space / ■ Keep the cube map fixed in world space / • otherwise the reflection appears glued to the camera / 1 // Fragment shader. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Environment Mapping in GLSL 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Environment Mapping in GLSL 1/2'?
Page 107	Environment Mapping in GLSL 2/2
Source cue	<pre>1 void main() / 2 { / 3 vec3 V = normalize(cameraPosition - vWorldPosition); / 4 vec3 N = normalize(vWorldNormal); / 5 vec3 R = reflect(-V, N);</pre>

Professor-style explanation	The slide 'Environment Mapping in GLSL 2/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: 1 void main() / 2 { / 3 vec3 V = normalize(cameraPosition - vWorldPosition); / 4 vec3 N = normalize(vWorldNormal); / 5 vec3 R = reflect(-V, N). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Environment Mapping in GLSL 2/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: 1 void main() / 2 { / 3 vec3 V = normalize(cameraPosition - vWorldPosition); / 4 vec3 N = normalize(vWorldNormal); / 5 vec3 R = reflect(-V, N). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Environment Mapping in GLSL 2/2' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Environment Mapping in GLSL 2/2' changes positions before rasterization?
Page 108	Practical Cube-Map Issues
Source cue	■ Cube-map reflections are an approximation / • environment is assumed to be far away / • object does not reflect itself / • other moving objects are usually missing / ■ Important implementation details

Professor-style explanation	The slide 'Practical Cube-Map Issues' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Cube-map reflections are an approximation / • environment is assumed to be far away / • object does not reflect itself / • other moving objects are usually missing / ■ Important implementation details. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Practical Cube-Map Issues. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Cube-map reflections are an approximation / • environment is assumed to be far away / • object does not reflect itself / • other moving objects are usually missing / ■ Important implementation details. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Practical Cube-Map Issues' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Practical Cube-Map Issues'?
Page 109	<code>1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);</code>
Source cue	<code>3 glTextureParameteri(envMap, / Glass Utah teapot rendering; image: Ronak19, CC / BY-SA 4.0. / Extrahierte Tabellen: / [Tabelle 1]</code>

Professor-style explanation	The slide '1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: 3 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS); image: Ronak19, CC / BY-SA 4.0. / Extrahierte Tabellen: / [Tabelle 1]. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about 1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: 3 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS); image: Ronak19, CC / BY-SA 4.0. / Extrahierte Tabellen: / [Tabelle 1]. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for '1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in '1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);'?
Page 110	Cube Mapping Pros and Cons
Source cue	■ Advantages / • direct hardware support / • limited distortion / • no pole singularities / • natural representation for directional data

Professor-style explanation	The slide 'Cube Mapping Pros and Cons' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: ■ Advantages / • direct hardware support / • limited distortion / • no pole singularities / • natural representation for directional data. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Cube Mapping Pros and Cons. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: ■ Advantages / • direct hardware support / • limited distortion / • no pole singularities / • natural representation for directional data. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Cube Mapping Pros and Cons' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Cube Mapping Pros and Cons' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Cube Mapping Pros and Cons' into a causal sentence instead of repeating the slide title?
Page 111	Spherical Environment Mapping Assumptions
Source cue	■ Environment assumed far from object to be textured / ■ Self-reflection not taken into account / • Object to be textured assumed a convex shape(even if it is not geometrically convex) / ■ No other moving objects in the environment / • Interreflection is not taken into account

Professor-style explanation	The slide 'Spherical Environment Mapping Assumptions' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Environment assumed far from object to be textured / ■ Self-reflection not taken into account / • Object to be textured assumed a convex shape(even if it is not geometrically convex) / ■ No other moving objects in the environment / • Interreflection is not taken into account. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Spherical Environment Mapping Assumptions. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Environment assumed far from object to be textured / ■ Self-reflection not taken into account / • Object to be textured assumed a convex shape(even if it is not geometrically convex) / ■ No other moving objects in the environment / • Interreflection is not taken into account. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Spherical Environment Mapping Assumptions' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping Assumptions'?
Page 112	Spherical Environment Mapping Conclusions
Source cue	■ Advantages / • Contains panoramic image of an environment in a single texture / • Filtering and LoD formation takes place within a single texture / ■ Disadvantages / • Singularity along the outer disc border

Professor-style explanation	The slide 'Spherical Environment Mapping Conclusions' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Advantages / • Contains panoramic image of an environment in a single texture / • Filtering and LoD formation takes place within a single texture / ■ Disadvantages / • Singularity along the outer disc border. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Spherical Environment Mapping Conclusions. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Advantages / • Contains panoramic image of an environment in a single texture / • Filtering and LoD formation takes place within a single texture / ■ Disadvantages / • Singularity along the outer disc border. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Spherical Environment Mapping Conclusions' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping Conclusions'?
Page 113	9.7 3D Textures and Volume Rendering
Source cue	Texturing volumetric data, solid materials, and participating media

Professor-style explanation	The slide '9.7 3D Textures and Volume Rendering' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: Texturing volumetric data, solid materials, and participating media. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about 9.7 3D Textures and Volume Rendering. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: Texturing volumetric data, solid materials, and participating media. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for '9.7 3D Textures and Volume Rendering' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '9.7 3D Textures and Volume Rendering'?
Page 114	3D Texturing: Relevant Topics
Source cue	■ Data model / • Regular voxel grids, channels, formats, memory footprint / ■ Mapping model / (s, t, r) / • Texture coordinates and object-to-texture transformations

Professor-style explanation	The slide '3D Texturing: Relevant Topics' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Data model / • Regular voxel grids, channels, formats, memory footprint / ■ Mapping model / (s, t, r) / • Texture coordinates and object-to-texture transformations. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about 3D Texturing: Relevant Topics. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Data model / • Regular voxel grids, channels, formats, memory footprint / ■ Mapping model / (s, t, r) / • Texture coordinates and object-to-texture transformations. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for '3D Texturing: Relevant Topics' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in '3D Texturing: Relevant Topics' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after '3D Texturing: Relevant Topics'?
Page 115	3D Texture as a Voxel Grid
Source cue	■ A 3D texture stores samples on a regular three-dimensional grid / ■ A sample is often called a voxel / ■ Each voxel can store scalar, vector, or multi-channel data / • Density, temperature, material ID / • Color and opacity

Professor-style explanation	The slide '3D Texture as a Voxel Grid' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ A 3D texture stores samples on a regular three-dimensional grid / ■ A sample is often called a voxel / ■ Each voxel can store scalar, vector, or multi-channel data / • Density, temperature, material ID / • Color and opacity. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about 3D Texture as a Voxel Grid. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ A 3D texture stores samples on a regular three-dimensional grid / ■ A sample is often called a voxel / ■ Each voxel can store scalar, vector, or multi-channel data / • Density, temperature, material ID / • Color and opacity. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for '3D Texture as a Voxel Grid' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '3D Texture as a Voxel Grid'?
Page 116	Object Space to Texture Space
Source cue	■ 3D texture coordinates are usually derived from object-space position / ■ For a volume-aligned bounding box / $p - p / \text{object min} / (s, t, r) =$

Professor-style explanation	The slide 'Object Space to Texture Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ 3D texture coordinates are usually derived from object-space position / ■ For a volume-aligned bounding box / $p - p / \text{object min} / (s, t, r) =$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Object Space to Texture Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ 3D texture coordinates are usually derived from object-space position / ■ For a volume-aligned bounding box / $p - p / \text{object min} / (s, t, r) =$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Object Space to Texture Space' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Object Space to Texture Space' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Object Space to Texture Space'?
Page 117	3D Texture Access
Source cue	■ For each fragment, texture coordinates are interpolated / ■ A 3D lookup fetches the neighboring voxel values / ■ Linear filtering in 3D performs trilinear interpolation

Professor-style explanation	The slide '3D Texture Access' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ For each fragment, texture coordinates are interpolated / ■ A 3D lookup fetches the neighboring voxel values / ■ Linear filtering in 3D performs trilinear interpolation. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about 3D Texture Access. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ For each fragment, texture coordinates are interpolated / ■ A 3D lookup fetches the neighboring voxel values / ■ Linear filtering in 3D performs trilinear interpolation. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for '3D Texture Access' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in '3D Texture Access' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after '3D Texture Access'?
Page 118	Trilinear Interpolation
Source cue	■ Trilinear interpolation is linear interpolation along three axes / ■ It blends the eight voxel values surrounding the lookup position / ■ Conceptually / • interpolate along / • interpolate the results along

Professor-style explanation	The slide 'Trilinear Interpolation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: ■ Trilinear interpolation is linear interpolation along three axes / ■ It blends the eight voxel values surrounding the lookup position / ■ Conceptually / • interpolate along / • interpolate the results along. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Trilinear Interpolation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: ■ Trilinear interpolation is linear interpolation along three axes / ■ It blends the eight voxel values surrounding the lookup position / ■ Conceptually / • interpolate along / • interpolate the results along. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Trilinear Interpolation' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Trilinear Interpolation' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Trilinear Interpolation' into a causal sentence instead of repeating the slide title?
Page 119	$f = (1 - \alpha) \cdot f + \alpha \cdot f$
Source cue	00 000 100 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 01 001 101 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 10 010 110
Professor-style explanation	The slide ' $f = (1 - \alpha) \cdot f + \alpha \cdot f$ ' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: 00 000 100 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 01 001 101 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 10 010 110. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.

Technical commentary	This slide is about $f = (1 - \alpha) \cdot f + \alpha \cdot f$. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: 00 000 100 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 01 001 101 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 10 010 110. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for ' $f = (1 - \alpha) \cdot f + \alpha \cdot f$ ' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer ' $f = (1 - \alpha) \cdot f + \alpha \cdot f$ ' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn ' $f = (1 - \alpha) \cdot f + \alpha \cdot f$ ' into a causal sentence instead of repeating the slide title?

Page 120	3D Texture Memory Footprint
Source cue	■ Memory grows cubically with resolution / memory = $w \cdot h \cdot d \cdot \text{bytes per voxel}$ / Resolution R8 R16F RGBA8 RGBA16F / 1 B/voxel 2 B/voxel 4 B/voxel 8 B/voxel / 2 MiB 4 MiB 8 MiB 16 MiB
Professor-style explanation	The slide '3D Texture Memory Footprint' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. The numerical values belong to one memory calculation: ■ Memory grows cubically with resolution / memory = $w \cdot h \cdot d \cdot \text{bytes per voxel}$ / Resolution R8 R16F RGBA8 RGBA16F / 1 B/voxel 2 B/voxel 4 B/voxel 8 B/voxel / 2 MiB 4 MiB 8 MiB 16 MiB. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations. For examination purposes, the important content is the chain width x height -> pixel count -> bits per pixel -> memory size; this chain also explains bandwidth and performance pressure.
Technical commentary	This slide is about 3D Texture Memory Footprint. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. The numerical values belong to one memory calculation: ■ Memory grows cubically with resolution / memory = $w \cdot h \cdot d \cdot \text{bytes per voxel}$ / Resolution R8 R16F RGBA8 RGBA16F / 1 B/voxel 2 B/voxel 4 B/voxel 8 B/voxel / 2 MiB 4 MiB 8 MiB 16 MiB. Resolution gives the number of pixels, color depth gives bits per pixel, and their product gives storage and bandwidth cost.
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.

Exam-grade answer	A strong answer for '3D Texture Memory Footprint' includes the formula relation width x height x bits per pixel, distinguishes bits from bytes, and links larger images to memory bandwidth.
Common trap	Do not confuse bits with bytes in '3D Texture Memory Footprint', and do not ignore that alpha or higher precision changes memory size.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in '3D Texture Memory Footprint'?
Page 121	3D Mipmaps and Filtering
Source cue	■ 3D textures can use mipmaps analogously to 2D textures ■ Each mip level halves width, height, and depth / $k + 1$ ■ Level contains one eighth of the voxels of level ■ Useful for
Professor-style explanation	The slide '3D Mipmaps and Filtering' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ 3D textures can use mipmaps analogously to 2D textures ■ Each mip level halves width, height, and depth / $k + 1$ ■ Level contains one eighth of the voxels of level ■ Useful for. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about 3D Mipmaps and Filtering. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ 3D textures can use mipmaps analogously to 2D textures ■ Each mip level halves width, height, and depth / $k + 1$ ■ Level contains one eighth of the voxels of level ■ Useful for. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for '3D Mipmaps and Filtering' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether '3D Mipmaps and Filtering' works per object, per image region, per ray, or per fragment?
Page 122	Solid Texturing

Source cue	■ 3D textures can define material throughout the interior of an object / ■ Surface fragments sample the material at their 3D position / ■ This avoids seams caused by 2D UV unwrapping / ■ Useful for materials with volumetric structure / • wood grain
Professor-style explanation	The slide 'Solid Texturing' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ 3D textures can define material throughout the interior of an object / ■ Surface fragments sample the material at their 3D position / ■ This avoids seams caused by 2D UV unwrapping / ■ Useful for materials with volumetric structure / • wood grain. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Solid Texturing. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ 3D textures can define material throughout the interior of an object / ■ Surface fragments sample the material at their 3D position / ■ This avoids seams caused by 2D UV unwrapping / ■ Useful for materials with volumetric structure / • wood grain. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Solid Texturing' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Solid Texturing'?
Page 123	Volumetric Data as Texture Data
Source cue	■ Volume data is often represented as a scalar field / $\rho = f(x, y, z)$ / ■ Examples / • CT or MRI intensity / • smoke density

Professor-style explanation	The slide 'Volumetric Data as Texture Data' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Volume data is often represented as a scalar field / $\rho = f(x, y, z)$ / ■ Examples / • CT or MRI intensity / • smoke density. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Volumetric Data as Texture Data. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Volume data is often represented as a scalar field / $\rho = f(x, y, z)$ / ■ Examples / • CT or MRI intensity / • smoke density. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Volumetric Data as Texture Data' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Volumetric Data as Texture Data'?
Page 124	Transfer Functions
Source cue	■ A transfer function maps data values to optical properties / $T(\rho) = (c, \alpha)$ / ■ Color emphasizes material classes or value ranges / ■ Opacity controls which structures become visible / ■ Common choices

Professor-style explanation	The slide 'Transfer Functions' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ A transfer function maps data values to optical properties / $T(\rho) = (c, \alpha)$ / ■ Color emphasizes material classes or value ranges / ■ Opacity controls which structures become visible / ■ Common choices. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Transfer Functions. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ A transfer function maps data values to optical properties / $T(\rho) = (c, \alpha)$ / ■ Color emphasizes material classes or value ranges / ■ Opacity controls which structures become visible / ■ Common choices. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Transfer Functions' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Transfer Functions'?
Page 125	Volume Rendering with 3D Textures
Source cue	■ 3D grid data is stored in a 3D texture / ■ Slice-based volume rendering draws proxy geometry through the volume / ■ Slices are usually orthogonal to the view direction / ■ Each slice samples the 3D texture and applies the transfer function / ■ Slices are blended back to front or front to back

Professor-style explanation	The slide 'Volume Rendering with 3D Textures' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ 3D grid data is stored in a 3D texture / ■ Slice-based volume rendering draws proxy geometry through the volume / ■ Slices are usually orthogonal to the view direction / ■ Each slice samples the 3D texture and applies the transfer function / ■ Slices are blended back to front or front to back. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.
Technical commentary	This slide is about Volume Rendering with 3D Textures. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ 3D grid data is stored in a 3D texture / ■ Slice-based volume rendering draws proxy geometry through the volume / ■ Slices are usually orthogonal to the view direction / ■ Each slice samples the 3D texture and applies the transfer function / ■ Slices are blended back to front or front to back. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Volume Rendering with 3D Textures' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Volume Rendering with 3D Textures'?
Page 126	Front-to-Back Alpha Compositing
Source cue	■ Volume rendering approximates absorption and emission along a ray / $c \propto$ / ■ For a sample with color i and opacity α / $C = C + (1 - \alpha) \cdot \alpha \cdot c$ / out in i

Professor-style explanation	The slide 'Front-to-Back Alpha Compositing' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ Volume rendering approximates absorption and emission along a ray / $c \cdot \alpha$ / ■ For a sample with color i and opacity α / $C = C + (1 - \alpha) \cdot \alpha \cdot c$ / out in in i. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Front-to-Back Alpha Compositing. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ Volume rendering approximates absorption and emission along a ray / $c \cdot \alpha$ / ■ For a sample with color i and opacity α / $C = C + (1 - \alpha) \cdot \alpha \cdot c$ / out in in i. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Front-to-Back Alpha Compositing' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Front-to-Back Alpha Compositing' works per object, per image region, per ray, or per fragment?
Page 127	Ray Marching Volume Rendering
Source cue	■ Instead of drawing slices, cast one ray per fragment through the volume / ■ Repeatedly sample the 3D texture along the ray / ■ Apply the transfer function at every sample / ■ Composite samples until the ray exits the volume or becomes opaque / 1 for (float $t = t_{\text{Entry}}$; $t < t_{\text{Exit}}$; $t += \text{stepSize}$) {

Professor-style explanation	The slide 'Ray Marching Volume Rendering' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ Instead of drawing slices, cast one ray per fragment through the volume / ■ Repeatedly sample the 3D texture along the ray / ■ Apply the transfer function at every sample / ■ Composite samples until the ray exits the volume or becomes opaque / 1 for (float t = tEntry; t < tExit; t += stepSize) {. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Ray Marching Volume Rendering. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ Instead of drawing slices, cast one ray per fragment through the volume / ■ Repeatedly sample the 3D texture along the ray / ■ Apply the transfer function at every sample / ■ Composite samples until the ray exits the volume or becomes opaque / 1 for (float t = tEntry; t < tExit; t += stepSize) {. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Ray Marching Volume Rendering' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Ray Marching Volume Rendering' works per object, per image region, per ray, or per fragment?
Page 128	Gradients and Volume Lighting
Source cue	■ Gradients estimate local orientation in the scalar field / ■ Central differences approximate the gradient / $\rho(x + \Delta, y, z) - \rho(x - \Delta, y, z)$ / ■ ■ / $\nabla \rho \approx$ ■ $\rho(x, y + \Delta, z) - \rho(x, y - \Delta, z)$ ■

Professor-style explanation	The slide 'Gradients and Volume Lighting' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Gradients estimate local orientation in the scalar field / ■ Central differences approximate the gradient / $\rho(x + \Delta, y, z) - \rho(x - \Delta, y, z)$ / ■ ■ / $\nabla \rho \approx$ ■ $\rho(x, y + \Delta, z) - \rho(x, y - \Delta, z)$ ■. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Gradients and Volume Lighting. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Gradients estimate local orientation in the scalar field / ■ Central differences approximate the gradient / $\rho(x + \Delta, y, z) - \rho(x - \Delta, y, z)$ / ■ ■ / $\nabla \rho \approx$ ■ $\rho(x, y + \Delta, z) - \rho(x, y - \Delta, z)$ ■. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Gradients and Volume Lighting' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Gradients and Volume Lighting'?
Page 129	Quality and Performance Trade-Offs
Source cue	■ Step size / • smaller steps improve quality but increase cost / • larger steps are faster but can miss thin structures / ■ Opacity correction / • keeps appearance stable when step size changes

Professor-style explanation	The slide 'Quality and Performance Trade-Offs' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: ■ Step size / • smaller steps improve quality but increase cost / • larger steps are faster but can miss thin structures / ■ Opacity correction / • keeps appearance stable when step size changes. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation. For examination purposes, the important content is the causal pattern: input object, operation or test, produced result, and the next stage that consumes it.
Technical commentary	This slide is about Quality and Performance Trade-Offs. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: ■ Step size / • smaller steps improve quality but increase cost / • larger steps are faster but can miss thin structures / ■ Opacity correction / • keeps appearance stable when step size changes. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer for 'Quality and Performance Trade-Offs' turns the title into a precise cause-and-effect statement with input, processing rule, output, and later use.
Common trap	Do not answer 'Quality and Performance Trade-Offs' by repeating the title; name what changes, why it changes, and what the next rendering step receives.
Check yourself	Can you turn 'Quality and Performance Trade-Offs' into a causal sentence instead of repeating the slide title?
Page 130	3D Texturing Takeaways
Source cue	■ 3D textures generalize 2D texture lookup from surfaces to volumes / ■ They are useful for solid materials, volumetric effects, and sampled scientific data / ■ Trilinear interpolation provides smooth access to voxel grids / ■ Volume rendering turns sampled data into color and opacity along viewing rays / ■ Transfer functions and compositing determine what becomes visible

Professor-style explanation	The slide '3D Texturing Takeaways' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ 3D textures generalize 2D texture lookup from surfaces to volumes / ■ They are useful for solid materials, volumetric effects, and sampled scientific data / ■ Trilinear interpolation provides smooth access to voxel grids / ■ Volume rendering turns sampled data into color and opacity along viewing rays / ■ Transfer functions and compositing determine what becomes visible. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about 3D Texturing Takeaways. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ 3D textures generalize 2D texture lookup from surfaces to volumes / ■ They are useful for solid materials, volumetric effects, and sampled scientific data / ■ Trilinear interpolation provides smooth access to voxel grids / ■ Volume rendering turns sampled data into color and opacity along viewing rays / ■ Transfer functions and compositing determine what becomes visible. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for '3D Texturing Takeaways' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether '3D Texturing Takeaways' works per object, per image region, per ray, or per fragment?
Page 131	■ Textures are sampled GPU data sources used by shaders
Source cue	• Color, opacity, normals, roughness, height, masks, and volume density / ■ Texture coordinates define where texture data is sampled / (s, t, r) / • UV mapping, texture transformations, and 3D coordinates / ■ Texture formats, channel semantics, and color space determine correct interpretation

Professor-style explanation	The slide '■ Textures are sampled GPU data sources used by shaders' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: • Color, opacity, normals, roughness, height, masks, and volume density / ■ Texture coordinates define where texture data is sampled / (s, t, r) / • UV mapping, texture transformations, and 3D coordinates / ■ Texture formats, channel semantics, and color space determine correct interpretation. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.
Technical commentary	This slide is about ■ Textures are sampled GPU data sources used by shaders. The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: • Color, opacity, normals, roughness, height, masks, and volume density / ■ Texture coordinates define where texture data is sampled / (s, t, r) / • UV mapping, texture transformations, and 3D coordinates / ■ Texture formats, channel semantics, and color space determine correct interpretation. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for '■ Textures are sampled GPU data sources used by shaders' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip '■ Textures are sampled GPU data sources used by shaders' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic '■ Textures are sampled GPU data sources used by shaders' points to for deeper study?
Page 132	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Literature and other sources used in this chapter' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Literature and other sources used in this chapter' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 133	■ Text Books
Source cue	• J. Foley, A. van Dam, S. Fine "Computer Graphics: Principles and Practice (3About Edition)", / Addison-Wesley 2013. / • T. Akenine-Möller, E. Haines: Real-Time Rendering, Addison-Wesley. / • P. Shirley, M. Ashikhmin S. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Others

Professor-style explanation	The slide '■ Text Books' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: • J. Foley, A. van Dam, S. Fine "Computer Graphics: Principles and Practice (3About Edition)", / Addison-Wesley 2013. / • T. Akenine-Möller, E. Haines: Real-Time Rendering, Addison-Wesley. / • P. Shirley, M. Ashikhmin S. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Others. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.
Technical commentary	This slide is about ■ Text Books. The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: • J. Foley, A. van Dam, S. Fine "Computer Graphics: Principles and Practice (3About Edition)", / Addison-Wesley 2013. / • T. Akenine-Möller, E. Haines: Real-Time Rendering, Addison-Wesley. / • P. Shirley, M. Ashikhmin S. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Others. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for '■ Text Books' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip '■ Text Books' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic '■ Text Books' points to for deeper study?

09.1 Texture Objects, Texels, Formats, and Color Space

Source location: Section 9.1

Commentary

A texture object stores sampled data on the GPU. A texel is one stored texture sample. The texture format decides what channels and numeric representation the samples use. Color space matters because color values may be stored nonlinearly, especially for sRGB images.

The key shift is to stop thinking of texture as only a picture. Textures can store color, normals, depth, lookup data, environment information, or volume data. The shader decides how the sampled values are interpreted.

Mental model

A texture is a GPU-accessible sampled data field.

Check yourself

Why can the same texture mechanism store both color maps and normal maps?

09.2 Texture Coordinates, UV Mapping, and Wrapping

Source location: Section 9.2

Commentary

Texture coordinates map surface points to locations in texture space. UV coordinates are usually interpolated across primitives during rasterization and then used by the fragment shader to sample a texture.

Wrapping rules decide what happens outside the normal coordinate range. Repeat, clamp, mirrored repeat, and border behavior are not visual afterthoughts; they define the sampling function outside the base domain.

Mental model

UVs are the address system that lets a fragment look up texture data.

Check yourself

What artifact might appear if a wrapping mode is wrong at the edge of a surface?

09.3 Sampling, Filtering, Mipmaps, and Anisotropy

Source location: Section 9.3

Commentary

Sampling turns continuous texture coordinates into values from discrete texels. Nearest filtering selects a nearby texel and can look blocky. Linear filtering blends nearby texels and looks smoother. Minification is harder because many texels may map to one pixel.

Mipmaps store prefiltered lower-resolution versions of a texture. They reduce aliasing and shimmer when textures are seen at small scale. Anisotropic filtering improves quality when a texture is viewed at a steep angle where footprint shape is elongated.

Mental model

Filtering reconstructs values; mipmaps choose an appropriate scale before reconstruction.

Check yourself

Why does a distant checkerboard shimmer without mipmapping?

09.4 Modern OpenGL Texture Pipeline

Source location: Section 9.4

Commentary

In modern OpenGL, textures are represented by texture objects, bound to texture units, connected to sampler uniforms, and accessed in shaders. The shader does not sample a filename; it samples a bound texture through a sampler.

Texture bugs often come from state mismatches: wrong active texture unit, wrong sampler uniform, missing mipmaps for a mipmap filter, wrong wrap mode, or wrong internal format.

Mental model

OpenGL texturing is a chain: object data -> texture unit -> sampler uniform -> shader lookup.

Check yourself

Why can a texture appear black when the shader code is mathematically correct?

09.5 Normal Mapping

Source location: Section 9.5

Commentary

Normal mapping stores normal directions in a texture so that lighting can vary at a finer scale than the geometry. The surface may have few triangles, but the shader receives detailed normals per fragment.

The hard part is coordinate space. Normal maps are often defined in tangent space, so the shader must transform or interpret them with the correct tangent, bitangent, and normal basis.

Mental model

Normal mapping changes the lighting normal, not the actual mesh silhouette.

Check yourself

Why does normal mapping not change the geometric outline of an object?

09.6 Environment Mapping and 3D Textures

Source location: Sections 9.6-9.7

Commentary

Environment mapping uses textures to represent surrounding illumination or reflections. A direction, not a surface UV alone, can be used to sample an environment map. This is a texture lookup driven by view/reflection geometry.

3D textures and volume rendering extend the same sampled-data idea into three dimensions. Instead of sampling a 2D image, the shader samples a volume. This is useful for data such as medical scans, density fields, or procedural volumetric effects.

Mental model

Textures are general sampled data; the coordinate dimensionality depends on the problem.

Check yourself

What coordinate type would you use to sample a 3D texture?

End-of-lecture summary

If you remember only one thing from Lecture 09, remember this: This lecture treats textures as sampled data for shading, not merely images glued onto objects. It connects UV coordinates, sampling, filtering, mipmaps, OpenGL texture state, and advanced texture uses.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 10 - Shadows

This lecture explains shadows as a visibility problem from the light source. A point is lit if the light can see it; it is shadowed if another object blocks that visibility.

Source chunk	course_text_parts/03_lectures/10_shadows.txt
Extracted slide pages	60
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.

Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Shadow Perception 1/3
Source cue	- Visual perception of an environment is influenced by shadows - Object object relationships - Object space relationships - Improved depth perception is made possible by shadows - Shadows provide appealing visual effects
Professor-style explanation	The slide 'Shadow Perception 1/3' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Visual perception of an environment is influenced by shadows - Object object relationships - Object space relationships - Improved depth perception is made possible by shadows - Shadows provide appealing visual effects. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Shadow Perception 1/3. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Visual perception of an environment is influenced by shadows - Object object relationships - Object space relationships - Improved depth perception is made possible by shadows - Shadows provide appealing visual effects. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Shadow Perception 1/3' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Shadow Perception 1/3' works per object, per image region, per ray, or per fragment?
Page 3	Shadow Perception 2/3
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Shadow Perception 2/3' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Shadow Perception 2/3. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Shadow Perception 2/3' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Perception 2/3'?
Page 4	Shadow Perception 3/3
Source cue	■ KerstenD. Knill, D. C., Mamassian, P. and Bülthoffl. Illusory motion from shadows, Nature, 379 (31), / 1996.
Professor-style explanation	The slide 'Shadow Perception 3/3' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. The terms describe light-space visibility: ■ KerstenD. Knill, D. C., Mamassian, P. and Bülthoffl. Illusory motion from shadows, Nature, 379 (31), / 1996. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image. For examination purposes, the important content is light-space visibility: a point is shadowed because another object blocks the path from the light.

Technical commentary	This slide is about Shadow Perception 3/3. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The terms describe light-space visibility: ■ KerstenD. Knill, D. C., Mamassian, P. and Bülthoffl. Illusory motion from shadows, Nature, 379 (31), / 1996. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Shadow Perception 3/3' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Perception 3/3'?

Page 5	Shadow Atmosphere
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Shadow Atmosphere' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Shadow Atmosphere. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Shadow Atmosphere' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Atmosphere'?
Page 6	■ Shadows createa certain atmosphere

Source cue	■ The Lighthouse (2019)
Professor-style explanation	The slide '■ Shadows createa certain atmosphere' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ The Lighthouse (2019). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about ■ Shadows createa certain atmosphere. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ The Lighthouse (2019). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for '■ Shadows createa certain atmosphere' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '■ Shadows createa certain atmosphere'?
Page 7	Shadow Atmosphere
Source cue	■ Shadows createa certain atmosphere / ■ The Lighthouse (2019) / ■ Kill Bill: Volume 1 (2003)

Professor-style explanation	The slide 'Shadow Atmosphere' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Shadows create a certain atmosphere / ■ The Lighthouse (2019) / ■ Kill Bill: Volume 1 (2003). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Shadow Atmosphere. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Shadows create a certain atmosphere / ■ The Lighthouse (2019) / ■ Kill Bill: Volume 1 (2003). Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Shadow Atmosphere' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Shadow Atmosphere'?
Page 8	Overview
Source cue	10.1 Definitions / 10.2 Ground Plane Shadows / 10.3 Light Maps / 10.4 Shadow Volumes / 10.5 Shadow Maps

Professor-style explanation	The outline 'Overview' gives the lecture its internal logic. The listed topics are the objects that will be connected during the chapter: first the problem space, then the mathematical or algorithmic tools, then the implementation consequences. The listed entries form a dependency order: 10.1 Definitions / 10.2 Ground Plane Shadows / 10.3 Light Maps / 10.4 Shadow Volumes / 10.5 Shadow Maps. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary. The order matters because later items rely on earlier definitions. For example, an OpenGL mechanism is much easier to understand once the corresponding pipeline object or mathematical operation has already been introduced. The outline is therefore a compact dependency graph of the lecture rather than a collection of isolated labels. For examination purposes, the important content is the dependency structure: which concept introduces the vocabulary, which later algorithm uses it, and which implementation problem it solves.
Technical commentary	This slide is about Overview. The listed items define the lecture sequence: the topic begins with a problem statement, introduces the required objects or algorithms, and then connects them to rendering or implementation consequences. The listed entries form a dependency order: 10.1 Definitions / 10.2 Ground Plane Shadows / 10.3 Light Maps / 10.4 Shadow Volumes / 10.5 Shadow Maps. Earlier entries introduce the vocabulary and problem setting; later entries build algorithms, API details, or consequences on top of that vocabulary.
Why it matters	Outlines tell you the dependency order. They are the safest way to avoid learning isolated bullet points.
Exam-grade answer	A strong answer for 'Overview' names the topics in order and explains at least one dependency between an earlier item and a later item.
Common trap	Do not memorize 'Overview' as a list of headings only; the exam-relevant part is how the headings depend on each other.
Check yourself	Can you explain where 'Overview' fits in the lecture order and what later section depends on it?

Page 9	10.1 Definitions
Source cue	About shadow-receiving and shadow-casting scene objects
Professor-style explanation	The slide '10.1 Definitions' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. The terms describe light-space visibility: About shadow-receiving and shadow-casting scene objects. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image. For examination purposes, the important content is light-space visibility: a point is shadowed because another object blocks the path from the light.

Technical commentary	This slide is about 10.1 Definitions. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The terms describe light-space visibility: About shadow-receiving and shadow-casting scene objects. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for '10.1 Definitions' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in '10.1 Definitions'?
Page 10	Classification of scene objects
Source cue	■ Distinction between / • Shadow-casting scene objects(Occluder) / • Shadow-receiving scene objects (Receiver) / ■ Classify shadow-casting and shadow-receiving objects in a preprocess? / • Self-shadowing results from same object casting and receiving shadows
Professor-style explanation	The slide 'Classification of scene objects' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. The terms describe light-space visibility: ■ Distinction between / • Shadow-casting scene objects(Occluder) / • Shadow-receiving scene objects (Receiver) / ■ Classify shadow-casting and shadow-receiving objects in a preprocess? / • Self-shadowing results from same object casting and receiving shadows. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image. For examination purposes, the important content is light-space visibility: a point is shadowed because another object blocks the path from the light.
Technical commentary	This slide is about Classification of scene objects. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The terms describe light-space visibility: ■ Distinction between / • Shadow-casting scene objects(Occluder) / • Shadow-receiving scene objects (Receiver) / ■ Classify shadow-casting and shadow-receiving objects in a preprocess? / • Self-shadowing results from same object casting and receiving shadows. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.

Exam-grade answer	A strong answer for 'Classification of scene objects' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Classification of scene objects'?
Page 11	Umbra and Penumbra
Source cue	■ Point light sources / • Result in hard shadows / • High intensity contrast between shadowed and non-shadowed region / ■ Area light sources / • Result in soft shadows
Professor-style explanation	The slide 'Umbra and Penumbra' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Point light sources / • Result in hard shadows / • High intensity contrast between shadowed and non-shadowed region / ■ Area light sources / • Result in soft shadows. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Umbra and Penumbra. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Point light sources / • Result in hard shadows / • High intensity contrast between shadowed and non-shadowed region / ■ Area light sources / • Result in soft shadows. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Umbra and Penumbra' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Umbra and Penumbra'?
Page 12	Hard and Soft Shadows

Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Hard and Soft Shadows' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Hard and Soft Shadows. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Hard and Soft Shadows' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Hard and Soft Shadows'?
Page 13	Description of Shadows
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Description of Shadows' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.

Technical commentary	This slide is about Description of Shadows. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Description of Shadows' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Description of Shadows'?

Page 14	■ Shadows can be described differently
Source cue	■ Shadows can be modeled as / • Separate objects / • Surfaces not seen when looking into scene from light source / • Volumetric regions with reduced lighting
Professor-style explanation	The slide '■ Shadows can be described differently' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Shadows can be modeled as / • Separate objects / • Surfaces not seen when looking into scene from light source / • Volumetric regions with reduced lighting. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about ■ Shadows can be described differently. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Shadows can be modeled as / • Separate objects / • Surfaces not seen when looking into scene from light source / • Volumetric regions with reduced lighting. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.

Exam-grade answer	A strong answer for '■ Shadows can be described differently' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '■ Shadows can be described differently'?
Page 15	Observations 1/2
Source cue	■ Possible shadow generation approaches / • Shadows as surfaces: exploit hidden-surface algorithms / • Shadows as projection: create shadows from A to B by projecting A onto B from light source / ■ Static vs. dynamic scenes / • Shadows are static in scenes with static lighting and scene objects (shadow computation)
Professor-style explanation	The slide 'Observations 1/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: ■ Possible shadow generation approaches / • Shadows as surfaces: exploit hidden-surface algorithms / • Shadows as projection: create shadows from A to B by projecting A onto B from light source / ■ Static vs. dynamic scenes / • Shadows are static in scenes with static lighting and scene objects (shadow computation. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Observations 1/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: ■ Possible shadow generation approaches / • Shadows as surfaces: exploit hidden-surface algorithms / • Shadows as projection: create shadows from A to B by projecting A onto B from light source / ■ Static vs. dynamic scenes / • Shadows are static in scenes with static lighting and scene objects (shadow computation. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.

Exam-grade answer	A strong answer for 'Observations 1/2' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Observations 1/2' changes positions before rasterization?

Page 16	Observations 2/2
Source cue	■ Shadow shape can be obtained by projection / • Trivial for projection on flat surfaces / • Algorithmically very complex for projection in general case / ■ Shadow intensity / • Shadows attenuate the incoming light used in illumination model
Professor-style explanation	The slide 'Observations 2/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: ■ Shadow shape can be obtained by projection / • Trivial for projection on flat surfaces / • Algorithmically very complex for projection in general case / ■ Shadow intensity / • Shadows attenuate the incoming light used in illumination model. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Observations 2/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: ■ Shadow shape can be obtained by projection / • Trivial for projection on flat surfaces / • Algorithmically very complex for projection in general case / ■ Shadow intensity / • Shadows attenuate the incoming light used in illumination model. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Observations 2/2' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.

Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Observations 2/2' changes positions before rasterization?
Page 17	Shadow Calculation Methods
Source cue	■ Ground plane shadows / • Static shadow polygons / • Dynamic shadow polygons / ■ Projected shadows / ■ Light maps
Professor-style explanation	The slide 'Shadow Calculation Methods' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Ground plane shadows / • Static shadow polygons / • Dynamic shadow polygons / ■ Projected shadows / ■ Light maps. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Shadow Calculation Methods. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Ground plane shadows / • Static shadow polygons / • Dynamic shadow polygons / ■ Projected shadows / ■ Light maps. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Shadow Calculation Methods' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Shadow Calculation Methods'?
Page 18	10.2 Ground Plane Shadows
Source cue	Static and dynamic shadow polygons

Professor-style explanation	The slide '10.2 Ground Plane Shadows' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. The terms describe light-space visibility: Static and dynamic shadow polygons. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image. For examination purposes, the important content is light-space visibility: a point is shadowed because another object blocks the path from the light.
Technical commentary	This slide is about 10.2 Ground Plane Shadows. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The terms describe light-space visibility: Static and dynamic shadow polygons. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for '10.2 Ground Plane Shadows' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in '10.2 Ground Plane Shadows'?
Page 19	Static Shadow Polygons
Source cue	■ Render dark polygons to represent shadow regions / • Cast ray from light source through center of shadow-casting objects / • Position dark polygon where ray intersects shadow-receiving object / ■ Disadvantages / • Static shapes cannot represent geometry of dynamic scene objects

Professor-style explanation	The slide 'Static Shadow Polygons' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ Render dark polygons to represent shadow regions / • Cast ray from light source through center of shadow-casting objects / • Position dark polygon where ray intersects shadow-receiving object / ■ Disadvantages / • Static shapes cannot represent geometry of dynamic scene objects. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Static Shadow Polygons. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ Render dark polygons to represent shadow regions / • Cast ray from light source through center of shadow-casting objects / • Position dark polygon where ray intersects shadow-receiving object / ■ Disadvantages / • Static shapes cannot represent geometry of dynamic scene objects. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Static Shadow Polygons' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Static Shadow Polygons' works per object, per image region, per ray, or per fragment?
Page 20	Render Static Shadow Polygons 1/2
Source cue	■ Depth value identical with that of shadow-receiving polygon (care must be taken wrt. z-buffer / fighting) / ■ Depth value smaller than that of shadow-receiving polygon (shadow can be too large)

Professor-style explanation	The slide 'Render Static Shadow Polygons 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Depth value identical with that of shadow-receiving polygon (care must be taken wrt. z-buffer / fighting) / ■ Depth value smaller than that of shadow-receiving polygon (shadow can be too large). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Render Static Shadow Polygons 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Depth value identical with that of shadow-receiving polygon (care must be taken wrt. z-buffer / fighting) / ■ Depth value smaller than that of shadow-receiving polygon (shadow can be too large). Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Render Static Shadow Polygons 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Render Static Shadow Polygons 1/2'?
Page 21	Render Static Shadow Polygons 2/2
Source cue	■ Disable depth test / • Cast ray to determine shadow visibility / • Show entire shadow polygon / ■ Shadow too large and possibly in wrong places / ■ Test corners for shadow visibility

Professor-style explanation	The slide 'Render Static Shadow Polygons 2/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ Disable depth test / • Cast ray to determine shadow visibility / • Show entire shadow polygon / ■ Shadow too large and possibly in wrong places / ■ Test corners for shadow visibility. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Render Static Shadow Polygons 2/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ Disable depth test / • Cast ray to determine shadow visibility / • Show entire shadow polygon / ■ Shadow too large and possibly in wrong places / ■ Test corners for shadow visibility. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Render Static Shadow Polygons 2/2' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Render Static Shadow Polygons 2/2' works per object, per image region, per ray, or per fragment?
Page 22	Static Shadow Polygons
Source cue	■ Good for car racing games (static scene geometry) / ■ With fast movements and proximity of casters and receivers, wrong shadows are often not detected / as such

Professor-style explanation	The slide 'Static Shadow Polygons' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. The terms describe light-space visibility: ■ Good for car racing games (static scene geometry) / ■ With fast movements and proximity of casters and receivers, wrong shadows are often not detected / as such. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image. For examination purposes, the important content is light-space visibility: a point is shadowed because another object blocks the path from the light.
Technical commentary	This slide is about Static Shadow Polygons. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The terms describe light-space visibility: ■ Good for car racing games (static scene geometry) / ■ With fast movements and proximity of casters and receivers, wrong shadows are often not detected / as such. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Static Shadow Polygons' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Static Shadow Polygons'?
Page 23	Dynamic Shadow Polygons
Source cue	■ Project shadow-casting scene objects onto shadow-receiving object by modifying shape / ■ Accurate procedure for semi-transparent shadows, but only if shadows do not overlap / ■ Restrictions / • Only planar polygons on shadow-receiving scene objects possible / • No self-shadowing possible

Professor-style explanation	The slide 'Dynamic Shadow Polygons' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. The terms describe light-space visibility: ■ Project shadow-casting scene objects onto shadow-receiving object by modifying shape / ■ Accurate procedure for semi-transparent shadows, but only if shadows do not overlap / ■ Restrictions / • Only planar polygons on shadow-receiving scene objects possible / • No self-shadowing possible. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image. For examination purposes, the important content is light-space visibility: a point is shadowed because another object blocks the path from the light.
Technical commentary	This slide is about Dynamic Shadow Polygons. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The terms describe light-space visibility: ■ Project shadow-casting scene objects onto shadow-receiving object by modifying shape / ■ Accurate procedure for semi-transparent shadows, but only if shadows do not overlap / ■ Restrictions / • Only planar polygons on shadow-receiving scene objects possible / • No self-shadowing possible. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Dynamic Shadow Polygons' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Dynamic Shadow Polygons'?
Page 24	Shadow Projection
Source cue	S L / ■ Shadow point (= vertex of the shadow polygon) lies on a line between light source and vertex / of shadow-casting scene object / $S = P - \alpha \cdot L$ / $S_z = 0$ $\alpha =$

Professor-style explanation	The slide 'Shadow Projection' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: $S = P - \alpha \cdot L$ / $S_z = 0 \Rightarrow \alpha = \frac{P_z}{L_z}$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations. For examination purposes, the important content is that rasterization creates fragment candidates from projected primitives; later tests decide whether those candidates affect pixels.
Technical commentary	This slide is about Shadow Projection. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: $S = P - \alpha \cdot L$ / $S_z = 0 \Rightarrow \alpha = \frac{P_z}{L_z}$. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Exam-grade answer	A strong answer for 'Shadow Projection' explains coverage, fragment creation, interpolation, and the difference between a generated fragment and a final pixel update.
Common trap	Do not call every fragment a pixel; fragments are candidates and can still fail tests or be blended before the framebuffer is updated.
Check yourself	Can you explain which samples/fragments are generated by 'Shadow Projection'?
Page 25	Rendering Dynamic Shadow Polygons
Source cue	- Matrix transforms object into its shadow polygon - Render scene object with shadows - Render scene object - Multiply current model-view -matrix with - Render scene object in shadow color with blending

Professor-style explanation	The slide 'Rendering Dynamic Shadow Polygons' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ Matrix transforms object into its shadow polygon / ■ Render scene object with shadows / • Render scene object / • Multiply current model-view -matrix with / • Render scene object in shadow color with b lending. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Rendering Dynamic Shadow Polygons. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ Matrix transforms object into its shadow polygon / ■ Render scene object with shadows / • Render scene object / • Multiply current model-view -matrix with / • Render scene object in shadow color with b lending. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Rendering Dynamic Shadow Polygons' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Rendering Dynamic Shadow Polygons' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Rendering Dynamic Shadow Polygons'?
Page 26	10.3 Light Maps
Source cue	Structured lighting with textures

Professor-style explanation	The slide '10.3 Light Maps' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: Structured lighting with textures. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about 10.3 Light Maps. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: Structured lighting with textures. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for '10.3 Light Maps' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '10.3 Light Maps'?
Page 27	Light Maps
Source cue	■ Static shadows can be precomputed and stored in light maps / ■ When precomputing light maps, shadows are calculated offline(e.g. with ray tracing) / ■ Soft shadows in lightmaps / • Interpolation of the lightmap creates soft shadow approximations / • Light source sampling results in better results,sending multiple rays to different points of the

Professor-style explanation	The slide 'Light Maps' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ Static shadows can be precomputed and stored in light maps / ■ When precomputing light maps, shadows are calculated offline(e.g. with ray tracing) / ■ Soft shadows in lightmaps / • Interpolation of the lightmap creates soft shadow approximations / • Light source sampling results in better results,sending multiple rays to different points of the. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Light Maps. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ Static shadows can be precomputed and stored in light maps / ■ When precomputing light maps, shadows are calculated offline(e.g. with ray tracing) / ■ Soft shadows in lightmaps / • Interpolation of the lightmap creates soft shadow approximations / • Light source sampling results in better results,sending multiple rays to different points of the. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Light Maps' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Light Maps' works per object, per image region, per ray, or per fragment?
Page 28	Soft Shadows with Light Maps 1/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Soft Shadows with Light Maps 1/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Soft Shadows with Light Maps 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Soft Shadows with Light Maps 1/2' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Soft Shadows with Light Maps 1/2'?
Page 29	Soft Shadows with Light Maps 2/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Soft Shadows with Light Maps 2/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.

Technical commentary	This slide is about Soft Shadows with Light Maps 2/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Soft Shadows with Light Maps 2/2' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Soft Shadows with Light Maps 2/2'?

Page 30	10.4 Shadow Volumes
Source cue	Representation of shadows by geometry enclosing them
Professor-style explanation	The slide '10.4 Shadow Volumes' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. The terms describe light-space visibility: Representation of shadows by geometry enclosing them. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image. For examination purposes, the important content is light-space visibility: a point is shadowed because another object blocks the path from the light.
Technical commentary	This slide is about 10.4 Shadow Volumes. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The terms describe light-space visibility: Representation of shadows by geometry enclosing them. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for '10.4 Shadow Volumes' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.

Check yourself	Can you explain what the light can or cannot see in '10.4 Shadow Volumes'?
Page 31	Shadow Volumes
Source cue	■ Shadows are considered as volume in space / ■ Originally developed by Crow in 1977 / ■ Extension by Brotman and Badler in 1984 / ■ Heidmann introduced the technology for efficient rendering of shadow volumes in 1991
Professor-style explanation	The slide 'Shadow Volumes' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. The terms describe light-space visibility: ■ Shadows are considered as volume in space / ■ Originally developed by Crow in 1977 / ■ Extension by Brotman and Badler in 1984 / ■ Heidmann introduced the technology for efficient rendering of shadow volumes in 1991. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image. For examination purposes, the important content is light-space visibility: a point is shadowed because another object blocks the path from the light.
Technical commentary	This slide is about Shadow Volumes. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The terms describe light-space visibility: ■ Shadows are considered as volume in space / ■ Originally developed by Crow in 1977 / ■ Extension by Brotman and Badler in 1984 / ■ Heidmann introduced the technology for efficient rendering of shadow volumes in 1991. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Shadow Volumes' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Volumes'?
Page 32	Initial Considerations
Source cue	■ Shadow volume shape depends on shape and position of light source and shadow-caster / ■ All points inside shadow volume are shadowed / ■ Shadow volume generation / • Find silhouettes of shadow-caster regarding light source / • Extrude silhouettes in light ray direction to create polygons

Professor-style explanation	The slide 'Initial Considerations' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ Shadow volume shape depends on shape and position of light source and shadow-caster / ■ All points inside shadow volume are shadowed / ■ Shadow volume generation / • Find silhouettes of shadow-caster regarding light source / • Extrudes silhouettes in light ray direction to create polygons. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Initial Considerations. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ Shadow volume shape depends on shape and position of light source and shadow-caster / ■ All points inside shadow volume are shadowed / ■ Shadow volume generation / • Find silhouettes of shadow-caster regarding light source / • Extrudes silhouettes in light ray direction to create polygons. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Initial Considerations' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Initial Considerations' works per object, per image region, per ray, or per fragment?
Page 33	Shadow Volume Geometry
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Shadow Volume Geometry' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Shadow Volume Geometry. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Shadow Volume Geometry' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Volume Geometry'?
Page 34	Shadow Volume Realization
Source cue	■ Silhouettes edges can be found by analyzing normals / • Silhouette edge lies between front and back face (as seen from light source) / ■ - Silhouette edges are stored as vertex list containing vertices / ■ For convex shadow-casters / L V

Professor-style explanation	The slide 'Shadow Volume Realization' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Silhouettes edges can be found by analyzing normals / • Silhouette edge lies between front and back face (as seen from light source) / ■ Silhouette edges are stored as vertex list containing vertices / ■ For convex shadow-casters / L V. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Shadow Volume Realization. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Silhouettes edges can be found by analyzing normals / • Silhouette edge lies between front and back face (as seen from light source) / ■ Silhouette edges are stored as vertex list containing vertices / ■ For convex shadow-casters / L V. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Shadow Volume Realization' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Shadow Volume Realization'?
Page 35	Shadow Volume Properties 1/2
Source cue	■ All points inside shadow volume are shadowed / ■ Shadowing can be determined for each view ray by intersection tests / • Assumption: Camera not in shadow volume / • At each intersection with front-face shadow polygon, intersection counter cnt is incremented / • At each intersection with back-face shadow polygon, intersection counter cnt is decremented

Professor-style explanation	The slide 'Shadow Volume Properties 1/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ All points inside shadow volume are shadowed / ■ Shadowing can be determined for each view ray by intersection tests / • Assumption: Camera not in shadow volume / • At each intersection with front-face shadow polygon, intersection counter cnt is incremented / • At each intersection with back-face shadow polygon, intersection counter cnt is decremented. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Shadow Volume Properties 1/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ All points inside shadow volume are shadowed / ■ Shadowing can be determined for each view ray by intersection tests / • Assumption: Camera not in shadow volume / • At each intersection with front-face shadow polygon, intersection counter cnt is incremented / • At each intersection with back-face shadow polygon, intersection counter cnt is decremented. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Shadow Volume Properties 1/2' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Shadow Volume Properties 1/2' works per object, per image region, per ray, or per fragment?
Page 36	Shadow Volume Properties 2/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Shadow Volume Properties 2/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Shadow Volume Properties 2/2. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Shadow Volume Properties 2/2' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Volume Properties 2/2'?
Page 37	Stencil Shadow Volume
Source cue	■ Generation of shadow volumes per frame / • Works well for scenes with few moving objects / • Expensive for complex shadow volumes / • Improvement by simplifying shadow-casting geometries / ■ Four rendering passes required

Professor-style explanation	The slide 'Stencil Shadow Volume' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. The terms describe light-space visibility: ■ Generation of shadow volumes per frame / • Works well for scenes with few moving objects / • Expensive for complex shadow volumes / • Improvement by simplifying shadow-casting geometries / ■ Four rendering passes required. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image. For examination purposes, the important content is light-space visibility: a point is shadowed because another object blocks the path from the light.
Technical commentary	This slide is about Stencil Shadow Volume. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The terms describe light-space visibility: ■ Generation of shadow volumes per frame / • Works well for scenes with few moving objects / • Expensive for complex shadow volumes / • Improvement by simplifying shadow-casting geometries / ■ Four rendering passes required. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Stencil Shadow Volume' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Stencil Shadow Volume'?
Page 38	Implementation
Source cue	■ Determine visible scene objects / • Render scene without writing in color buffer / ■ Represent shadow volume in stencil buffer / • Initialize stencil buffer with 0 / • Render front-face shadow polygons and increment stencil buffer by 1

Professor-style explanation	The slide 'Implementation' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Determine visible scene objects / • Render scene without writing in color buffer / ■ Represent shadow volume in stencil buffer / • Initialize stencil buffer with 0 / • Render front-face shadow polygons and increment stencil buffer by 1. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Implementation. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Determine visible scene objects / • Render scene without writing in color buffer / ■ Represent shadow volume in stencil buffer / • Initialize stencil buffer with 0 / • Render front-face shadow polygons and increment stencil buffer by 1. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Implementation' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Implementation'?
Page 39	Special Case: COP in the shadows
Source cue	■ When COP lies in a shadow volume, the situation can be unclear / • Near - or far -plane can intersect shadow volume / ■ Possible solutions / s = 0 / • Test COP against all shadow volumes cnt

Professor-style explanation	The slide 'Special Case: COP in the shadows' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. The terms describe light-space visibility: ■ When COP lies in a shadow volume, the situation can be unclear / • Near - or far -plane can intersect shadow volume / ■ Possible solutions / $s = 0$ / • Test COP against all shadow volumes cnt. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image. For examination purposes, the important content is light-space visibility: a point is shadowed because another object blocks the path from the light.
Technical commentary	This slide is about Special Case: COP in the shadows. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The terms describe light-space visibility: ■ When COP lies in a shadow volume, the situation can be unclear / • Near - or far -plane can intersect shadow volume / ■ Possible solutions / $s = 0$ / • Test COP against all shadow volumes cnt. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Special Case: COP in the shadows' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Special Case: COP in the shadows'?
Page 40	Shadow Volume Example
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Shadow Volume Example' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Shadow Volume Example. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Shadow Volume Example' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Volume Example'?
Page 41	Shadow Volume Conclusions
Source cue	■ Advantages / • Shadow-casting and shadow-receiving objects can have arbitrary shapes / • Applicable for multiple light sources / • Level-of-detail adaptable through approximated geometries / • Shadow volumes for static pairs of objects and lights can be precalculated

Professor-style explanation	The slide 'Shadow Volume Conclusions' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Advantages / • Shadow-casting and shadow-receiving objects can have arbitrary shapes / • Applicable for multiple light sources / • Level-of-detail adaptable through approximated geometries / • Shadow volumes for static pairs of objects and lights can be precalculated. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Shadow Volume Conclusions. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Advantages / • Shadow-casting and shadow-receiving objects can have arbitrary shapes / • Applicable for multiple light sources / • Level-of-detail adaptable through approximated geometries / • Shadow volumes for static pairs of objects and lights can be precalculated. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Shadow Volume Conclusions' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Shadow Volume Conclusions'?
Page 42	10.5 Shadow Maps
Source cue	Depth test from the point of view of the light source

Professor-style explanation	The slide '10.5 Shadow Maps' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: Depth test from the point of view of the light source. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about 10.5 Shadow Maps. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: Depth test from the point of view of the light source. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for '10.5 Shadow Maps' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether '10.5 Shadow Maps' works per object, per image region, per ray, or per fragment?
Page 43	Shadow Mapping
Source cue	■ Based on z-Buffer (contains depth relative to the light source) / ■ First used within Pixar's Renderman

Professor-style explanation	The slide 'Shadow Mapping' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: ■ Based on z-Buffer (contains depth relative to the light source) / ■ First used within Pixar's Renderman. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Shadow Mapping. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: ■ Based on z-Buffer (contains depth relative to the light source) / ■ First used within Pixar's Renderman. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Shadow Mapping' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shadow Mapping'?
Page 44	Projected Shadows
Source cue	■ depth texture / ■ Rendering algorithm / • Render shadow-casting objects as seen from the light source into depth texture / • Project depth texture using projective texturing when rendering shadow-receiving objects / ■ Characteristics of the process

Professor-style explanation	The slide 'Projected Shadows' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ depth texture / ■ Rendering algorithm / • Render shadow-casting objects as seen from the light source into depth texture / • Project depth texture using projective texturing when rendering shadow-receiving objects / ■ Characteristics of the process. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Projected Shadows. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ depth texture / ■ Rendering algorithm / • Render shadow-casting objects as seen from the light source into depth texture / • Project depth texture using projective texturing when rendering shadow-receiving objects / ■ Characteristics of the process. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Projected Shadows' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Projected Shadows' works per object, per image region, per ray, or per fragment?
Page 45	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 46	Depth Comparison
Source cue	■ For each vertex in world coordinates determine associated depth value in shadow map • Transform into camera's eye space • Use light source's 4x4 transformation matrix to transform into light's clip space • Compare depth value with associated depth value in shadow map

Professor-style explanation	The slide 'Depth Comparison' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: ■ For each vertex in world coordinates determine associated depth value in shadow map / • Transform into camera's eye space / • Use light source's 4x4 transformation matrix to transform into light's clip space / • Compare depth value with associated depth value in shadow map. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Depth Comparison. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: ■ For each vertex in world coordinates determine associated depth value in shadow map / • Transform into camera's eye space / • Use light source's 4x4 transformation matrix to transform into light's clip space / • Compare depth value with associated depth value in shadow map. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Depth Comparison' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Depth Comparison' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Depth Comparison'?
Page 47	Coordinate Systems
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Coordinate Systems' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Coordinate Systems. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Coordinate Systems' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Coordinate Systems' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Coordinate Systems'?
Page 48	Pseudo Code
Source cue	// generate shadow map w.r.t light source / generateShadowMap(); // render scene from COP / for all rasterized fragments do { / Transform fragment xyz into lights coordinate system

Professor-style explanation	The slide 'Pseudo Code' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: // generate shadow map w.r.t light source / generateShadowMap(); / // render scene from COP / for all rasterized fragments do { / Transform fragment xyz into lights coordinate system. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear. For examination purposes, the important content is space awareness: name the source space, the matrix operation, and the target space before applying any formula.
Technical commentary	This slide is about Pseudo Code. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: // generate shadow map w.r.t light source / generateShadowMap(); / // render scene from COP / for all rasterized fragments do { / Transform fragment xyz into lights coordinate system. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Exam-grade answer	A strong answer for 'Pseudo Code' names source space, target space, the transformation type, and whether the object is a point, direction vector, normal, or coordinate frame.
Common trap	Do not multiply matrices mechanically in 'Pseudo Code' without naming coordinate spaces and order of operations.
Check yourself	Can you state the coordinate space before and after 'Pseudo Code'?
Page 49	Properties
Source cue	■ Shadow maps are static when scene and lighting are static / ■ Shadow quality depends on shadow map resolution / ■ Shadow mapping has three main problems / • View volume problem / • Bias problem

Professor-style explanation	The slide 'Properties' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. The terms describe local shading inputs: ■ Shadow maps are static when scene and lighting are static / ■ Shadow quality depends on shadow map resolution / ■ Shadow mapping has three main problems / • View volume problem / • Bias problem. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type. For examination purposes, the important content is the vector and material relation: normal, light direction, view direction, reflection term, and color contribution.
Technical commentary	This slide is about Properties. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. The terms describe local shading inputs: ■ Shadow maps are static when scene and lighting are static / ■ Shadow quality depends on shadow map resolution / ■ Shadow mapping has three main problems / • View volume problem / • Bias problem. Surface position, normal, light direction, view direction, and material response combine to produce the final color contribution.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Exam-grade answer	A strong answer for 'Properties' names the surface point, normal, light direction, view direction, material parameters, and the resulting ambient, diffuse, or specular contribution.
Common trap	Do not mix up normal direction, light direction, and view direction; changing one changes the lighting term in a different way.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Properties'?
Page 50	View Volume Problem
Source cue	■ When vertex lies outside light source's view volume, required depth information is missing / ■ Possible remedies / • Use cubic shadow map (similar to cubic environment map) / • Use only spotlights

Professor-style explanation	The slide 'View Volume Problem' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ When vertex lies outside light source's view volume, required depth information is missing / ■ Possible remedies / • Use cubic shadow map (similar to cubic environment map) / • Use only spotlights. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about View Volume Problem. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ When vertex lies outside light source's view volume, required depth information is missing / ■ Possible remedies / • Use cubic shadow map (similar to cubic environment map) / • Use only spotlights. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'View Volume Problem' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'View Volume Problem' works per object, per image region, per ray, or per fragment?
Page 51	Bias Problem 1/2
Source cue	Incorrect self-shadowing occurs for / all fragments with $\text{ShadowMap}(x',y') \approx z'$ / Can be corrected by / increasing depth precision of z-buffer / applying z-bias

Professor-style explanation	The slide 'Bias Problem 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Incorrect self-shadowing occurs for / all fragments with $\text{ShadowMap}(x',y') \approx z'$ / Can be corrected by / increasing depth precision of z-buffer / applying z-bias. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image. For examination purposes, the important content is state plus binding plus shader interface: OpenGL draws from the currently bound objects and active settings.
Technical commentary	This slide is about Bias Problem 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Incorrect self-shadowing occurs for / all fragments with $\text{ShadowMap}(x',y') \approx z'$ / Can be corrected by / increasing depth precision of z-buffer / applying z-bias. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Exam-grade answer	A strong answer for 'Bias Problem 1/2' names the OpenGL object or state involved, where the data lives, which shader or pipeline stage consumes it, and how a wrong binding would show up visually.
Common trap	Do not assume an OpenGL call is enough by itself; the result depends on the complete active context state and currently bound resources.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Bias Problem 1/2'?
Page 52	Bias Problem 2/2
Source cue	It is difficult to choose a good z-bias / $\text{ShadowMap}(x',y') + \text{bias}$? / ■ good / ■ too low / ■ too high

Professor-style explanation	The slide 'Bias Problem 2/2' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. The terms describe light-space visibility: It is difficult to choose a good z-bias / $\text{ShadowMap}(x',y') + \text{bias} ?$ / ■ good / ■ too low / ■ too high. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image. For examination purposes, the important content is light-space visibility: a point is shadowed because another object blocks the path from the light.
Technical commentary	This slide is about Bias Problem 2/2. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. The terms describe light-space visibility: It is difficult to choose a good z-bias / $\text{ShadowMap}(x',y') + \text{bias} ?$ / ■ good / ■ too low / ■ too high. A light, an occluder, a receiver, and a stored or computed visibility representation determine where illumination is blocked.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Exam-grade answer	A strong answer for 'Bias Problem 2/2' names the light, blocker, receiver, visibility representation, and the reason a region receives reduced direct illumination.
Common trap	Do not explain a shadow only from the camera view; the key visibility test is usually from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Bias Problem 2/2'?
Page 53	Aliasing Problem 1/3
Source cue	■ Aliasing occurs when shadow map is undersampled(i.e., a texel in shadow map covers several / pixels) / ■ Particularly bad when camera and light source are facing each other

Professor-style explanation	The slide 'Aliasing Problem 1/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: ■ Aliasing occurs when shadow map is undersampled(i.e., a texel in shadow map covers several / pixels) / ■ Particularly bad when camera and light source are facing each other. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes. For examination purposes, the important content is the camera-to-screen chain: view coordinates, projection matrix, clip coordinates, perspective divide, normalized device coordinates, and viewport mapping.
Technical commentary	This slide is about Aliasing Problem 1/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: ■ Aliasing occurs when shadow map is undersampled(i.e., a texel in shadow map covers several / pixels) / ■ Particularly bad when camera and light source are facing each other. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Exam-grade answer	A strong answer for 'Aliasing Problem 1/3' explains the mapping from view space through clip space and perspective divide to normalized and viewport coordinates.
Common trap	Do not confuse projection with viewport mapping; projection creates clip coordinates and the perspective divide comes before final screen mapping.
Check yourself	Can you explain how 'Aliasing Problem 1/3' changes positions before rasterization?
Page 54	Aliasing Problem 2/3
Source cue	■ Filter (= weighted average formation over neighbors) of a depth valuedoes not make sense / ■ Percentage Closest Filtering (PCF) / • Filter results of shadow tests (weighted average of comparison results) / • Bias selection is made even more difficult / ■ Choosing the filter kernel

Professor-style explanation	The slide 'Aliasing Problem 2/3' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: ■ Filter (= weighted average formation over neighbors) of a depth value does not make sense / ■ Percentage Closest Filtering (PCF) / • Filter results of shadow tests (weighted average of comparison results) / • Bias selection is made even more difficult / ■ Choosing the filter kernel. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about Aliasing Problem 2/3. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: ■ Filter (= weighted average formation over neighbors) of a depth value does not make sense / ■ Percentage Closest Filtering (PCF) / • Filter results of shadow tests (weighted average of comparison results) / • Bias selection is made even more difficult / ■ Choosing the filter kernel. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for 'Aliasing Problem 2/3' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether 'Aliasing Problem 2/3' works per object, per image region, per ray, or per fragment?
Page 55	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?

Page 56	Aliasing Problem 3/3
Source cue	■ Averaging Filter / ■ Percentage Closest Filtering
Professor-style explanation	The slide 'Aliasing Problem 3/3' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. The terms describe sampled data access: ■ Averaging Filter / ■ Percentage Closest Filtering. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations. For examination purposes, the important content is coordinate-driven lookup: texture coordinates select stored texels, sampler state reconstructs values, and the shader interprets the result.

Technical commentary	This slide is about Aliasing Problem 3/3. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. The terms describe sampled data access: ■ Averaging Filter / ■ Percentage Closest Filtering. Coordinates select texels, sampler state controls reconstruction, and the shader interprets the fetched value as color, normal, material, or other data.
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Exam-grade answer	A strong answer for 'Aliasing Problem 3/3' names the texture data, texture coordinates, filtering or wrapping state, and shader interpretation of the sampled value.
Common trap	Do not reduce texturing to 'putting an image on an object'; coordinates, sampler state, mipmaps, and shader meaning are part of the concept.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Aliasing Problem 3/3'?

Page 57	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Exam-grade answer	A strong answer links this visual or title page to the closest surrounding concept slide and names the topic transition it introduces.
Common trap	Do not invent details that are not visible in the extracted text; use this page as a boundary marker and rely on adjacent slides for technical content.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 58	■ Shadows in Visual Perception

Source cue	• Enhance depth perception and object relationships / • Create appealing visual effects and atmosphere / ■ Shadow Definitions and Types / • Shadow-casting (Occluder) vs. shadow-receiving (Receiver) objects / • Self-shadowing: Same object casts and receives shadow
Professor-style explanation	The slide '■ Shadows in Visual Perception' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: • Enhance depth perception and object relationships / • Create appealing visual effects and atmosphere / ■ Shadow Definitions and Types / • Shadow-casting (Occluder) vs. shadow-receiving (Receiver) objects / • Self-shadowing: Same object casts and receives shadow. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing. For examination purposes, the important content is the comparison rule: which candidate is visible according to depth, ordering, image subdivision, or ray intersection.
Technical commentary	This slide is about ■ Shadows in Visual Perception. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: • Enhance depth perception and object relationships / • Create appealing visual effects and atmosphere / ■ Shadow Definitions and Types / • Shadow-casting (Occluder) vs. shadow-receiving (Receiver) objects / • Self-shadowing: Same object casts and receives shadow. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Exam-grade answer	A strong answer for '■ Shadows in Visual Perception' identifies the competing candidates, the space in which the algorithm works, and the rule that selects the visible result.
Common trap	Do not assume the first generated primitive is visible; visibility requires an ordering, depth, region, or ray comparison.
Check yourself	Can you decide whether '■ Shadows in Visual Perception' works per object, per image region, per ray, or per fragment?
Page 59	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail. For examination purposes, this page should be linked to the nearest surrounding text-heavy slides: it introduces a topic boundary or visual example, while the neighboring pages provide the precise vocabulary and algorithmic details.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for 'Literature and other sources used in this chapter' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip 'Literature and other sources used in this chapter' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 60	■ Text Books
Source cue	• P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Research Publications / • Crow, F., Shadows Algorithms for Computers Graphics, SIGGRAPH 1977. / • Lance Williams: Casting curved shadows on curved surfaces, SIGGRAPH 1978.

Professor-style explanation	The slide '■ Text Books' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. The listed sources support the chapter content: • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Research Publications / • Crow, F., Shadows Algorithms for Computers Graphics, SIGGRAPH 1977. / • Lance Williams: Casting curved shadows on curved surfaces, SIGGRAPH 1978. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide. For examination purposes, the important content is source attribution and vocabulary: references tell where precise definitions and standard algorithms come from.
Technical commentary	This slide is about ■ Text Books. The slide lists source material or chapter references that support the technical content and give names for further reading. The listed sources support the chapter content: • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Research Publications / • Crow, F., Shadows Algorithms for Computers Graphics, SIGGRAPH 1977. / • Lance Williams: Casting curved shadows on curved surfaces, SIGGRAPH 1978. They point to the books, papers, or resources behind the definitions and algorithms used in the lecture.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Exam-grade answer	A strong answer for '■ Text Books' identifies what kind of source is listed and which course concept or algorithm that source supports.
Common trap	Do not skip '■ Text Books' if it names a standard algorithm or source that defines terminology used later in the chapter.
Check yourself	Can you identify which source or topic '■ Text Books' points to for deeper study?

10.1 Definitions

Source location: Section 10.1

Commentary

A shadow is not simply a dark texture. It is evidence that light visibility is blocked. Important distinctions include hard versus soft shadows, umbra versus penumbra, and geometric versus precomputed approaches.

For real-time graphics, shadows are usually approximated. The approximation must answer a visibility question cheaply enough for interactive rendering.

Mental model

Shadows are light-space visibility made visible in the camera image.

Check yourself

Why is shadow computation related to visibility determination?

10.2 Ground Plane Shadows

Source location: Section 10.2

Commentary

Ground plane shadows project an object onto a receiving plane. This is simple and can look convincing in restricted scenes, but it assumes a known planar receiver and does not handle general shadowing between arbitrary objects.

The method is useful pedagogically because it makes the geometric nature of shadows explicit: a light source, an occluder, and a receiver define where the shadow lands.

Mental model

A planar shadow is projected geometry on a known receiver.

Check yourself

What scene limitation makes ground plane shadows less general than shadow maps?

10.3 Light Maps

Source location: Section 10.3

Commentary

Light maps store precomputed lighting or shadow information in textures. They can be efficient at runtime, but they are limited when lights, objects, or geometry move dynamically.

This is another example of a recurring graphics tradeoff: precompute for speed, but lose flexibility. Real-time rendering often mixes precomputed and dynamic techniques.

Mental model

A light map spends storage and preprocessing to save runtime shading work.

Check yourself

Why are light maps less suitable for fully dynamic moving lights?

10.4 Shadow Volumes

Source location: Section 10.4

Commentary

Shadow volumes construct the volume of space blocked from a light by an occluder. The camera view can then determine whether visible points lie inside that volume. Stencil buffer techniques are often associated with this method.

The strength is geometric precision for hard shadows. The cost is handling silhouette edges, volume construction, and robust stencil operations. It is a good contrast to shadow maps, which are image-based from the light view.

Mental model

A shadow volume is the 3D region where the light cannot reach.

Check yourself

Why are silhouette edges important for constructing shadow volumes?

10.5 Shadow Maps

Source location: Section 10.5

Commentary

Shadow mapping renders the scene from the light's point of view and stores depth. During the camera pass, a surface point is transformed into light space and compared against the stored depth. If it is farther than what the light saw, it is in shadow.

This method is practical and widely used, but it has artifacts. Shadow acne comes from precision/self-comparison issues. Bias can reduce acne but too much bias causes detached shadows. Resolution, filtering, and light projection strongly affect quality.

Mental model

Shadow maps reuse the depth-buffer idea, but from the light's camera.

Check yourself

What exactly is stored in a shadow map?

End-of-lecture summary

If you remember only one thing from Lecture 10, remember this: This lecture explains shadows as a visibility problem from the light source. A point is lit if the light can see it; it is shadowed if another object blocks that visibility.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.